高等学校本科应用型教材

计算机网络原理

张基温

高 等 教 育 出 版 社

内 容 提 要

本书是一本采用全新体系结构的计算机网络基础教材。全书共分为4篇，分别从4个角度观察计算机网络，理解计算机网络的工作原理：第1篇是在一个平面上观察计算机网络，分别介绍计算机网络的两个基本元素——链路和节点上的基本通信技术；第2篇是从立面上观察计算机网络，主要介绍几种计算机网络的体系结构；第3篇是从计算机网络工作时通信双方的关系上观察计算机网络的工作原理；第4篇是从实现的角度观察计算机网络的工作原理。这4篇将计算机网络的基本原理分解成相对独立的4个层次。读者每学习完一个层次的内容，对计算机网络工作原理的认识就会上升到一个新的高度。通过以上4个方面的学习，读者将会建立全面的、较为深刻的计算机网络的基本概念，掌握计算机网络的基本技术原理。

本书结构清晰、概念清楚、取材贴近现实技术，适合作为高等院校计算机专业、信息类专业、自动控制专业、管理工程专业及相关专业的教材，也可供有关工程技术人员学习参考。

前　言

20 世纪 50 年代末,通信与计算这样两个原来似乎互不相干的领域几乎同时向对方伸出了热情之手。一方面,为了提高计算机系统的可靠性并使用户能够享用远地的计算资源,开始用通信线路将计算机连接起来;另一方面,长期受机械式自动交换机容量、成本和扩展不便所困扰的通信系统,开始到与计算机技术密不可分的数字技术领域去寻找出路。从此,计算机技术和通信技术开始联姻。可是,当时谁也没有想到,这样一个结合,竟产生了一个奇迹,在短短的 20 年间,电子商务、电子政务、虚拟图书馆、数字地球、远程教育……这些建立在计算机网络基础上的"社会",从根本上改变了人们的时空观念和生活方式,将人类社会推向了一个新的时代——信息时代,或者称为信息网络时代。在这场技术革命中,最有影响力的当是计算机网络。

计算机网络诞生于 20 世纪 60 年代。然而 1983 年著名的开放系统互连参考模型 OSI/RM 的出现,才使它进入一个新的发展时期。从此,计算机网络的科学体系开始建立,并逐渐形成有关专业的一门独立课程。自然,这门课程的核心就是 ISO 制定的 OSI/RM 7 层协议。20 多年过去了,在这 20 多年间,计算机技术和通信技术在各自的迅速发展中进一步融合,出现了通信技术计算机化、计算机技术网络化的新局面,一些新的、有前途的技术和现象正在浮出水面,淡化着原来的课程体系,并丰富着它的内容。它们是:

Internet 的迅速扩张和渗透,造成了 TCP/IP 一统天下的现实;

宽带、高速、高可靠性的光传输技术;

方便、简单的无线通信技术;

以三网合一为目标的宽带网络技术及其所引发的接入技术;

已经广泛应用的 ATM 交换和 IP 交换技术;

基于客户机/服务器模式的应用架构以及其他新的应用技术。

目前,计算机网络已经成为一个重要的研究和学习领域。它不仅是一套理论,也是一种技术。对于广大学习者来说,要学习好这门课程,不仅要学习一些概念,掌握计算机网络的基本原理;掌握一些技能,具备工程实践能力;最好还应当清楚它的实现方法,具备研究开发能力。对于一个教学单位来说,要教好这门课程,不仅要安排课堂教学,还要安排一定的工程实践教学和供学生进行研究开发的实验。实践证明,将计算机网络课程的教学分为理论教学、工程实训和实验

研究三个教学环节是非常有益的。作者的实践,得到了社会的认可,被列为江苏省“十五”教育科学规划项目,也得到了高等教育出版社的大力支持。本书就是本人构思的计算机网络的三个教学环节中用于理论教学的教材。

计算机网络的理论和知识本身就是网络结构的,既丰富复杂,又相互交织,这给教学带来不少困难。瞎子摸象是一个富有寓意的故事。它告诉我们,要认识事物,如果能从更多的方面进行观察,认识就会比较全面。在教学实践中,受这个故事的启迪,作者把计算机网络的理论学习分为4个层次,让学生分别从不同的角度去揣摩计算机网络,每一个层次都会在前一个层次的基础上,从更高的视角上,在更深的理论水平方面把学习推向新的高度和深度。这4个学习层次分别是:

从平面结构上观察计算机网络,分别讨论在节点和链路上的通信原理,让学习者构建最基本的网络通信知识;

从立面上观察计算机网络,讨论计算机网络的体系结构,将学习推向一个新的高度;

从计算机网络的工作模式观察计算机网络的工作方式和原理,建立客户机/服务器工作模式的概念,为计算机网络的应用和下一篇的学习打下基础;

从设计与实现的角度讨论网络程序设计的基本方法,为学习者进行计算机网络的开发奠定基础。

这4个层次分别对应本书的1~4篇,为学习者构建一个学习计算机网络的知识环境。

协议是计算机网络的灵魂。在本书的写作中,突出了协议,并且以TCP/IP为核心,突出了基于TCP/IP协议的应用开发能力的培养。同时也注意了对其他先进技术(如ATM)的介绍。为了能够在有限的学时内介绍完核心内容,本书忽略了一些已经陈旧(如x.25、帧中继等)以及在别的课程中要介绍的(如安全技术)内容。

本书作为建立新的计算机网络教材体系的尝试,必定会存在许多不足。作者殷切地期望得到更多的评比和改进意见,以使本书得到进一步的完善。

庄春兴、陈瞡、杨波、陶利民、江森林参加了本书的校对、校订等工作。高等教育出版社的编辑同志在本书的出版过程中付出了辛勤的劳动。在此谨向他(她)们表示诚挚的谢意。

张基温

2003年7月

目　　录

第1篇　计算机网络组成原理

第1章　计算机网络概述 …… (3)
1.1　计算机网络及其分类 …… (3)
1.1.1　计算机网络及其功能 … (3)
1.1.2　计算机网络的分类 …… (3)
1.2　计算机网络元素 …… (7)
1.2.1　网络节点 …… (7)
1.2.2　传输链路 …… (9)
1.2.3　协议…… (10)
习题 …… (12)
第2章　链路上的通信技术 …… (13)
2.1　基本通信方式 …… (13)
2.1.1　交互方式…… (13)
2.1.2　并行传输与串行传输…… (14)
2.1.3　串行通信中的同步控制…… (14)
2.2　数据信号分析与编码 …… (18)
2.2.1　数据信号分析与信道参数…… (18)
2.2.2　信道的主要性能参数…… (20)
2.2.3　数字信号的模拟调制…… (25)
2.2.4　模拟信号的数字编码——脉冲编码调制技术…… (27)
2.2.5　数字编码…… (28)
2.3　信道的多路复用技术 …… (30)
2.3.1　频分多路复用 FDM 技术…… (30)
2.3.2　时分多路复用 TDM 技术…… (31)
2.3.3　光波分多路复用 WDM 技术…… (43)
2.4　差错控制 …… (45)
2.4.1　差错产生的原因与基本对策…… (45)
2.4.2　循环冗余码校验…… (46)
2.4.3　差错控制协议…… (48)
2.5　流量控制与滑动窗口协议 … (50)
2.5.1　发送器窗口的工作原理…… (50)
2.5.2　接收器窗口的工作原理…… (50)
2.6　高级数据链路控制协议 HDLC …… (52)
2.6.1　数据链路连接管理方式…… (52)
2.6.2　HDLC 配置和数据传输工作方式…… (53)
2.6.3　HDLC 帧格式 …… (54)
习题 …… (55)
第3章　中间节点上的通信技术 …… (57)
3.1　交换节点上的通信 …… (57)
3.1.1　电路交换…… (57)
3.1.2　存储－转发交换…… (58)
3.1.3　分组交换的虚电路服务和数据报服务…… (60)
3.1.4　交换机的功能…… (63)
3.1.5　交换机实现技术…… (64)
3.2　路由节点上的通信 …… (67)
3.2.1　路由表与路由算法…… (68)

3.2.2 路由器的基本结构…… (72)
3.2.3 路由器技术的演进…… (73)
习题 …… (75)

第 2 篇 计算机网络体系结构

第 4 章 ISO/OSI参考模型 …… (79)
4.1 关于网层次结构的基本概念 …… (79)
4.1.1 分层原则…… (79)
4.1.2 服务与接口…… (79)
4.1.3 各层设计中的流量控制…… (82)
4.2 ISO/OSI参考模型结构 …… (84)
4.2.1 OSI的低层 …… (86)
4.2.2 OSI的高层 …… (89)
4.2.3 传输层…… (90)
4.2.4 OSI/RM 各层中的数据流动…… (96)
习题 …… (98)
第 5 章 IEEE 802 模型与局域网 … (100)
5.1 局域网模型 …… (100)
5.1.1 IEEE 802 模型 …… (100)
5.1.2 IEEE 802 协议标准 …… (102)
5.1.3 信道的多点共享访问控制 …… (102)
5.2 令牌网 …… (103)
5.2.1 令牌环网与 IEEE 802.5 标准 …… (103)
5.2.2 令牌总线网与 IEEE 802.4 标准 …… (106)
5.3 以太网 …… (106)
5.3.1 CSMA/CD 协议 …… (107)
5.3.2 IEEE 802.3 与 10 Mb/s 以太网 …… (111)
5.3.3 100 Mb/s以太网 …… (115)
5.3.4 IEEE 802.3z 与千兆位以太网 …… (118)
5.4 交换式局域网 …… (119)
5.4.1 局域网的分段与交换式局域网 …… (119)
5.4.2 交换式局域网工作原理 …… (122)
5.4.3 虚拟局域网 VLAN …… (125)
5.5 无线局域网 WLAN …… (127)
5.5.1 WLAN 传输方式和频段范围 …… (127)
5.5.2 无线局域网的结构 …… (128)
5.5.3 IEEE 802.11 协议 …… (129)
习题 …… (132)
第 6 章 Internet 与 TCP/IP 体系结构 …… (133)
6.1 Internet 概述 …… (133)
6.1.1 TCP/IP 概述 …… (133)
6.1.2 TCP/IP 与 OSI/RM 的比较 …… (135)
6.2 网络接口协议 SLIP 和 PPP … (136)
6.2.1 SLIP 协议 …… (137)
6.2.2 PPP 协议 …… (138)
6.3 网际层 …… (138)
6.3.1 IP 地址结构 …… (139)
6.3.2 IP 地址解析 …… (141)
6.3.3 IPv4 分组格式 …… (144)
6.3.4 网际控制消息协议 ICMP …… (146)
6.3.5 IPv6 …… (149)
6.4 IP 路由 …… (155)
6.4.1 路由器工作概述 …… (155)
6.4.2 路由信息协议 RIP …… (159)
6.4.3 开放式最短路优先

协议 OSPF …… (160)
6.4.4 边界网关协议 BGP …… (165)
6.4.5 基于路由器的防火墙技术 …… (168)
6.5 TCP 层 …… (170)
6.5.1 TCP 协议 …… (170)
6.5.2 UDP 协议 …… (176)
6.5.3 TCP/UDP 端口号的分配方法 …… (177)
习题 …… (178)
第 7 章 ATM 网络 …… (180)
7.1 ATM 技术特点 …… (180)
7.1.1 ATM 与 STM …… (180)
7.1.2 信元传输 …… (180)
7.1.3 虚路径和虚信道 …… (180)
7.1.4 服务质量 QoS …… (182)
7.1.5 ATM 网络接口 …… (183)
7.2 ATM 协议参考模型 …… (185)
7.2.1 物理层 …… (185)
7.2.2 ATM 层 …… (187)
7.2.3 ATM 适配层 (AAL) …… (188)
7.3 ATM 局域网仿真 LANE …… (189)
7.3.1 LANE 网络的组成及体系结构 …… (189)
7.3.2 LANE 的帧格式 …… (191)
7.3.3 LANE 中的连接 …… (191)
7.3.4 LANE 的初始化 …… (192)
7.3.5 地址解析与数据传播 …… (193)
习题 …… (194)

第 3 篇　计算机网络工作模式

第 8 章 进程间通信的客户机/服务器模式 …… (197)
8.1 进程 …… (197)
8.1.1 进程及其状态 …… (197)
8.1.2 进程的特点 …… (198)
8.2 客户机/服务器通信模型 …… (198)
8.2.1 客户机/服务器模型概述 …… (198)
8.2.2 客户机/服务器计算模式的优点 …… (200)
8.2.3 客户机/服务器的应用方式 …… (200)
8.3 中间件 …… (201)
8.3.1 中间件的概念 …… (201)
8.3.2 中间件的类型 …… (202)
习题 …… (203)
第 9 章 客户机/服务器模式应用举例 …… (204)
9.1 远程登录 TELNET …… (204)
9.1.1 本地注册与远程登录 …… (204)
9.1.2 NVT 字符集 …… (206)
9.1.3 TELNET 工作方式和用户命令 …… (210)
9.2 文件传输协议 FTP …… (211)
9.2.1 FTP 模型 …… (211)
9.2.2 FTP 文件传输过程 …… (212)
9.3 电子邮件传送协议 …… (218)
9.3.1 电子邮件系统的基本原理 …… (218)
9.3.2 简单邮件传输协议 SMTP …… (220)
9.3.3 其他几个重要的电子邮件协议 …… (222)
9.4 简单网络管理协议 SNMP …… (224)
9.4.1 网络管理概述 …… (224)
9.4.2 SNMP 管理模型 …… (226)
9.4.3 SMI …… (227)
9.4.4 MIB …… (229)

9.4.5 SNMP 的工作机制 …… (231)
9.5 超文本传输协议 HTTP …… (235)
9.5.1 超文本与 WWW …… (235)
9.5.2 B/S 计算模式与浏览器结构 …… (236)
9.5.3 HTTP 的工作机制 …… (239)
习题 …… (243)

第 4 篇 计算机网络应用程序设计

第 10 章 插口 API 基础 …… (247)
10.1 基本概念 …… (247)
10.1.1 插口 API …… (247)
10.1.2 UNIX/Linux 进程 …… (248)
10.1.3 信号及其处理 …… (251)
10.2 基本插口函数 …… (253)
10.2.1 创建 Socket——服务绑定：socket() …… (254)
10.2.2 本地地址绑定：bind() …… (256)
10.2.3 建立插口连接——绑定远端服务器地址：connect() …… (260)
10.2.4 插口被动转换：listen() …… (262)
10.2.5 从被动插口的完成队列中接收一个连接请求：accept() …… (264)
10.2.6 基本插口 I/O 函数 …… (266)
10.2.7 关闭插口通道与撤销插口 …… (268)
习题 …… (270)
第 11 章 TCP/UDP 的插口程序设计 …… (271)
11.1 TCP 有限状态机 …… (271)
11.1.1 TCP 有限状态机的概念 …… (271)
11.1.2 状态转换分析 …… (272)
11.1.3 TCP 连接的建立和释放 …… (273)
11.2 TCP 插口编程初步 …… (281)
11.2.1 TCP 的 C/S 模型时序图 …… (281)
11.2.2 一个简单的 TCP 网络通信程序 …… (282)
11.3 TCP 输入/输出模式 …… (291)
11.3.1 阻塞模式下的输入/输出与超时控制 …… (291)
11.3.2 非阻塞模式下的输入/输出模式 …… (299)
11.3.3 TCP 输入/输出多路复用 …… (303)
11.4 基于 UDP 的插口程序设计 …… (307)
11.4.1 UDP 编程模式 …… (307)
11.4.2 一个简单的 UDP 客户机/服务器程序 …… (309)
11.4.3 非阻塞模式下的 UDP 客户/服务器程序 …… (312)
习题 …… (315)
附录 名词术语英汉对照表 …… (316)
参考文献 …… (322)

第 1 篇

计算机网络组成原理

计算机网络,说复杂,其实也很简单。把它看成一个平面,实际上只有两种物理元素:链路和节点。计算机网络中的通信过程就是在链路和节点上实现的。为了实现通信,必须建立节点和链路上的通信规则,这些通信规则称为协议。这一篇主要介绍在链路和节点上的通信技术。主要内容有:

计算机网络的基本概念、分类及基本元素(第 1 章);

链路上的通信技术(第 2 章);

中间节点上的通信技术(第 3 章)。

第 1 章　计算机网络概述

本章首先介绍计算机网络的基本概念、建立它的目的以及对它进行分类的各种方法,以便读者对它有一个初步的认识。

1.1　计算机网络及其分类

1.1.1　计算机网络及其功能

计算机网络是计算机技术与通信技术相结合的产物,或者说是用通信介质将多台计算机连接起来所形成的计算机系统。这里,连接有两重含义:一是指通过传输介质和传输设备建立的物理上的连接;一是由一些网络软件实现的逻辑上的连接。之所以要进行连接,是为了实现下列功能:

(1) 通信。即在计算机之间传送数据。例如,文件传送(FTP)、E-Mail、网络传呼(ICQ、OICQ)、IP 电话、WWW、电子布告栏等。

(2) 资源共享。即实现计算机硬件资源、软件资源和信息资源的异地互用。"共享"是指可以互通有无和异地使用。例如,使用异地的大型计算机进行本地计算机无法进行的计算,使用浏览器从其他计算机中获取信息等。这样,除互通有无外,还能均衡负载,使网络上各资源的"忙"、"闲"得到合理调整。

(3) 提高计算机系统的可靠性。在计算机网络中各台计算机间可以互为后备,从而提高了计算机系统的可靠性。

1.1.2　计算机网络的分类

计算机网络是一种复杂的系统。人们对于复杂系统的认识,一开始只能像盲人摸象,认识一个片面,但若能从不同的角度触摸几次,认识就会趋向全面。下面介绍计算机网络的各种分类方法,目的还在于为初学者拓宽视野。

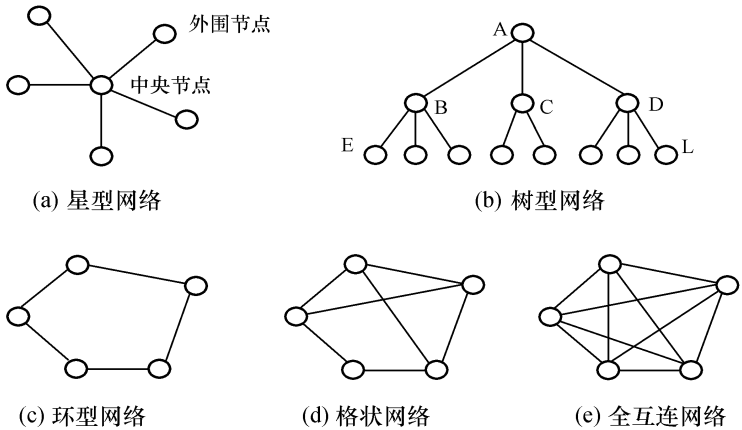
1. 按拓扑结构分类

拓扑结构是计算机网络的重要特性。从拓扑学的观点,网络是由一组节点(node)和连接节点的链路(link)组成。在计算机网络中,计算机作为节点,连接

计算机的通信线路作为链路,形成计算机的地理分布和互连关系上的几何排序(几何构形)。这种计算机与链路之间的拓扑关系,称为计算机网络的拓扑结构。计算机网络的拓扑结构有许多种。但是,按照网络中一条链路所能连接的计算机的台数,可以把计算机网络归结为两大类:链路型网络(点到点的结构)和广播型网络(多点共享链路结构)。

(1) 链路型网络(点到点的结构)

在点到点的结构中,一条链路只能连接两个节点。这样,两点之间要么直接通信,要么必须通过中间节点转发。图 1.1 所示为几种点到点的计算机网络结构。



(a) 星型网络　(b) 树型网络

(c) 环型网络　(d) 格状网络　(e) 全互连网络

图 1.1　点到点的网络拓扑结构

星型结构是一种以中央节点为中心,把若干外围节点连接起来的辐射式互连结构,中央节点实施对全网的控制,并分别通过单独的链路与各个外围节点相连接。其拓扑特点是,中央节点与多条链路连接,其余节点只与一条链路连接。如图 1.1(a) 所示,由于各外围节点分别用线缆与中央节点直接连接,因而在星型结构中数据的传输不会在线路上发生碰撞,并且系统比较容易扩充,但中央节点会成为系统的“瓶颈”和可靠工作的最薄弱环节。

树型结构由星型结构衍变而来。如图 1.1(b) 所示,它实际上是多个星型结构的级联组合。树型结构的特点是网络中有多个中心节点,但主要的数据流通是在网络的各分支之间进行,形成一种分级管理的集中式网络,适宜于各种管理部门进行分级数据传送的场合。其拓扑特点是,多个中心节点与多条链路连接,其余节点(末端节点)只与一条链路连接。树型结构的优点是连接容易、管理简单、维护方便,缺点是共享能力差、可靠性低。

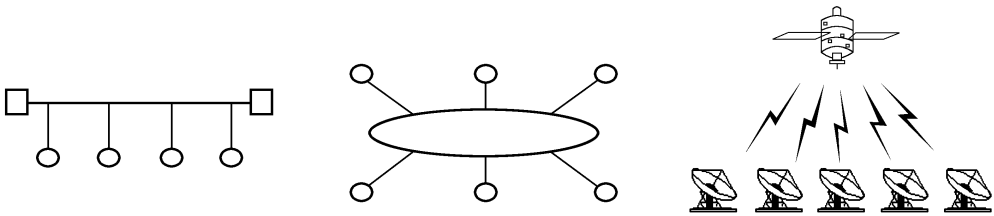
环型结构(如图 1.1(c) 所示)是把所有节点连接成环形。其拓扑特点是,每一个节点都与两条链路连接。在这种结构中,任意两点之间形成两条路径,当某

一链路有故障时,还可以通过另一条路径进行通信。

格状结构(如图1.1(d)所示)是所有节点具有两个或两个以上直接通路的拓扑结构。全互连结构(如图1.1(e)所示)是所有节点之间都有直接通路的拓扑结构。这两种结构也称网状结构,具有较高的可靠性,但网络结构复杂,链路多,投资大。

(2) 广播型网络

广播型网络的特点是,通信线路为多个节点共享。这样,一个节点发送的信息可以传输到其他所有的节点;而当有两个以上节点同时发送信息时,便会引起冲突。总线型结构是一种应用最普遍的广播型网络拓扑结构。图1.2给出了3种典型的广播型网络结构。



(a) 总线型网络　　(b) 环型总线网络　　(c) 无约束型网络

图1.2　广播型网络结构

在总线型结构中各个计算机网络节点的设备用一根总线挂接起来。总线型结构目前在局域网中应用很广,有如下一些特点:

节点的插入或拆卸方便,易于扩充;

不需要中央控制器,有利于分布式控制,某个节点发生故障时对整个系统影响很小,网络的可靠性高;

总线自身的故障对系统是毁灭性的,因而要求较高的安装质量。

当网络的总线首尾相连成闭合的环路时,这种总线型结构称为环型总线结构,如图1.2(b)所示。

卫星通信和微波通信采用电磁波传输信息。这种结构属无约束型或称任意型的广播式传输结构,如图1.2(c)所示。

2. 按覆盖地域分类

网络按照其覆盖地域的大小可以分为微微网(Piconet)、个人局域网(PAN, Personal Area Network)、局域网(LAN, Local Area Network)、城域网(MAN, Metropolitan Area Network)和广域网(WAN, Wide Area Network)。

(1) 微微网是由采用蓝牙技术的设备以特定方式组成的网络,这种网络的

建立是从两台设备（如笔记本电脑和移动电话）的连接开始，同一时刻最多可以激活 8 台设备。

⑵ 个人局域网 PAN 是近年来随着各种短距离无线电技术的发展而提出的一个新概念，一般覆盖距离为 100 m 以内。

⑶ 局域网（又称局部地域网）LAN 指通信距离通常在中等规模的地理区域内（一般在 10 km 范围内）的网络，例如一幢办公楼、一座仓库、一所学校中的计算机网络。它能借助于具有中高速数据传输率的物理通信信道实现可靠通信。

⑷ 城域网 MAN 指地理覆盖范围大约为一个城市的网络，其通信距离一般在 5～50 km 之内。

⑸ 广域网 WAN 又称远程网，一般指跨地区甚至延伸到整个国家和全世界的网络。

3. 按照网络中主机的台数分类

这是 Internet 中的分类方法，它按主机台数将网络分为 A、B、C 三类：

A 类是大型网络，网内主机最多可达 1 600 万台；

B 类是中型网络，网内主机最多可达 65 534 台；

C 类是小型网络，网内主机最多可达 254 台。

4. 其他分类方法

除上述分类方法外，还可以按下列的方法进行分类：

⑴ 按使用权限分类

按照网络系统的使用权限，计算机网络可以分为公用网和专用网（或称私用网）。公用网由电信部门组建，一般也由电信部门管理和控制，可以作为社会公用设施。专用网是不允许其他部门或单位使用的内部专用网络。专用网可以由使用部门或单位组建，也可以租用电信部门的传输线路建成。

⑵ 按公司的命名分类

如 IBM 网、ARPANet、ChinaNet、微软网等。

⑶ 按操作系统及其版本分类

如 UNIX 网、Linux 网、Windows NT、Novell NetWare 386 V3.12、NetWare 4.01 等。

⑷ 按所用技术分类

如 x.25、ALOHA、DDN、帧中继网、ISDN、ATM 等。

⑸ 按使用的介质分类

如无线网、光纤网等。

⑹ 按业务范围分类

如校园网、企业网、金桥网、教育科研网、经济网、科技网、医卫网等。

1.2　计算机网络元素

如前所述，网络可以抽象成由节点和链路组成的平面拓扑结构。

为了保证物理元素间正常地通信，使网络协调地工作，必须使元素之间遵守约定的规程和规则。这些规程和规则就是协议。

这一节介绍节点、链路和协议的基本概念。

1.2.1　网络节点

在复杂的网络中进行数据传输，从源节点往往要经过多个中间节点才能传送到目的节点。中间节点大致有4种：端节点、中继节点、交换节点和路由节点。

1. 端节点

端节点是进行数据处理的节点。在计算机网络中，这些节点应当具有两种功能：数据处理和通信。图1.3是两个端节点通过网络通信的示意图。其中：

数据终端设备DTE（Data Terminal Equipment）为具有一定数据处理能力的发送、接收设备，如计算机或各种终端设备；

数据通信设备DCE（Data Communication Equipment）为通信接口设备，在DTE与通信网之间提供信号变换及编码功能，并负责建立、维护和释放物理连接，如波形变换器、基带传输器、调制解调器等。

图1.3　端节点结构

2. 中继节点

信号在介质中传输时，随着传输距离的增加，幅度将会逐渐衰减，波形将会产生失真。中继器（Repeater）用于同类网络介质之间的互连，起到信号再生、放大作用。再生就是通过对失真的但仍可以辨认的波形的分析，重新生成原来的波形；放大就是将信号衰减了的幅度加以恢复。通过再生和放大能够使网络传输的距离范围得以扩大。图1.4简单地说明了中继器的基本工作原理。

图 1.4　中继器的工作原理

中继有一通一的中继和一通多的中继。图 1.4 所示就是一通一的中继。而集线器 (Hub) 就是一通多的中继器,或称多端口的中继器。Hub 所连接的各条链路之间的信号具有共享性,一个端口的输入信号,经整形、放大后可以转发到其他所有连接的链路或网段上,形成星型结构。目前 Hub 主要用于在星型结构的中央节点上连接多条无屏蔽双绞线,当某条线路或节点有故障时,可以简单地卸掉,不扩大故障范围,不影响其他节点的正常工作,便于维护。Hub 按其配置形式分,有独立型集线器、模块化集线器和堆叠式集线器三种;按其端口分,有 8 端口、16 端口、24 端口等几类;按其传输速率分,有 10 Mb/s、100 Mb/s 和 10/100 Mb/s 集线器等几种。

应当强调的是,经过集线器的信号都是重新整理过后再传送出去。不同的通信介质有不同的中继器,例如微波中继器、卫星中继器、用于同轴电缆的双口中继器 (连接两段同轴电缆) 和多口中继器 (用于扩展 3 个或 3 个以上网段) 以及用于多条双绞线连接的集线器等。

3. 交换节点

一个通信过程往往要经过多条链路之间的转接才能实现。转接由交换 (Switching) 节点实现。如在图 1.5 中,节点 A 到节点 B 之间的通信,要经过中间节点 C、D 的转接。简单地说,交换节点的功能是将一条链路上送来的数据有选择地转送到另外的一条链路上。复杂拓扑结构网络中的通信信道就是经过多个中间节点的转发—连接而实现的。

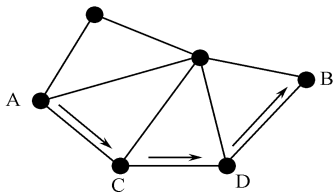


图 1.5　通过交换节点连接的通信

4. 路由节点

路由节点是一种特殊的节点。它位于网络之间,起连接网络的作用,属于所

连接的网络共有。如图1.6所示,节点R1连接了网络A、B、C、D,节点R2连接了网络C、E,节点R3连接了网络D、E。正是由于路由节点,才使互联网得以形成。如果把每个网络看成一个通路——链路,那么路由节点的分布就形成了互联网络的拓扑结构或框架。

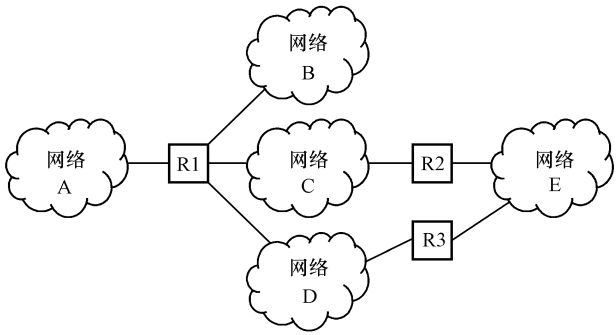


图1.6　用路由节点连接网络

路由器的网络连接功能主要体现在它能为到达的数据选择到达目的节点的路由。这就是路由节点称为路由器(Router)的原因。

通过第2篇的介绍读者将会知道,网络是按照层次结构设计和建造的,不同级别的功能被分布在不同的层次中实现。路由和交换虽然具有相似之处,但是是在不同的层次中实现的。从这点上讲,它们的概念有所不用。

1.2.2　传输链路

传输链路是网络中连接两个节点的直接信息通路,简称链路(Link)。数据在计算机网络中传输,往往要经过多条链路一段一段地传输。中间可能经过中继节点,也可能经过交换节点。

链路可能是物理的,也可能是逻辑的。采用多路复用技术可以把一条物理链路划分成多条逻辑链路。

通信介质就是搭载信号的传输媒介,是链路的物理基础。通信信号是10^4～10^{16} Hz频率范围内的电磁波。它们的基本传播途径有两种:

有线传输——在有限空间内传输;

无线传输——在自由空间中传输。

通信介质分为硬介质(双绞线电缆、同轴电缆、光缆等)和软介质(即空间介质,如微波通信、卫星通信、红外通信等)。表1.1列出了几种通信介质的性能比较。

表 1.1　几种通信介质的性能比较

性能＼通信介质	双绞线	同轴电缆(基带)	同轴电缆(宽带)	光纤	地面微波	卫星
带宽(Hz)	＜1 G	＜100 M	＜300 M	＜300 G	0.3～300 G	500 M
距离(km)	＜0.3	＜2.5	＜100	100	40～50	不受限制
抗强电干扰性	较差	高	高	极高	差	差
安装难易程度	易	中	中	较难	易	易
布局多样性	好	较好	较好	中	好	好
保密性	一般	好	好	极好	差	差
经济性	低	较低	较低	较高	中	较高
时延	小	小	小	小	小	大

1.2.3　协议

除了节点和链路这样两种物理元素外,网络还有一种非物理的元素——协议。

1. 协议及其内涵

凡是两个以上的对象或系统相互进行联系,都需要建立联系的规则或约定。例如,在大海中用旗语通信、聋哑人用哑语交流等都需要一套约定的规则和标准。这些为了通信或联系所制定的约定、规则和标准就被称为协议(Protocol)。实际上,人类的每一种语言都是一种协议。协议存在于任何通信过程中,它包含了所传输信息的格式(语法)、语义和定时(传输顺序)三个方面的约束。

(1) 语法。语法是关于信息格式的规定。可以说,在计算机网络中传输的信号是一些二进制数字流,为了让系统能够识别这些数字流,需要将信息按一定的格式进行组织和传输。例如,一个数据分组有多长、具体划分为几个字段等。两个不同的网络互连,即当一个数据分组被转换为数字信号以后从一个网络进入另一个网络时,必须进行数据格式的转换,这就需要根据两种网络协议的规定进行数据的分割与组装。

(2) 语义。在计算机网络中,传输的数据有用户信息和控制命令两种。语义用来规定命令和应答的含义。

(3) 定时。命令和应答需要一定的次序关系。定时用于规定命令、应答和状态变化的顺序和时间要求。

2. 协议的基本功能

在计算机网络中,协议的目的是确保通信过程顺利、安全、可靠地进行,因此

它的基本功能有以下几点：

(1) 分割与组装。在通信网络中，为了提高传输效率以及克服传输能力的限制，在发送端要将报文分割成一些适合网络传输的数据包——分组，并在分组时加上控制信息；在接收端要将数据分组重新组装成报文。

(2) 规定格式和特性。规定数据格式，规定网络接口的机械、物理、功能和规程特性等。

(3) 传输服务。满足一定的性能需求，如在传输过程中要求控制数据优先于信息数据等。不同的通信系统有不同的传输服务，例如，为了在一条线路上同时传输多路信号，就会要求系统按适当的顺序发送和接收报文，并规定同步方式和寻址方式等。

(4) 传输控制。包括流量控制、差错控制和连接控制。流量控制用于控制数据传输过程中发送方的发送速度和数量，解决信道上的拥塞问题；差错控制用于减少、检测和纠正错误；连接控制用于通信实体间连接的建立或拆除。

3. 协议的实现——网卡和网络操作系统

网络协议通常是由网卡和网络操作系统共同实现的。

(1) 网卡

网络接口卡(NIC，Network Interface Card)简称网卡，又称网络适配器，该电路板安装在计算机扩展槽中以连接计算机与网络，是网络设备(如工作站、服务器等)与网络的通信介质进行连接服务的设备。文件服务器、工作站都需使用一块网卡和通信介质进行连接，它主要实现网络协议中与通信相关的功能。网络正常工作时，网卡通过通信介质的端口监视网络的状态，接收传输介质上的信号。当网卡接收到有效的数字信号时，网卡会判断是否发给本站(是，则将数据通过网络设备的接口传输到网络设备；否，则放弃或按原方向转发)；另一方面，要适时将所连设备需发送的数据发送到网上，以实现网络设备间的通信。

网卡的选择应注意以下几个方面：

(a) 网卡的速度。网卡的速度表示它接收和发送数字信号的最高速度，如10 Mb/s、100 Mb/s、1 000 Mb/s等。普通小型共享式局域网中一般使用10 Mb/s的网卡；在交换式局域网或要传输宽带信号的网络应选用速度较快(如100 Mb/s)的网卡。

(b) 网卡的总线类型。目前，网卡按总线类型可分为ISA网卡和PCI网卡等。ISA网卡的数据宽度为16位，速度一般在10 Mb/s级，价格较低，多应用于小型局域网中；PCI网卡的数据宽度为32位，速度有10 Mb/s和100 Mb/s两种。文件服务器上所配网卡，一定要是性能良好的16位以上的网卡。

(c) 网卡的接口。网卡与不同的线缆连接时，需要不同的固定接口，如与双

绞线连接需有 RJ－45 接口，与细同轴电缆连接时需有 BNC 接口，等等。

(c) 网卡的驱动程序。网卡上的驱动程序应与操作系统相匹配，最好具有 PnP (即插即用) 功能。

为了适应网络设计的灵活性和对系统扩充后的适应能力，早期的各种网卡上都有多组开关或跳接线，用于网络接口参数的设置，而现在新型网卡多用无跳线结构，采用软件设置各种参数。此外，每种网卡都有相应的驱动程序。

(2) 网络操作系统

网络操作系统 (NOS，Network Operating System) 是网络用户与通信网络之间的接口。最早的网络操作系统仅仅用于网络上的文件管理，随着计算机网络的发展，网络操作系统的功能不断丰富和完善，为网上用户提供了便利的操作和管理平台。目前市场上的几种主要 NOS 为：Linux、UNIX 和 Microsoft 公司的 Windows XP 等。

习　　题

1.1　试给出一个计算机网络的定义。

1.2　计算机网络可以实现哪些功能？

1.3　计算机网络可以从哪几方面进行分类？

1.4　计算机网络的拓扑结构有哪几种？在不同的计算机网络拓扑结构中各采用什么样的通信控制方法？

1.5　比较中继节点和交换节点上传输方式的不同以及使用设备的不同。

1.6　查阅资料，比较各种不同的传输介质的适用范围。

1.7　列举生活中使用协议的例子。

1.8　查阅资料，列举操作系统和网卡的类型。

第 2 章　链路上的通信技术

一般说来,链路是指物理链路,即一条无源的点到点的物理线路段,中间没有任何交换节点。本章讨论一段链路中的数据传输技术,即将数据从一个节点传送到另一个节点的技术。这些技术包括：

源节点与目标节点之间的交互方式(单工、双工或半双工)；

数据的传输形式(并行传输/串行传输)；

同步控制方式(同步传输/异步传输)；

信号形式(信号分析与编码)；

在一条物理信道中传输多路信号——多路复用技术；

差错控制技术；

流量控制技术；

典型传输规程。

本章还将使用另一个术语——信道。读者应当注意信道与链路的微小区别。

2 1　基本通信方式

这一节先介绍链路上通信要解决的三个问题：

通信双方的交互形式；

数据线提供的传输条件是一个字节中的各位同时传输——并行传输,还是只能一位一位地传输——串行传输；

链路所连接的两方如何取得同步。

2 1 1　交互方式

信道的交互方式是指在一条物理信道上所允许的信号流动方向,有三种形式:信道两端的用户只能一方收,另一方发;双方可以同时进行收、发;双方虽然都可以进行收、发,但每一方的收和发只能交替进行,而不可同时进行。这样的三种情形分别称为单工方式、全双工方式和半双工方式。图 2 1 是这三种交互方式的示意图。

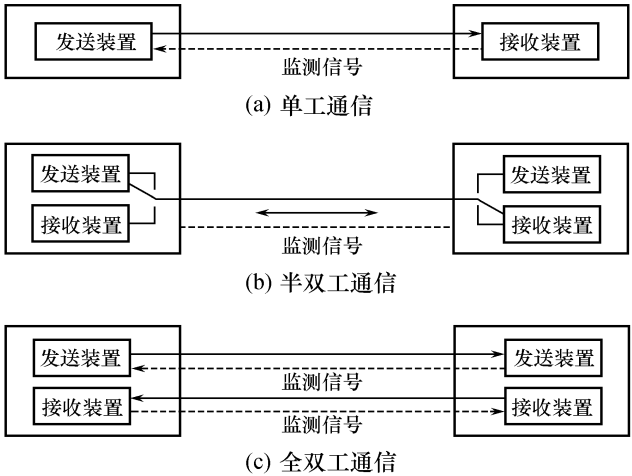


图 2 1　三种交互方式示意图

⑴　单工方式

单工方式的特点是单向通信,信号只能进行一个方向的流动,即只能固定地由一端发出,由另一端接收。如无线广播、电视等。

⑵　全双工方式

全双工方式的特点是同时双向,即通信双方都可同时发送和接收信号。全双工通信线路相当于 2 个单工线路的组合,例如电话就是全双工通信方式。

⑶　半双工方式

半双工方式的特点是不同时双向,也称双向交替方式,即通信双方都可收、发信息,但不可同时进行,同一时刻只限一个方向的信号传输,只能由一端发出,另一端接收。例如对讲机,讲话和听话不能同时进行。

2 1 2　并行传输与串行传输

在计算机中通常以字节或字为单位进行数据处理。在数据传输时,一条线路同一时刻只能传输信号的 1 位。如果将信号的各位分别在多条线路同时传输,则称为并行传输;如果只用一条线,让信号以位为单位顺序传输,就称为串行传输。图 2 2 为串/并行传输的示意图。

2 1 3　串行通信中的同步控制

1　同步的概念

为了让计算机的各部件间协调工作,必须靠时钟对它们分别定时和定序。

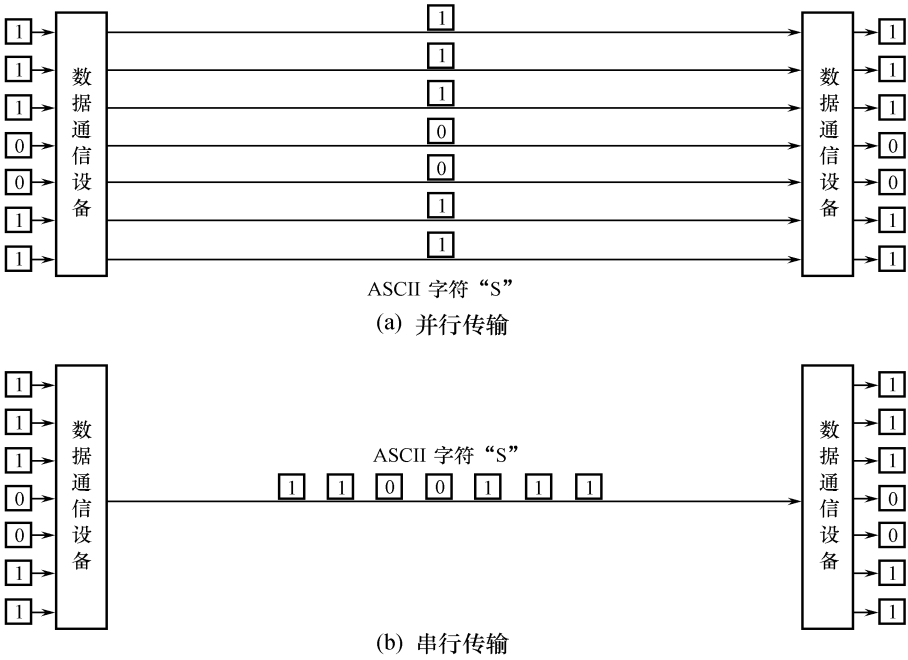


图2.2　并行传输与串行传输

在计算机网络中,接收方和发送方的时序协调也要依靠时钟定时,即发送方依靠时钟来决定每一位的起始和终止,接收方也要依靠定时时钟来确定对信号每一位的采样取值位置和时间间隔。然而,任何两个系统间时钟完全准确地同步(一致)是不太可能的。随着时钟漂移(不一致)的积累,就会产生接收方取值位置和时间间隔的错位,要么在信号某一位上多采了一次样,要么信号的某一位被跳过。为解决这一问题,就需要像对表一样,每隔一个时间间隔,发收两端校对一下时间。发收两端校对时间就称为收发两端的同步。

2. 帧级同步、字节级同步和位级同步

按照传输的数据的长度,可以考虑从三个级别上进行收发两端的同步:帧级、字节级和位级。

(1) 帧级同步

帧(Frame)是在链路连接的两端每次所传输的数据块,通常由多个字节组成,具体长度由链路上所使用协议规定。帧的同步,就是在数据块的两端加上前文(Preamble)和后文(Postamble),表示帧的起始和结束。前文和后文的特性也取决于所用的协议,并可以分为面向字符和面向位两大类。

(a) 在面向字符的同步传输中,帧头包含一个或多个同步字符——SYN。

SYN 是一个控制字符，后面是控制和数据字节。接收端发现帧头，便开始接收后面的数据块，直至遇到另一个同步字符。IBM 的二进制同步规程（BSC 或 Bisync）是具有代表性的面向字符的同步传输规程。

(b) 在面向位的同步传输中，将数据块看作数据流，并用序列“01111110”作为开始和结束的标志。为了避免在数据流中出现序列“01111110”时引起混乱，发送方总是在其发送的数据流中每出现 5 个连续的“1”，就插入一个附加的“0”；接收方则每检测到 5 个连续的“1”并且其后有一个“0”时，就删除该“0”。目前应用最普遍的面向位的同步传输规程是 HDLC 和 SDLC。

显然，同步传输的传输效率要比异步传输高。图 2.3 所示为同步传输的两种帧格式。

SYN	SYN	SOH	字符序列	FCS
帧头		控制信息	数据块	校验序列

(a) 面向字符的同步帧格式

01111110	C	位流	FCS	01111110
帧头	控制信息	数据块	校验序列	帧尾

(b) 面向位的同步帧格式

图 2.3　同步传输的两种帧格式

采用帧同步时，接收端时钟与发送端时钟会有微小的差异，会造成帧内各数据位的漂移。由于帧一般较长，位漂移的积累将造成一个帧内后面部分的数据位无法正确接收，所以帧同步不能独立使用。

(2) 字节级同步

字节级同步就是两端每次以字节为单位传输数据，并在字节的两端加上特殊记号，表示字节的开始和结束，以实现收发两端同步。通常采用如图 2.4 所示的方法：

不传送时，信道一直处于高电平，表示停止（状态“1”）；

用 1 位低电平（状态“0”）表示起始位；

接着传送 1 个字符；

最后用 1 位或 2.5 位或 2 位的高电平表示停止位。

在字节级同步方式中，接收方知道了字节起止后，尽管会由于时钟的差异造成字节内位的漂移，但由于字节较短，还不至于漂移到使接收端脉冲信号不能正确接收的程度，因而它可以直接独立使用。

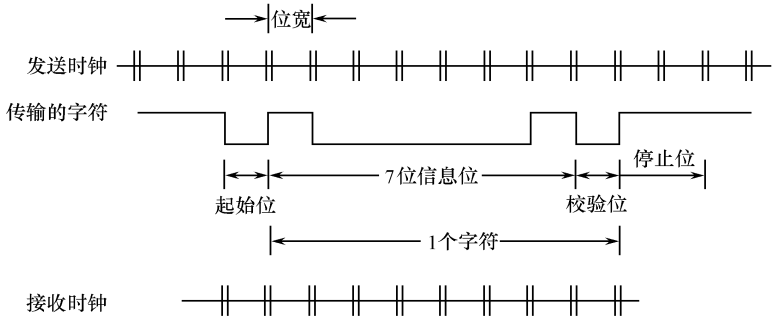


图2-4　异步传输过程

(3) 位级同步

前面两种方法都是采用附加冗余数据信号来实现同步。但是位级同步不能再采用附加冗余数据信号的方法了。因为这样不仅冗余太大,而且几乎无法实现。因此,位级同步要采用同步时钟实现,即用发送方时钟直接控制接收方时钟,使双方达到完全同步。发送方对接收方的同步可以通过两种方法进行:

(a) 外同步法

外同步法在发送方和接收方之间提供单独的时钟线路,发送方在每个比特周期都向接收方发送一个同步脉冲。接收端根据这一串同步脉冲来调整自己的接收时序,把接收时钟的重复频率锁定在同步频率上,以便在接收数据的过程中始终与发送端同步。这种方法在短距离传输中比较有效;长距离传输中,会因同步信号失真而失效。

(b) 自同步法

自同步法利用特殊编码(如曼彻斯特编码或微分曼彻斯特编码)让数据信号携带时钟信号。

由于实际上不可能以位为单位进行传输,所以位级同步方式也不能独立使用。而字节级同步又可以独立使用,所以位级同步只能与另外一个不能独立使用的同步方式——帧同步配合使用。

3. 同步传输和异步传输

在实际应用中,通常用位级同步来弥补帧同步的不足,而字节级同步可以独立使用。于是实际应用的同步方式只有两种。位级同步和帧同步结合的同步方式最显著的特点是接收端与发送端具有共同的时钟。这种方式被称为同步传输方式;相对而言,字节级同步方式中接收端与发送端不需要严格的时间标准,因而称为异步传输,或称起止同步方式。

异步传输实现简单,设备技术开销小,价格便宜。但由于传输时每个字符都

要附加 2～3 位用于起止位，字节之间还有间隔，因此传输效率低。

2.2　数据信号分析与编码

在一个信道上进行数据传输的成败和质量，不仅决定于信道的质量，还与信号自身的形式有关。这一节介绍不同形式的信号对传输质量的影响以及将不同的数据变换为不同形式的信号的技术。

2.2.1　数据信号分析与信道参数

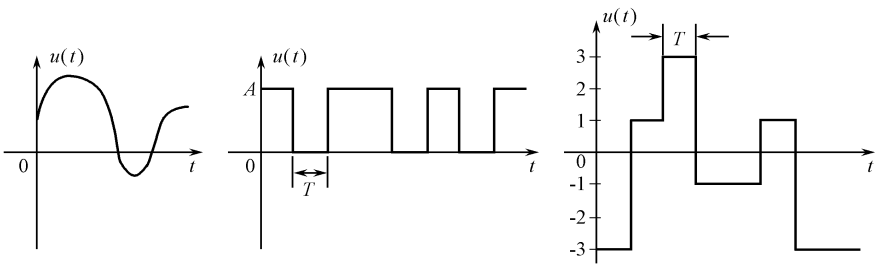
信号是数据在载体中的存在形式，为了便于分析，可以用时域法或频谱法表示。

1. 时域表示法

时域表示法就是把信号的幅值表示成时间函数。用时域表示法表示信号时，可以清楚地看出信号随时间变化的规律。图 2.5 所示为三种不同波形的信号。其中：

图 (a) 所示信号的特点是幅值连续，即在一定的时间区间内幅值可以取无限多个值。这类信号称为模拟信号。

图 (b) 和 (c) 所示信号的特点是幅值不连续，即幅值只有有限个值。如图 (b) 所示信号只能在 0，A 之中取值，图 (c) 所示信号只能在 (3，1，－1，－3) 4 个值之中取值。这类信号称为数字信号。



(a) 模拟信号　　(b) 二进制数字信号　　(c) 四进制数字信号

图 2.5　模拟信号与数字信号

总之，模拟信号具有以下特征：

波形是连续的、圆滑的、没有突跃的变化；

幅值在某一时间范围内可以取任一值。

典型的模拟信号是正弦波信号,频率、幅值和相位是正弦波的3个属性。

2. 频谱表示法

频谱表示法是把信号的幅值表示成频率的函数。这是基于傅里叶分析的一种方法。傅里叶分析表明,基频为 *f* 的任意周期函数 $g(t)$ 都可以由(无限个)正弦函数和余弦函数合成,即

$$g(t)=\frac{1}{2}C+\sum_{n=1}^{\infty}a_n\sin(2\pi nft)+\sum_{n=1}^{\infty}b_n\cos(2\pi nft)$$

式中,*f* 称为基频,*C* 是常数,a_n、b_n 是第 n 次谐波的幅值,n=1 的分量波称为基波。

如果把一个持续时间有限的数据信号想像成一个反复重复的模式,就可以将任意信号作为周期信号进行傅里叶分析,分析出它的谐波组成。一个信号的频率范围称为该信号的绝对带宽。在现实中,许多信号具有无限带宽,即它的傅里叶分析结果是无穷级数之和。不过,信号的大部分能量都集中在某一段频带之中,这个频带称为该信号的有效带宽,简称带宽。

用这种表示方法,可以看出一种信号的频率范围——带宽,以及每一个频率分量的信号能量的大小。同一信号的时域表示与频谱表示之间有一定的关系。图2.6为5种不同信号时域表示与频谱表示的对应图。它表明,越接近正弦波,信号的带宽就越小,对信道的质量要求就越低;而越接近数字信号,信号的带宽就越宽,对信道的质量要求就越高。

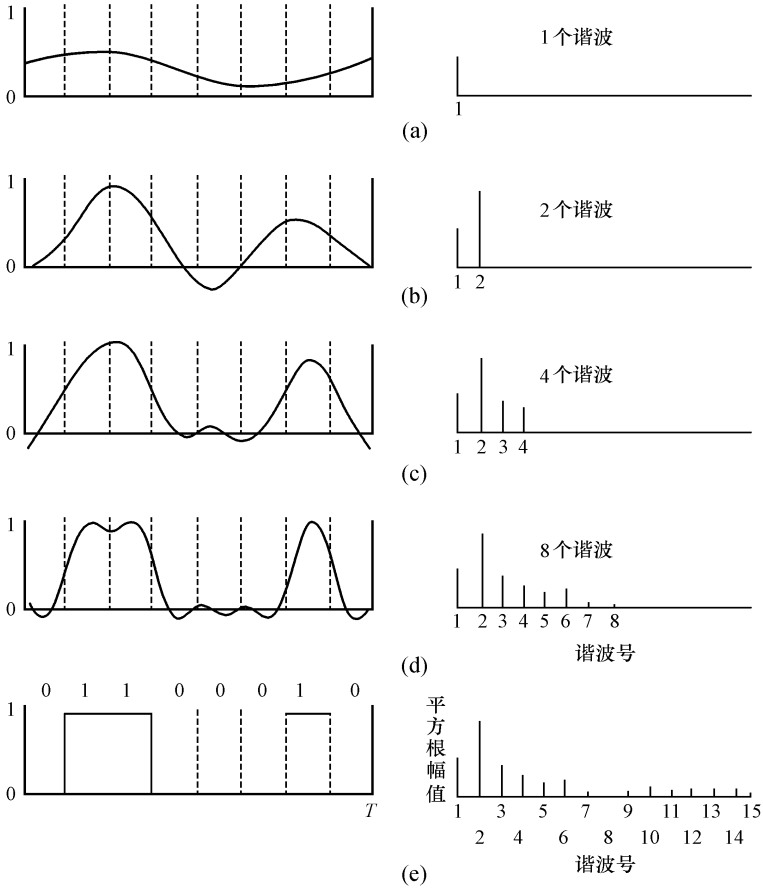
图中,纵坐标用幅值的平方根表示,因为信号分量幅值的平方根与所传输的能量成正比。

即使同是数字信号,因周期脉冲比(T/τ)不同,所含谐波分量的数量(频带宽度)和各分量的幅值比也不相同。图2.7为3种不同周期脉冲比的矩形波信号频谱图。可以看出,周期脉冲比越大的信号,所含高次谐波就越丰富,带宽就越大,对信道的要求就越高。

为了有效地解决信号传输中不同频率的谐波分量共存所引起的技术问题,人们把信号分为基带(Base Band)信号和频带信号两大类。

频带信号,一般就指音频模拟信号,即带宽为300 Hz～3 400 Hz的普通电话信号。这种信号对信道质量要求较低。

关于基带信号,有三种层次的定义:最广义的概念是指一切原汁原味的不经任何变换(调制)的原始信号;稍低层次的概念是指最高频率与最低频率相差很大的信号,它既可以是模拟信号(如电视信号,基本频带为0～6 MHz),也可以是数字信号;在计算机数据通信中,使用的是狭义的概念,主要指数字信号,它是一种矩形波,其分量的频率范围相差很大(见图2.6(e))。



(a)~(c) 为几个连续的接近于原始信号的谐波
(e) 为一个二进制信号及其平方根幅值

图 2 6　不同形状信号的时域表示与频谱表示

2 2 2　信道的主要性能参数

数据传输的必经之路称为信道。它与链路有联系，也有区别，因为可以采用多路复用技术在一条链路中划分出多条逻辑信道。具体内容将在 2 3 节介绍。信道在实际传输信号时，会面临三大损害：衰减、延迟失真和噪声。衰减(Attenuation)是指信号在传输过程中的能量损耗，通常随传输距离的增加而增加，用单位 dB/km 度量。延迟失真(Delay Distortion)是指由于信道对信号中的各傅里叶分量，因其频率不同所引起的衰减程度不同而造成的波形失真。噪声(Noise)是指信道上的非发送方期待的能量。噪声分为热噪声、串音和脉冲噪

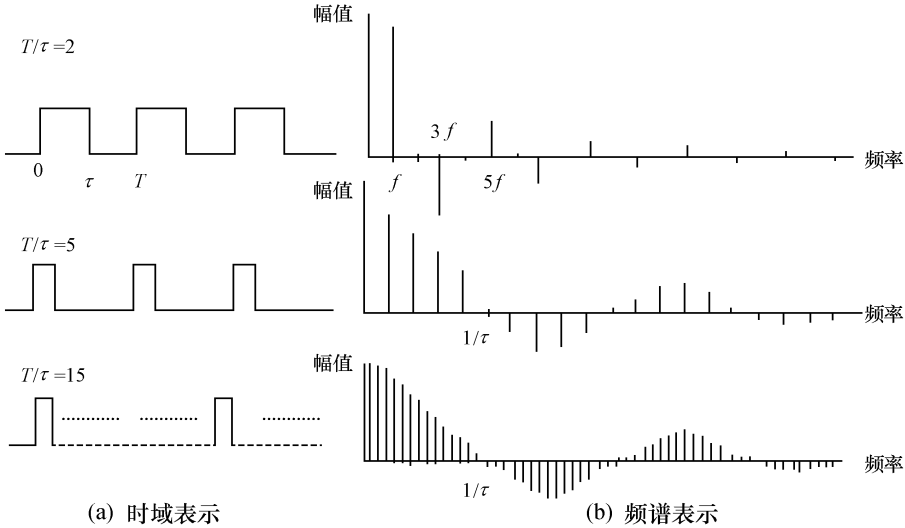


图2.7　不同周期脉宽比的矩形脉冲频谱

声。热噪声是由于线路(主要是电气线路)中的电子自由运动加剧而产生的能量。串音是由于临近线路上传输的信号耦合感应传递的能量;脉冲噪声是由于外界强力的电磁信号(如雷电、电力线路上的尖峰等)传递的能量。

信道的性能可以从信道的传输速率、信道带宽、信道容量、吞吐量、出错率等方面衡量。

1. 传输速率

数据的传输速率是指单位时间内所传输的数据量的多少。为了能够统一度量,可以采用两种方法作为传输速率的单位。一种是用单位时间内所传输的符号数作单位,称为码元速率,也称为波特(Baud)率。记为

$$B = 1/T$$

但是,不同符号集中的码元所含信息量是不同的。一个符号所含的信息量取决于它在符号集中出现的概率:出现的概率越小,能消除接收方不确定性的程度越大,所含信息量就大;出现的概率越大,能消除接收方不确定性的程度越小,所含信息量就小。设某条消息由 M 个符号 $(x_1, x_2, \cdots, x_M)$ 描述,每个符号在排列中出现的概率与其他符号无关,分别用 $P(x_1), P(x_2), \cdots, P(x_M)$ 表示,并且有 $\sum_{i=1}^{M} P(x_i) = 1$,则符号 x_i 所携带的信息量为 $I(x_i) = \log\frac{1}{P(x_i)}$,而该条消息发生时,所携带的信息量为

$$I(x_1) + I(x_2) + \cdots + I(x_M) = \log\frac{1}{P(x_1), P(x_2), \cdots, P(x_M)}$$

当每个符号以等概率出现时，对有 *M 个码元的符号集，每个符号出现的概率均为* 1/M。每个符号携带的信息量均为：$I=\log_2 M$。若以比特作为单位，有

$$I=\log_2 M$$

在通信系统中，不仅要考虑信号传输的速度，还要考虑符号本身所携带的信息量，从而引入了信息速率即数据传输速率。数据传输速率是信号每秒钟传输的信息量，通常以 b/s (bit per second，比特/秒) 为单位，称比特率，有

$$R=(1/T)\log_2 M$$

当单位脉冲仅表示 0 和 1 两种状态时，$M=2$，$R=1/T$；若 $M=4$，即采用四进制码时，$R=2/T$。显然，采用的码制越大，数据传输速率越高，因为这时每一位码所含的信息量大。

2. 带宽

衰减和延迟失真与信号中所含傅里叶分量频率相关，并由信道的质量决定。任何信道都不是理想的，每一个信道都对能通过信号的频率范围有一个限制。信道允许通过的信号频率范围，即可传送的信号最高频率与最低频率之差，被称为信道的通频带宽，亦即信道带宽，单位为 Hz。

信道对信号的限制会导致信号失真，限制信号的传输速率，增加信号识别难度。图 2.8 表明信道带宽对数字信号传输波形的影响：信道带宽越低，信号的畸变越大。为了保证信号传输的质量，信道只能传输比自己带宽小的信号。

随着计算机网络的普及，人们对带宽的要求与日俱增，于是宽带 (Broadband) 技术应运而生。宽带技术主要用于带宽很宽 (一般大于 2.5 Gb/s) 的信道，这样就可以在同一传输介质上实现多重 (并行) 传输的高速数据传输通道。图 2.9 所示为 ADSL 上的 3 个信息通道：POTS (话音) 通道 (4 kHz)、上行通道 (10 kHz～50 kHz) 和下行通道 (1 MHz 以上)。或者说，ADSL 技术就是在一对通信线上划分出这样 3 个信息通道的技术，从而在一对通信线上既可以打电话，又可以上网。

上面所说的“宽带”，目前国际上还没有标准定义。一般可以将速度在 1.54 Mb/s 以上的数据传输叫做宽带。从运营商的角度，一般只把速度超过 2 Mb/s 的数据传输称做宽带传输。

注意，使用接入速率为 2 Mb/s 的宽带网，并不等于每秒钟最高可以下载 2 Mb 的数据。应当是 2 Mb/s＝2 048 kb/s＝256 kB/s，再减去数据传输所需要的校验位，实际最高传输速率为 200 kb/s 左右。

3. 信道容量

信道容量即信道上所允许的最大数据传输速率，这是信道的一个极限参数，即信道传输数据的速率不能超过其最大数据传输速率，它与信道带宽以及噪声

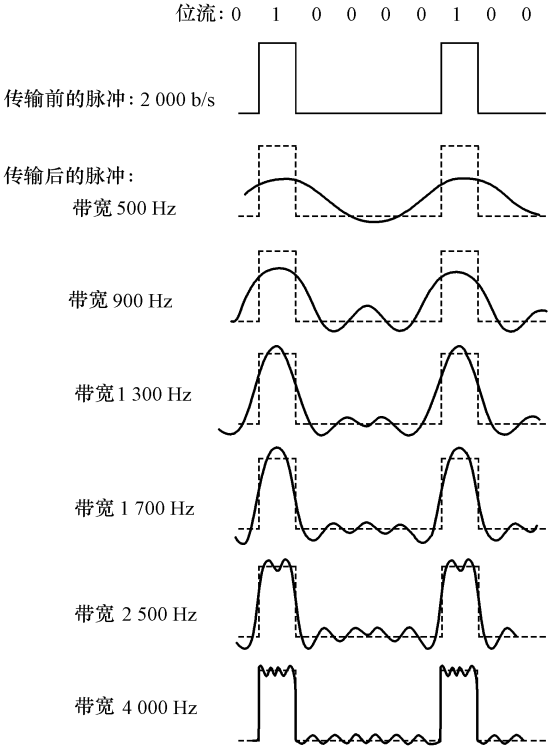


图2.8　信道带宽对数字信号的影响

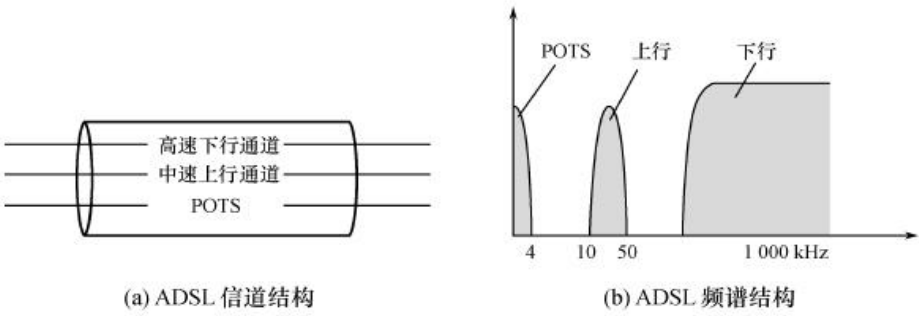


图2.9　ADSL上的3个信息通道

强度有关。也就是说,限制数据传输速率也就是限制带宽,反之亦然。表2.1给出了一些常用数据传输速率与基波周期、一次谐波频率和最高次谐波次数之间的关系。例如,在一条话音级线路上以9 600 b/s的速率传输数据,对应最高次的谐波次数仅为2。

表 2.1　数据传输速率与谐波数之间的关系

数据传输速率 (b/s)	基波周期 (ms)	一次谐波频率 (Hz)	最高次谐波次数
1 200	6.67	150	20
2 400	3.33	300	10
4 800	1.67	600	5
9 600	0.83	1 200	2
19 200	0.42	2 400	1
38 400	0.21	4 800	0

早在 1924 年,奈奎斯特 (H. Nyquist) 就意识到信道对于数据传输速率的限制,并推导出一个被称为奈奎斯特准则的有限带宽无噪声信道的最大数据传输速率表达式:

$$S_{max} = 2 \cdot W \cdot \log_2 M \quad (b/s)$$

式中,*M 为信道上所传输信号的可取离散值的个数 (即信号电平的级数)*,*W 是*有限带宽无噪声信道的带宽,2*W* 为采样频率。例如,在带宽为 3 kHz 的信道上传输二进制码,则信道的最大数据传输速率为 6 kb/s。如果在该信道上传输八进制信号,则信道的最大数据传输速率为 18 kb/s。

奈奎斯特准则考虑的是一个理想的低通信道,如果考虑噪声,问题就复杂多了。在奈奎斯特准则基础上,1948 年香农 (Shannon) 进一步研究了有限带宽的随机 (服从高斯分布的) 噪声干扰信道的极限数据传输速率:

$$\text{信道的极限数据传输速率} = W \cdot \log_2 (1 + S/N) \quad (b/s)$$

式中,*W 是有限带宽无噪声信道的带宽,S 为信道上数据信号的平均功率,N 为*信道内部的噪声功率,S/N 为信噪比。香农公式表明,信道的带宽越宽、信噪比越大,信道的极限数据传输速率就越高。例如,信道的带宽为 3 kHz,信噪比为 1 000 ($10 \cdot \log_{10} 1\,000 = 30$ dB),则其极限数据传输速率为 $3\,000 \cdot \log_2 (1 + 1\,000) \approx 30$ kb/s。

为了便于应用,已经制定了一些信道容量标准。ITU－T 规定的标准信道系列有:

A 信道——4 kHz 带宽的标准模拟话路信道;

B 信道——64 kb/s 的数字 PCM 话音或数据信道;

C 信道——8/16 kb/s 的数字信道;

D 信道——用作通信控制 (带外信令) 的数字信道;

E 信道——64 kb/s 内部 ISDN 信令的数字信道;

H 信道——数字信道。

其中最常用的是B(Bearer)信道、D(Data Link)信道和H(High)信道。D系列信道为16 kb/s(D0)或64 kb/s(D1)。常见的H系列有:H0=384 kb/s(6个B信道)、H10=1.472 kb/s(23个B信道)、H11=1.536 kb/s(24个B信道)、H12=1.920 kb/s(30个B信道)。

在干线级,各地常用的传输标准如表2.2所示。

表2.2　各地常用的干线速率标准(Mb/s)

北美	欧洲	日本
T1=1.544	E1=2.048	1.544
T2=4×T1=6.312	E2=4×E1=8.448	6.312
T3=7×T2=44.736	E3=4×E2=34.368	32.064
T4=6×T3=274.176	E4=4×E3=139.264	97.728
	E5=4×E4=564.992	397.200

4. 信道的误码率

误码率是指信道传输信号的出错率,是数据通信系统在正常工作情况下的传输可靠性指标,用下面的公式表示:

$$P = N_E / N$$

式中,*N为数据传输的总位数,*N_E为数据传输过程中出错的位数。通常计算机网络要求误码率低于10^{-6},即每传送1兆位数据,不能出现多于一个错误。

2.2.3　数字信号的模拟调制

将不适合信道传输的信号变换为适合信道传输的信号称为调制(Modulate)。数字信号的调制是将数字信号转换为频带信号,以便能将数字数据在传统的模拟线路(电话网)上进行传输,其传输过程如图2.10所示。调制的逆过程称为解调(Demodulate)。由于通信多是双向的,所以在实际应用中调制与解调两部分功能要做在一个设备——调制解调器(Modem)中。最基本的数字信号调制方法是采用移动键控技术实现正弦信号载波数字信号。

下面介绍数字—模拟调制的具体方法。

设用于载波数字信号的正弦波为

$$u(t) = u_m \sin(\omega t + \phi_0)$$

在这个式子中,除时间*t外,还有*3个参数:幅值u_m、角频率ω和相位ϕ_0。因此,可以采用调幅、调频、调相3种移动键控技术来进行数字数据的模拟调制,

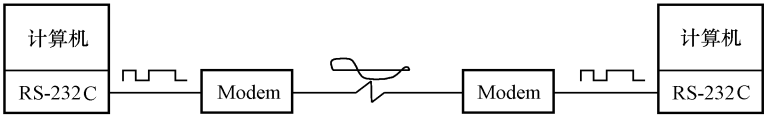


图 2.10　用 Modem 进行计算机通信

分别称为幅移键控（ASK，Amplitude－Shift Keying）、频移键控（FSK，Frequency－Shift Keying）、相移键控（PSK，Phase－Shift Keying），如图 2.11 所示。

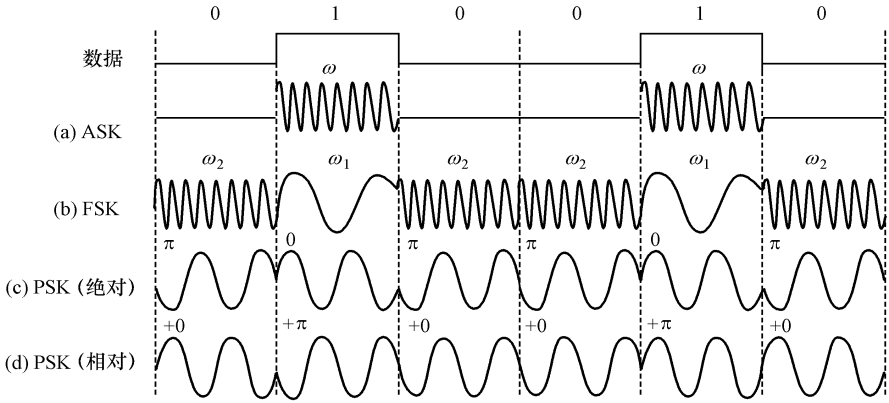


图 2.11　数字—模拟信号调制

1. 幅移键控 ASK

在 ASK 方式中，用不同幅值的正弦载波信号来分别表示数字“1”和“0”。例如，用某一幅值的正弦载波信号表示数字“1”，用零幅值（无载波信号）表示数字“0”。

ASK 的技术简单、实现容易，但抗干扰能力差。

2. 频移键控 FSK

在 FSK 方式中，用不同角频率的正弦载波信号来分别表示数字“1”和“0”。FSK 的技术简单、实现容易、抗干扰能力强，是目前最常用的方法。

3. 相移键控 PSK

在 PSK 方式中，用不同初相位的正弦载波信号来分别表示数字“1”和“0”。它的抗干扰能力强，但实现技术复杂。具体的实现方法有：绝对调相（用相位的绝对值表示数字“1”、“0”），相对调相（用相位的相对偏移值表示数字“1”、“0”）和多相调相（用不同的相位值表示“0”、“1”码组合，如用相位相差 π/2 的相位值分别表示 00，01，10，11）。

2.2.4　模拟信号的数字编码——脉冲编码调制技术

模拟数据的数字编码是将连续的信号波形用有限个离散(不连续)的值近似代替的过程。简单地说,就是将模拟信号用数字信号近似地代替,其中最常见的方法是脉冲编码调制(PCM,Pulse Code Modulation)技术,简称脉码调制PCM。PCM基本步骤是:

采样,即将原波形的时间坐标离散化,得到一系列的样本值;

量化,对采样得到的样本值按量级分级并取整;

编码,将分级并取整的样本值转换为二进制(0,1)码。

图2.12所示为PCM的一个实例。

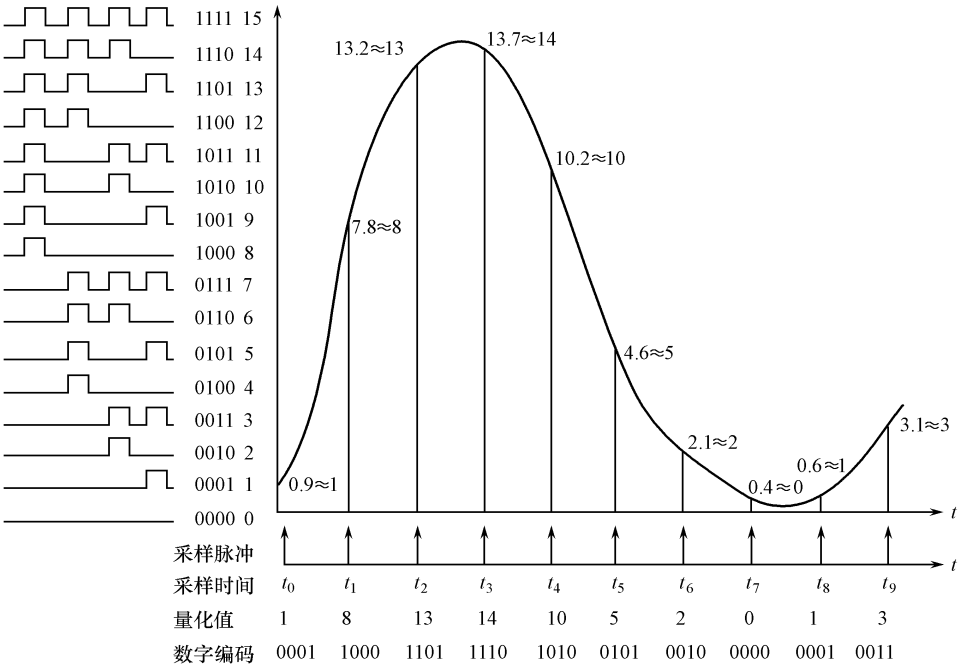


图2.12　波形的采样与量化

数字化的质量取决于下列技术参数:

(1) 采样频率

采样频率,即一秒钟内的采样次数,它反映了采样点之间的间隔大小。间隔越小,丢失的信息越少,采样后的图形越细腻和逼真。根据奈奎斯特采样定律,只要采样频率高于信号最高频率的两倍,就可以从采样准确地重现原始信号的波形。

(2) 测量精度

测量精度是样本在垂直方向的精度,是样本的量化等级,它通过对波形垂直方向的等分而实现。由于数字化最终是要用二进制数表示,所以常用二进制数的位数表示样本的量化等级。若每个样本用 8 位二进制数表示,则共有 $2^8=256$ 个量级;若每个样本用 16 位二进制数表示,则共有 $2^{16}=65\ 536$ 个量级。量级越多,采样精度越高。

应当指出,采样频率和测量精度的提高都是以存储容量为代价的。此外,当考虑通信双方的同步、信息保密、信息压缩等问题时,还要对这种基本编码进行一些变换。

2.2.5 数字编码

虽然"1"和"0"是两个非常简单的码,但实际传输和使用的信号形式有很大的差别。图 2.13 给出了 6 种具有代表性的二进制数字信号的编码方式。

1. 单极性码和双极性码

图 2.13 中的 (a) 和 (c) 表示的是两种单极性码,它们的特点是只在表示"1"时才发出电流,表示"0"时不发出电流,电流只有一个极性 (方向)。

图 2.13 中的 (b) 和 (d) 表示的是两种双极性码,它们的特点是表示"1"时发出正电流,表示"0"时发出负电流,电流具有两个极性 (方向)。

2. 归零码和不归零码

图 2.13 中的 (c) 和 (d) 表示的是两种归零码 (RZ,Return to Zero),它们的特点是,每次进行 0—1 变换或 1—0 变换时,都要在无电流处停留一下。

图 2.13 中的 (a) 和 (b) 表示的是两种不归零码 (NRZ,Non—Return to Zero),它们的特点是,每次进行 0—1 变换和 1—0 变换都是直接的,不在无电流处停留。

3. 曼彻斯特码和差分曼彻斯特码

曼彻斯特 (Manchester) 码的特点是将每个比特周期分为两部分:前半个比特周期传送该比特的原码,后半个周期传送该比特的反码,于是在每个比特周期的中间产生一个电平跃变。这个跃变信号既可以用做同步信号,也可以用做表示数据 (如图 2.13 (e) 所示,用正跃变表示为"0",用负跃变表示为"1")。

差分曼彻斯特 (Different Manchester) 码是对曼彻斯特码的的改进,它用每一码元的开始边界处有无跃变来区别"0"和"1",如图 2.13 (f) 所示,有跃变表示为"0",无跃变表示为"1"。这时,每个比特周期中间的跃变仅仅用做同步时钟。

评价数字信号编码方式的优缺点可以从以下几点考虑:

脉冲的宽度。脉冲宽度大,信号的能量就大,对于提高接收端的信噪比

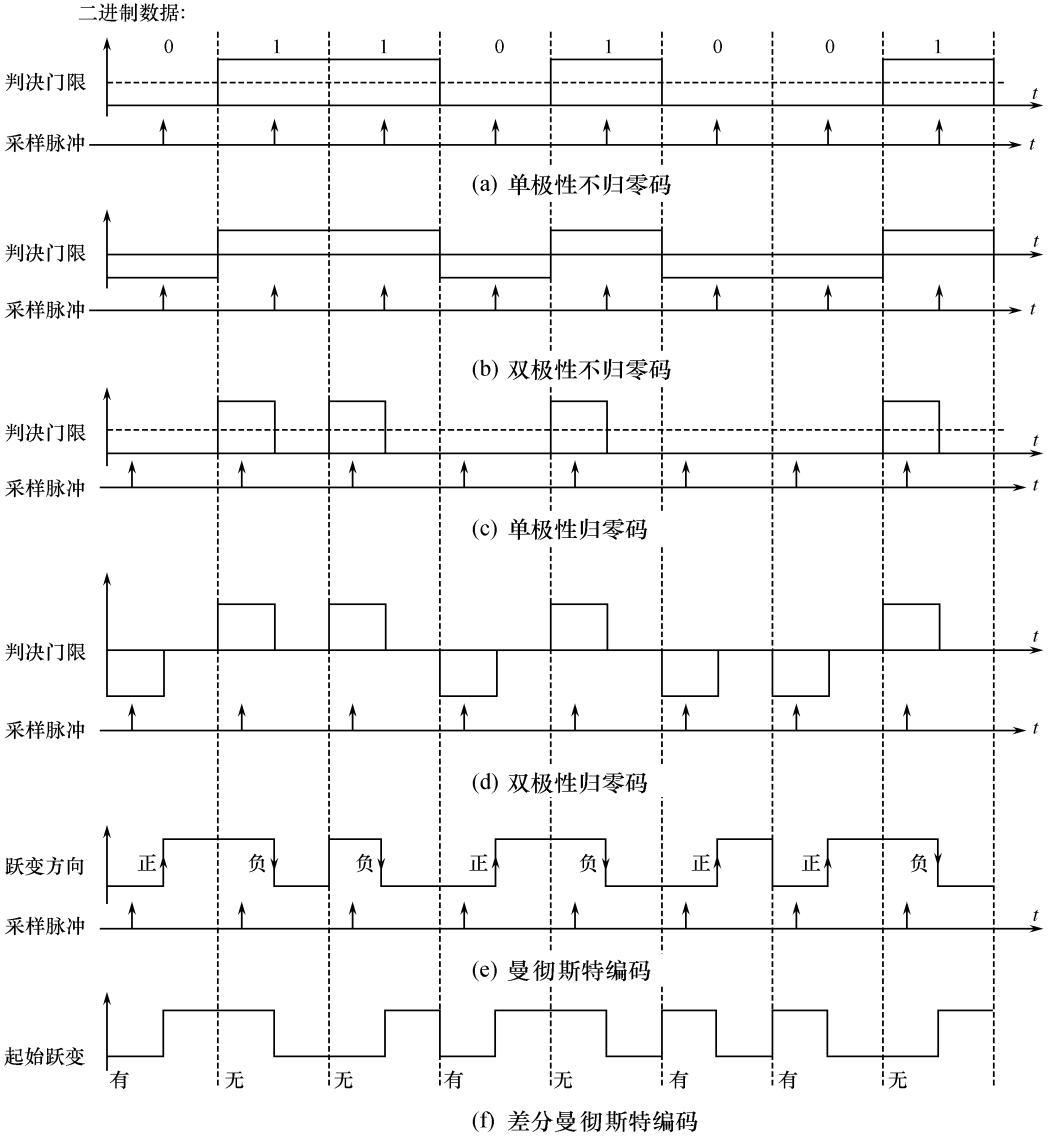


图2.13　6种二进制数字信号的编码方式

有利。

占用的频带宽度。脉冲宽,占用的频带就窄,如归零码比全宽码占用的频带要宽。

直流分量的成分。直流分量低有利于传输,如双极性码的直流分量较低,曼彻斯特码和差分曼彻斯特码的每个码元中都有跃变,因而没有直流分量。

自同步能力。曼彻斯特码和差分曼彻斯特码的每个码元中都有跃变,可

以提供自同步能力。如在IEEE802.5中,正常的信号编码都采用差分曼彻斯特码,只有起始和结束字段中各有4位“特殊比特”,这些特殊比特的码元中间没有跃变,要么是全高电平,要么是全低电平,以“特殊比特”来作为信号编码起始和结束的标志。

2.3 信道的多路复用技术

在通信系统中,信道有逻辑信道与物理信道之分。物理信道是实实在在的物理通路,如有线信道、无线信道、卫星信道等。物理信道与逻辑信道之间的关系有点像铁路与车次之间的关系,物理信道好比是铁路,逻辑信道好比是车次。北京到重庆之间并没有一条专线,但可以通过连接京广线—陇海线—宝成线—成渝线,开出一趟或几趟北京至重庆的列车来。而其中的京广线、陇海线、宝成线、成渝线上,还可以分别跑北京至广州、上海至乌鲁木齐、西安至昆明、成都至南宁的列车。逻辑信道是建立在物理信道基础上的:一条物理信道通过载波、分时或改变连接方式等,有可能分为几条逻辑信道(好像同一条铁道上同时运行多趟列车一样);在复杂拓扑结构的网络上,两点之间的通信并不一定要有一条专门的物理线路,而可以由其内部节点间的连接来实现。通常把逻辑信道的实现称为“连接”。

多路复用(MUX)源于拉丁语multi(许多)和plex(混合)。它指在一个物理信道上同时传送多个信号,或者说是把一个物理信道设法分成多个逻辑信道,以提高信道利用率。

2.3.1 频分多路复用(FDM)技术

FDM(Frequency Division Multiplexing)是模拟传输中常用的一种多路复用技术。它把一个物理信道划分为多个逻辑信道,各个逻辑信道占用互不重叠的频带,相邻信道之间用“警戒频带”隔离,以便将不同路的信号调制(滤波)分别限制在不同的频带内,在接收端再用滤波器将它们分离,就好像在大气中传播的无线电信号一样,虽同时传送多个频率信号,但互不重叠,可以分辨。图2.14所示为将一个物理信道频分为3路进行复用的情形,每个逻辑信道分配4 000 Hz带宽,并只传送3 000 Hz左右的载波频带信号。

目前,FDM技术已经在一定程度上实现了标准化。其中一个广泛使用的国际标准是把12×4 000 Hz的音频信道(用户使用3 000 Hz,再加两个500 Hz的防护频带)复用到60 Hz~108 Hz的频带上,其单位称为群(Group)。

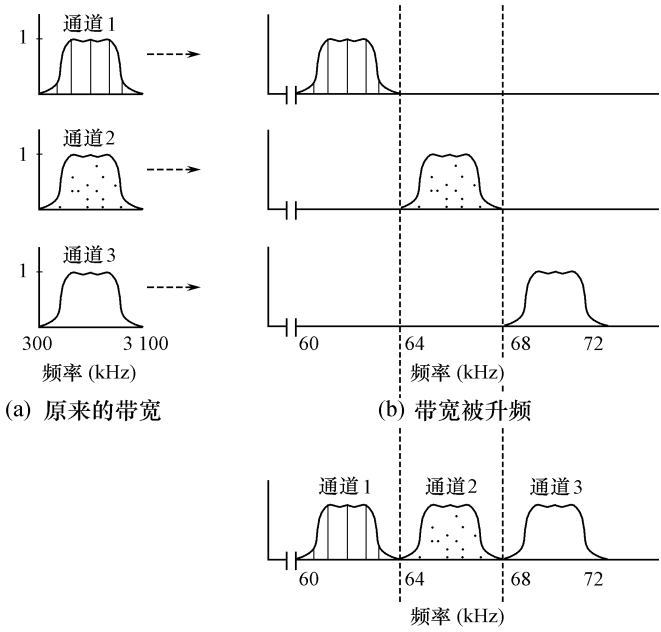


图2.14　频分多路复用

最典型的频分多路复用技术的应用是普通手机。

2.3.2　时分多路复用(TDM)技术

1. TDM基本原理

与FDM的同时发送多路信号相比,时分多路复用(TDM,Time Division Multiplexing)是一种非同时发送的多路复用技术。如图2.15所示,它将一个传送周期划分为多个时隙,让多路信号分别在不同的时隙内传送,形成每一路信号在连续的传送周期内轮流发送的情形。

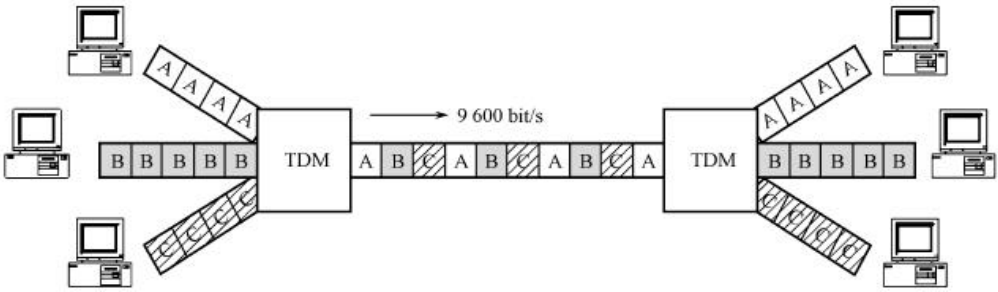


图2.15　时分多路复用

数字信号的时分复用也称为复接，参与复接的信号称为支路信号，复用后的信号称为合路信号，从合路信号中将原来的支路信号分离出来称为分接。

通常，话音信号是用脉码调制来编码的。由于典型的电话通道是 4 kHz，按照奈奎斯特定理，为了用数字信号精确地表示一个模拟信号，对话音模拟信号的采样频率至少要达到 8 000 Hz。用一个 8 位字来代表每个采样，则话音信号数字化的结果便是一个 8 000×8（位字）的数据流，数据传输速率为 64 kb/s。上述方法称为 PCM 复用。为了提高数码率，对 PCM 复用后的数字信号再进行时分复用，形成更多路的数字通信，目前广泛采用这种数字复用方法来提高通信容量。

图 2.16 所示为 ITU－T 推荐的数字速率等级和复接等级，它们都是基于传输速率 64 kb/s（称为零次群）的数字信号的。两种等级不同之处在于，一类是用 TDM 技术将 24 路零次群复用到一条线路上，形成数据传输速率为 1.544 Mb/s 的一次群（称为 T1 次速率，主要在北美应用），并在此基础上形成其二次群、三次群、四次群等；另一类是用 TDM 技术将 30 路零次群复用到一条线路上，形成数据传输速率为 2.048 Mb/s 的一次群（称为 E1 次速率，主要在欧洲应用），并在此基础上形成其二次群、三次群、四次群等。

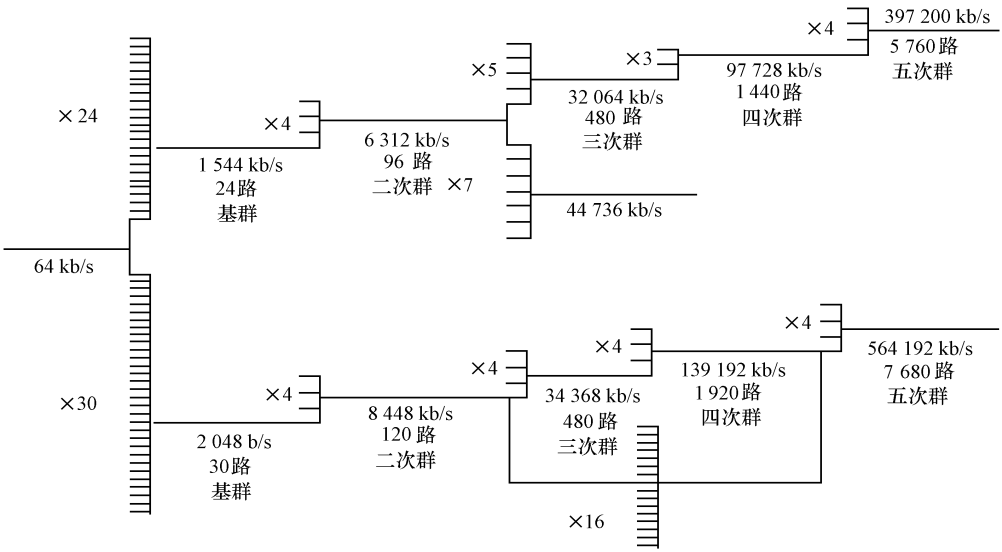


图 2.16　ITU－T 推荐的数字速率等级和复接等级

2. 复接方式

TDM 可以设计成按位、按字节、按字符、按字或按任意多位的方式来对每个终端进行扫描复接。图 2.17（a）、（b）分别为按位和按字节进行扫描复接的示意图。

图 2.18 为对 4 路 PCM30/32 基群信号按位复接和按字复接的示意图。

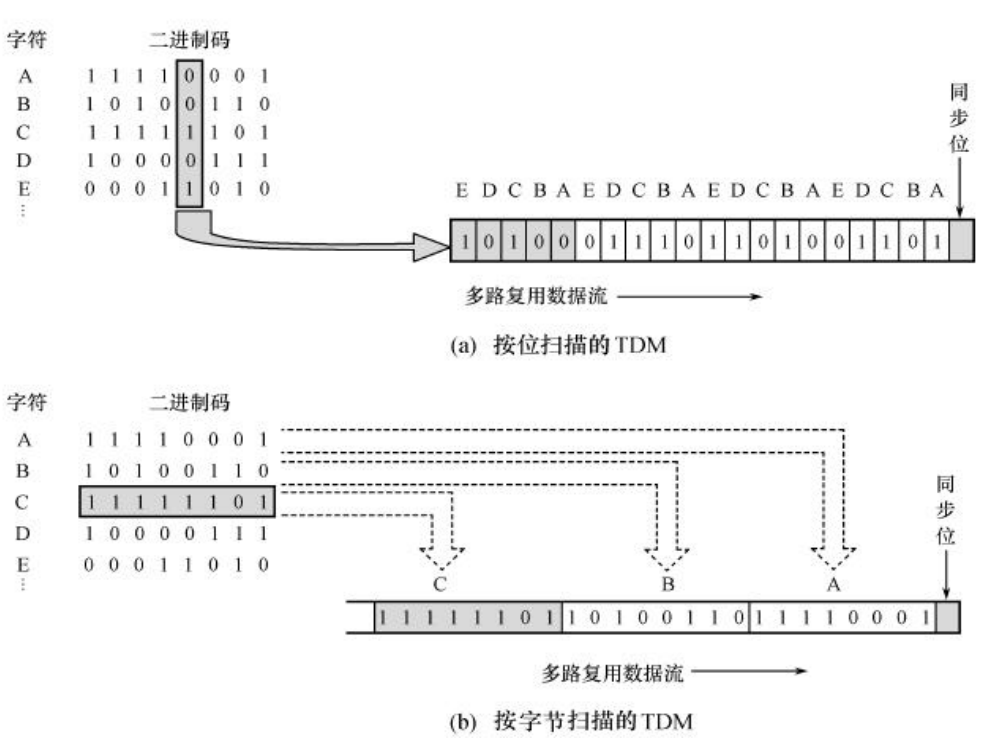


图2.17　两种扫描方式

按位复接又称为比特复接,即复接时每支路依次复接一个比特。这是目前广泛使用的方法,它的设备简单,需容量小,且容易进行,但对信号的交换不利。

按字节复接的方法如下:对基群来说,一个码字有8位,复接前先将8位码存起来,在规定的时间内一次复接;4个支路轮流复接。这种方法需要较大的存储容量,适合于数字电话交换。

按帧复接是每次复接一个支路的帧(256比特)。这种方法不破坏原来的帧结构,有利于交换,但需要更大的存储容量。

复接实际上就是通过脉冲采样再生成一定宽度的数字信号的过程,但由于要保持原来的数字信号周期,因此复接生成的方波宽度一定要按复接支路数进行分频。如在图2.18中,对4路数字信号进行复接后,在合路上的位宽减小到原来的1/4,使得4路的总宽等于原来1位的宽度,这样才能保持信号的传输速率不变。

3. 同步时分复用和异步时分复用

图2.19给出了时分多路复用的两种同步方式——同步时分多路复用(STM)和异步时分多路复用(ATM)。同步时分多路复用是指时分方案中的时间

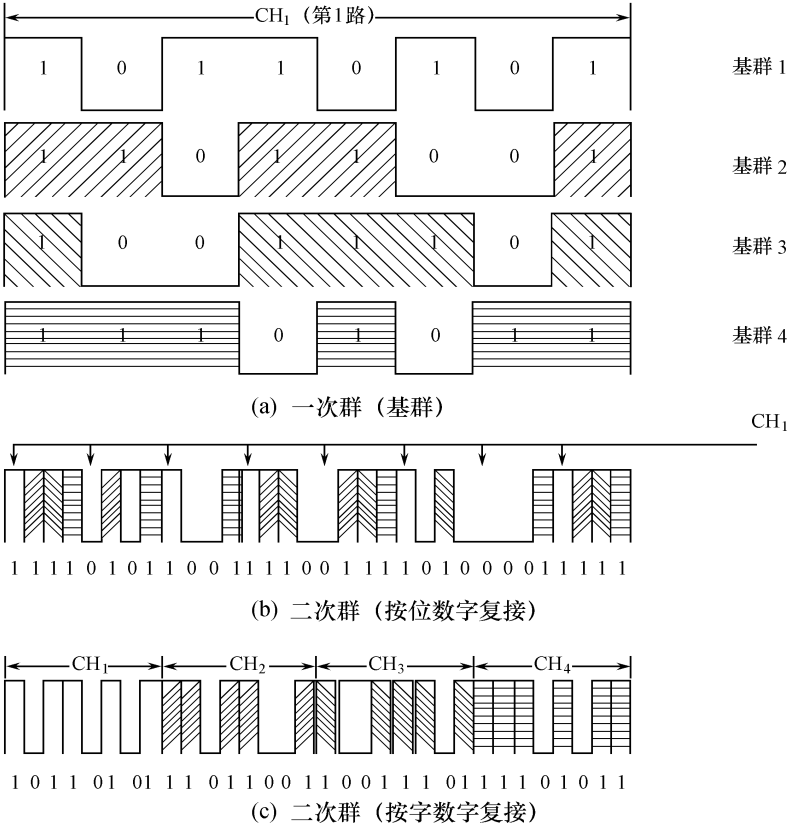


图 2.18　对 4 路 PCM30/32 基群信号按位复接和按字复接

片是预先分配好的，时间片与数据源是一一对应的，不管某一个数据源有无数据要发送，对应的时间片都是属于它的；或者说，各数据源的传输定时是同步的。在接收端，根据时间片的序号来分辨是哪一路数据，以确定各时间片上的数据应当送往哪一台主机。

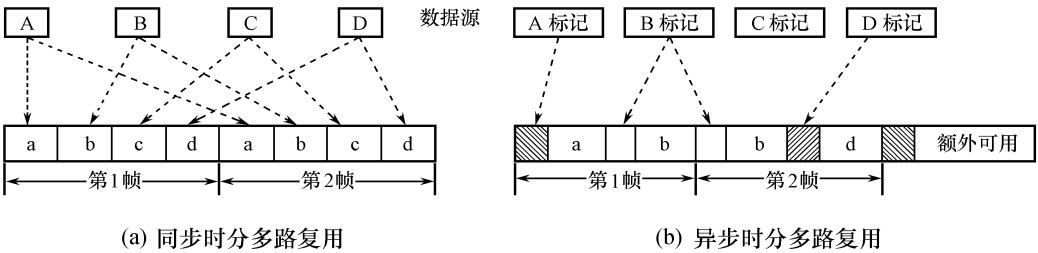


图 2.19　同步时分多路复用与异步时分多路复用

采用异步时分多路复用时，各时间片与数据源无对应关系，系统可以按照需要动态地为各路信号分配时间片，各时间片与数据源无对应关系。为使数据传输顺利进行，所传送的数据中需要携带供接收端辨认的地址信息，因此异步时分多路复用也称为标记时分多路复用。ATM技术中的传输就是这种方式。

4. 复接时系统间的同步问题

当由几个低次群数字信号复接成一个高次群数字信号时，如果各低次群采用独立的时钟，即使每个低次群所使用的时钟的标称数码率相同，也会由于线路长短不同产生的时延差异等原因，造成瞬时数码率的差异，从而形成如图2.20所示的重叠或错位现象，使复接合成后的数字信号无法分接恢复成为原来的低次群数字信号。

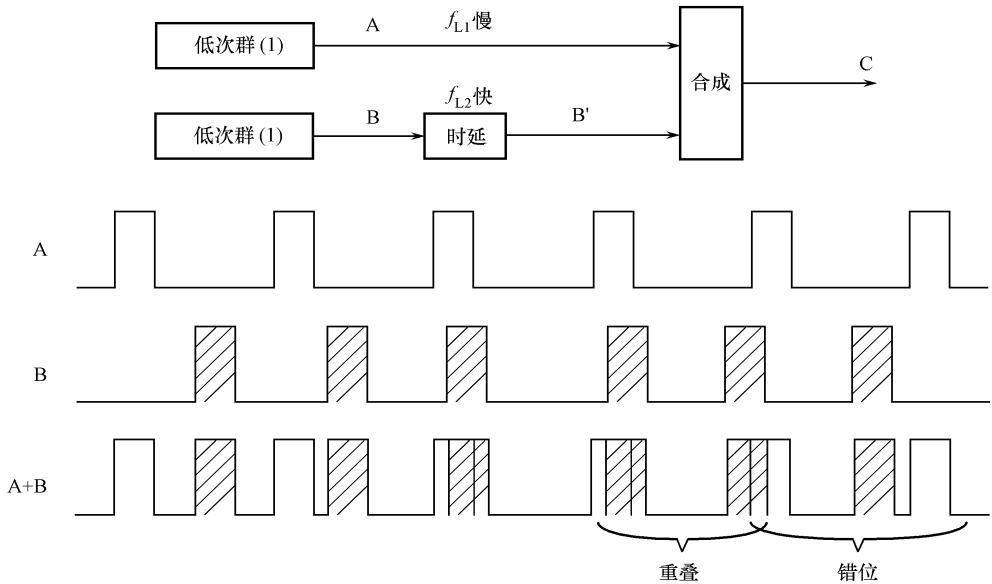


图2.20　瞬时数码率差异造成的重叠和错位

为避免这个问题，需要解决系统与系统间的同步问题。系统间的同步可以用两种方法实现：一种是用一个高稳定的主时钟来控制被复接的几个低次群，使它们的数码率统一在主时钟的频率上，从而达到同步的目的；另一种，也是最常用的，是称为码速调整的方法。码速调整方法是让各低次群仍然使用自己的时钟，并在复接前插入一些码元。例如，要将支路速率为2 048 kb/s的4个基群信号复接到速率为8 448 kb/s的二次群上，就必须首先对基群的数据速率进行调整，使之与8 448 kb/s的4分频速率2 112 kb/s同步，然后再进行同步复接。这种调整后的数码率高于调整前的数码率的码速调整方法称为正码码速调整。

图 2.21 所示是调整到 2 112 kb/s 的基群流帧结构，称为二次群复接子帧结构。图中：

(a) C_{ij}为插入标志码，用于分接时判断所在基群中的复接子帧中有无插入码速调整位。

有插入位，取全“1”；

无插入位，取全“0”。

(b) V_i 为插入码——为调整码率而插入的码元。

支路速率偏低，在该时隙插入一个不含信息的脉冲；

支路速率正常，该时隙仍传输支路信息。

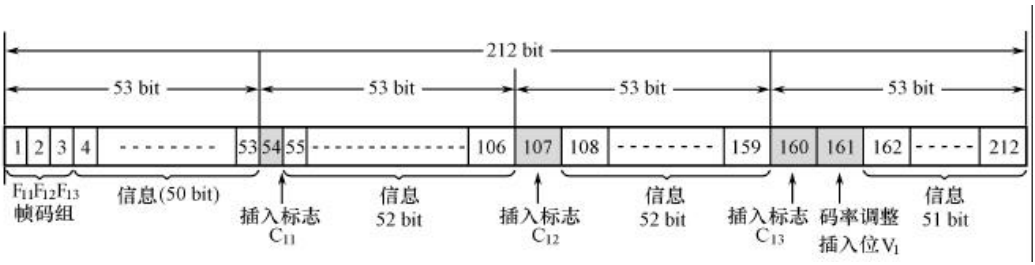


图 2.21　调整到 2 112 kb/s 基群流的二次群复接子帧结构

在支路级速率调整的基础上，进行二次复接帧的速率调整。图 2.22 所示为 PCM 二次群复接帧结构。从图中可以看出，二次群的数码率为 8.448 Mb/s，帧周期约为 100 μs，帧周期内共 848 个比特。其中：

第 1～10 位为插入帧同步码 1111010000（依据 ITU－T 规定）；

F_{33}（第 11 位）作对端告警码；

第 12 比特备用；

C_{11}、C_{21}、C_{31}、C_{41}、C_{12}、C_{22}、C_{32}、C_{42}、C_{13}、C_{23}、C_{33}、C_{43}为插入标志码；

V_1、V_2、V_3、V_4 为插入码——为调整码率而插入的比特。

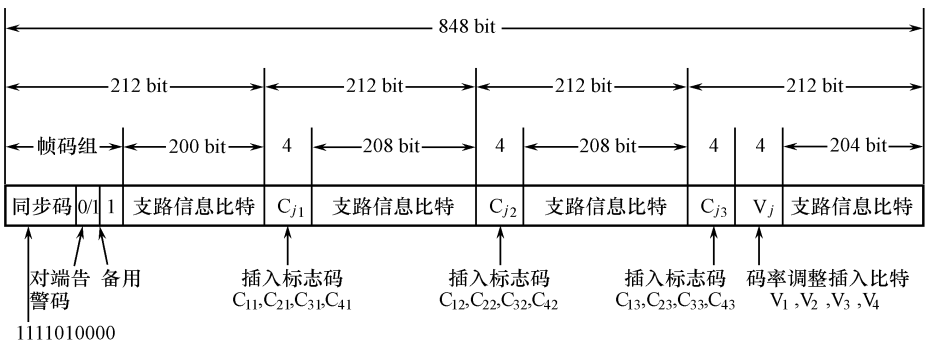


图 2.22　PCM 二次群复接帧结构

在复接器中为调整码率要插入插入码，在分接器中要将它们去掉——消插。分接器进行消插的依据是“插入标志码”。当插入标志码的4位中有“000”时，表示无插入码；当插入标志码的4位中有“111”时，表示有插入码。这种方法叫做“大数判决法”。

实际上，同步复接中也要使用插入码。与异步复接不同的是，它在每一个帧中都要插入插入码，目的是使低次群的码速能在复接后符合高次群的要求。

5. 准同步数字系列 PDH

准同步数字系列 PDH (Plesiochronous Digital Hierarchy) 是曾广泛使用的数字传输技术，它采用异步复接，而且大多数采用正码码速调整，即通过插入一些非信息比特使各支路信号与复接设备信号同步，并复接成高次群信号。这种方式的最大缺点是很难从高次群信号中直接识别和提取低次群信号。要分接出支路信号或插入支路信号，惟一的方法是将高次群信号一步一步地分接成所要提取的低次群信号，然后再一步一步地复接成高次群信号。图2.23示出了在PDH系统中从PCM四次群传输速率为140 Mb/s信号中提取传输速率为2 Mb/s信号的过程。

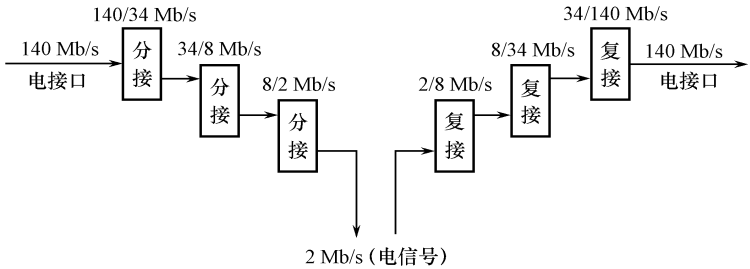


图2.23　在PDH系统中提取一个低次群信号示例

6. 同步数字系列 SDH

1988年，ITU－T通过了有关同步数字系列 (SDH, Synchronous Digital Hierarchy) 的三个基本建议：G.707、G.708、G.709，标志着数字传输技术的一个新的转折，从此传送网开始进入“光信网”的时代。

SDH包含极为丰富的内容，它既是一套新的国际标准，又是一个组网原则，也是一种复用方法。更重要的是，SDH提供了公认的框架，在这个框架的基础上，可以建成一种灵活、可靠并能进行遥控管理的传输主干网。

下面介绍与之有关的几个概念。

(1) STM－1

SDH传输网所传输的信号由不同等级的欧规同步传输模块 STM

(Synchronous Transfer Module) 信号组成。根据 G.707 建议,SDH 以同步传输模块 STM－1 为最基本的信号模块。STM－1 具有 9 行×270 列的帧结构 (见图 2.24) ,其速率为 9×270×8×800＝155.520 Mb/s。STM－1 由低速率的净负荷 (Payload) 、段开销 SOH (Section Overhead) 和管理单元指针 AU PTR (Administrative Unit Pointer) 组成。

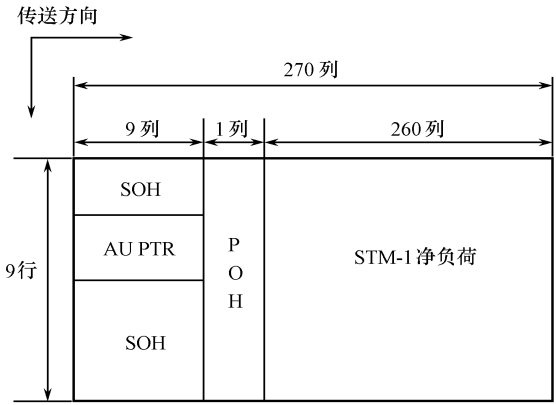


图 2.24 STM－1 帧结构

(a) 信息净负荷

信息净负荷简称净负荷,是 STM 帧中用于存放各种信息的空间,也包含了少量的通道开销 POH (Path Overhead) ——用于通道性能监视、管理和控制。POH 通常作为净负荷的一部分与数据码流一起传送。

(b) 段开销 SOH

SOH 是在 STM 帧结构中用于保证净负荷正常灵活地传送所必需的附加字节,主要供网络运行管理和维护使用。

(c) 管理单元指针 AU PTR

AU PTR 主要用来指示信息净负荷的第 1 个字节在 STM 帧中的准确位置,同时也用于借助调整技术解决网络节点间的时钟偏差,使 SDH 系统能够在接收端正确地分离信息净负荷。

N 个 STM－1 信号可以同步复用成高速率的信息模块 STM－*N*。目前国际标准取 *N*=1,4,16,64。4 个 STM－1 同步复用成速率为 622.080 Mb/s 的 STM－4;16 个 STM－1 同步复用成速率为 2 488.320 Mb/s 的 STM－16;64 个 STM－1 同步复用成速率为 10 Gb/s 的 STM－64,相当于 12 万条话路。表 2.3 列出了 STM 系列标准、美规 ANSI 标准和光载波标准 OC (Optical Carrier) 间的对照关系。其中,OC 和 ANSI 相同,都以 51.84 Mb/s 为基本速率。

表2.3　STM(SDH)系列标准与ANSI(OC)标准对照

ITU－T	ANSI(OC)	传输速率
	STS－1(OC－1)	51.84 Mb/s
STM－1	STS－3(OC－3)	155.52 Mb/s
STM－4	STS－12(OC－12)	622.08 Mb/s
STM－16	STS－48(OC48)	2 488.32 Gb/s
STM－64	STS－192(OC－192)	9 953.28 Gb/s

每个STM－*N*中都有相应的净负荷、段开销和管理单元指针。

(2) 数字交叉连接设备DXC与SDH分插复用器ADM

数字交叉连接原是在PDH中引入的，用来代替人工配线架功能。但是PDH的数字交叉连接设备只能交叉连接低等级数字信号，并且随着交叉连接系统端口数量的增加价格将急剧上涨。

SDH的DXC(Digital Cross Connect)是一种具有多功能的传输设备，它综合了SDH的复接、分接、自动化配线调线、可靠的网络保护恢复以及传输网管理等多种功能，能充分发挥SDH的组网能力。

分插复用器ADM(Add Drop Multiplexer)是SDH网中使用最普遍的设备。它能从主信号流中灵活地分接出一些支路信号(下电路)、插入一些支路信号(上电路)，从而大大简化复接/分接过程。图2.25为SDH用ADM进行复接/分接的示意。

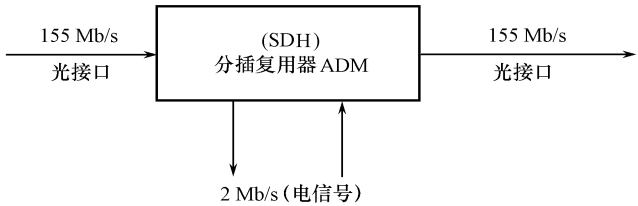


图2.25　SDH用ADM进行复接/分接

(3) SDH传输网分层模型

如图2.26所示，SDH传输网的分层模型自上向下可分为电路层网络、通道层网络和传输媒介层网络。

(a) 电路层网络

电路层网络面向公共交换业务，直接为用户提供通信服务。

(b) 通道层网络

通道层网络提供传送服务，能支持一个或多个电路层网络。可提供的传输链路的信息单元为VC－11、VC－12、VC－2、VC－3和VC－4。

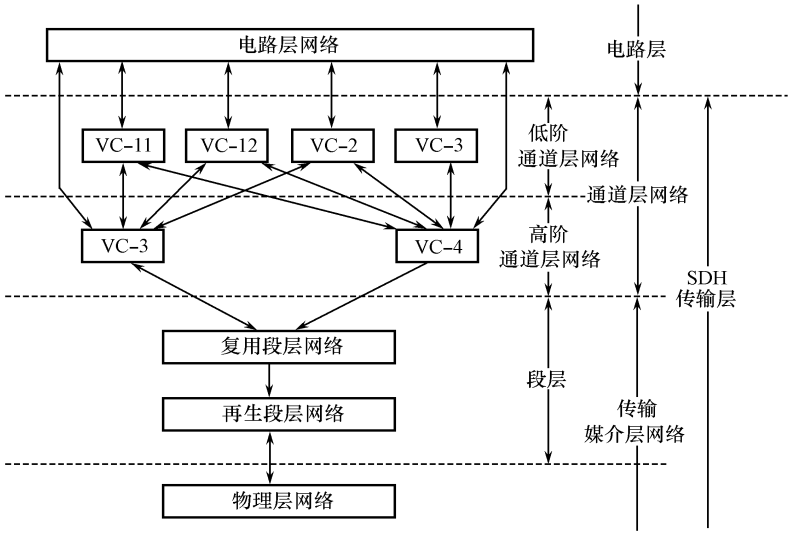


图 2.26　SDH 传输网分层模型

通道层网络进一步划分为高阶通道层网络（VC－3、VC－4）和低阶通道层网络（VC－11、VC－12、VC－2、VC－3）。

(c) 传输媒介层网络

传输媒介层网络与通信介质（光缆或无线）有关，它支持一个或多个通道层网络，为通道层网络节点提供合适的通道容量，并用 STM－*N* 作为通信介质层网络的标准等级容量。

(4) SDH 复用结构

SDH 复用结构用于表述 SDH 的信息流是怎样被复用的，也说明哪些标准接口的信号可以进入 SDH 体系。图 2.27 所示为中国的 SDH 基本复用映射结构。它以基于 2 Mb/s 的 PDH 系列作为 SDH 的有效负荷。

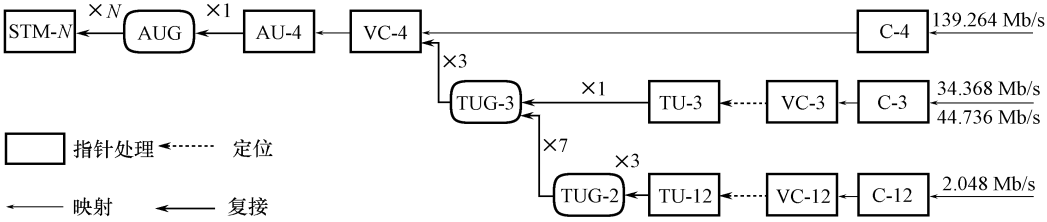


图 2.27　中国 SDH 复用结构

(a) 容器 C

SDH 用不同等级的信息容器来装载不同速率的数字信号，完成速率调整等

适配功能,并为各种 PDH 信号提供接口,用 C－*n* 表示,其中 *n* 表示 PDH 系列的速率等级。表 2.4 列出了 C－*n* 分别允许装载的 PDH 支路信号速率。

表 2.4　容器 C 的装载标准 (单位:Mb/s)

C－4	C－3	C－2	C－12	C－11
139.264	44.736/34.368	6.312	2.048	1.544

(b) 虚容器 VC

虚容器 VC－*n* 是用来支持通道层连接的信息结构,其中 *n* 表示 PDH 系列的速率等级。虚容器 VC－*n* 由净负荷和通道开销 (POH) 组成一个矩形的块状帧结构,如:VC－3 帧为 9 行×85 列的数据块,VC－4 帧为 9 行×261 列的数据块。数据在 SDH 中传输时,VC－*n* 可作为一个独立的实体在通道中的任意一点取出或插入,也可以进行同步复用或交叉连接处理。所有的 VC－*n* 都将被安放在 STM－*N* 的净负荷区内传输。

VC－*n* 分高阶虚容器 (n=3,4) 和低阶虚容器 (n=1,2)。低阶虚容器 VC－2、VC－12 和 VC－11 分别来自容器 C－2、C－12 和 C－11。

(c) 支路单元 TU

支路单元是一种在低阶通道层和高阶通道层网络间提供适配功能的信息结构,也提供 PDH 信号的灵活接口,有 TU－3、TU－2、TU－12 和 TU－11 四种,它们分别由 VC－3、VC－2、VC－12 和 VC－11 加上指针 TU－3 PTR、TU－2 PTR、TU－12 PTR 和 TU－11 PTR 后得到。这些指针的作用是为了指明各个 VC 在相应的复用 TU 帧中的起始点位置,并用来进行频率调整,即有:

$$\text{TU}-n=\text{VC}+\text{TU}-n\,\text{PTR}$$

这是 SDH 与 PDH 的重要区别。在 PDH 中,解决各支路间同步工作的方法是比特填充。这给支路的取出或插入带来困难。SDH 使用指针指示 VC 在净负荷区的起始位置,使 VC 在净负荷区内浮动。这样,在 STM－*N* 信号中,通过计算,用指向每种类型支路信号的指针值,就能得到该支路信号的位置,便于实现 SDH 各支路信号的交叉连接与交换,简化了复接、分接处理,不仅省去了大量配线架,还可以消除相关时延。

G.708 规定,传输速率为 2.048 Mb/s 的信号按照 SDH 的复用规范,首先被"映射"到 VC－12 再进入 TU－12 中。

(d) 支路单元群 TUG

支路单元群有 TUG－3 和 TUG－2 两种,它由一个或多个在高阶 VC 净负荷中占据固定而确定位置的支路单元组成,即有

$$m\times TU-n\rightarrow TUG-2,m=1,2\text{ 或 }4,n=11,12\text{ 或 }2$$

$$7\times TUG-2\rightarrow TUG-3$$

$$1\times TU-3$$

(e) 管理单元 AU

管理单元 AU 是在高阶通道层网络和复用段层网络之间提供适配功能的信息结构,它由信息净负荷(高阶 VC)和指示净负荷帧起点相对于复用段起点偏移量的管理单元指针组成,即有

$$AU-n=VC-n+AU-n\ PTR$$

(f) 管理单元组 AUG

管理单元组 AUG 由一个或多个在 STM－N 帧中占据固定而确定位置的管理单元指针 AU PTR 和信息净负荷组成。N 个 AUG 是按字节交织的方法复合在一起的。

例 2.1　将 PDH 四次群信号(139.264 Mb/s)异步映射进 VC－4 模块,并由 AU－4 定位于 STM－1 的过程(图 2.28)。

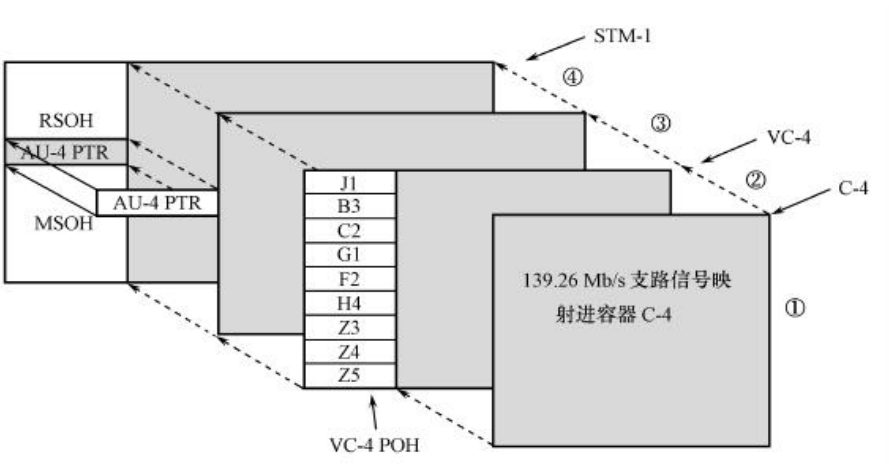


图 2.28　传输速率 139.264 Mb/s 的信号定位至 STM－1 的过程

① PDH 的传输速率 139.264 Mb/s 的信号映射进容器 C－4,经速率调整后输出 149.760 Mb/s;

② C－4 加上 9 个字节的 POH(9 字节/帧,576 kb/s),输出速率150.336 Mb/s 的数字信号;

③ 在 AU－4 内加上 POH(9 字节/帧,576 kb/s),输出速率 150.912 Mb/s 的数字信号;

④ 一个 AUG(因为 $N=1$)加上段开销 SOH(4.608 Mb/s),输出 150.520 Mb/s 的数字信号,即 STM－1。

⑥ 基本复用映射操作

映射：一种在SDH网络边界处使支路信号适配进相应的虚容器的过程。

定位：一种将帧偏移信息收进TU或AU的过程，依靠TU PTR和AU PTR实现。

复用：一种使多个低阶通道层信号适配进高阶通道，或将高阶通道信号适配进复用层的过程，基本的方法是字节间插。

2.3.3　光波分多路复用（WDM）技术

1. WDM的基本原理

光波分多路复用（WDM，Wavelength Division Multiplexing）技术是在一根光纤中能同时传输多个光波信号的技术。WDM的基本原理如图2.29所示，是在发送端将不同波长的光信号组合起来，复用到一根光纤上，在接收端又将组合的光信号分开（解复用），并送入不同的终端。

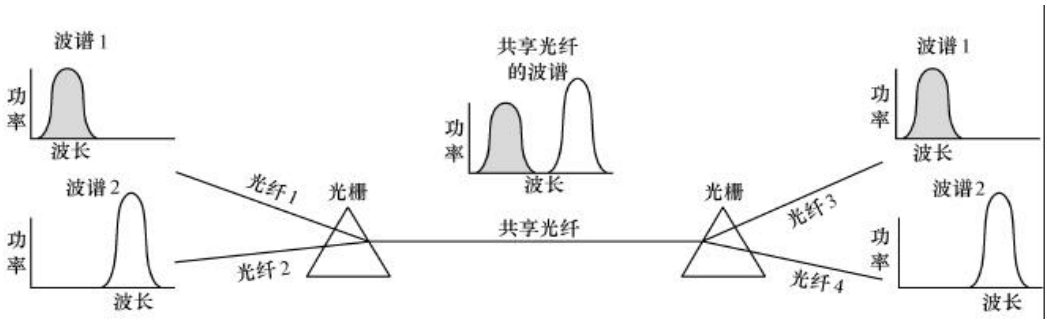


图2.29　光波分多路复用单纤传输

WDM技术对网络的扩容升级、发展宽带业务、挖掘光纤潜力、实现高速通信等具有十分重要的意义。

WDM亦称波分多路复用，而DWDM即密集波分多路复用，是1998年才宣告成功的一项技术，是当前研制出的传输速率最高的网络技术，可以处理传输速率高达80 Gb/s的业务，并将传输速率提高到了800 Gb/s。

DWDM可以大大提高单模光纤传输信息的能力。它把发射机中发射的红外光分解成多个波长，以提高红外光的传输能力。现在，人们已讨论在1 550 nm～1 570 nm的单模光纤上开256个窗口的问题。专家们认为，信息高速公路需要无限容量之时，DWDM的作用才会充分地展现。

2. 光WDM系统原理

简单地说，光WDM就是将一条单纤转换为多条“虚纤”，每条虚纤工作在不

同的波长上。光 WDM 系统有 3 种基本结构:光多路复用单纤传输系统、光双向单纤传输系统和光分路插入传输系统。

(1) 光多路复用单纤传输系统

图 2.30 为光多路复用单纤传输系统的结构图。在这种系统中,发送端将不同波长的已调光信号 $\lambda_1, \lambda_2, \cdots, \lambda_n$ 通过复用器 M 组合在一起,在一条光纤中单向传输;接收端使用解复用器 D 将不同波长的信号分开,从而完成信号传输任务。

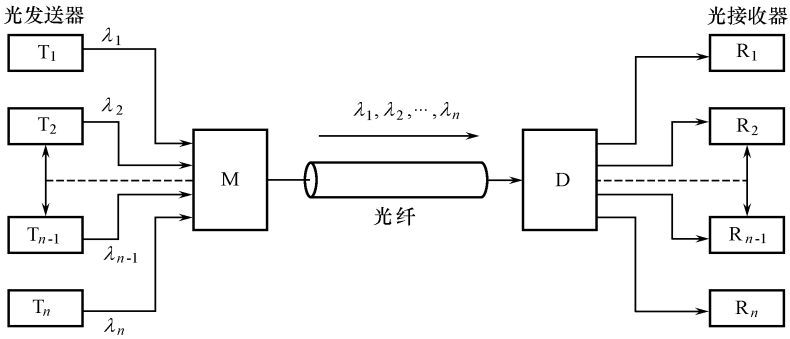


图 2.30　光多路复用单纤传输系统结构

(2) 光双向单纤传输系统

图 2.31 为光双向单纤传输系统的结构图。在这种系统中,用一条光纤实现两个方向信号的同时传输,因而也称为单纤全双工通信系统。实现这种系统的关键是两端都需要一组复用/解复用器 MD。

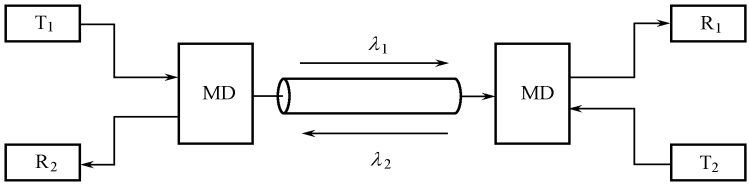


图 2.31　光双向单纤传输系统的结构图

(3) 光分路插入传输系统

图 2.32 为光分路插入传输系统的结构图。在这种系统中,两端都需要一组复用/解复用器 MD,复用器将光信号 λ_3、λ_4 插接到光纤中,解复用器将光信号 λ_1、λ_2 从光纤中分接出来,通过不同波长光信号的合流与分流实现信息的上、下通路。

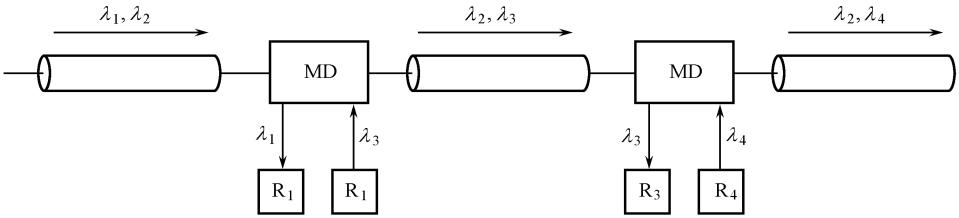


图2.32　光分路插入传输系统的结构图

2.4　差错控制

2.4.1　差错产生的原因与基本对策

在数据传输中，接收到的数据与原来发送的数据不一致称为传输差错。信道噪声引起传输信号的畸变是产生差错的主要原因。如图2.33所示，信道噪声

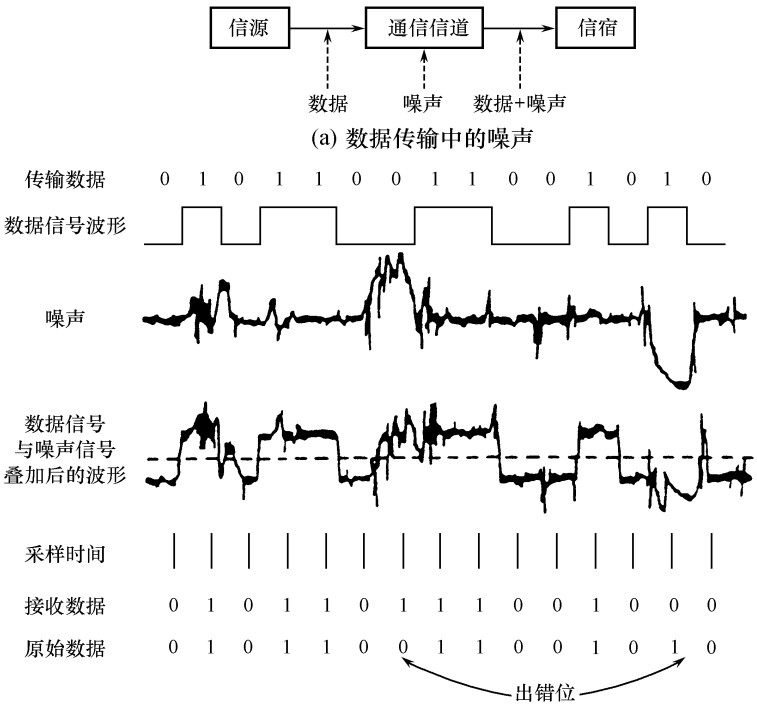


图2.33　数据传输差错的产生

将在数据信号上叠加一些高次谐波,从而引起接收端判断错误。

通信信道的噪声分为两类:热噪声和冲击噪声。

热噪声是内部噪声,由介质中电子热运动而引起,随机产生,强度与频率无关,频谱很宽,产生随机差错;

冲击噪声是外部噪声,由外界干扰引起,其幅值较大,呈突发性,产生突发性差错。

在数据通信中,差错的基本应对策略有 3 个:

(1) 提高信道质量

使用高质量的信道,即使用具有热噪声小、信号屏蔽能力强等优点的信道;

使用中继器,中继器的作用是每经过一定的传输距离将数据信号重新复制一次。

(2) 提高数据信号的健壮性

纠错码,为传输的数据信号增加冗余码,以便能自动纠正传输差错;

检错码,为传输的数据信号增加冗余码,以便查出哪一位出错。

(3) 采用合适的差错控制协议

与检错码相比,纠错码具有自动纠错功能,但实现复杂、造价高、传输效率低。通常是采用检错码检查出差错,由合适的差错控制协议来补救。

2.4.2 循环冗余码校验

目前,主要的检错技术是奇偶校验(Odd-Even Check)和循环冗余码校验(CRC,Cyclic Redundancy Check)。

奇偶校验码分为垂直奇偶校验码、水平奇偶校验码和水平垂直奇偶校验码。它们的冗余位少、方法简单,但纠错能力差,一般只用于要求较低的通信场合。这里只介绍循环冗余码校验 CRC。

循环冗余码是一种能力相当强的检错、纠错码,并且实现编码和检码的电路比较简单,常用于串行传送(二进制位串沿一条信号线逐位传送)的辅助存储器与主机的数据通信和计算机网络中。

循环码通过某种数学运算实现有效信息与校验位之间的循环校验。

1. 编码步骤

① 将待编码的 *n 位信息码组* $C_{n-1}\ C_{n-2}\cdots C_i\cdots C_2\ C_1\ C_0$ 表达为一个 n-1 阶的多项式

$$M(x) = C_{n-1}x^{n-1} + C_{n-2}x^{n-1} + \cdots + C_i x^i + \cdots + C_1 x^1 + C_0 x^0$$

② 将信息码组左移 *k 位,形成* $M(x)\cdot x^k$,即 n+*k 位的信息码组*

$$C_{n-1+k}C_{n-2+k}\cdots C_{i+k}\cdots C_{2+k}C_{1+k}C_k00\cdots00$$

③ 用 $k+1$ 位的信息码组生成多项式 $G(x)$ 对 $M(x)\cdot x^k$ 作模2除，得到一个商 $Q(x)$ 和一个余数 $R(x)$。显然，有

$$M(x)\cdot x^k = Q(x)\cdot G(x) + R(x)$$

生成多项式 $G(x)$ 是预先选定的，关于它稍后再进行介绍，这里先介绍一下模2除。

模2运算指以按位模2加为基础的四则运算，运算时不考虑进位和借位。模2加的规则为：两数相同为0，两数相异为1。模2除，就是求用2整除所得到的余数，每求1位商应使部分余数减少1位。取商的原则是：当部分余数为1时，商取1；当部分余数为0时，商取0。如：

```
          1 0 1 ……                    商
101 ) 1 0 0 0 0 ……                    部分余数为 1
      1 0 1
        0 1 0 ……                    部分余数为 01
        0 0 0
          1 0 0 ……                  部分余数为 10
          1 0 1
            0 1 ……                  余数
```

④ 将左移 k 位的待编码有效信息与余数 $R(x)$ 作模2加，即形成循环冗余校验码。

例2.2　对4位有效信息1100作循环冗余校验码，选择生成多项式 $G(x)$ 为1011（k=3）。

① $M(x) = x^3 + x^2 = 1100$

② $M(x)\cdot x^3 = x^6 + x^5 = 1100000$　（k=3，即加了3个0）

③ 模2除，$M(x)\cdot x^k / G(x) = 1100000/1011 = 1110 + 010/1011$，即 $R(x) = 010$

④ 模2加，得到循环冗余校验码 $M(x)\cdot x^3 = Q(x)\cdot G(x) + R(x) = 1100000 + 010 = 1100010$

2. 纠错原理

由于 $M(x)\cdot x^k = Q(x)\cdot G(x) + R(x)$，根据模2加的规则

$$M(x)\cdot x^k + R(x) = M(x)\cdot x^k - R(x) = Q(x)\cdot G(x)$$

所以合法的循环冗余校验码应当能被生成多项式整除，如果循环冗余校验码不能被生成多项式整除，就说明出现了信息差错。并且，有信息差错时，循环冗余校验码被生成多项式整除所得到的余数与出错位有对应关系，因而能确定出错位置。表2.5为例2.2所得到的循环冗余校验码的出错模式。

表2.5 循环冗余校验码的出错模式

	D7 D6 D5 D4 D3 D2 D1	余数	出错位
正确	1 1 0 0 0 1 0	000	—
错误	1 1 0 0 0 1 1	001	1
	1 1 0 0 0 0 0	010	2
	1 1 0 0 1 1 0	100	3
	1 1 0 1 0 1 0	011	4
	1 1 1 0 0 1 0	110	5
	1 0 0 0 0 1 0	111	6
	0 1 0 0 0 1 0	101	7

进一步分析还会发现,当循环冗余校验码有1位出错时,用生成多项式作模2除,将得到一个不为0的余数,将余数补0继续作模2除,又得到一个不为0的余数,再补0再作模2除……于是余数形成循环。如上例,最终形成001,010,100,011,110,111,101;001,010,100,011,110,111,101……的余数循环,这也就是“循环码”的来历。

3. 生成多项式

并不是任何一个多项式都可以作为生成多项式。从检错和纠错的要求出发,生成多项式应能满足下列要求:

任何一位发生错误都应使余数不为0;

不同位发生错误应使余数不同;

对余数继续作模2运算应使余数循环。

生成多项式的选择主要靠经验。有3种多项式已经成为标准,具有极高的检错率,即

$$CRC-12 = x^{12} + x^{11} + x^{3} + x^{2} + x + 1$$

$$CRC-16 = x^{16} + x^{15} + x^{2} + 1$$

$$CRC-ITU-T = x^{16} + x^{12} + x^{5} + 1$$

2.4.3 差错控制协议

差错控制协议ARQ(Automatic Request for Repeat)是链路层的一种传输控制协议,它具有流量控制和差错控制等功能。ARQ的差错控制原理是反馈重发,按具体的实现方法可以分为停等ARQ和连续ARQ。

1. 停等ARQ

停等ARQ的工作原理如图2.34所示。当主机A发送一个数据帧到主机B

时,若B正确地收到,便会立即发一个确认应答帧ACK给A,A接到确认应答帧,就可以再发下一个数据帧;若B收到的数据帧不正确,便立即发一个否认应答帧NAK给A,A接到否认应答帧,就将数据帧重发一次。

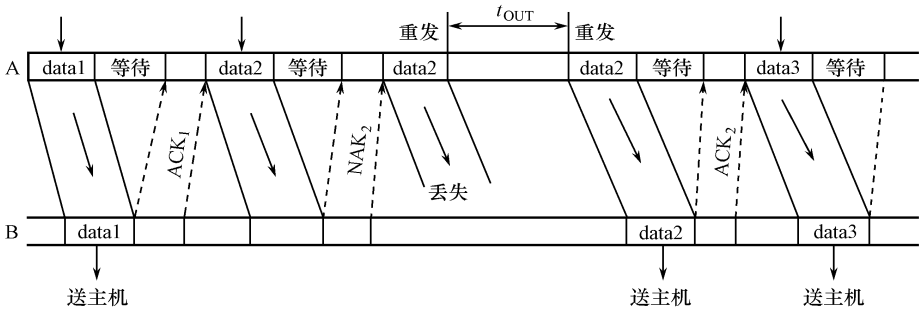


图2.34　停等ARQ的工作原理

这里还有两个问题要解决：

一是当A发出的数据帧丢失,B收不到时是不会发任何应答帧的。这时A一直等待,当等待时间超过一个限度 t_{OUT}时,就将数据帧重新发送一次。

二是B虽然收到了A发来的数据帧,也发出了应答帧(可能是ACK,也可能是NAK),A却没有收到。这种情况下,A等待超过一定时限时,也要将数据帧重新发送一次。

停等ARQ协议简单,但系统效率较低。

2. 连续ARQ

连续ARQ是在发完一个数据帧后,不再等待,而是连续地发送若干个数据帧,具体实现方式有拉回方式和选择重发方式。

(1) 拉回连续ARQ

拉回连续ARQ的工作原理如图2.35所示。在发送方A连续地发送数据帧的同时,接收方对接收到的数据帧进行校验,并向A方发送应答帧;当A接收到一个数据帧对应的应答帧为NAK时,就从这个数据帧开始将此后所发送过的数据帧重发一遍。例如,A方发送了1～5号数据帧,其中第2号数据帧出错,B将其后已收到的帧(2～5号)丢弃,A收到NAK2后,要进行拉回重发。出现超时时,也要拉回重发,如3号帧丢失后,要将已发送的3～6号帧拉回重发。

(2) 选择重发连续ARQ

选择重发连续ARQ与拉回连续ARQ的不同之处在于它只重发出错的数据帧。

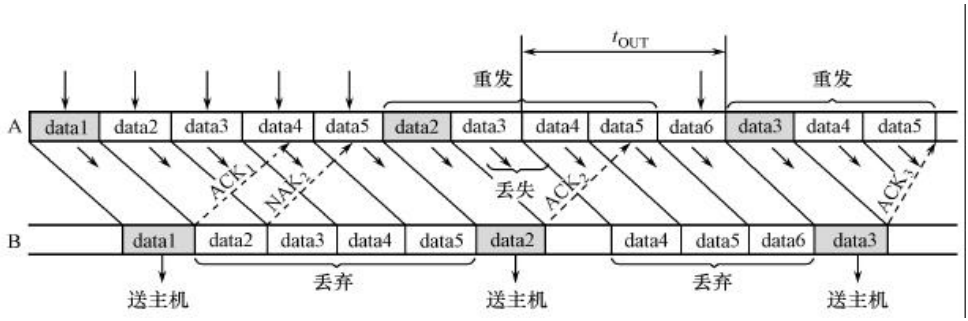


图 2.35　连续 ARQ 的工作原理

2.5　流量控制与滑动窗口协议

在链路上进行流量控制的目的，主要是根据链路的传输水平协调发送端与接收端的关系，使发送端的发送在接收端的控制下进行，使发送能力与接收能力相适应。目前，典型的流量控制技术是采用滑动窗口协议。

滑动窗口协议是从发送和接收两方来限制用户资源需求，并通过接收方来控制发送方的发送数量。其基本思想是：某一时刻，发送方只能发送编号在规定范围内，即落在发送窗口内的几个数据单元，接收方也只能接收编号在规定范围内，即落在接收窗口内的几个数据单元。这不仅可以用于流量控制，还兼有差错控制的功能。

使用滑动窗口协议，要涉及到两个方面的问题：

数据单元的编号问题（这与数据单元中用于编号的位数有关）；

窗口的大小，即缓冲区大小问题。

下面用数据单元 3 位进行数据单元的编码（即数据单元采用模 8 编码），发送窗口的大小为 5，接收窗口的大小为 4，来说明滑动窗口协议的工作原理。

2.5.1　发送器窗口的工作原理

发送器窗口的大小（宽度）规定了发送方在未接到应答的情况下允许发送的数据单元数。也就是说，窗口中能容纳的逻辑数据单元数就是该窗口的大小。

图 2.36 说明了发送窗口移动的规则，其窗口大小为 5。

2.5.2　接收器窗口的工作原理

图 2.37 说明了接收窗口的移动规则，其窗口大小为 4。

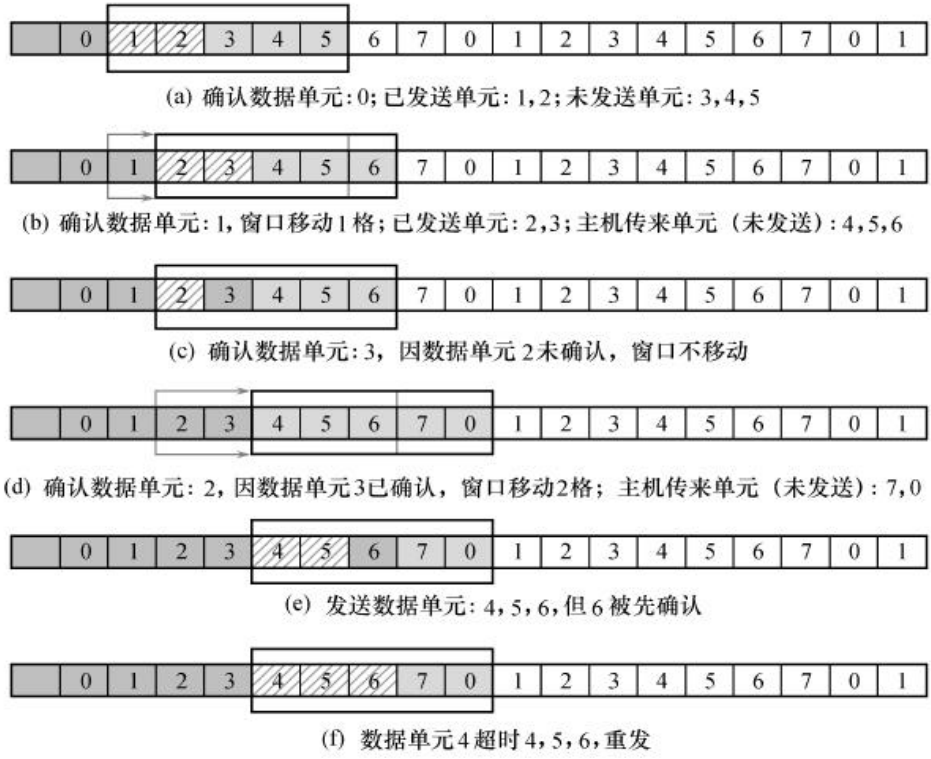


图2.36　发送器窗口的工作原理

图2.37　接收器窗口的工作原理

前面介绍了用滑动窗口进行流量控制的基本原理,具体实现时还有一些问题要处理,如：

窗口宽度的控制是预先固定,还是可适当调整；

窗口位置的移动控制是整体移动,还是顺次移动；

接收方的窗口宽度与发送方相同还是不同。

2.6　高级数据链路控制协议 HDLC

2.6.1　数据链路连接管理方式

数据链路(Data Link)是指可以用来传输数据的链路,它包含两方面的内容:一是一条物理链路,二是一些必要的协议。或者说,把实现数据传输协议的硬件和软件加到链路上,就形成了数据链路。

数据链路层的功能是计算机网络中最基本的功能,是任何网络都必有的层次,也是最成熟的层次。数据链路层的控制规程是在不太可靠的物理链路基础上实现可靠的数据传输,也就是在物理链路的支持下要重点考虑数据传输的可靠性。

在计算机网络中,数据链路层的主要功能是为网络层提供连接服务。其基本的连接管理方式有两种:面向连接的服务和无连接的服务。

1. 面向连接的服务

面向连接的服务也称连接型服务,这种服务方式把一次数据传送过程分为 3 个阶段:

建立数据链路,让双方的链路层都同意并作好传送准备;

传送数据帧,将网络层移交的数据传输到对方;

拆除数据帧,拆除数据链路的连接。

在通常情况下,面向连接的服务在数据传送阶段对每个帧都要进行确认,发送方收到确认标志信号后才能发送下一个帧。这种确认称为应答。应答有 3 种方式:

正向应答,只在接收到正确帧时才返回 ACK,其他情况不应答,靠超时重发;

负向应答,只在接收到受损帧时才返回 NAK;

双向应答,既可返回 ACK,也可返回 NAK。

面向连接的服务也有无应答的情况。

2. 无连接的服务

在这种服务中,发送前不建立连接,源主机可以在任何时候发送链路层数据单元。无连接的服务可以有应答和无应答两种情形:

有应答,目的主机的数据链路层接收帧时要进行帧校验,并向发送方返回应答;

无应答,目的主机接收到帧即将其交给网络层。

2.6.2　HDLC的配置和数据传输工作方式

为了适应不同配置和不同的数据传输方式,HDLC定义了3种类型的站、2种链路配置和3种数据传输方式。

1. 通信站

(1) 主站

主站的主要功能是发送命令(帧),也接收响应帧,负责对整个链路进行控制。在一个通信链路上,同一时刻只有一个站作为主站。

(2) 次站

次站接收来自主站的命令(帧),并向主站发送响应帧。次站受主站控制,只能同主站交换数据,而不能同其他次站交换数据。

(3) 复合站

复合站兼有主站和次站的功能,既能发送命令帧,又能接收命令帧,能对全链路进行控制。

2. 两种链路配置

(1) 非平衡配置

非平衡配置是由主站和次站组成的链路,适用于点到点、点到多点的通信,支持半双工或全双工通信。

(2) 平衡配置

平衡配置是由两个复合站组成的链路,支持半双工或全双工通信,仅适用于点到点的通信。

3. HDLC数据传输工作方式

当通信站处于信息传送状态时,一般有3种数据传输的工作方式:

(1) 正常响应方式(NRM)

NRM是用于非平衡配置的数据传输方式,只有主站才可以把数据传送给次站,而次站只能传输响应信号。

(2) 异步平衡方式(ABM)

ABM是用于平衡配置的数据传输方式,在没有接收其他复合站指令信号的情况下,任何复合站均可以先开始传输数据。

(3) 异步响应方式(ARM)

ARM是用于非平衡配置的数据传输方式,当主站没有明确发送等待命令的情况下,次站可以先开始传送数据,但主站仍然具有对通信链路的指挥管理功能。

2.6.3 HDLC 帧格式

链路上的全部传输都是以帧的形式进行的，帧是数据链路协议的数据单元。HDLC 帧结构如图 2.38 所示，它由报头、数据字段、报尾三部分组成。

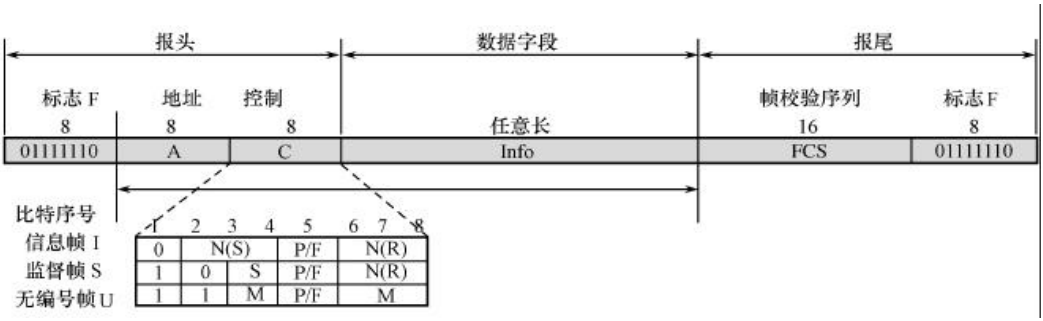


图 2.38 HDLC 帧结构

在帧的组成中，除数据字段外，其他都是控制信息，下面介绍这些控制信息的用途。

1. 帧同步控制与透明传输

数据链路层中的帧同步控制，就是从接收到的比特流中正确地判断一个帧的开始和结束。HDLC 中采用 8 位的标志 F (Flag) 来作为帧的开始和结束标志，它们都为“01111110”。

在两个标志字段之间，完全有可能出现与标志字段相同的字段，即出现“01111110”，这样系统就有可能将其作为结束符，从而出现错误。为此，在发送一个帧时，若发现有“0”后面跟着 5 个“1”，就加一个“0”，以区别标志 F。在接收过程中，先确定帧的起止边界，然后对比特流进行扫描，发现连续的 5 个“1”，就将其后的 1 个“0”删除，将比特流还原。这样，出现任何比特组合都不会引起对帧边界的错误判断。具有这种特点的传输称为透明传输。

2. 寻址

HDLC 在帧中设置 8 位的地址字段来填写次站 (对非平衡式) 或应答站 (对平衡式) 的地址。在 256 个可用地址中，有两个特殊地址：全“1”为广播方式，全“0”为无效地址。其余 254 个地址表明一条链路上最多可以连接 254 个次站。一般情况下 256 个地址足够使用了。在特殊情况下，如分组无线电的用户很多时，可以对地址字段进行扩充。

3. 差错控制

HDLC 设置 16 位的帧校验序列 (FCS, Frame Check Sequence) 字段，采用循环

冗余校验,生成多项式 $G(x)=x^{16}+x^{12}+x^{5}+1$。图2.38标出了校验区间A,C,Info。

4. 发送序号和接收序号

发送序号和接收序号主要用于监视所传送的信息帧有无丢失或重发。由3位的N(S)和N(R)编号,分别可以编排8个序号。N(S)为发送序号(只有信息帧中才有)。N(R)为期望接收到的对方发来帧的接收序号,带有确认的意思,可以在发送信息帧中捎带确认信息。

5. 数据与控制信息的识别

HDLC规定了3种帧:

(1) 信息帧,C字段的第1位为"0",主要用于完成数据传输和捎带确认信息。

(2) 监督帧,C字段的第1位和第2位分别为"1"和"0",用于监视链路的常规操作,共4种——接收端的就绪(RR)、未就绪(RNR)、拒绝(REJ)和选择拒绝(SERJ),由两位的S字段取值决定。监督帧也可以捎带确认信息。

(3) 无编号帧,C字段的第1位和第2位分别为"1",用两个M字段的编号表示不同的无编号帧,主要用于链路的建立、释放、恢复、响应等。由于不带有N(S)和N(R),所以称为无编号帧。

6. 询问/终止

P/F(Poll/Final)为询问/终止。主站或复合站向次站或复合站发出的命令帧中,P为1,表示询问,允许对方发送数据。次站或复合站响应询问,可以发一连串的响应帧,但若将F置1,表示为最后一个响应帧,传送终止。

习　　题

2.1　何谓单工、半双工和全双工?它们各用于何种场合?

2.2　一个经填充的位串为011110111110111111110,其输出串是什么?

2.3　数字信号与模拟信号有何区别?

2.4　设有4个符号:X1,X2,X3,X4,分别以1/2,1/4,1/8和1/8的独立概率出现。试计算消息Y=X2X4X1所携带的信息量。

2.5　试述信道的传输速率、带宽和容量三者之间的关系。

2.6　数字信号的编码方式有几种类型?各有何特点?

2.7　试绘出二进制数据00110101的下列编码波形:

(1) 单极性不归零码;

(2) 双极性不归零码;

(3) 单极性归零码;

(4) 双极性归零码；

(5) 曼彻斯特码；

(6) 差分曼彻斯特码。

2.8 说明最基本的几种调制方法的特点。

2.9 讨论链路、物理信道与逻辑信道之间的区别和联系。

2.10 何谓基带传输、频带传输和宽带传输？

2.11 电视频道的带宽为 6 MHz,假定没有热噪声,当数字信号取 4 种离散值时,可获得的最大数据率是多少？

2.12 对一个 4 kHz 的无噪声信道每 0.1 ms 采样一次,可以得到的最大传输速率有多少？

2.13 在 50 kHz 的信道上传输 T1 信号,需要的信噪比最少要多大？

2.14 有一信道,频率范围为 60 kHz～108 kHz。假定每路信号的带宽为 3.2 kHz,各路信号间的防护间隔为 0.8 kHz,若采用频分多路复用,最多可以同时传输几路信号？

2.15 PCM 复用与数字复接有何区别？当前广泛采用数字复接技术的理由是什么？

2.16 同步复接和异步复接时,是否都要进行码速调整,为什么？

2.17 已知 PCM 三次群的数码率为 34 368 kb/s,它由 4 个速率为 8 448 kb/s 的二次群复接而成。试计算在原二次群支路中平均每帧 (125 μs) 内插入了多少个附加码字？这些码字起什么作用？

2.18 说明 SDH 中 C、VC、TU、TUG、AU、AUG、STM－N 的含义及其在网络中的位置和作用。

2.19 OC－1 (STS－1) 是一个每 125 μm 产生 90 列×9 行的矩形帧的同步传输信号。其每一帧的前 3 列被留作系统管理使用 (前 3 行包含段开销,后 6 行包含线路开销),同步载荷的第 1 列是通路开销。OC－3 相当于 3 个 OC－1 复合在一起。试计算 OC－3 的用户数据传输速率。

2.20 简述光 WDM 的工作原理与特点。

2.21 已知 CRC 生成多项式为 $G(x) = x^4 + x + 1$,设要传送的码字为 10110 (从左到右发送),试计算校验码。

2.22 某一数据通信系统采用 CRC 校验,并且生成多项式 $G(x)$ 的比特序列为 11001,目的节点接收到的比特序列为 1101111001 (含 CRC 校验码)。请判断传输中有无差错,并说明道理。

2.23 为什么数据链路层协议要把 CRC 放在帧的尾部？

2.24 一个 4 kHz 的信道,传播延迟为 20 ms,为了使停等协议至少有 50% 的效率,帧的大小应当在什么范围内？

2.25 在数据通信系统中,完整的差错控制应包含哪些方面？

2.26 试述流量控制的策略。

2.27 简述滑动窗口协议的工作原理。

2.28 试述 HDLC 的几种数据传输方式,说明 HDLC 的帧格式及其各字段的意义。

第 3 章　中间节点上的通信技术

如前所述,在计算机网络中,可以将节点分为端节点、中继节点、交换节点和路由节点。端节点上的通信在第 2 章已经讨论了一些,更详细的内容将在第 2 篇讨论。中继节点的功能就是放大和整形,没有什么更多的内容。本章主要讨论交换节点和路由节点上的通信技术。

3.1　交换节点上的通信

在多节点的网络中,为了提高线路的利用率,任意两个节点间的通信不可能都是建立在一条直接通路上,在许多情况下要使用两条以上的线路。把一条线路上的数据转接到另一条线路上,称为数据交换。或者说,交换的基本功能就是转发业务流。如图 3.1 所示,交换是通过交换节点中的交换机构实现的。交换机构的功能是将一条输入信道上的数据转送到另外的输出信道上,将输入端口与输出端口对应起来。

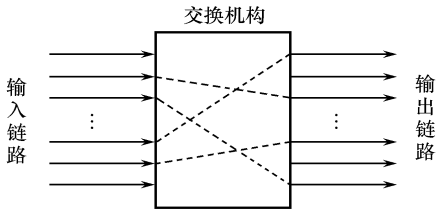


图 3.1　交换机构的功能

这一节介绍数据交换的基本方式和数据交换机构的基本工作原理。

3.1.1　电路交换

在电路交换 (Circuit Switching 或 Circuit Exchanging) 方式下,通信的双方在进行数据传送之前先要建立一个实际的物理电路连接,连接的电路被通信的一对用户独占,只有通信结束电路释放后,才能被别人使用。简单地说,它要经过 3 步:建立连接 (呼叫) ,数据传送,线路拆除 (释放) 。在通信过程中,交换机为通信

双方提供物理电路连接。图3.2演示了在连续的4个节点之间用电路交换方式进行数据传输的基本过程。

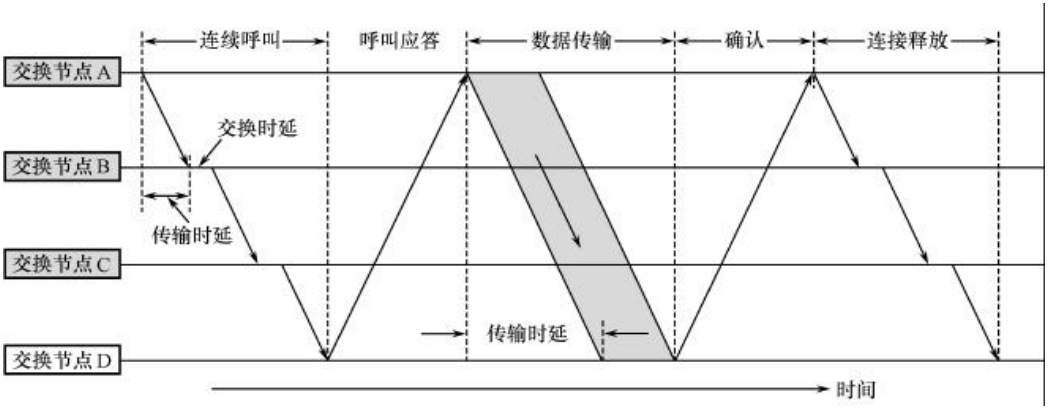


图3.2　电路交换的基本过程

电话系统是一种电路交换的网络,通信双方一旦接通,便独占一条线路,即使没有数据传送,别的用户也无法利用。严格地说,它只是物理层的一个延伸,在电话系统内部并没有上层协议保障其差错控制能力。在传输数据时,这些功能一部分要由调制解调器承担。

电路交换有如下特点:

(1) 由于需要连接过程,建立连接需要时间,故适合传输大量数据,而传输少量数据时效率不高;

(2) 连接一旦建立,便可以固定的速率传输数据,除了传输延迟外,不再有别的延迟;

(3) 一旦连接成功,就建立了一条临时专线,即使不通话,也被占用,他人不可使用。

3.1.2　存储-转发交换

1. 存储-转发交换的概念

存储-转发交换(Store-and-Forward Switching)是一种不要求建立专用物理信道的交换技术。当发送方要发送信息时,应把目的地址先加到报文中,然后从发送节点起,按地址把报文逐节点地转送到目的节点;在转送过程中,中间节点要先把报文暂时存储起来,然后在线路不忙时将报文转发出去,这就是将其称为存储-转发交换的缘由。存储转发交换不像电路交换那样要独占一条固定的信道,线路利用率高,同时可以根据网络中的流量分布动态地选择报文的通过路

径,系统效率高,因而得到了广泛的应用。

为了实现存储－转发,每个交换节点要为每一个端口分别设置一个输入缓冲区和一个输出缓冲区。

2. 分组交换系统的发展

早期的存储－转发交换以报文(Message)形式进行,称为报文交换。图3.3演示了在连续的4个节点之间用报文分组交换方式进行数据传输的基本过程。

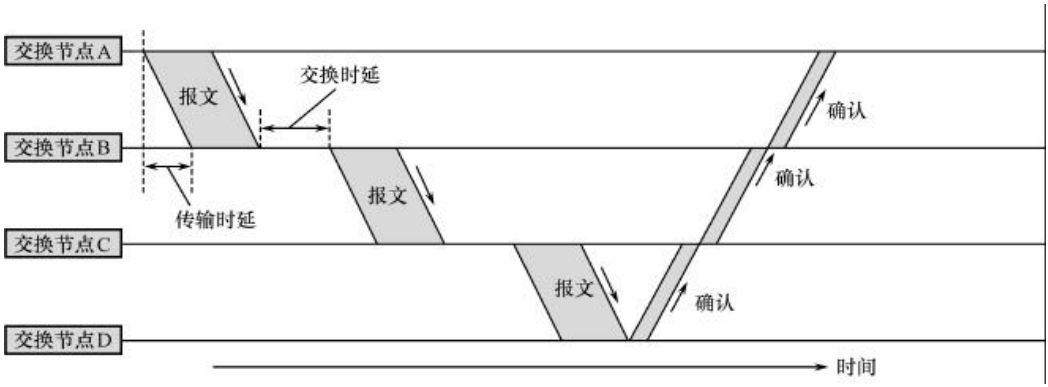


图3.3　报文交换的基本过程

一般说来,报文较大,因而传输延时较长,不适合传输语音等实时信号;并且,在传输过程中容易因为个别错误而导致整个报文传输作废。例如,某一线路允许的差错率为10^{-5},若传输的报文长度为100 kB(B表示字节),则每次传输中都可能有1个字符出错。这样的报文很难传到目的节点。为此,在报文交换的基础上,研制出分组交换(Packet Switching),也称包交换。报文分组后,就可以避免上述两个缺点。

在分组交换网中,要先把一个报文分割成规定长度的信息组——分组打包,然后在每个分组上贴上标签——报头,按编号一批一批地将"数据分组"发出去;在每一个中间节点上,都要先存储、后转发;传送到达目的地后,再重新装配成完整的报文。图3.4为报文分组的示意图。

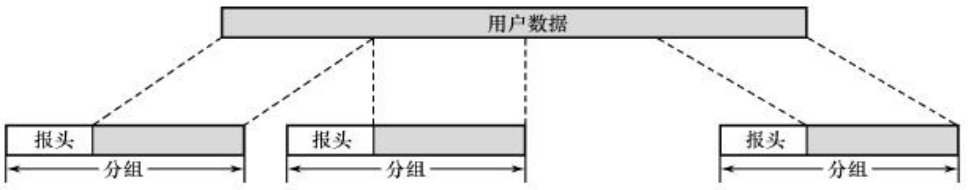


图3.4　报文分组

ATM、TCP/IP 都是分组交换。但是，它们的分组大小和具体交换方式各不相同。

3.1.3　分组交换的虚电路服务和数据报服务

分组交换有虚电路（Virtual Circuit）和数据报（Datagram）两种服务方式。

1. 虚电路服务

虚电路的服务过程与下面的部队从 A 地向 D 地转移的过程相似。

① 先派侦察兵侦察出一条安全的通道（如图 3.5 中的 A－B－C－D），侦察工作完成后，向指挥部发回报告；

② 部队接到通道已经侦察好的报告，开始以支队为单位沿侦察好的通道转移，并且是先出发的先到达；

③ 部队转移结束，A 地通知 D 地，该通道的任务完成，停止使用。

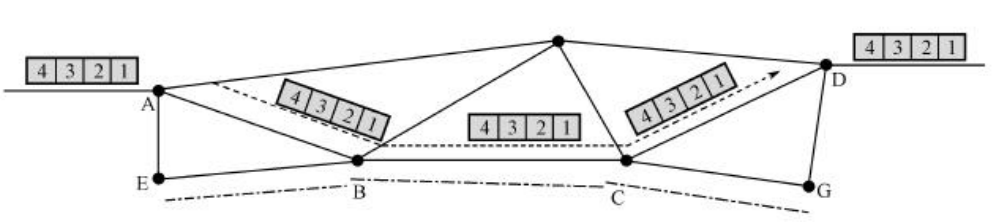


图 3.5　虚电路的通信方式

虚电路服务也分 3 个阶段。图 3.6 说明在 A－B－C－D 4 个节点之间以虚电路方式进行数据传输的时序关系。

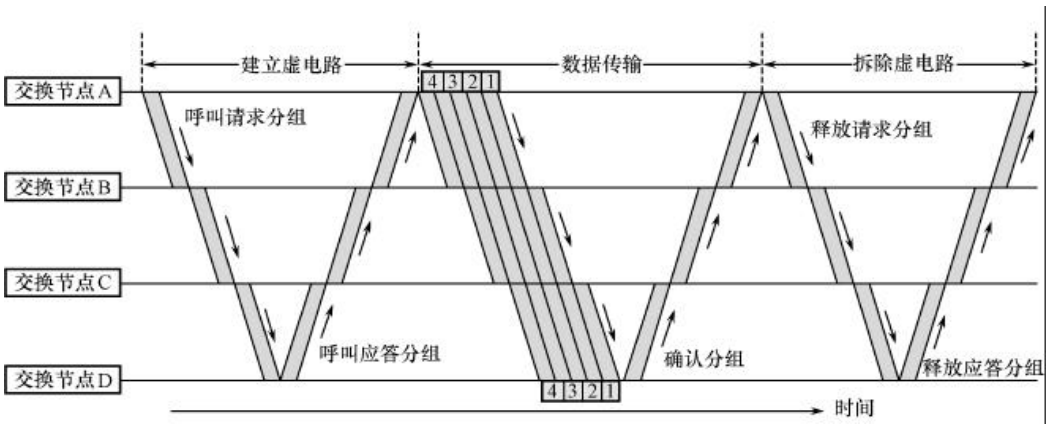


图 3.6　虚电路工作时的时序关系

(1) 建立虚电路

A启动路由算法,选择下一个节点B,并向B发送一个“呼叫请求分组”;接着B也启动路由算法,选择下一个节点C,并向C转发“呼叫请求分组”。于是“呼叫请求分组”沿A－B－C的路径到达目的节点D。目的节点D则沿D－C－B－A的路径返回“呼叫应答分组”。至此虚电路建立。

(2) 数据传输

A将报文分组打包,不需再加目的主机的全地址,只需带上到B的虚电路号(A对可能到达的每个节点编号)即可,按照该虚电路号将各数据分组发送到B。B如法炮制,将各数据分组发送到C,由C再发送到D。

这里之所以要使用分组(包),而不是将报文作为整体发送,是因为:如果在某个节点上检验出某个分组有错,只要重发该分组即可,不需将全部报文重发,这是分组交换比报文交换的优越之处;不会在物理上占用每一段电路,如图3.5中的B－C段就可以供E到G通信时建立虚电路使用。这是存储－转发交换比电路交换的优越之处,也是虚电路与逻辑电路的区别,从复用的角度也可以将其称为分组复用。此外,采用虚电路方式可以连续地发送数据分组,数据分组上附加的地址信息少,且接收到的数据分组顺序与发送的顺序一致,又有些像电路交换,可以保证数据分组的实时性。所以说,虚电路是电路交换和存储－转发交换的结合。但是,若在某一对相邻节点间的传送中有一个分组不能被正确接收,其他分组就要等待,从而降低了传送效率。

(3) 拆除虚电路

报文传送完毕,A沿A－B－C－D路径发送“释放请求分组”到D;D返回“释放应答分组”,将虚电路拆除。

虚电路可以是临时的,也可以是永久的。临时虚电路也称交换虚电路(SVC),它在每次数据传输时建立一次连接,每次数据传输结束便拆除。永久虚电路(PVC)则是将连接持续到其中一台计算机关机时为止,两端用户可以随时使用该逻辑信道。

2. 数据报服务

数据报服务方式犹如大部队化整为零后分头向某地行进集结,每个分组都加有目的地地址,并且按“各自为战”的策略,依据网络的运行情形各自选择合适的路径,向目的地进发。这样,就会呈现以下几个传输特点。

(1) 每个分组所走过的路径可能是不相同的,例如对于有A、B、C、D 4个节点的全连通网络,要从节点A发送数据到节点D,则可能有下列传输途径:

A－B－D;

A－C－D;

A－B－C－D；

A－C－B－D；

A－D。

图 3.7 示出了一个只有 A、B、C、D 4 个节点的网络在数据报方式下进行传输的示意图。

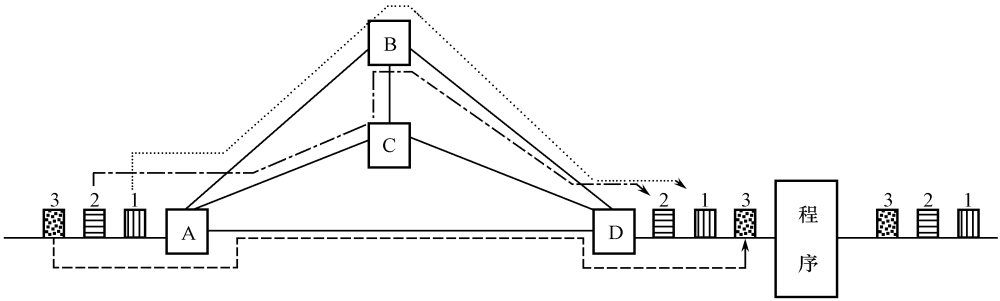


图 3.7　数据报工作原理示意图

⑵ 由于传输路径不同，各节点交换处理的时间不等，到达目的地的时间也不相同，于是就会出现到达目的地后各分组的顺序与发送时的顺序不同，必须重新排序，再装配成报文。

⑶ 数据报服务不需要进行连接，因此也说它提供无连接的服务。后面要介绍的 IP 就只提供数据报服务。

⑷ 数据报服务要求在传送过程中每一个数据分组都要带有目的地址和源地址。同时在传送到目的地后，有可能出现乱序、重复与丢失现象，因而传输延时较大，适合于突发性通信，不适合于长报文通信。

3. 电路交换、虚电路与数据报的比较

下面主要从三个过程、三个参数上对交换方式进行比较。

⑴ 三个过程

连接过程；

传输过程；

释放过程。

电路交换和虚电路都有连接和释放过程。但是，电路交换所连接的是物理信道，而虚电路连接的是逻辑电路。虚电路服务，就是当两个节点之间要传输数据时，必须首先进行路由选择，建立一条虚电路连接；虚电路连接建立之后，所有的分组将沿该路径传送；数据传送结束后，要拆除虚电路连接。换句话说，在虚电路方式中，所有的分组将沿同一条路径传输，整个的传输过程只是连接时进行

一次路由选择。

数据报没有连接过程。各分组传输路径不一定相同,是一种各自为政的方式,并且每做一次转发都要进行一次路由选择。为了不断地进行路由选择,每个分组都必须具有目的端的完整地址。对于大型网络,目的地址字段会非常长。而对虚电路来说,由于在数据传输前路径已经确定,每个分组中不一定再需要完整的地址字段,而只用简短的虚电路号代之。虚电路号表明经过一个节点时,下一步将从哪一条逻辑电路上发出去。为此,要为每个节点建立一张节点逻辑电路管理表。

(2) 三个参数

(a) 时延

呼叫时延:呼叫连接过程花费的时间;

传输时延:从一个节点传输到下一个节点所用的时间;

节点时延(处理延迟):节点进行数据交换和处理花费的时间。

(b) 顺序

分组到达目的节点的顺序是否与源节点发送时的顺序一致。

(c) 通用性

是否适合不同类型、不同速率的计算机或终端之间通信。

电路交换一经连接即可以恒定速率进行数据直通传输,只有传输延迟,没有节点延迟。分组交换,不管是数据报还是虚电路,都有节点延迟,因为每一个中间节点都要进行存储、转发等处理,因而不仅有传输延迟,还有可变节点延迟。其中,由于每个数据报都是独立发送的,因而数据包节点延迟要比虚电路长;另外,由于各数据报报文分组,经过的路径不同,节点延迟不同,因而分组的到达顺序可能会与发送时不同,传给主机前还要进行重装。

3.1.4　交换机的功能

现代通信网络按照有无交换功能可以分为两大类:交换网与传输网。传输网没有交换功能,仅仅用于数据传输。例如DDN(数字数据网)就没有交换功能,一般用于向用户提供专线服务。在交换网中,数据交换是由交换机实现的。因此,交换机是交换网的交通枢纽。简单地说,交换机的作用是接收数据,然后有选择地将数据转发出去,实现交换网中的数据流控制。进一步说,它的功能包括:

数据转发;

物理编址——定义数据帧的物理地址;

网络拓扑结构设定——定义设备物理连接所形成的网络拓扑结构;

差错验证——错误发生时发出告警；

数据帧整序；

流量控制——延缓数据的传输能力。

3.1.5 交换机实现技术

构成交换机构的最基本元件是交换单元。交换单元的基本实现技术有两种：空间交换和时隙交换。

1. 空间交换

空间交换是指交换单元按照链路的空间分布进行交换，将分组从一条链路转送到另一条链路上。最典型的空间交换机构是图3.8所示的交叉点矩阵结构，也称纵横制交换（crossbar switching）机构。它的*N路输入线和N路输出线之间有N^2个接点*。通常这些接点是触点开关或电子开关，只要适当地控制这些开关的接通或断开，即可在任一路入线和出线之间构成通路。

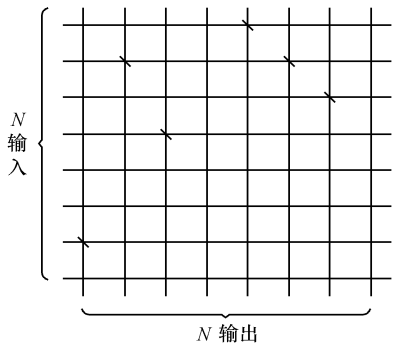


图3.8 一个简单的交叉点矩阵

2. 时隙交换

时隙交换是按时间顺序进行交换，是指将输入链路上某个时隙上的分组安排到输出链路上的另一个时隙上。

(1) STM传输模式的交换机

STM模式的交换机按照时隙顺序进行交换。图3.9给出一种称为ATOM的共享介质型交换单元结构。在共享介质型交换单元中，各个输入端口的分组被同步地分时复接到一组高速介质（如时分复用总线等）上，各个输出端口也与这组高速介质相连。输出端口利用地址过滤器（AF）将发往本端口的分组接收下来。很像商店里出售处理品的柜台，售货员将各种要处理的商品倒在摊位上，顾客经过过滤挑选自己需要的商品。

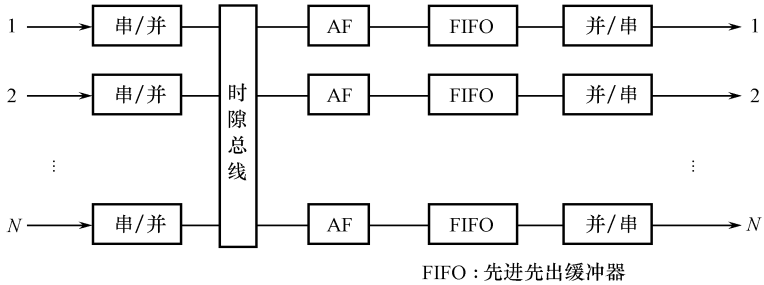


图3.9　ATOM 模块结构

共享介质型交换机有一个高速背板，用于交换单元间的互连。因而共享介质结构常用于模块化的交换机中，以实现较高的端口密集度。

(2) ATM 传输模式的交换机

ATM 传输模式的交换机是按照分组标记进行交换。图3.10所示为共享内存型交换单元结构，其核心是一个 RAM。各个输入端口的分组流复接成一条高速分组流后，根据分组中的标记写入 RAM 中与每个输出端口对应的队列中。输出时，轮流从各个队列读出一个分组，输出分组流经分接后到达各个输出端口，完成交换过程。

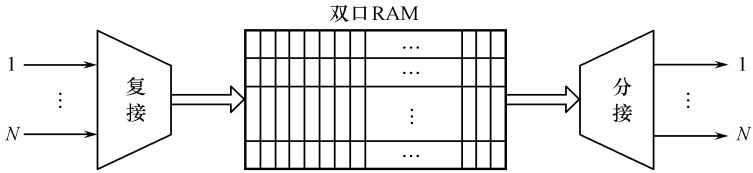


图3.10　共享内存型交换单元结构

共享内存型交换单元是低价小型交换机中常见的一种结构，其优点是能够在一台交换机中同时支持不同类型和速率的局域网。

3. 多级互联网络交换结构 MIN

为了减少交叉节点的数目，可以把大容量交换网络分成小容量的交换单元，并将这些交换单元连接成多级互联网络（MIN，Multi－stage Interconnect Network）。图3.11示出了一个由2×2的基本交换单元构成的3级 MIN 结构，每一级有4个基本单元。级间要求全互连，以使每一条入线都能到达每一条出线。

当然，基本交换单元不一定是2×2的，也可以是16×16的或其他。显然，构成同样的大容量交换结构，采用大的交换单元比采用小的交换单元，所使用的交换单元数量要少。

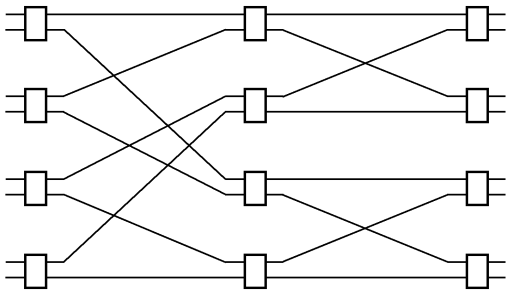


图 3.11　多级交换结构

MIN 最初是在 20 世纪 50 年代由 C.Clos 发明并应用到电路交换网络中的，20 世纪 70 年代开始应用于分组交换网络。

4. 分组交换结构的发展

计算机网络中的交换系统最早是由计算机实现的，如 ARPAnet，它为每一个端接口分别设置一个分组输入队列和一个分组输出队列，交换过程就是将某个端接口输入队列中的分组放到另一个端接口的输出队列中。为了减轻主计算机的负荷，后来在系统中增加了前端机 FEP（Front－End Processor），执行较低级别的规程，如链路差错控制等，形成图 3.12 所示的结构。

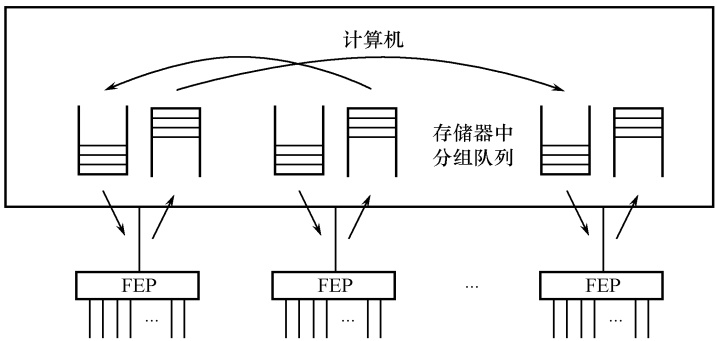


图 3.12　基于计算机的分组交换系统

分组交换系统进一步发展，前端处理机的功能不断增强，计算机退出对分组交换过程的直接干预，仅仅用于虚电路的建立，并通过总线与各前端处理机交换信息，形成图 3.13 所示的结构。这时，系统中计算机瓶颈消除，但由于采用共享介质分时地进行分组传输，使吞吐量受到介质带宽的限制。

计算机网络发展的历史，就是计算机技术和通信技术在发展中融合的历史。在分组交换系统中引入电话交换系统，是迟早的事情。这种系统的结构如图

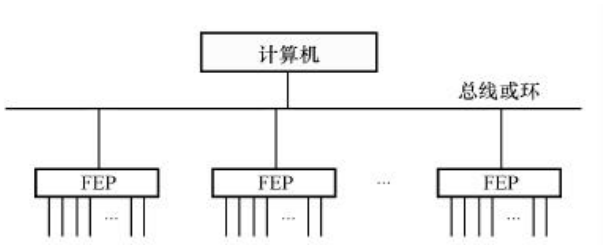


图3.13　基于独立前端处理机的分组交换系统

3.14所示。它用较小的基本交换单元构成多级交换网络，在控制处理机的控制下进行并行处理，大大地提高了吞吐量。这就是今天广泛应用的交换机系统。

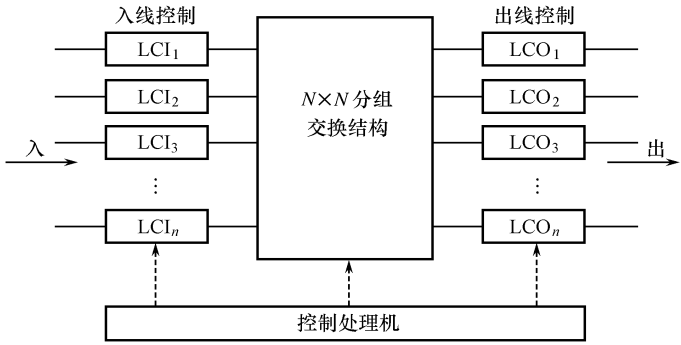


图3.14　交换机系统结构

图中：

LCI为输入线路控制器，LCO为输出线路控制器，用于对各分组的逻辑信道号进行转换，并在各分组头上加上和去除经过交换结构时需要的路由信息；

交换结构根据路由信息将分组送往相应输出端口，将一时无法送出的分组存放在缓存中；

控制处理机控制呼叫连接建立、释放、管理和维护。

3.2　路由节点上的通信

路由节点是联结不同网络的节点。当数据分组传输到路由节点时，路由节点不是对其进行简单的交换；由于它处于几个网络的边界，因此首先要判断该数据分组的目的地是否与送来的节点在同一个网络——称为本地；如果是，就本地提交；如果不是，就转发出去。由于它可能联结多个网络，因此转发时还要考虑

从哪一条路径上转发出去较好。在路由节点上承担路由工作的设施是路由器。如果将每个网络看成一条链路,那么网络与路由节点组成的拓扑结构就是互联网的拓扑结构。所以说,路由器的配置决定了互联网的结构框架。

3.2.1 路由表与路由算法

路由器的主要工作就是为经过路由器的每个数据帧寻找一条最佳传输路径,并将该数据帧有效地传送到目的站点。为此,每个路由器中都要维护一个路由表(Routing Table),供路由选择时使用。路由表中的每一个表项均为四元组:〈目的地,下一站地址,下一站网络接口号,距离〉。以目的地址作为关键字,就可以从路由表中查出下一站路由器的地址以及它所在的接口。因此,路由表就成为路由器的中枢,它决定了每个数据包的转发方向。于是路由表的建立和维护就成为路由器技术的关键。建立和维护的算法称为路由算法。

路由选择是一个极为复杂的问题。因为网络中每一个节点的工作都是置于网络上其他节点形成的环境中,并且网络上各个节点、各条路径的工作状态往往是随机的。此外,当网络处于拥塞状态时,路由选择应当有利于拥塞的缓解。下面介绍几种常用的路由选择算法。从总的方面看,它们分别属于静态路由和动态路由。

1. 静态路由

静态路由用预先编好的程序定义通过网络的路径,其算法通常由网络管理员人工实现。该管理员负责发现和传播通过网络的路由。下面介绍几种常用的静态路由算法。

(1) 洪泛(Flooding)算法

洪泛算法也称扩散式算法。它的基本思想是,每个节点收到分组后,即将其发往除分组来的节点之外的其他各相邻节点。可以想像,按照这种算法,网络上的分组会像洪水一样泛滥起来,造成大量的分组冗余,导致网络出现拥塞现象。因此要限制分组复制的数目。可以有3种方法:一是在每个分组的头部设一个计数器,用来统计分组到达节点的数量,当计数器超过规定值(如端到端最大段数)时,将之丢弃;另一种方法是,在每个节点上建立一个分组登记表,不接收重复的分组;还有一种方法是只选择距目标节点近的部分节点发送分组。

洪泛算法具有很好的健壮性和可靠性,适用于规模较小、可靠性要求较高的场合。

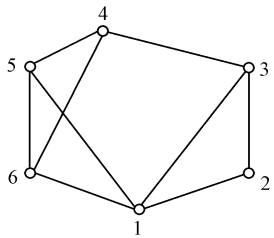
(2) 热土豆(Hot Potato)算法

通常每个节点要为其连接的相邻节点各建一个分组队列。热土豆算法是,

节点收到一个分组后，为了尽快脱手，要将其放在最短的队列中，而不管该分组的目标节点是什么。

(3) 固定路由算法

固定路由算法也称查表法，它是在网络的每个节点上都存放一个预先计算好的路由表，给出本节点到所有可能的目标节点的最短路径。图3.15所示为一个固定路由算法的例子。这样，一个节点每收到一个分组，通过查表就可找出相应的转发出口。



(a) 通信子网

后继节点		当前节点					
		1	2	3	4	5	6
目标节点	1	/	1	1	5/6	1	1
	2	2	/	2	3	1	1
	3	3	3	/	3	1/4	1/4
	4	3/6	3	4	/	4	4
	5	5	1	1/4	5	/	5
	6	6	1	1/4	6	6	/

(b) 全路由表

图3.15　一个固定路由算法的例子

固定路由算法简单、实现容易，选择的是最短路径，但是不适应网络的拓扑变化，当被选路由出现故障时，影响正常传送。改进的方法是，为每个节点提供一个次佳路由，当第一路由失效时，起用次佳路由。

固定的路由是静态路由，它由网络管理员事先设置，一般是在系统安装时根据网络的配置情况预先设定的，不随未来网络结构的改变而改变。当网络的拓扑结构发生变化时，要由网络管理员手工修改路由表中的相关路由。并且，静态路由信息在缺省情况下是私有的，不会传递给其他的路由器。不过，网络管理员也可以将其设置为共享的。

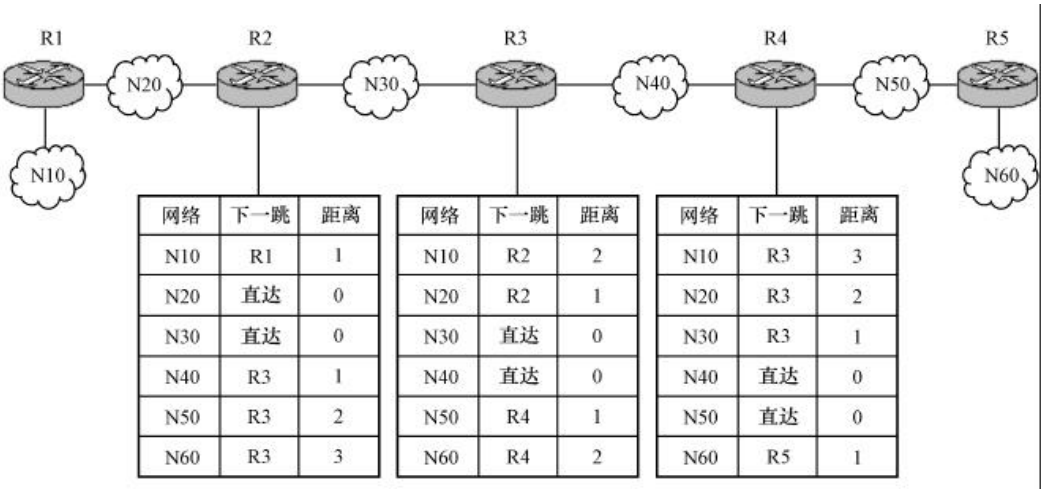
静态路由适合比较简单的网络环境。在这样的环境中，网络管理员易于清楚地了解网络的拓扑结构，设置正确的路由信息以及网络拓扑变化时进行路由信息的调整不需太多的精力。此外，静态路由具有较高的安全保密性。

2. 动态路由

动态(Dynamic)路由表是路由器根据网络系统的运行情况自动调整的路由表。通常是通过路由器与相邻路由器交换网络信息来动态地进行路由表维护的。路由器间相互交换网络信息的规范由路由协议定义。根据发现和计算到达目的网络的新路径的方法，可以把动态路由算法分为3种：

(1) 距离向量 (Distance Vector) 算法

距离向量算法的原理非常简单:它把每经过一个路由器称为一跳。一条路由上的跳数称为“距离”,然后动态地选择最短距离作为路径。图 3.16 所示为一个简单的距离向量路由表示的例子。图中给出了 R2、R3 和 R4 三个路由器的距离向量路由表。



网络	下一跳	距离
N10	R1	1
N20	直达	0
N30	直达	0
N40	R3	1
N50	R3	2
N60	R3	3

网络	下一跳	距离
N10	R2	2
N20	R2	1
N30	直达	0
N40	直达	0
N50	R4	1
N60	R4	2

网络	下一跳	距离
N10	R3	3
N20	R3	2
N30	R3	1
N40	直达	0
N50	直达	0
N60	R5	1

图 3.16　距离向量示例

距离向量算法的主要优点是易于实现和调试,主要用于小型网络中。

(2) 链路状态 (Link State) 算法

链路状态是路由器上的接口 (网络地址和网络类型等) 描述及其与相邻路由器关系的总称。这些链路状态的集合形成了一个链路状态数据库 (Link-State Database) 。

用链路状态算法确定网络拓扑的过程非常直截了当。

在 Internet 中广泛使用的最短路径优先 SPF (Shortest Path First) 就是一种分布式链路状态协议 (Link State Protocol) 。它提供了网络的树状表示。树根是运行 SPF 的设备,用来计算到达每个目的网络的最短路径列表。图 3.17 所示为在路由器 R1 上执行最短路径算法的例子。

(a) 每个路由器标识与它直接相连的网络上的所有路由设备。

(b) SPF 将实际网络、路由器和链路集抽象成有向图,两个路由器间的一系列链路由一对有向弧表示,各指向一方,且它们的权值可能不同。然后根据有向弧上的权值计算最短路径。这些权值用专用名词“度” (Metric) 称呼,其值可以是 1～65 535 中的任何一个无量纲的数。“度”的具体内容可以是费用、距离、时延、带宽等,它们都由网络管理人员决定。在图 3.17 (b) 中,R5-3 表示到 R5 的路径

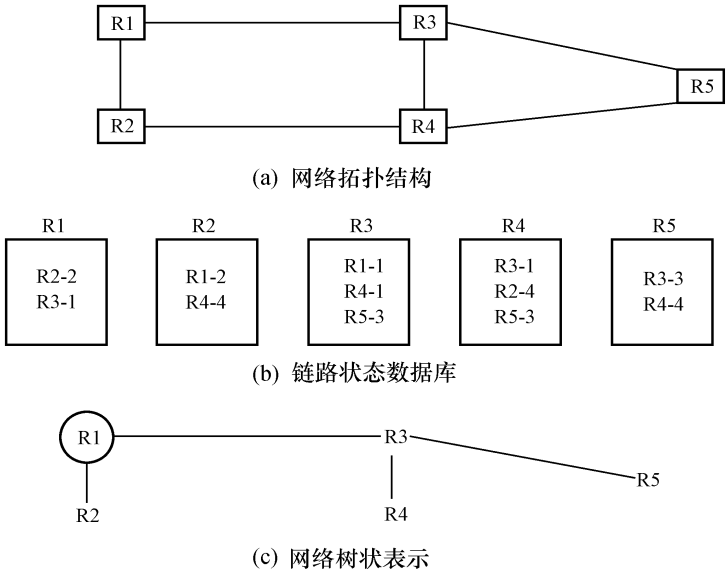


图3.17　在路由器 R1 上执行最短路径算法

上的度为3

SPF 处理服务类型路由的办法是保留多张有向图，一张标注以时延为度，一张标注以吞吐量为度，一张标注以可靠性为度（虽然3张有向图的计算工作量是一张图的3倍，但却提供了分别按时延、吞吐量和可靠性优先选择路由的可能）。

(c) 每个路由器通过与网络中的其他路由器交换 LSA（Link State Advertisement，链路状态公告）通知所有与之直接相连的网络链路以及每个链路关联的度。

每个路由器使用这些 LSA 建立一个详细记录当前网络拓扑结构的链路状态数据库（因为每个路由器处理相同的 LSA 集合，所以每个路由器建立的链路状态数据库是相同的）。只要网络拓扑发生任何变化，该链路状态数据库就能很快进行更新，并且通过各路由器之间频繁地交换信息，维持链路状态数据库在全网的一致性，即维持各路由器链路状态数据库的同步。

(d) SPF 提供了网络的树状表示。树根是运行 SPF 的设备，用来计算到达每个目的网络的理想路径。虽然每个路由器的链路状态数据库相同，但是由于每个设备占据了网络中的一个不同位置，SPF 将为每个路由器产生不同的树。

(3) 混合（Hybrid）路由算法

混合算法试图综合距离向量算法和链路状态算法的优点：

使用“度”把优先权分配给一个路径（吸取距离向量算法的优点）；

通过事件驱动更新路由而不是定期更新路由（吸取链路状态算法的优

点)。

3.2.2 路由器的基本结构

路由器是一种设备,其基本结构都包括路由器接口(网卡)、路由器软件、接收/发送缓冲区等。下面介绍各部分的功能。

1. 网络接口

路由器的网络接口用作将路由器连接到网络。对路由器来说,接口的主要功能是输入和输出。

从大的方面来说,网络接口可以分为局域网接口和广域网接口两种。局域网接口主要包括 10 Mb/s 以太网接口、100 Mb/s 以太网接口、1 000 Mb/s 以太网接口、10/100 Mb/s 自适应以太网接口、快速以太网接口、令牌环、令牌总线等网络接口。广域网接口主要包括:通用串行口(可转换成 X.21 DTE/DCE、V.35 DTE/DCE、RS232 DTE/DCE、RS449 DTE/DCE、EIA530 DTE)、E1/T1,E3/T3,DS3,ATM 接口(2M、25M、155M、633M 等)、POS 接口(155M、622M)等。

网络接口通常由网卡提供,一块网卡一般支持 4、8 或 16 个端口。

2. 路由器软件

路由器软件一般实现路由协议功能、查表转发功能和管理维护功能等。由于 Internet 规模庞大,路由表一般也非常庞大,可能会有几十万条。路由协议是路由器软件的核心,主要用作建立和维护路由表。

3. 路由交换单元

路由器还可以与交换开关结合在一起,组成一个路由交换单元。交换开关可以使用多种不同的技术来实现。最简单的开关是使用一条总线来连接所有输入和输出端口,总线开关的缺点是其交换容量受限于总线的容量以及为共享总线仲裁所带来的额外开销。交换开关通过开关提供多条数据通路,具有 $N\times N$ 个交叉点的交换开关可以被认为具有 $2N$ 条总线。如果一个交叉点是闭合的,则输入总线上的数据在输出总线上可用,否则不可用。交叉点的闭合与打开由调度器来控制,因此,调度器限制了交换开关的速度。在共享存储器的路由器中,进来的包被存储在共享存储器中,所交换的仅是包的指针,这提高了交换容量,但是开关的速度受限于存储器的存取速度。尽管存储器容量每 18 个月能够翻一番,但存储器的存取时间每年仅降低 5%,这是共享存储器交换开关的一个固有限制。

4. 路由器队列

路由器是基于 IP 分组交换的设备,在每个接口上进行带宽统计复用。所以

路由器必须在每个端口上维护一个或多个队列,以处理多个数据包发向一个端口而端口接收能力有限的矛盾。队列管理的能力直接影响路由器的性能以及拥塞管理能力。下面是三种常用的队列管理算法：

(1) 基于时标的IP分组调度算法

基于时标的IP分组调度算法为每个IP分组维持两个时标:起始时标(Start Time Stamp)和完成时标(Finish Time Stamp)。路由器根据这两个时标来决定下一个转发数据包,常见的有WFQ、WF2Q等。

(2) 基于轮转的调度算法

轮转调度算法又称时间片法,系统赋予每个队列一个时间片。转发操作在各队列间轮转地进行,常见的有WRR、DRR等。

(3) 基于优先级的调度算法

基于优先级的调度算法是根据预先规定的优先级对不同队列的数据包进行调度转发。

3.2.3 路由器技术的演进

路由器的具体结构随路由技术的发展而发展。表3.1所示是华为公司关于路由器体系结构演进的总结。

表3.1　路由器体系结构的演进

路由器演进	技术特点	应用业务	典型产品
第一代	集中转发	SOHO	Cisco 2500系列
	总线交换	数据类业务	华为Quidway R2500系列
第二代	集中+分布转发	中小企业网	Cisco 4500系列
	接口模块化,总线交换	数据、少量语音类业务	华为Quidway R3600系列
第三代	分布转发	大中企业网/城域网	Cisco 7500系列
	总线交换	数据、少量语音类业务	华为Quidway NetEngine 16/08/05系列
第四代	ASIC分布转发	行业骨干网/大型城域网	Cisco GSR12400/12000系列
	网络交换	数据类业务	JUNIPPER M40/160系列
第五代	网络处理分布转发	行业骨干网/大型城域网	华为Quidway NetEngine
	网络交换	高质量业务	80/40/20系列

1. 第一代路由器技术

第一代路由器主要由一个CPU和多个固定的网络接口组成,如图3.18所

示,CPU与网络接口之间通过内部总线相连接。网络接口收到报文后,通过内部总线传送给CPU。CPU负责所有事务处理,包括路由收集、转发处理、设备管理等。CPU处理后,从另一个网络接口传送出去。

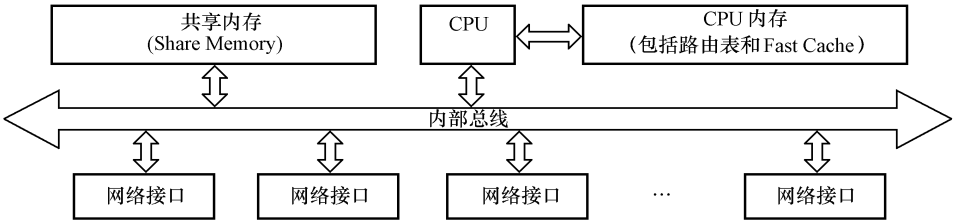


图3.18 第一代路由器结构

第一代路由器的接口是固定的。这样,就难于满足网络链路经常变化的需求,给网络设备投资保护和维护管理带来不便。

2. 第二代路由器技术

第二代路由器把网络接口做成可插拔的活动模块,使用户可以根据需要增减网络接口模块,为路由器升级扩容提供了便利。但是,它还是使用一个CPU进行集中转发。

3. 第三代路由器技术

如图3.19所示,第三代路由器是在各网络接口上增加CPU,使路由器呈现分布式结构。这样,每个接口业务处理模块都用自己的CPU进行转发和业务处理,它只负责少量网络接口。

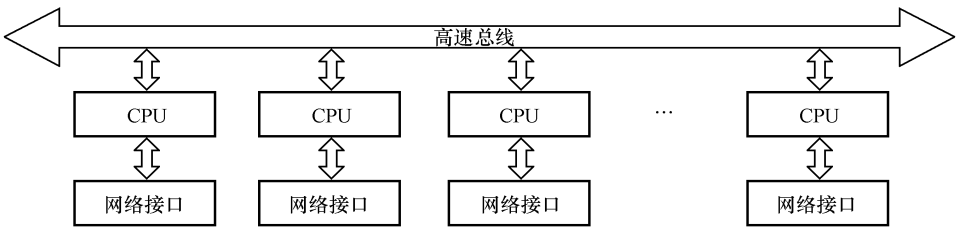


图3.19 网络接口上配置路由交换单元的路由器结构

4. 第四代路由器技术

前面3种路由器都采用基于CPU的软件转发模式。第四代路由器则是在第三代实现分布式转发的基础上,将基于CPU的软件转发模式转换为基于ASIC(Application-Specific Intergrated Circuit,专用集成电路)技术的硬件转发。在此基

础上,还采用了交换网络结构,即各接口板只需要一条高速链路与交换网芯片连接。

5. 第五代路由器技术

第五代路由器基于第四代路由器的硬件转发和交换网式体系结构,在关键的IP转发和业务流程处理上采用如下两大技术替代了原来的ASIC技术:

采用可编程序的、专为IP网络设计的网络处理器(NP),实现IP报文处理和转发;

采用大容量的交换网络结构,可通过升级软件增加新的处理功能,满足不同的应用需求。

图3.20所示为华为公司的一个第五代路由器的结构示意图。

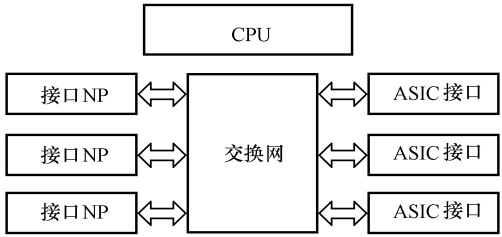


图3.20　第五代路由器的结构

习　　题

3.1　试从实时性、可靠性、透明性以及通信成本等几个方面对电路交换、报文交换和分组交换进行比较。

3.2　试从效率、可靠性以及网络与主机间工作的复杂程度等三个方面,对虚电路和数据报两种通信方式进行比较。

3.3　考虑由 n 个节点构成的分组交换网络,组成下面三种拓扑结构:

(a) 中央交换的星型拓扑结构;

(b) 不带交换功能的全连接网络;

(c) 双向环型结构。

试分别求它们的最好、最差和平均传输跳数(路径个数)。

3.4　在一个网络中,从节点A到节点B共有 K 条链路,每段链路的电波传输时延均为 t 秒,数据传输率为 B(b/s);它可以以电路交换和分组交换两种方式工作:

(a) 按照电路交换方式工作时,端-端电路建立时间为 S 秒;

(b) 按照分组交换方式工作时,分组长度为 P 比特。

若可以忽略每个节点上的分组排队时间(网络负载极轻),那么在什么条件下对于从节点

A 到节点 B 传输数据的总时延来说，分组交换方式比电路交换方式短？

3.5 对于交换网络可以定义下列参数：

N——两给定节点间的转接链路数；

L——报文长度(bit)；

B——数据传输速率(b/s)；

P——分组长度(bit)；

H——分组开销(报头，bit)；

S——(电路交换或虚电路)呼叫建立时间(s)；

D——每个链路上的传输延迟(s)。

对于某一交换网络，若有 $L=3\,200$，$B=9\,600$，$P=1\,024$，$H=16$，$D=0.000\,5$；从节点 A 到节点 B，$N=6$，并且网络负载极轻，各节点上的处理时间可以不计。试分别计算电路交换、报文交换、虚电路交换和数据报分组交换时，从节点 A 到节点 B 的端到端延迟时间。

3.6 已知某一交换机有 1 024 路输入和 1 024 路输出，每个通路均以 OC12(622 Mb/s)的速率传输数据，并能提供 594 Mb/s 的用户速率。问该交换机所需的合成带宽是多少？若所处理的分组是 ATM 信元(长度为 53 字节)，问每秒钟可以处理多少信元？

3.7 查阅资料，阐述当前交换技术发展趋势。

3.8 查阅资料，对尽可能多的几种路由算法进行比较。

第 2 篇

计算机网络体系结构

这一篇主要从立面上观察计算机网络的层次结构和工作原理。

计算机网络是一种非常复杂的系统。实践证明,按照层次结构建立的系统具有较高的稳定性和可管理性。为此,计算机网络也按照层次结构进行研究和建造。

另一方面,复杂系统的设计不是惟一的。出于不同技术背景、不同需求等因素的考虑,人们可以构建不同结构的计算机网络,并可以采用不同的协议。每一种计算机网络的层次结构连同其所使用的协议体系一并称为这种网络的体系结构(Architecture),或者说是这种计算机网络的模型。不同的计算机网络体系结构,具有各自特有的在建立和使用通信硬件和软件时所需的一套统一规则和规范,按照这些规则和规范可以建立整个网络系统的逻辑组成并进行功能的分配。但是要注意,某个网络模型并不是具体的网络。

这一篇介绍下面几个典型的计算机网络体系结构和相应的网络:

ISO/OSI 参考模型；

IEEE 802 模型与局域网；

Internet 与 TCP/IP 体系结构；

ATM 网络。

第 4 章　ISO/OSI 参考模型

4.1　关于网层次结构的基本概念

4.1.1　分层原则

在层次结构的计算机网络系统中,每一层都具有相对独立的通信功能。各层间相对独立的通信就好像国家之间的往来,分为元首级的通信、部长级的通信、工作人员级的通信,他们各以不同方式进行不同内容的通信;同一级的通信都是对等的,并且都是在各自的下一级的通信支持下进行的。在计算机网络中的通信也是如此,通信的双方都可以分为不同的层次,每一层之间进行对等的通信。

基于不同的现实和考虑原则,计算机网络可以有不同的分层方法。但是,一般说来有一些原则基本上是共同的,如:

(1) 每层的功能应当明确、各层相对独立,使得当某层的方法变更时,只要保持接口关系不变,不会影响其他层的结构和方法;

(2) 各层功能的确定应当有助于网络协议国际标准的制定;

(3) 层间接口要清晰,尽量减少跨越接口的信息量;

(4) 层数适当,不可太少,只要增加一个不同等级的抽象,就应当为网络增加相应的层次;也不可太多,否则使网络结构过于复杂。

4.1.2　服务与接口

用于同层间通信的协议称为同层协议,并且每一层间通信的协议都是在下一层间通信的协议提供的服务基础上进行工作的。因此,协议和服务是两个不同的概念:协议是"水平的",是控制对等实体之间通信的规则;服务是"垂直的",是下层通过接口向上层提供的支持。

1. (N)服务与(N)协议

与前述国家间的交往相似,在对等层间形成虚拟通信。在通信过程中,第(N)层向第(N+1)层提供服务,而第(N)层的服务除了要使用(N)层的功能

——(N)功能外,还需要第(N-1)层以下服务的支持。这里,将第(N)层对第(N+1)层提供服务称为(N)服务,显然,(N)服务是(N)层及其以下各层服务的综合。对于第(N+1)层来说,只关心(N)服务,而不必关心(N-1)服务。

对等层之间的虚拟通信是通过执行(N)功能实现的。这时要使用一组规则(语义)和格式(语法)。这组规则(语义)和格式(语法)称为(N)协议。

2. 接口

由前面的讨论可知,上、下层间通过接口交换信息。图4.1表示了层间接口中的活动。

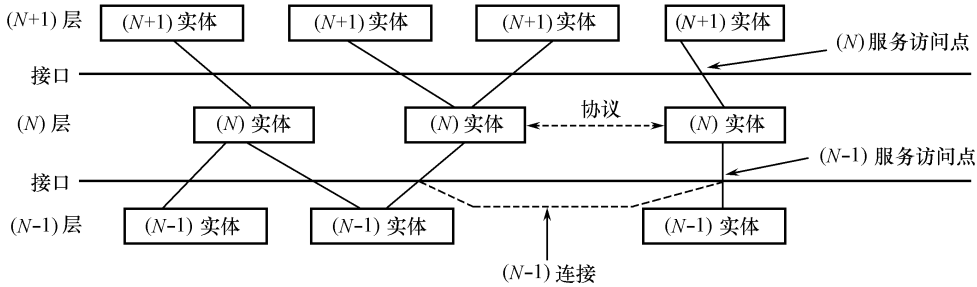


图4.1　层间接口中的活动

可以看出,两个相邻层间的信息交换,实际上是由两层间的实体通过服务访问点相互作用的。这里,(N)实体是N层中一个活动的元素,可以是软实体(如一个进程),也可以是硬实体(如智能输入输出芯片)。

接口以一个或多个服务访问点(SAP,Service Access Point)的形式存在,并通过访问服务点来实现其功能。(N)SAP位于N层和N+1层的逻辑交界面上,是(N)实体向(N+1)实体提供服务的地方,或者说是(N+1)实体请求N层服务——(N)服务的地方。每个SAP有一个惟一的标识标明它的地址,如在物理层的SAP地址可以是异步通信口的地址。

OSI模型是面向连接的。两个(N)实体利用一种协议并通过(N-1)连接实现通信。但是,一个(N)SAP一次只能连接一个(N)实体和一个(N+1)实体,而一个(N)实体可以同时通过多个(N)服务连接一个或多个(N+1)实体。

3. 服务原语

(1) 原语的概念

服务在形式上由一组原语(Primitive)描述。原语规定了应完成的功能,并用于传送数据以及控制信息,它们供用户和其他实体访问某服务时调用。它们通知服务提供者采取某些行动或报告某个对等实体的活动。

(N) 服务原语可以由 (N) 实体向 (N+1) 实体发送，也可以由 (N+1) 实体向 (N) 实体发送。ISO/OSI 定义了表 4.1 所示的 4 种类型的服务原语。

表 4-1　ISO/OSI 定义的 4 种类型的服务原语

名称	格式	发送方向	含义
请求原语	(N)_PRIMITIVE NAME.request(a_0,…,a_{n-1})	(N+1) → (N)	请求 (N) 实体的服务
指示原语	(N)_PRIMITIVE NAME.indication(a_0,…,a_{n-1})	(N) → (N+1)	通知服务已开始
响应原语	N_PRIMITIVE NAME.response(a_0,…,a_{n-1})	(N+1) → (N)	已响应最近的一次指示
证实原语	N_PRIMITIVE NAME.confirm(a_0,…,a_{n-1})	(N) → (N+1)	请求的服务已完成

大部分原语都带有参数。例如一个运输连接请求原语可以写为

T_CONNECT.request(被叫地址，主叫地址，…)

这里，T_CONNECT 是原语名称，request 是原语类型，中间用圆点分隔，“被叫地址”和“主叫地址”是原语参数。

(2) 有证实服务和无证实服务

服务有有证实 (Confirmed) 和无证实 (Unconfirmed) 之分。有证实服务包括请求、指示、响应和证实 4 个原语，无证实服务只包括请求、指示 2 个原语。图 4.2 以张给李打电话通知开会为例，说明有证实服务和无证实服务的交互过程。

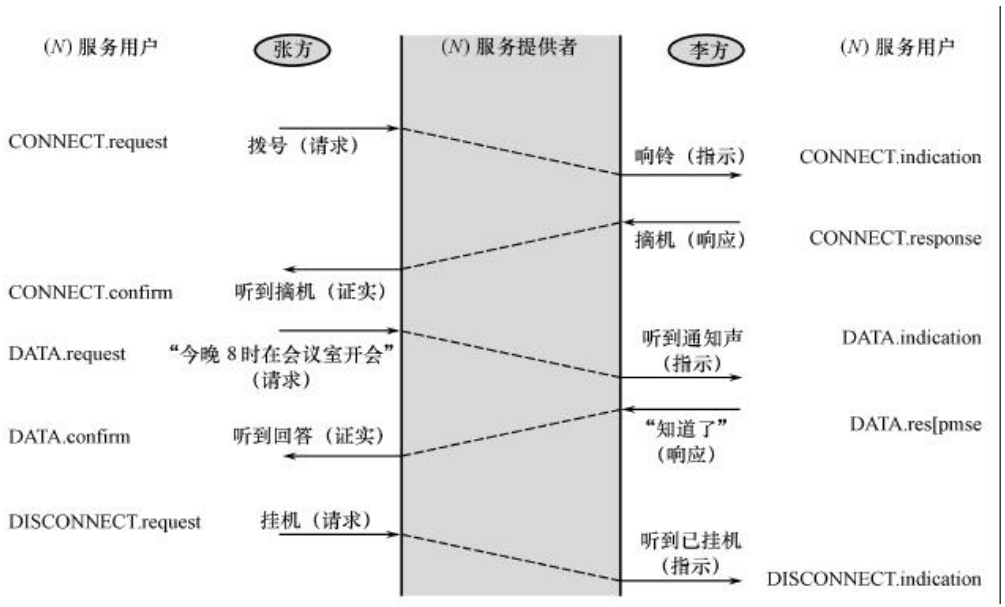


图 4.2　有证实服务和无证实服务的交互过程

这里,CONNECT 和 DATA 过程是有证实服务,DISCONNECT 是无证实服务。

(3) 原语的参数

大多数原语都有参数。例如:连接请求的参数可能要指明是与哪一台计算机连接,需要的服务类别,以及在该连接上使用的最大报文长度;连接指示原语的参数可能包括呼叫者标识,需要的服务类型,以及建议的最大报文长度等。在典型情形下,当接收到一个服务原语时,该层的协议实体读原语中的参数,并把它们与附加的协议控制信息相结合形成该层的 PDU,所产生的 PDU 再放到带有附加参数的服务原语的用户段中,以便传给相邻的下层。

4. 协议与服务的关系

服务与协议关系非常密切,但却是完全不同的概念,然而又却容易被混淆。

服务定义了相邻两层之间的接口关系,上层是服务的用户,下层是服务的提供者,并用一组原语描述服务内容——下层能够为它的上层给予的支持。

协议是同层实体之间数据交换的规则。实体利用协议来实现它们的服务的定义。

在某一层上,只要不改变提供给上层的服务,实体可以使用不同的协议。这样,就把服务同协议完全分离开来。OSI 的一大贡献也就是实现了服务同协议的分离。

4.1.3 各层设计中的流量控制

在第 2 章介绍了用滑动窗口协议实现链路上数据流量控制的方法。实际上,不仅在数据链路层,在网络层和运输层都要进行流量控制,如路由器也有流量控制的功能。这一小节对流量控制做一些较系统的分析,以便使读者在不同层次上了解流量控制的相同点和不同点。

1. 拥塞与死锁

在公路上,车流量超过一定限度,大家的速度就不得不减慢;再超过一定的限度,就会出现谁也走不动的现象。计算机网络也是如此,无论是计算机装置还是通信设备,对数据的处理能力总是有限的。当网上传输的数据量增加到一定程度时,网络的吞吐量下降,这种现象称为“拥塞”(Congestion)。传输数据量急剧增加时,丢弃的数据帧不断增加,从而引起更多的重发;重发数据所占用的缓冲区得不到释放,又引起更多的数据帧丢失;这种连锁反应将很快波及全网,使通信无法进行,网络处于“死锁”(Deadlock)状态,陷于瘫痪。图 4.3 表明了网络系统吞吐量与输入负载的关系。

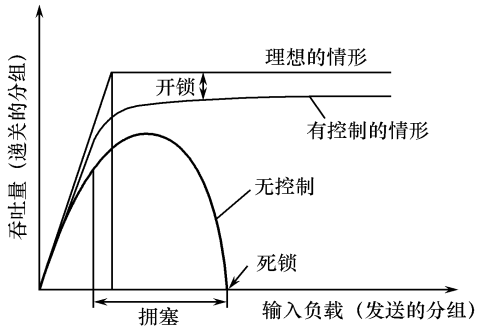


图4.3　网络系统吞吐量与输入负载的关系

2. 流量控制的基本原理

防止拥塞和死锁的办法是制定网上交通规则，进行流量控制。从原理上说，拥塞和死锁是网上可用资源不能满足各用户资源需求造成的，即

$$\Sigma\text{用户资源需求} \geq \text{可用资源}$$

由此，可以得到进行流量控制的基本策略：

增加用户可用资源；

限制用户资源需求。

3. 流量控制的级别

计算机网络是一个复杂的系统。如图4.4所示，它的流量控制可以从两种角度来研究。从网络的组成环节上看，涉及相邻节点间（段级）的流量控制、节点间（入口出口级）的流量控制、主机与节点间（进网级）的流量控制和主机间（传输级）的流量控制；从网络的体系结构上看，涉及到物理层、链路层、网络层和传输层。这二者之间具有以下几点联系：

物理层提供了系统最基本的可用资源；

段级流量控制在链路层实现；

入口出口级的流量控制在网络层实现；

进网级的流量控制在网络层和链路层实现；

传输级的流量控制在传输层实现。

其中，物理层流控的主要目的是提供网络的基本可用资源，基本策略有：

合理选择路径，均衡网上负荷；

增加通道带宽，增加信息速率；

增加缓冲区，等等。

其他层的流控策略主要是限制用户资源需求。

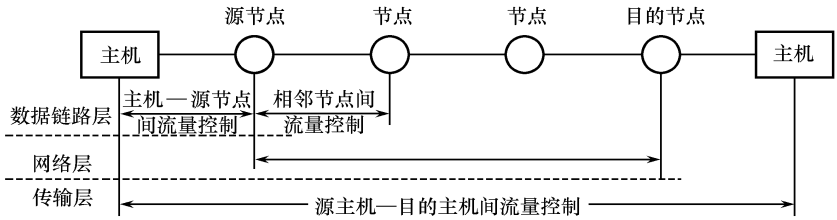


图 4.4　流量控制的级别

4. 用户资源需求限制策略

下面介绍几种通过限制用户资源需求来控制流量的策略。

(1) 等待传输法

当接收节点的缓冲区将被占满、死锁来临之前，向发送节点发送暂停发送信息；当危险解除后，再通知发送节点恢复发送。

(2) 预约缓冲区法

源主机在开始传输数据之前，首先了解目的主机的可用缓冲区大小，预约一定大小的缓冲区，根据分配的缓冲区大小控制数据的发送量。当缓冲区用完后，等待对方重新分配缓冲区。

(3) 数据单元丢弃法

当目的主机有缓冲区时就接收数据单元，无缓冲区时就将数据单元丢弃，被丢弃的数据单元由于源主机得不到确认而重发。

(4) 许可证法

这是一种全网流量控制策略。其基本方法为：网络初启时，给每个节点各分配一定数量的许可证；一个节点上的主机要发送数据，必须从该节点获得一个许可证，将许可证与数据一起发送，这时发送节点将减少一个许可证，没有许可证可用的节点不能发送数据；目标节点收到一个数据的同时，也收到一个许可证，供自己的主机发送数据使用。为了防止某一个节点积累的数据太多，超过限量时，将把多余的许可证单独发送或让数据单元捎带到别的节点上。

4.2　ISO/OSI 参考模型结构

网络发展的早期，各个计算机网络厂家都分别有自己的网络体系结构，如 IBM 的 SNA 网络体系，DEC 的 DNA 网络体系和 UNIVAC 的 DCA 分布式网络体系等。不同的网络体系结构分别有自己的层次结构和协议，给网络标准化和互连

造成很大困难。为此,ISO (International Standards Organization,国际标准化组织) 于 1978 年 2 月开始研究解决这个问题的方法草案,于 1982 年 4 月形成一个开放系统互连参考模型 (Open System Interconnection/Reference Model,OSI/RM) 的国际标准草案。OSI/RM 是一个七层模型,如图 4.5 所示。

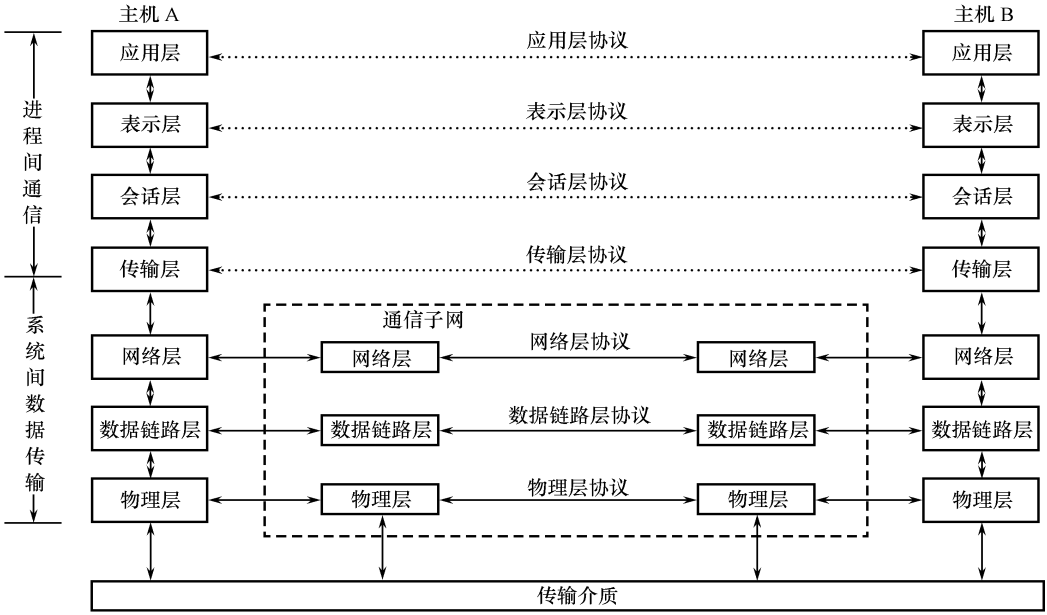


图 4.5　开放系统互连参考模型

在 OSI 参考模型中,每一层的功能都在下一层的支持下实现,并支持上一层。OSI/RM 中的低三层通常归入通信子网的功能范畴,通常靠硬件做成通信卡或由通信处理机实现;网络中的高四层归入资源子网的功能范畴,通常用软件方法实现,形成网络操作系统 (最高层常由应用软件实现)。如果按相互通信的对象进行划分,可将 OSI 参考模型的通信分为计算机系统间的通信和计算机系统中进程间的通信。由会话层、表示层、应用层构成的上三层为进程间的通信,主要解决两台通信的主机间信息传输问题;由物理层、数据链路层、网络层构成的下三层为系统间的通信,主要解决通信子网中的数据传输问题;传输层处于两部分的中间,可以看作系统通信与进程通信间的接口层。

应当注意,OSI 模型仅仅是“参考模型”,而不是网络体系结构的全部;它仅仅描述了每一层的功能,而没有描述每一层使用哪些协议和承担哪些服务。现在,ISO 已经为各层制定了标准,不过这些标准并不是 OSI 模型的一部分,而是独立存在的。这一节在介绍各层功能的同时,虽然也顺便介绍一些有关协议,但读者一定不要认为它们也是 OSI 模型的一部分。

4.2.1 OSI 的低层

OSI 的低层包括物理层、数据链路层和网络层。这一部分的功能主要是实现通信功能,也称为通信子网部分。

1. 物理层

ISO 给出的物理层(Physical Layer)定义为:"物理层为启动、维护和释放数据链路实体之间二进制位传输而进行的物理连接提供机械的、电气的、功能的和规程的特性。这种物理连接可以通过中间系统在物理层内进行二进制位传输。它允许进行全双工或半双工的二进制位传输,对物理数据单元(即二进制位)可以以同步方式或异步方式进行传输。"

从这段定义可以看出,物理层的功能是解决"物理连接"的标准问题,它包含了如下几个方面的含义。

(1) 它建立在传输介质之上,并不考虑传输介质的具体敷设问题,而只关心介质两端的连接,或者说它只关心链路两端点的物理特性。这些物理特性包括:

机械特性——插头尺寸、各脚位置;

电气特性——表示"1"和"0"的电压各为多少,一个比特的宽度;

功能特性——某一根线上出现某一电平所代表的意义;

规程特性——不同功能的各种可能事件出现的顺序。

(2) 建立物理层的目的是提供一条物理链路上"1"、"0"码传输的物理条件——为数据链路层提供这个服务。

(3) 同时还要涉及下列问题:

数据传输的单位——串行(1bit),并行(nbit);

数据传输方式——全双工或半双工以及同步方式(这些概念见第 2 章);

服务质量参数——出错率、服务可用性、传输速率和延迟;

物理层本身的管理问题,等等。

常用的物理层协议有 EIA-232-D、V.35 和 X.21 等。例如,在使用电缆进行设备连接时,都要使用连接头,即插头和插座,有名的 EIA-232-D(前身是 RS-232C)就是关于接口的协议之一。它按照 ISO 2110 标准定义的 DB-25 接插件,规定了如图 4.6 所示的外形尺寸——机械特性,还规定 DB-25 的阳件(带插针的件)安装在 DTE(Data Terminal Equipment,数据终端设备)方,阴件(带插孔的件)安装在 DCE(Data Communication Equipment,数据通信设备)方。

为了保证正确地传输二进制数据并使设备控制能有效地进行,EIA-232-D 提供了数据信号和控制信号的电压标准——电气特性,具体分配如图 4.7 所示。

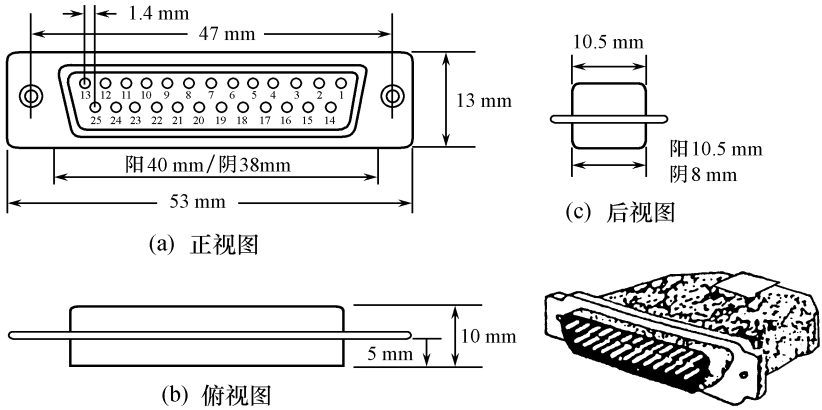
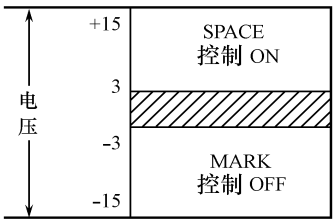


图 4.6　DB－25 外形尺寸



说明：

电压状态(V)	信号条件	数据值	控制功能
+3~+15	SPACE	0	ON
–3~–15	MARK	1	OFF

图 4.7　EIA－232－D 的电气特性

EIA－232－D 标准还描述了引脚的功能分配——功能特性，如表 4.2 所示。

表 4.2　EIA－232－D 接口引脚功能

引脚号		助记符	信号功能	信号方向	类型	简要描述
25 脚	9 脚					
1	5		屏蔽地			接至机壳或机架，消除静电干扰；可不接
2	3	TXD	发送数据	DTE→DCE	数据	DTE 发送数据，不发送数据时为负电压
3	2	RXD	接收数据	DCE→DTE	数据	DTE 接收数据，无接收数据时为负电压
4	7	RTS	请求发送	DTE→DCE	控制	请求发送/准备好接收
5	8	CTS	清除发送	DCE→DTE	控制	DCE 同意接收，对 RTS 的应答
6	6	DSR	数据设备就绪	DCE→DTE	控制	DCE (数据设备) 准备好
7		GND	信号地		公共	必不可少的公共接地线
8	1	CD	载波测试	DCE→DTE	控制	接收方导线信号检测器

续表

引脚号		助记符	信号功能	信号方向	类型	简要描述
25 脚	9 脚					
9			保留			
10						
11						
12			次信道载波检测		控制	
13			次信道清除发送		控制	
14			次信道发送数据		数据	
15		TXC	发送定时	DTE→DCE	计时	
16			次信道接收数据		数据	
17		RXC	接收定时	DCE→DTE	计时	
18			保留		控制	
19			次信道请求发送		控制	
20	4	DTR	数据终端准备好	DTE→DCE	控制	DTE 准备好
21			信号质量检测		控制	
22	9	RI	振铃指示	DCE→DTE	控制	
23		DRI	信号速率检测		控制	
24		TXC	发送定时	DTE→DCE	计时	
25			保留		控制	

V.35 和 X.21 则是由国际电报电话咨询委员会 ITU－T 提出的两种物理层协议，前者主要用于模拟传输，后者用于数字传输。

物理层的功能实现后，表示两点之间的链路已连接好，就可以传输 0、1 码了。

2. 数据链路层

在物理层实现了透明的 0、1 码传输的基础上，数据链路层（data link layer）将加强这些原始比特的传输，使之成为一条无错的数据传输链路。

为了达到“无错的数据传输链路”的目标，对链路层的设计通常要解决如下 3 方面的问题：

(1) 成帧。帧（Frame）就是一段或一组位串。从发送方来讲，网络层的数据是以分组的形式传输的。这些分组传输到数据链路层后，首先要解决这些分组的同步传输问题，于是要加上一个头尾标志，经过这样包装的分组就称为帧。或者说，帧解决了同步传输问题。也可以说，同步传输问题是在数据链路层解决的。帧送到物理层后，是以位串形式传输的。对于接收端来说，数据链路层从物理层接收到的是一串 0、1 码，到了数据链路层后，要首先根据帧头和帧尾判定出帧中的数据部分（要传向网络层的分组）。图 4.8 所示为一般帧的格式。

F	A	C	I	FCS	F
帧起始标志	目的站	控制段	数据段	帧校验序列	帧结束标志
8位	8位	8位	任意长	16位	8位

图4.8　一般帧格式

(2) 确定帧的顺序,以便数据拼接。

(3) 差错校验——根据校验序列对帧进行校验。

(4) 流量控制和差错控制(如停等协议、滑动窗口协议)。

(5) 数据链路管理(链路的建立、维护和拆除)。

目前,典型的数据链路层协议有:面向字符的协议(如ISO-1745基本型控制规程、IBM的二进制同步通信协议BSC等)和面向位的协议(如IBM研制的同步数据链路控制协议SDLC,以及SDLC的修改版本——ISO的高级数据链路控制协议HDLC、ITU-T的链路访问协议LAP等)。

3. 网络层

数据链路层从一条传输链路的角度来解决传输中的可靠性问题。网络层则是从整个网络的角度来处理数据传输中的有关问题,处理问题过程中需要考虑通信双方的终端节点及中间节点间的关系,具体内容包括:

(1) 路由选择。如何在组成网络的各子网之间找到到达传送目的地的最佳路径。

(2) 流量和拥塞控制。防止在子网中间出现过多的分组,造成通路阻塞,形成瓶颈。

(3) 差错及故障的恢复。

网络层中有代表性的标准协议有ITU-T的X.25、Internet中使用的IP协议等。

4.2.2　OSI的高层

OSI的高层包括会话层、表示层和应用层,属于资源的分配和处理的部分,通常称为资源子网部分。

1. 会话层

会话层是建立在用户与网络间的接口,好像两台主机间的联络官,主要处理通信双方交互的建立、组织,并协调、控制会话过程的进行(会话服务)。

2. 表示层

表示层处理两个应用实体间数据交换的语法问题,解决数据交换中存在的数据格式不一致和数据表示方法不同等问题。例如IBM系统采用EBCD编码,

而其他一些系统用户采用 ASCII 编码，表示层要为它们的转换服务，转换后传输给会话层。此外，表示层还进行加密、压缩或终端仿真处理。图 4.9 形象地描述了表示层的功能。

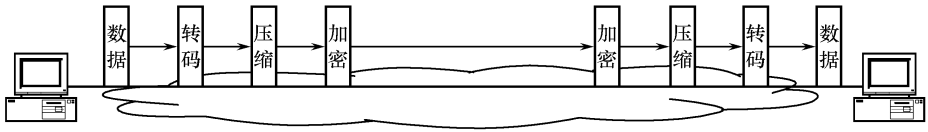


图 4.9　表示层的功能

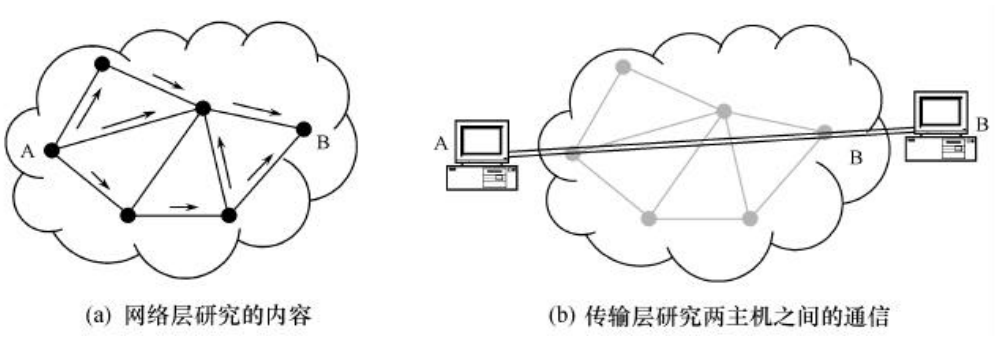
3. 应用层

应用层主要进行应用管理和系统管理，直接为用户服务，在信息网络用户之间形成一个交换信息的界面 k k 用户应用程序。简单地说，就是接收用户数据。信息网络系统接受了用户的数据后，立即把它们传输给表示层。

4.2.3　传输层

1. 传输层的功能

传输层是站在比网络层更高的角度来研究问题。如图 4.10 所示，网络层考虑的是网络的细节，而传输层屏蔽了这些细节，只从主机间逻辑连接（或进程间通信）的角度来处理数据的传输问题。



(a) 网络层研究的内容　　(b) 传输层研究两主机之间的通信

图 4.10　网络层与传输层的研究角度

传输层与数据链路层不同，数据链路层处理的是相同网络中两个节点间的数据通信；而传输层处理的是主机间的数据通信，是源节点到目的节点之间的传输，它考虑的具体内容有：

(1) 两个主机间数据传输的方式。虚电路方式，与连接有关的数据传输服务，像打电话；数据报方式，与连接无关的数据传输服务，像邮局传输信件。

(2) 把数据分割成数据包,分组,为会话层提供最佳的数据传输服务,以便充分地利用网络。

传输层是计算机网络中非常关键的一层。可以说,当在传输层中解决了两台主机间的通信问题后,在其上就可以进一步解决通信双方的交互问题、数据的表示问题和具体应用了。

2. 传输模型

传输层是整个协议层次结构的中心,它下靠传输服务提供者(Transport Service Provider)——通信子网的支持,上为传输服务用户(Transport Service User)——上面的 3 层提供从源端机到目的端机的可靠的、价格合理的数据传输服务。图 4.11 是传输层的一个抽象模型。其中,TSAP(Transport Service Access Pointer)为传输层服务访问点,NSAP(Network Service Access Pointer)为网络层服务访问点,传输实体(Transport Entity)为完成传输层任务的硬件和软件,TPDU(Transport Protocol Data Unit)是传输协议数据单元。

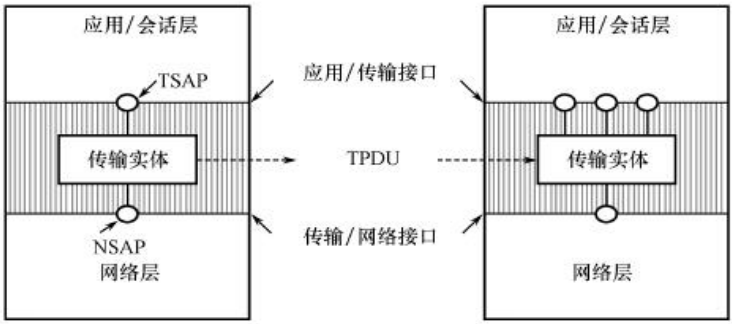


图 4.11　传输层抽象模型

3. 传输层的基本开发思想

传输层建立在网络层之上。在通信子网范畴中,连接通常叫做虚电路(Virtual Circuit),无连接的独立报文通常叫做数据报。网络层建立在数据链路层的基础上,向传输层提供两种服务:面向连接的服务和无连接的服务。通常把面向连接的服务也叫做虚电路服务,把无连接的服务叫做数据报服务。由于网络层是通信子网的边沿,采用哪一种服务方式,实际上就是将报文的差错和顺序控制等复杂的功能放在通信子网中,还是放在主机(传输层)中。这代表了两种建网的思想。Internet 是采用无连接服务的代表者,这类网络开发者的基本思想是:由于计算机处理设备已经非常便宜,把复杂性放在主机中是非常合算的;而且,对于数据准确性要求很高的用户,对子网的可靠性总是留有戒心,为了保护

自己,总要在传输层设置差错和报文顺序的控制功能,再在子网中设置这些功能就没有多大意义了。ATM 则是采用面向连接服务的代表者,这类网络开发者的基本思想是:通信子网是一项重要的公共基础设施,应当能为多种需求服务,其中包括对子网质量要求高的用户(如银行系统)的服务,这对子网的可靠性提出了较高的要求;另一方面,这样重要的基础设施不能朝建夕改,应当有一定的超前性和稳定性。这两种思路,孰优孰劣,极难定断,它们各自都在向前发展。

传输层与网络层一样,也有两种服务:面向连接的服务和无连接的服务。这两种服务与网络层是一致的,因为传输层是对网络层在可靠性方面最后的补充。也就是说,在传输层之上不再考虑传输的可靠性问题。为了让用户更放心,为了给应用于各种网络的应用程序提供更可靠的网络服务,使这些应用程序能够采用一个标准的原语集来编写,而不必担心不同的通信子网接口和不可靠的数据传输,才设置了传输层。

4. 传输层协议类型与网络层服务质量的关系

传输层的主要功能是增强和优化网络层提供的服务质量。因此,传输层的工作情形依赖于网络层提供的服务质量。网络层提供的服务愈完善,传输层协议就愈简单;反之,网络层提供的服务愈简单,传输层协议就愈复杂。

按照质量,可以把网络层的服务分为 A,B,C 三级:A 级网络提供完善的服务,分组的丢失、重复和错序极少。C 级网络提供的服务几乎是不可靠的,它会出现分组丢失和重复分组。B 级网络服务介于 A 级和 C 级之间。基于 A 级网络服务的传输层协议非常简单,基于 C 级网络服务的传输层协议非常复杂。广域网几乎不能提供 A 级服务,面向连接的广域网常提供 B 级服务,面向无连接的广域网则提供 C 级服务;少数局域网可以提供 A 级服务。根据网络层提供的服务,ISO/OSI 把传输层协议分为 0~4 等 5 类,如表 4.3 所示。

表 4.3 网络层的服务等级与传输层协议类型

传输层协议类型	网络层等级	说明
0 类:简单类	A	最简单:不进行排序和流控
1 类:基本错误恢复类	B	不需错误检测和流量控制
2 类:多路复用类	A	允许多个连接向一个网络发送数据
3 类:出错恢复和多路复用类	B	集中了 1、2 两类协议的特点
4 类:出错检测和恢复类	C	最复杂:必须能处理分组丢失、重复、出错

5. 传输服务过程

传输层向传输服务用户提供面向连接的服务和无连接的服务,并且服务是

通过建立的两个传输实体之间所用的传输协议实现的。下面通过介绍面向连接的传输服务过程,使读者了解传输协议的基本要素。

(1) 建立连接

面向连接的服务是从使用服务原语建立连接开始的。然而,一个应用程序要与一个远程应用程序建立连接,首先必须明确是与哪个远方应用程序建立连接,也就是要确定一对传输地址 TSAP。每一个传输地址都与一定的功能相联系。一般说来,本地传输地址不难确定,可以依据传输服务用户(进程)所依附的 TSAP 确定。而远端的 TSAP 确定不这么简单。

传输地址 TSAP 有点像电话的号码,每个电话号码是与特定的用户或用途相联系的。但是,有的人有电话,还可能有几部电话;而有的人没有固定电话可用,别人要找他,就得借助一些固定电话。传输地址也有这样的情况,有的 TSAP 是约定俗成具有不变功能的,有的则是临时分配的,传输服务用户无法直接知道。

(a) 与远端固定传输地址的连接

端－端通信向上提供的服务最终表现为面向应用程序的服务。应用程序间的通信具体由多个进程实现,在多用户、多任务的网络中要求一台主机能并发地处理多个进程。为此,传输层顶端提供了多个"端口"(传输地址)的服务,一个端口对应一个进程,以满足一台主机同时并发地处理多个进程的要求。

首先讨论与远端固定传输地址连接的情形。设源端机上一个依附于 8 号 TSAP 的进程需要远端的一个时间进程的服务;远端时间进程依附于 123 号 TSAP,并处于休眠状态。连接过程如下:

① 源主机上的进程需要查询时间时,发出连接请求——源传输地址为 8 号 TSAP,目标地址为 123 号 TSAP。

② 源主机传输层收到连接请求后,在源主机和目标主机上选择 NSAP,并在它们之间建立一个网络连接。

③ 源主机上的传输实体利用上述连接,将服务质量参数通知目的主机上的传输实体。

④ 目的主机上的传输实体发送连接指示给时间进程。

⑤ 如果时间进程同意,就发出连接响应原语。

⑥ 源主机的传输实体收到确认信号,连接便建立。

(b) 与非固定传输地址的连接

一个人要是没有固定电话,而别人要与他通信,就得借助某个固定电话(如秘书的电话)。具体的实现可能有多种方式。例如,请求通话者先打一个电话到某固定电话(秘书处),让该固定电话通知他,如果他愿意与请求者通话,(秘书)

就为他选定一台电话并把这台电话的号码通知请求者,再由请求者打过来。

通过公用传输地址 TSAP 与非固定传输地址进行连接,也有许多方法。其中一种方法是:主机为远程进程的使用提供注册程序或进程连接服务程序(相当于秘书);该程序可常驻内存并附在固定 TSAP 地址上(这个地址可以在网络使用手册中找到)。任何进程要访问远程服务,步骤如下:

① 首先与注册进程连接(在 ARPA 网中称为初始连接)——相当于先接通秘书的电话。

② 请求进程给注册进程发消息,说明要服务的程序——相当于告诉对方秘书要找谁。

③ 注册进程按照消息要求产生一个新的进程运行该服务进程,并为它分配一个未用的传输地址——相当于分配一部临时可用电话。

④ 注册进程将分配的传输地址发给远端请求进程,并释放与请求进程的传输连接,然后进入休眠状态——相当于把临时分配的电话号码告知来电者,然后挂机。

⑤ 请求进程收到传输地址后,释放与注册进程的连接——相当于也挂机。

⑥ 请求进程与服务进程连接——相当于再拨给欲联系者的临时电话。

(c) TSAP 地址和 NSAP 地址结构

一般地,通用 TSAP 地址具有下面的层次结构:

＜国家＞＜部门＞＜网络＞＜主机＞＜端口＞

实际上,也就是:

TSAP 地址＝NSAP 地址＋＜端口＞

因此,当一个传输实体收到一个 TSAP 地址并想与之连接时,只要使用 TSAP 地址中的 NSAP 地址,就可以首先建立一个网络连接,进而到达适当的远端传输实体。也就是说,网络连接是传输连接的一部分,传输连接是基于网络连接的。

(2) 数据传输

两个要通信的传输服务用户进程一旦建立了连接,便可以立即开始数据传输。传输层要向传输用户提供可靠的、透明的和价格合理的数据传输。下面介绍传输实体在数据传输过程中要处理的一些问题。

(a) 数据封装

在源端,传输层接收来自高层的服务数据单元 TSDU,然后加上报头进行封装,使之成为传输层的协议数据单元 TPDU。这时,如果报文太长,则应将其分段;如果报文太短,应将它们组装。与之对应,在目的端再恢复原状。

(b) 拼接和分割

从传输层往下看,TPDU 实际上是要通过网络层进行实际的数据传输。为

此,要按网络层的 NPDU 大小,对传输层的 TPDU 进行拼接或分割。

随着分组一层一层地向下层传输,数据分组被进行一层一层的封装,最后到达物理层按比特流传输。图 4.12 表明了这一层层封装的过程。

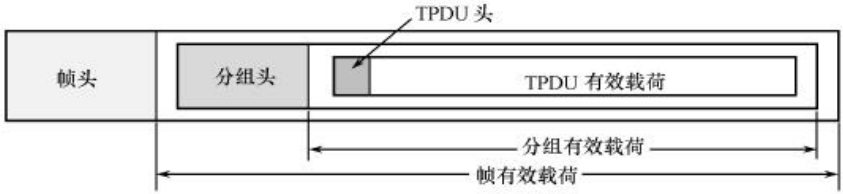


图 4.12　TPDU 的层层封装过程

(c) 多路复用和分流

传输连接是建立在网络连接基础上的,网络连接是建立在虚电路概念上的。多路复用是指当传输服务进程产生的数据流较少时,可以将几条传输数据流映射到同一物理连接(即一条虚电路)上,以充分利用网络带宽,减少网络连接,降低费用。分流是指当用户进程发送的数据量大于网络连接的容量时,该传输连接可以打开多个网络连接(即多条虚电路)来提高带宽。

(d) 流量控制和缓冲管理

流量控制就是控制发送数据量,使其不超过接收者的能力。这一点与数据链路层的流量控制概念相同,并且它们都是基于滑动窗口技术的。但是,数据链路层线路较少,通信量大,常采用固定大小的窗口;传输层则由于所需的连接个数远比数据链路层要多,但同时建立连接的可能性又比较小,为了提高缓冲区的利用率,常采用可变大小的窗口以及动态缓冲分配技术。

(3) 释放连接

当数据传输完毕或出现异常情况时,都需要释放连接。

6. 传输服务原语

(1) 传输服务原语的参数

在传输服务原语中,要涉及如下参数:

callee,被呼方传输实体使用的地址(即被呼方 TSAP 的地址);

caller,呼叫方传输实体使用的地址(即呼叫方 TSAP 的地址);

exp_wanted,是否加快数据发送(布尔值);

qos,希望的服务质量;

user_data,传输服务用户数据;

responder,目标方作为响应的传输地址;

reason,连接释放原因。

(2) 传输服务原语的类型

每种传输服务各有自己的访问原语。ISO/OSI 服务原语是提供给面向连接的和无连接的服务使用的。传输服务原语也分这样两大类:面向连接的传输服务原语和无连接的传输服务原语。

(a) 面向连接的传输服务原语

连接服务原语

T_CONNECT.request(callee,caller,exp_wanted,qos,user_data)

T_CONNECT.indication(callee,caller,exp_wanted,qos,user_data)

T_CONNECT.response(qos,responder,exp_wanted,user_data)

T_CONNECT.confirm(qos,responder,exp_wanted,user_data)

连接释放服务原语

T_DISCONNECT.request(user_data)

T_DISCONNECT.indication(responder,user_data)

数据正常传输服务原语

T_DATA.request(user_data)

T_DATA.indication(user_data)

数据加速传输服务原语

T_EXPEDITED_DATA.request(user_data)

T_EXPEDITED_DATA.indication(user_data)

(b) 无连接的传输服务原语

T_UNIDATA.request(callee,caller,exp_wanted,qos,user_data)

T_UNIDATA.indication(callee,caller,exp_wanted,qos,user_data)

(3) 传输服务原语使用的时序

图 4.13 表示了几种典型传输服务过程的原语时序关系。图中,两条竖线将空间分为 3 部分:中间为传输服务提供者,两侧为进行通信的两个传输服务用户。

4.2.4 OSI/RM 各层中的数据流动

从 OSI/RM 中还可以看出,在每一层中数据的格式是不相同的。图 4.14 描绘了从用户应用程序进程将数据送入应用层开始,直到在物理层中按比特流进行实际传送,数据在各层中被所在层的协议加工、流动的过程。

下面进一步加以说明。

假定要将数据 M 从用户应用进程 P_A 传向用户应用进程 P_B。其过程如下:

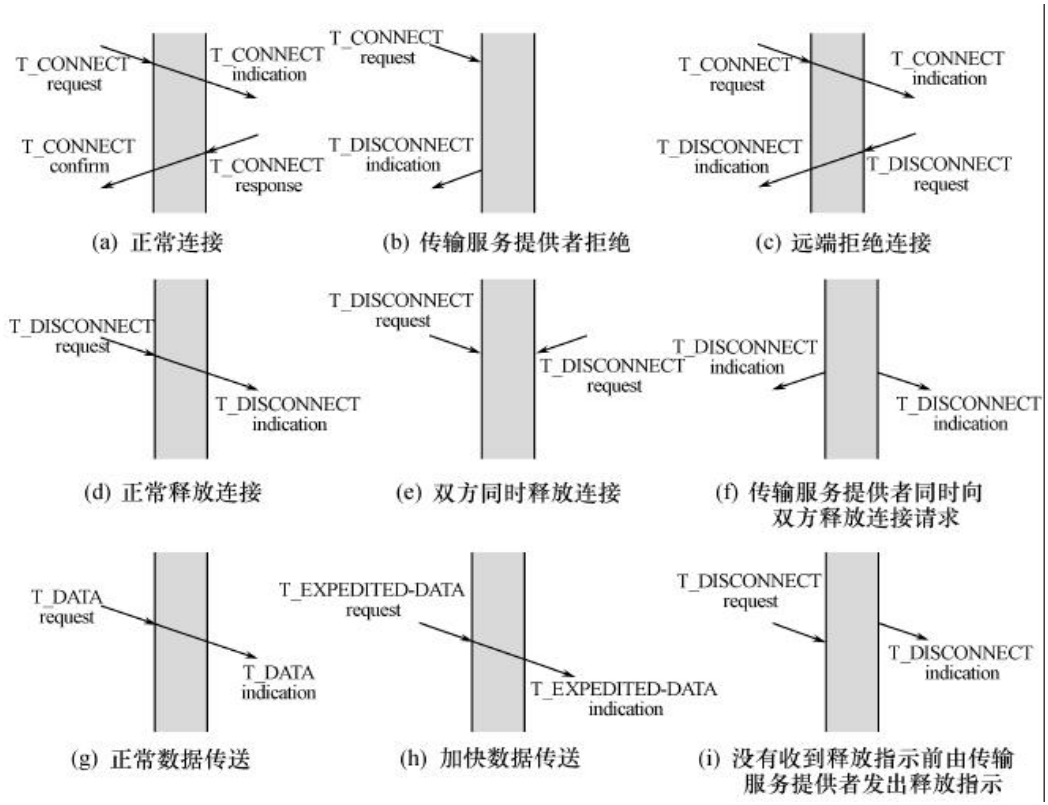


图 4.13　几种典型传输服务过程的原语时序关系

在发送端进程 P_A 首先要将数据 AP 送到应用层；

在应用层，加上一些控制信息组成的报头 H_7，形成第 7 层的报文——协议数据单元 PDU_7，再将 PDU_7 传到表示层；

在表示层，加上第 6 层协议要求的控制信息组成的报头 H_6，形成第 6 层的协议数据单元 PDU_6，再将 PDU_6 传到会话层；

在会话层，加上第 5 层协议要求的控制信息 H_5，形成第 5 层的协议数据单元 PDU_5，传到传输层；

在传输层，如果报文太长，应将其分为若干分组，即 $P_1, P_2, \cdots, P_n$，再分别为它们加上第 4 层的控制信息 H_4，形成第 4 层的协议数据单元 PDU_4，传到网络层；

在网络层，把第 4 层传来的分组及报头看成是高层数据单元，再加上第 3 层的报头 H_3，发往数据链路层；

在数据链路层，分别在头部和尾部加上报头 H_2 和报尾 T_2，形成第 2 层的数据单元——帧，发往物理层；

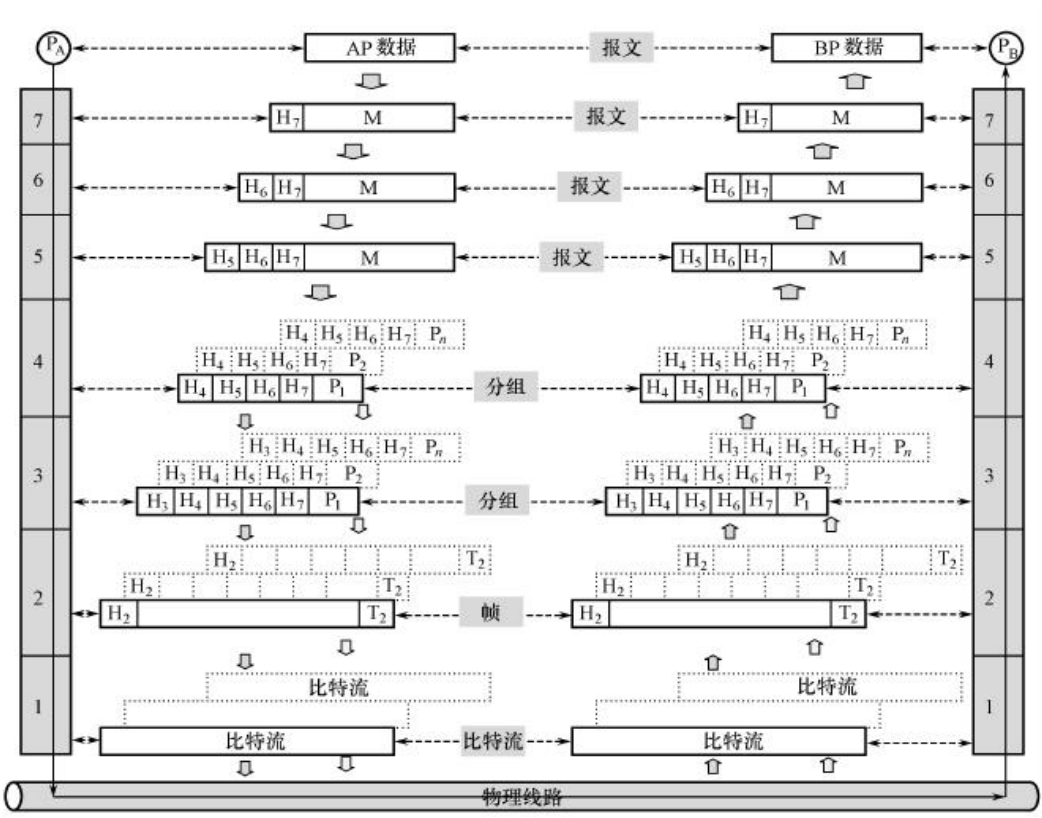


图 4.14　OSI/RM 中的数据流动示意图

在物理层，不再加控制信息，按比特流进行实际传送。

各层的协议控制信息 H_i，因协议不同、传送的内容不同，分别有不同的内容和格式要求。

比特流到达节点 B 后，传送到其数据链路层，剥去报头 H_2 和报尾 T_2，形成 PDU_3，传到 3 层；再剥去 H_3 形成 PDU_4，传到 4 层……，直到将数据 M 传给 P_B。

习　　题

4.1　试述协议和服务之间的区别与联系。

4.2　试举例说明日常生活中碰到的分层结构。

4.3　在 OSI/RM 中，对 N－PDU 的封装是在传输层分组中、网络层分组中，还是帧中实现？

4.4　在下列各描述后面的括弧中填入 OSI 中对应的层次名称。

定义帧（　　）　　注册和注销的过程（　　）

路由确定（　）
分组交换（　）
网络选择（　）
流量控制（　）
用户的同步（　）
纠错和重传（　）
重装数据分组（　）
格式和代码的转换（　）
确保数据可靠传输（　）
与外部世界的接口（　）
可靠的端到端的传输（　）
建立、维护和终止会话（　）
负责相邻节点的信息传输（　）
在物理媒体上传输比特流（　）
直接与用户应用程序通信（　）
机械、电气和功能的接口（　）
ASCCII 与 EBCDIIC 的转换（　）
为用户提供到网络的接入（　）
给用户提供服务，如电子邮件等（　）
从数据表示的差别中提供独立性（　）

第 5 章　IEEE 802 模型与局域网

局域网是一种覆盖地理范围较小的网络。它们具有较低的传输时延和误码率,同时由于其投资少、回报快、灵活、方便,因而获得了越来越广泛的应用。

局域网的出现要归功于个人计算机广泛应用,个人计算机的进一步普及和性能的提高,又推动了局域网的进一步发展。在过去的 30 多年间,计算机的处理速度提高了百万倍,而网络数据传输率才提高了上千倍。从理论上讲,一台微通道或 EISA 总线的微型计算机的数据流量可达 250 Mb/s,而早期的 Ethernet 的数据传输率只有 10 Mb/s,显然,开发高速局域网势在必行。由于传统的局域网技术是建立在"共享介质"的基础上的,因而随着站点的增加,网络效率与网络规模之间的矛盾就随着连网需求的增加日益突出。为了解决网络规模与网络效率之间的矛盾,人们从以下几个方面进行了努力:

局域网的标准化;

提高网络的传输效率;

用网桥或路由器将网段微化,使子网中接点数减少,以改善网络性能;

将"共享介质方式"改为"交换方式";

将一些广域网技术,如 ATM 技术,应用到局域网中。

本章介绍组织局域网的基本技术和知识。

5.1　局域网模型

5.1.1　IEEE 802 模型

IEEE (Institute of Electrical and Electronic Engineers,美国电子与电气工程师协会) 于 1980 年 2 月成立了局域网标准委员会,简称 IEEE 802 委员会,专门从事局域网的标准化工作,该委员会为局域网制定了一系列标准,统称为 IEEE 802 标准。

1. IEEE 802 模型的特点

局域网的体系结构与广域网相比,有很大的不同。

(1) 局域网种类繁多,使用的传输介质各种各样,接入方法也不相同,为此 IEEE 802 在数据链路层中专门划分出一个传输介质访问控制 (MAC,Medium Access Control) 子层来进行传输介质访问控制,并用逻辑链路控制 (LLC,Logical Link Control) 子层处理逻辑上的链路。LLC 子层与具体局域网使用的介质访问方式无关,主要为高层协议与局域网介质访问控制 MAC 子层之间提供统一的接口。

(2) 局域网的拓扑结构比较简单,且多个站点共享传输信道,在任意两个节点之间只有惟一的一条链路,不需要进行路由选择和流量控制,因而它不需要定义网络层,只具备 OSI/RM 低两层的功能就可以了。由于考虑到局域网要互连,所以在 LLC 子层之上设置了网际层。

(3) 其他高层功能往往与具体的实现有关,通常包含在网络操作系统中。

(4) 物理层还是需要的,并且物理层往往也分为两个子层。下面的子层用于对传输介质进行说明;上面的子层作为传输介质的访问单元,用于发送/接收比特、编码以及介质处理,类似于 OSI/RM 的物理层。

这样一来,就使问题简化了。图 5.1 给出了 IEEE 802 模型及其与 OSI/RM 的比较。

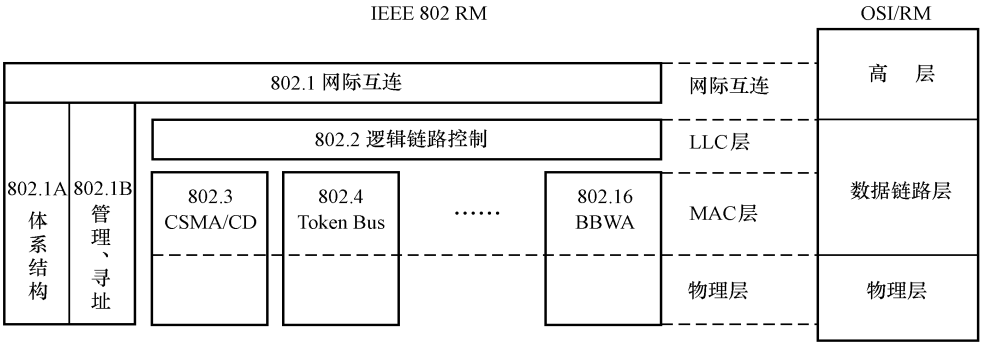


图 5.1　IEEE 802 模型及其与 OSI/RM 的比较

2. MAC 子层的主要功能

MAC 层主要处理与传输介质有关的问题,同时还负责在物理层传输比特的基础上的无差错通信。因此,MAC 层的主要功能有:

将上层交下来的数据封装成帧进行发送 (接收时相反,将帧拆卸递交到上层);

按 MAC 地址 (即帧地址) 寻址;

进行差错检测;

MAC 层的维护和管理。

3. LLC子层的主要功能

LLC层处理与接入介质无关的而又属于数据链路层处理的问题。主要功能如下：

提供与高层的接口；

实现数据链路层的差错控制；

给帧加上序号；

为高层提供数据链路层逻辑连接的建立和释放服务。

5.1.2 IEEE 802协议标准

IEEE 802是一个标准系列，不断增加了新的标准，现有的标准有：

IEEE 802.1A，概述和体系结构；

IEEE 802.1B，寻址、网际互连及网络管理；

IEEE 802.2，LLC协议；

IEEE 802.3，CSMA/CD访问方法及物理层规范；

IEEE 802.4，令牌传送总线访问方法及物理层规范；

IEEE 802.5，令牌传送环访问方法及物理层规范；

IEEE 802.6，城域网（MAN）标准；

IEEE 802.7，宽带局域网标准；

IEEE 802.8，光纤局域网标准；

IEEE 802.9，综合数据/语音网络标准；

IEEE 802.10，网络安全与保密标准；

IEEE 802.11，无线局域网标准；

IEEE 802.12，100BASE－VG标准；

……

IEEE 802.14，有线电视网（CATV Broadband）标准；

IEEE 802.15，无线个人网络（WPAN）标准；

IEEE 802.16，无线宽带局域网（BBWA）标准。

5.1.3 信道的多点共享访问控制

多点共享与多路复用不同，多路复用是将一条物理信道分割成多条逻辑信道，使多个用户信息在同一信道上同时传输的技术；多点共享技术则是多个节点使用同一条信道时的控制策略，也称多点共享控制技术、多点投入技术、多点访问技术或介质共享技术。简单地说，多点共享技术就是在某一时刻只允许传送

一个用户数据的情况下,为解决多个用户争相使用引起的信道冲突 (Collision,也称碰撞) 而采用的介质访问控制方案。多点共享技术可以有效地提高传输介质的利用率,主要用于局域网中,因为在局域网环境下,线路距离不长,没有必要使用复杂的多路复用/解复用设备。

目前有两种多点访问控制方式:

(a) 无竞争 (受控) 方式,各个节点必须在某一控制原则下接入,形成一种无冲突的访问控制方式。在受控多点访问技术中,多点线路既可以采用分散式控制,也可以采用集中式控制。

(b) 竞争方式,各节点以竞争方式来取得介质的使用权。

5.2 令　牌　网

令牌传递是一种受控访问控制方法,按照网络拓扑结构可以分为令牌环 (Token Ring) 介质访问控制和令牌总线 (Token Bus) 介质访问控制两种方法。

5.2.1　令牌环网与 IEEE 802.5 标准

1. 令牌环的工作原理

令牌环 (Token Ring) 是令牌通行环 (Token Passing Ring) 的简写。令牌就是一个具有特殊格式的帧,一个典型的例子是它为一个 8 位的二进制数,它一直在环上按一个方向从一个节点到另一个节点流动。如图 5.2 所示,每个节点上都有一个通过电缆、介质接口连接器和转发器连接到环上的工作站。

令牌有"闲"和"忙"两个状态,开始时为闲 (如将状态寄存器置"1") 。一个节点有数据要发送,必须等空闲令牌到来;检测到空闲令牌到来,便将之截获下来,置令牌的状态为"忙" (将状态寄存器置"0") ,并把要传送的数据等字段加上去,令其继续往前传送;每到一个站点,该站点的转发器便将帧内的目的地与本站的地址进行比较,如果两地址符合,则复制该帧,并在帧中置入"已收到"标志,然后让帧继续传送;当传送回发源站点时,若没有检查到"已收到"标志则继续发送当前帧,若检查到"已收到"的标志就停止传送,撤销所发送的数据帧并立即生成一个新的令牌发送到环上 (这时还有数据就继续发送,否则生成空闲令牌) 。这种由发送站回收令牌的策略具有广播性,允许多个站点接受同一数据帧。

令牌传送方式是一种无冲突的介质共享方式,常用于负载较重、通信量较大的网络,地理范围也比以太网大。不像以太网那样,随着负荷的增加和冲突增

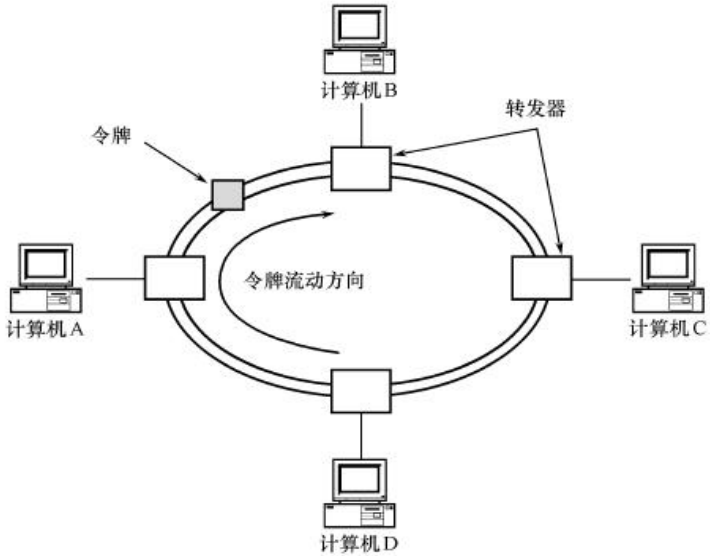


图 5.2　令牌环模型

多,网络效率急剧下降。令牌环网的缺点是,管理要比竞争方式复杂得多。为了防止令牌的损坏、丢失或出现两个甚至多个令牌等错误,网络必须有错误检测能力和恢复机制等。此外,令牌环网采用了集中管理方式,而该网络控制站一旦出现问题便会造成一些麻烦。

2. IEEE 802.5 的 MAC 帧

IEEE 802.5 是令牌环网的访问方法和物理层的标准,它定义了令牌环网 MAC 子层的两个基本帧:令牌帧和数据帧(非令牌帧)。令牌环的帧结构如图 5.3 所示。

组成令牌环帧的字段有如下一些。

(1) 起始字段和结束字段

它们各占 1 个字节,其中各有两对"JK"的特殊比特,用于起始和结束的识别标志。

对于数据帧来说,在结束字段中,要用 1 位 I 作为后继比特,当 I=1 时,表示还有数据要发送;当 I=0 时,表示是最后一帧数据。同时,还要用 1 位 E 作为差错比特,开始时发送站将 E 置 0,环上所有的经过站点都要对经过的帧进行校验,检测到错误,即将 E 置 1。

(2) 访问控制字段

该字段占 1 个字节。

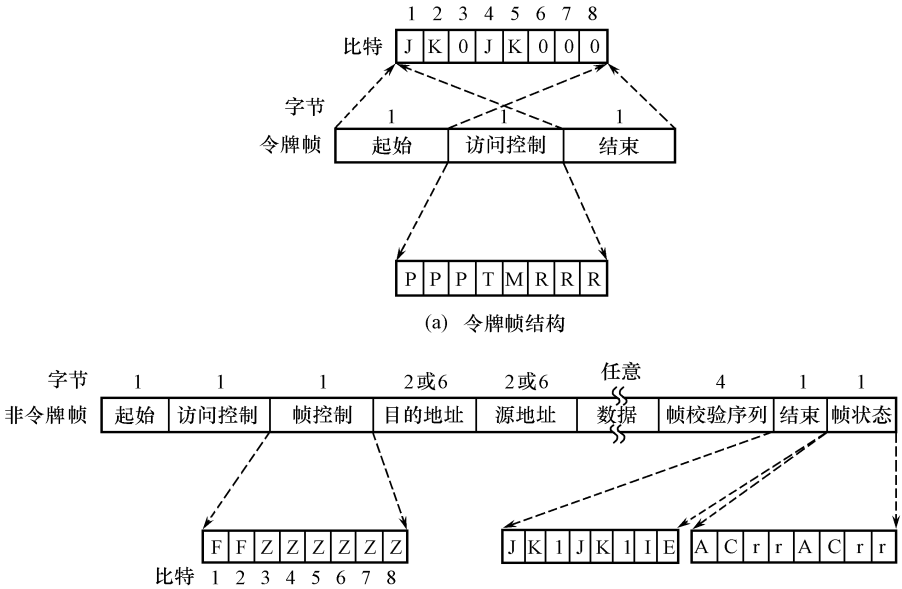


图 5.3　令牌环的帧结构

① 优先级比特 PPP（占 3 位），将优先级分为 8 级，使优先级高的站点可以通过预约取得下一次的发送权。

② 预约比特 RRR（占 3 位），用于按优先权预约下次发送权。一个站要发送数据时，可以在经过本站点的数据帧的 RRR 字段中进行预约，以取得下一次的发送权，当然，必须是本站的优先权高于当前数据帧中 RRR 字段的优先权。

③ 令牌比特 T（占 1 位），用于表示令牌的“闲”和“忙”。

④ 监督比特 M（占 1 位），为防止忙令牌的无限循环而设置。

如图 5.3(a) 所示，令牌帧只有起始（结束）字段和访问控制字段，而非令牌帧还有下面的一些字段。

(3) 帧控制字段

帧控制字段占 1 个字节，前 2 位 FF 表示帧的类型，根据 FF 的的取值，帧分以下 3 种类型：

FF＝00，为 MAC 控制帧，无数据字段；

FF＝01，为一般信息帧，只发送数据给地址字段指定的目的站点；

FF＝11 或 FF＝10 未定义。

帧控制字段中后 6 位 ZZZZZZ 为控制帧的种类。

(4) 目的地址和源地址

局部地址占 2 个字节,全局地址占 6 个字节。

(5) 数据字段

最小长度为 0,最大受令牌轮转一周的最长时间限制。

(6) 帧校验序列字段 FCS

占 4 个字节,采用 CRC 码。

(7) 帧状态字段 FS

最后 1 个字节,其中:

A,地址识别指示比特;

C,帧已复制指示比特;

r,未做规定。

5.2.2　令牌总线网与 IEEE 802.4 标准

令牌总线网的物理拓扑为总线,其基本原理是:让令牌一站接着一站地在总线上传递,到最后一个站点时反绕到第一个站点,形成逻辑上的环,如图 5.4 所示。

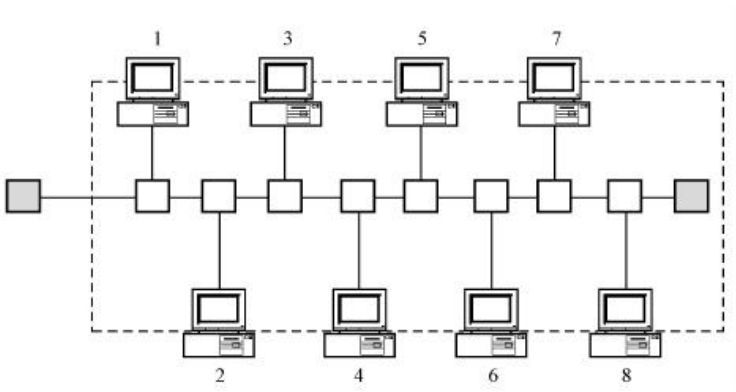


图 5.4　令牌总线网的物理和逻辑结构

令牌总线网的 IEEE 802.4 标准的工作原理与令牌环网相似,这里不再赘述。请读者分析其特殊性。

5.3　以　太　网

以太网(Ethernet)是目前应用最广泛的局域网。本节介绍它的基本原理和发展变形。

5.3.1　CSMA/CD 协议

CSMA/CD (Carrier Sense Multiple Access/Collision Detection) 即带有冲突检测 (CD) 的载波侦听 (CS) 多路访问 (MA) ,它是目前广泛应用的以太网上所采用的数据传输控制技术。CSMA/CD 发送方的工作原理如图 5.5 所示,简单地可以用下面的重复过程描述:

讲前先听——忙则等待,无声则讲——边讲边听,冲突即停,后退 (等待一段随机时间) 重传——多次无效 (仍冲突) ,放弃发送。

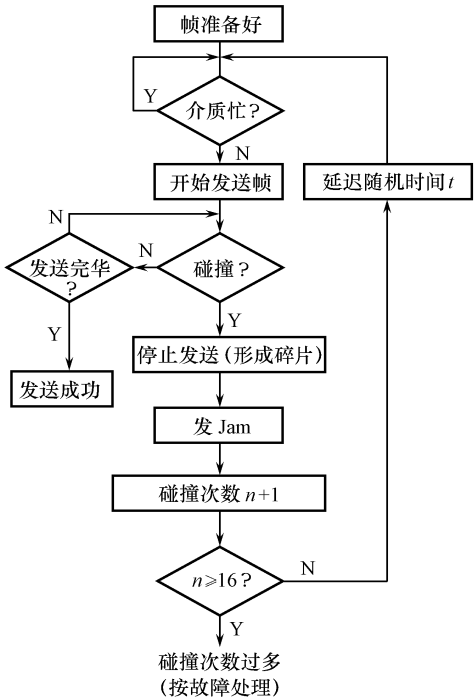


图 5.5　CSMA/CD 介质存取方法工作流程

1. CSMA

(1) 载波侦听 (CS, Carrier Sense)

载波侦听是欲发送数据的节点进行的第一件工作,即在数据发送之前先要检测线路上有无信号正在传送,这就是"讲前先听"。在不同的介质上采用不同的侦听方式。一般而言,在基带 (Baseband) 机制下,以传输线路上有无脉冲波变化为判断依据;在宽带 (Broadband) 机制下,则以有无载波信号来判断是否已有信号在传输。

如果线路上无信号传输,就可以发送自己的信号,这就是"无声则讲"。但是,以太网进行帧传送时,为确保前后两个帧互不重叠干扰,要求帧间必须保留有 12 个字节 (96 位) 的帧间隙 (Interframe Gap) 。这个帧间隙,对 10 Mb/s 以太网来说为是 9.6 μs 的时间间隔,对 100 Mb/s 以太网来说是 0.96 μs 的时间间隔。

(2) 坚持与不坚持算法

当监听到信道上有信号时,可以用坚持和不坚持两种算法进行处理,坚持就是还要一直监听下去,不坚持就是暂不再监听。具体又演化为如下三种 CSMA 方法。这三种 CSMA 方法可以用打电话的方式理解——打电话时,若对方空闲,则立即就讲;若对方占线,可以有三种方法处理:

不停地拨对方的号码,直到对方空闲。这是坚持要打——"贪婪"的算法。

发现对方占线,不立即拨号,等一段时间再拨。这是不坚持立即打——

“理智”的策略。

在处理对方占线的情形时,根据本站点的地位和线路情形,立即再拨的概率为 P,不立即再拨的概率为 $1-P$。这是折中的方法。

下面具体分析这三种方法。

(a) 1-坚持 CSMA

当一个站点要发送数据时,首先要侦听信道,看是否有其他站点正在发送数据。

如果信道忙,就等待;一旦发现信道空闲,即将数据送出。

若发生冲突,就等待一个随机长时间,再试。

由于一旦发现信道空,发送数据的概率为1,故称1-坚持 CSMA。“坚持”,即在信道忙时,要坚持地侦听。

(b) 非坚持 CSMA (nonpersistent CSMA)

非坚持 CSMA 不像1-坚持 CSMA 那样“贪婪”地等待发送,而是采取了“理智”的策略:

当一个站点要发送数据时,首先要侦听信道,看是否有其他站点正在发送数据。

如果信道空闲,即将数据送出;若信道在使用中,就不再继续侦听,等待一个随机长时间,再试。

(c) P-坚持 CSMA

P-坚持 CSMA 用于分隙信道。其工作过程如下:

当一个站点要发送数据时,首先要侦听信道,看是否有其他站点正在发送数据。

如果信道空闲,即将数据以 P 概率传送,并以概率 $1-P$ 把该次发送推迟到下一时隙;若下一时隙信道仍空闲,便再以 P 概率传送,并再以概率 $1-P$ 把该次发送推迟到下下时隙;如此重复,直到发送成功或另一站开始发送。

若信道忙,就继续侦听,到下一时隙重新开始上述过程。

假如介质忙时有 N 个站点等待数据发送,等到介质空闲时,还有 PN 个站点企图发送。与1-坚持型 CSMA 相比,冲突减少。冲突的多少取决于 P 的选择。如果 P 选得较大,冲突发生的几率就大;而 P 选得太小,则信道的利用率就会降低。

无论坚持还是不坚持,都不能立即传信号,要等到线路上出现闲置,这就是“忙则等待”。

2. 冲突检测 (CD, Collision Detection)

(1) 冲突的产生与检测方法

即使已经检测到线路空闲而发送信号后,还有可能在信号传送过程中产生

碰撞——冲突(Collision)。冲突产生的情形有如下两种：

(a) 两个以上节点都准备发送信号，并同时进行载波侦听又在侦听到线路空闲后都把信号发送到线路上，因而造成冲突。

(b) 一个节点(如 A)先检测到线路空闲后发送了信号，但由于信号传输延迟(Propagation Delay)，信号没有到达的节点(如 D)因没有检测到节点 A 已发送信号，也发送了信号，造成冲突。

冲突会造成已发送帧的破坏，为此，在发送帧的过程中，应当“边讲边听”，并且“冲突即停”——停止本次发送，丢弃受损帧，等待下一个随机时间再发送。

但是，有时即使发生冲突，也会检测不出——漏检。漏检的发生与帧的发送时间以及传输延迟时间两者的关系有关，其分析如图 5.6 所示。

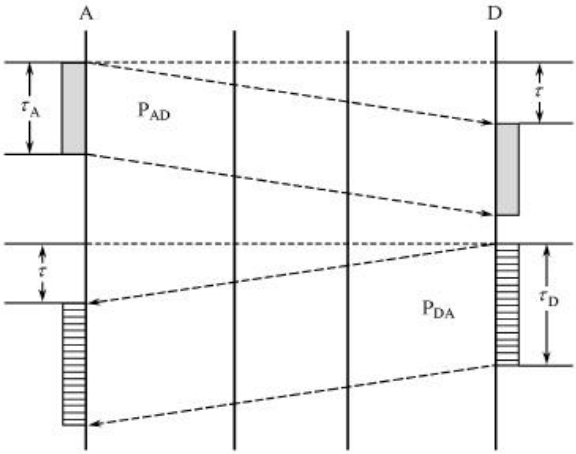


图 5.6　帧长度和传输延迟对冲突的影响

图中，P_{AD}为由节点 A 传送到节点 D 的一个帧，τ_A 为其发送时间(从发送开始到发送结束)，τ为传输延迟时间，T_A 为其传输总时间(从传输开始到全部传到)，显然有

$$T_A = \tau_A + \tau$$

同样，对 P_{DA}有

$$T_D = \tau_D + \tau$$

现在讨论 P_{AD}与 P_{DA}在两端同时发送帧时，A、D 两端对冲突的检测：

(a) $\tau_A < \tau$且 $\tau_D < \tau$，即 P_{AD}发送完时 P_{DA}还没有到 A，且 P_{DA}发送完时 P_{AD}还没有到 D，这时 A、D 都不能检测出冲突——A、D 都漏检。

(b) $\tau_A < \tau$且 $\tau_D > \tau$，即 P_{AD}发送完时 P_{DA}还没有到 A，而 P_{DA}发送完时 P_{AD}已经到 D，这时 D 可以检测出冲突，而 A 漏检。

(c) $\tau_A > \tau$ 且 $\tau_D < \tau$,即 P_{AD} 发送完时 P_{DA} 已经到 A,而 P_{DA} 发送完时 P_{AD} 还没有到 D,这时 A 可以检测出冲突,而 D 漏检。

(d) $\tau_A > \tau$ 且 $\tau_D > \tau$,即 P_{AD} 发送完时 P_{DA} 已经到 A,且 P_{DA} 发送完时 P_{AD} 也已经到 D,这时 A、D 都可以检测出冲突。

因此,要使冲突的两端都能检测出,就应当使两端都满足发送时间大于传输延迟时间的条件,并且越大传输效果越好。为了考虑传输延迟的影响,倾听时间要考虑两个因素:一是信号在线路上最远两点间来回传输所需的时间,二是以太网时隙时间(Slot Time)——512 位(以太帧格式所规定的最小帧长度)的发送时间。

由于在最坏的情况(发送点与碰撞点分别在两端)下,发送点检测到碰撞已经持续了 2τ 的时间,因此废帧的最大长度就是 2τ。为了避免数据帧与废帧混淆,数据帧的长度应大于 2τ。

数据帧也不能太长,如果数据帧太长,有可能出现一个故障站点连续发送,而其他站点无法通信的情形。为此要对数据帧的最大长度加以限制,并使用物理层的发送装置对帧的长度进行检测,发现超长帧就自动停止发送。

(2) 后退等待

一旦检测出冲突,应立即停止发送,同时发出一个干扰信号(Jamming Signal),清除(丢弃)已发出的帧,并通知所有站点"冲突已经发生",请所有站点都暂停发送,以免冲突持续造成网络瘫痪。这个等待停滞称为退让(Backoff),其中发出的干扰信号是一串 32~48 位的"1"。

退让时间(Backoff Time)的选择要兼顾使各站点尽快将帧发出和能够实现网络的拥塞控制。显然,大家都采用一个固定的退让时间是不行的,那样会一而再、再而三地发生冲突。因此各站点采用随机退让时间,以便有效地减少冲突的连续发生。

最常用的后退算法是截断二进制后退算法,它用下面的公式计算退让时间:

$$t = R \times 2\tau,\ R\text{ 的取值范围为 }[0, 2^N - 1]$$

式中,*R 为随机数,N 为重发次数。*

第 1 次重发时,R 的范围为 [0,1];

第 2 次重发时,R 的范围为 [0,3];

……

这样,等待时间就与冲突历史相关,发生过的冲突次数越多,就说明发生冲突的可能性越大,退避时间就应越长,以免无谓之功。表 5.1 为退避时间举例。

表 5.1　退避时间的有关数据(对 10 Mb/s 的以太网络)

重传次数	工作站点数	随机数范围	退避时间
1	1	0～1	0～51.2 μs
2	3	0～3	0～153.6 μs
3	7	0～7	0～358.4 μs
4	15	0～15	0～768 μs
5	31	0～31	0～1.59 ms
6	63	0～63	0～3.23 ms
7	127	0～127	0～6.5 ms
8	255	0～255	0～13.1 ms
9	511	0～511	0～26.2 ms
10～15	1 023	0～1 023	0～52.4 ms

一般情况下,重传 16 次后,仍发生冲突,就放弃发送。由于 MAC 子层上的帧还不是 MAC 帧,因此当 MAC 子层传到物理层时,还要加上一个由硬件生成的 8 字节的前导同步码,表示一个帧的开始。它是“1010……”交替码,只是到了最后一位将 0 变为 1,表示前导同步码的结束。

3. 接收处理

接收处理主要有两项工作:接收校验和本地处理。接收校验包括碎片校验、目的地址校验和完整性校验:

碎片校验,长度小于 512 位的帧是冲突碎片。

目的地址校验用于判断是否是本地地址;

完整性校验包括校验是否是畸形帧(长度＞1 518 字节)、CRC 校验和定界符(长度必须是 8 位的整数倍)。

5.3.2　IEEE 802.3 与 10 Mb/s 以太网

1975 年 Xerox 公司的 Palo Alto 研究中心推出了它们的第一个局域网——以早先人们想像的传播电磁波的介质 Ether 命名的以太网 Ethernet。由于它具有结构简单、工作可靠等优点而得到广泛的应用。1980 年美国的 DEC,Intel,Xerox 三家公司联合研制公布了 Ethernet 技术规范。1985 年 IEEE 计算机通信委员会颁布的 IEEE 802.3 局域网络协议(CSMA/CD)基本上和 Ethernet 技术规范一致。于是,Ethernet 技术规范成为世界上第一个局域网络技术规范。

1. IEEE 802.3 的特点

IEEE 802.3 标准规定了 CSMA/CD 访问方法和物理层技术规范。该规范有

三大特点。

(1) 采用 1－坚持 CSMA/CD 协议。

(2) 规定 MAC 帧的长度范围为 64～1 518 字节：要求按 CSMA/CD 方法接收数据时，每个节点必须检测通过该节点的所有数据帧。如果数据帧中包含的地址与本节点的地址一致，则在接收该数据帧之前先检测该数据帧的完整性：是否太长（超过 1 518 字节），不超长则进行 CRC 校验，再检测是否太短（少于 64 字节为帧碎片）。完整时，才接收。

(3) 采用截断二进制后退算法。

2. 以太网帧结构

以太网的帧有两种格式：DIX（取最早设计以太网的三家公司的首字母）格式和 IEEE 802.3 格式。图 5.7 所示为 IEEE 802.3 帧的结构。两者区别仅在于 IEEE 802.3 格式中的 16 bit 的数据长度字段，在 DIX 格式中是 16 bit 的类型字段。

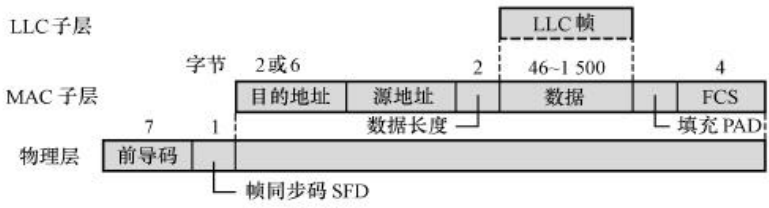


图 5.7　IEEE 802.3 帧结构

(1) 前同步码由 7 字节的前导码（1010…10）和 1 字节的帧同步码 SFD（10101011）组成。56 个“01”用于“唤醒”接收者，使接收者与发送者进行时钟同步，最后一个“11”用于“提醒”接收者开始接收。

(2) 源地址是指发送站的网卡地址；目的地址是指接收以太网帧的网卡地址。每个以太网卡地址中的前 3 个字节由 IEEE 统一分配做厂家编码，称为 BlockID；后 3 个字节由厂家统一编码，称为设备 ID。

(3) 类型：以太网封装的消息协议类型。标识该帧交给哪个高层协议（如 IP、IPX、ARP 等）。

(4) 数据长度：帧的长度。

(5) FCS：帧的循环冗余校验序列。

3. 以太网的组成

目前，许多网络都称为以太网。但是只有遵守 IEEE 802.3 的网络（即基带总线网）技术才能真正地称为以太网。图 5.8 为以太网的结构示意图。它由传输介质、收发器、网卡和计算机——工作站组成。

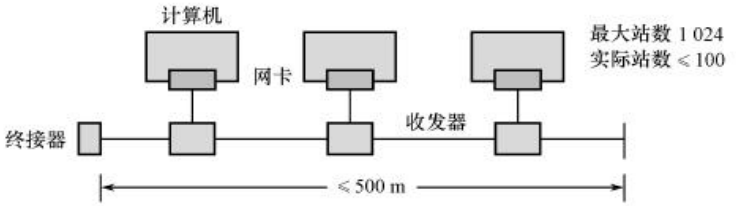


图 5.8　以太网的结构

(1) 传输介质

以太网的传输介质有粗缆 (10 Mb/s,传输距离 100 m)、细缆 (10 Mb/s,传输距离较近)、3 类 4 芯双绞线 (10 Mb/s,传输距离 100 m)、5 类双绞线 (100 Mb/s,传输距离 100 m)。也有用 3 类 8 芯双绞线来达到 100 Mb/s 的。

(2) 收发器

收发器具有如下功能：

接收或发送数据；

检测在总线上发生的数据帧的冲突；

在总线和总线接口的电子设备之间进行电气隔离；

超长功能控制。当收发器所连接的网卡或总线接口出现故障时,有可能向总线不停地发送无规律的数据,使总线上的所有站都无法工作。为此,要为所有的发送帧设置一个长度上限。收发器检测到某个数据帧超过此上限,即自动停止发送数据,以保护总线不受影响。

收发器与工作站之间用 5 组分别屏蔽了的双绞线连接：

一组供给收发器电源和地线；

数据两组——收、发各一组；

两组控制线——一组用于通知出现冲突,一组用于使工作站主动发送数据的路径与总线隔离。

收发器电缆不可超过 50 m,其上的信号采用曼彻斯特编码。

(3) 网卡

网卡是实现工作站之间通信的关键部件。它的一端与计算机的总线连接 (插在计算机的一个扩展槽中),另一端通过收发器电缆与收发器连接。网卡主要实现下列功能：

(a) 数据的封装和解封。发送时,将 LLC 子层传下来的 LLC 帧封装成带有地址和差错校验的 MAC 帧;接收时,拆去 MAC 帧的首部和尾部,送交 LLC 子层。

(b) 链路管理,实现 CSMA/CD 协议。

(c) 编码和解码。对送到收发器上的信号进行曼彻斯特编码,对从收发器

收到的信号进行曼彻斯特解码。

上述 (a) 和 (b) 是 MAC 子层的功能，(c) 是物理层的功能。

4. 10 Mb/s 以太网的物理层标准

10 Mb/s 以太网可以有多种物理层标准。目前有 5 种：

10 Base－5 标准，也称标准以太网（原始的 IEEE802.3）或粗缆以太网，采用粗同轴电缆、总线连接；

10 Base－2 标准——IEEE802.3a，也称便宜以太网或细缆以太网，采用细同轴电缆总线连接；

10 Base－T 标准——IEEE802.3i，也称双绞线以太网，采用无屏蔽双绞线、星型方式连接；

10 Base－F 标准——IEEE802.3j，也称光缆以太网，星型方式连接；

10 Broad36 标准，宽带传输以太网。

5. IEEE 802.3 以太网体系结构

为了便于物理层功能的实现，802.3 进一步将物理层分为两个子层。

(1) PLS 子层

PLS（Physical Signaling，物理信令）子层负责：

向 MAC 层提供服务；

曼彻斯特码的编码和解码；

载波侦听。

(2) PMA 子层

PMA（Physical Medium Attachment，物理介质附件）子层负责：

向 PLS 层提供服务；

冲突检测；

超长控制；

发送/接收串行比特。

图 5.9 给出了 802.3 以太网的两种体系结构。它们的区别在于 PLS 和 PMA 在不在同一个设备中。

图中：

MDI（Medium Dependent Interface，介质相关接口）定义了 PMA 与传输介质连接的电缆段、连接电缆段的连接器以及电缆末段负载的特性。

MAU（Medium Attachment Unit，介质附件单元）——PMA 与 MDI 的通称。

AUI（Attachment Unit Interface，附件单元接口）定义了连接 PLS 和 MAU 的电缆及其连接器的电气/机械特性以及该接口上的电信号特性。

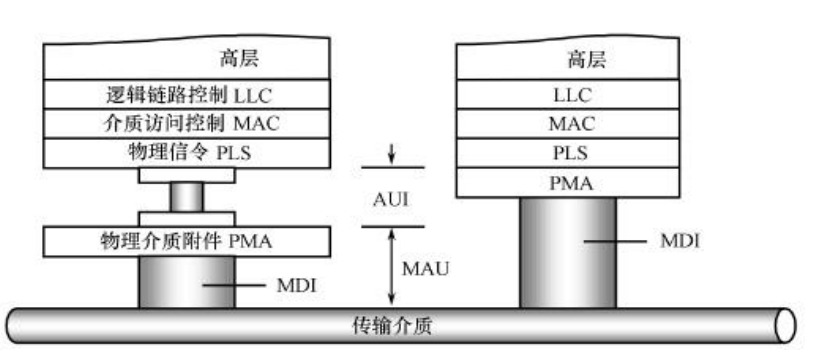


图 5.9　802.3 以太网的两种体系结构

图 5.10 所示为典型的以太网节点结构。

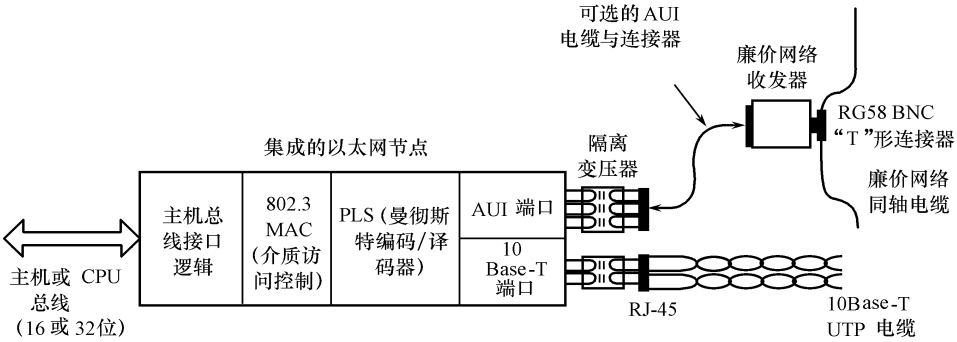


图 5.10　典型的以太网节点结构——10 Base－T 网卡的连接

5.3.3　100 Mb/s 以太网

1. 100Base－T 与 IEEE802.3u

100Base－T 也称快速以太网 (Fast Ethernet)，是在 10Base－T 基础上发展起来的、用以太网原始数据率 10 倍的速度运行的技术。它的相应标准是 IEEE802.3u。

100Base－T 保留了以太网 MAC 层的 CSMA/CD 介质控制协议，把每个比特的发送时间从 100 ns 降低到 10 ns，从而将数据率从 10 Mb/s 提高到 100 Mb/s。其网速的提高，一方面要求相应的协议调整，另一方面要求相应的通信介质。下面从协议调整和介质两个方面来介绍 100Base－T 的技术特征。

(1) 100Base－T 的物理层标准

与 10 Mb/s 以太网类似，为了支持各种拓扑结构并满足网络市场需求，

100Base－T 开发了 4 种类型的收发器，与之对应的有 4 种物理层标准：

100Base－TX，支持 2 对 5 类非屏蔽双绞线或 2 对 1 类屏蔽双绞线，数据速率 100 Mb/s，媒体段长度 100 m，网络跨度 200 m。

100Base－T4，支持 4 对 3、4 或 5 类非屏蔽双绞线，其中 3 对用于数据传输，1 对用于冲突检测，数据速率 100 Mb/s，媒体段长度 100 m，网络跨度 200 m。

100Base－FX，支持 2 条光纤，数据速率 100 Mb/s，媒体段长度 100 m，网络跨度 400 m。

100Base－T2，支持 2 对 3 类非屏蔽双绞线，数据速率 100 Mb/s，最大距离 100m。

(2) IEEE 802.3u CSMA/CD 模型

图 5.11 为 IEEE 802.3u 10/100 Mb/s DTE 的层次模型示意图。

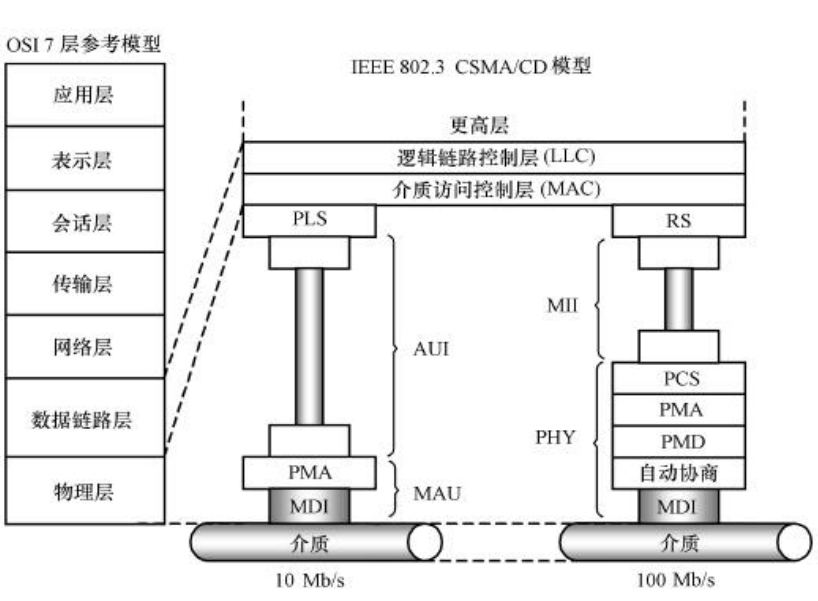


图 5.11　IEEE 802.3u 10/100 Mb/s DTE 的层次模型

从图中可以看出，在 100Base－T 中，用介质无关接口 MII (Medium Independent Interface) 代替了 10 Mb/s 以太网中的 AUI，并在 MII 与 MAC 层之间增加了一个协调子层 RS (Reconciliation Sublayer)。

用 MII 代替 AUI 的主要原因是 AUI 不能在达到 100 Mb/s 的速率下正常工作。AUI 采用的是串行接口，而 MII 的数据接口采用半字节模式 (一次传输 4 位数据) 接口，从而将带宽需求降低到 1/4。同时，MII 还提供了附加的强功能，以提供 MAC 层和物理层之间的专用报错和管理信号。当然，MII 也可以在 10 Mb/s

速度下工作。

由于 MAC 层只能提供位串行接口，而 MII 可以提供一个具有半字节宽度的发送/接收数据接口，所以要在 MAC 和 MII 之间增加一个协调子层，在原始以太网 MAC 和 MII 之间映射，以保证 MAC 层是一个标准实体。

此外，快速以太网还在下列方面对 10 Mb/s 做了改进：

(a) 不再使用曼彻斯特编码，而采用简单的不归零码。曼彻斯特编码是一种自含同步时钟的编码。但是它会产生高次谐波，这一现象会随着信号数据率的提高而加剧，产生严重的电磁干扰和射频干扰。因此，在 MII 中不再使用曼彻斯特编码。例如，在 100Base－4T 中使用 4 根彼此独立的 3 类无屏蔽双绞线，电缆上的时钟速率最大可以达到 40 Mb/s，为了将线位传输率减小至 30 Mb/s 以下而保证符号率为 100 Mb/s，采用了 8B6T 的三电平（+、－、0）编码方法。表 5.2 为 8B6T 编码表。

表 5－2　8B6T 编码

8 位数据	6T 码组	8 位数据	6T 码组	8 位数据	6T 码组	8 位数据	6T 码组
00000000	+－00+－	00001000	－+00+－	00010000	+0+－－0	00011000	0+－0+－
00000001	0+－+－0	00001001	0－++－0	00010001	++0－0－	00011001	0+－0－+
00000010	+－0+－0	00001010	－+0+－0	00010010	+0+－0－	00011010	0+－++－
00000011	－0++－0	00001011	+0－+－0	00010011	0++－0－	00011011	0+－00+
00000100	－0+0+－	00001100	+0－0+－	00010100	0++－－0	00011100	0－+00+
00000101	0+－－0+	00001101	0－+－0+	00010101	++00－－	00011101	0－++ +－
00000110	+－0－0+	00001110	－+0－0+	00010110	+0+0－－	00011110	0－+0－+
00000111	－0+－0+	00001111	+0－－0+	00010111	0++0－－	00011111	0－+0+－

(b) 增加自适应功能，通过自动协商进行工作模式的自动配置，在 10 Mb/s 和 100 Mb/s 的带宽上自适应。

(c) 可以使用全双工。与半双工相比，可以消除延迟限制，支持更大的网络半径。

(d) 定义了新的中继器规范，减少了中继器延迟。

(e) 可管理性得到加强。

2. 100Base－VG

100Base－VG 原名 100VG－AnyLAN，由 HP、IBM 和 AT&T 三家公司共同推出，目的是解决冲突型以太网中因争夺总线产生的瓶颈问题。它采用新的“按需优先轮询（Demand Priority Polling）”方法来处理网络请求，以免冲突。它同时支持

3 类、4 类和 5 类三种非屏蔽双绞线。

如图 5.12 所示，在 100Base－VG 中，100 Mb/s 的位流通过多路复用成 4 路 25 Mb/s 的数据流在 4 对双绞线上发送。这时需要使用 5B/6T 在集线器与节点之间传送以太网数据，使每一路双绞线的传输速率提高到 30 Mb/s。

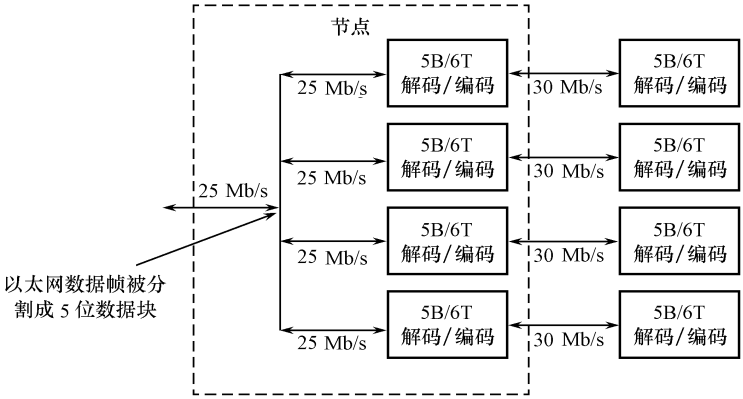


图 5.12 100Base－VG 中位流的编码

5.3.4 IEEE 802.3z 与千兆位以太网

1. 千兆位以太网的层次结构

千兆位以太网（Gigabit Ethernet）采用 IEEE 802.3z CSMA/CD 模型，如图 5.13 所示，其层次结构与快速以太网相似。

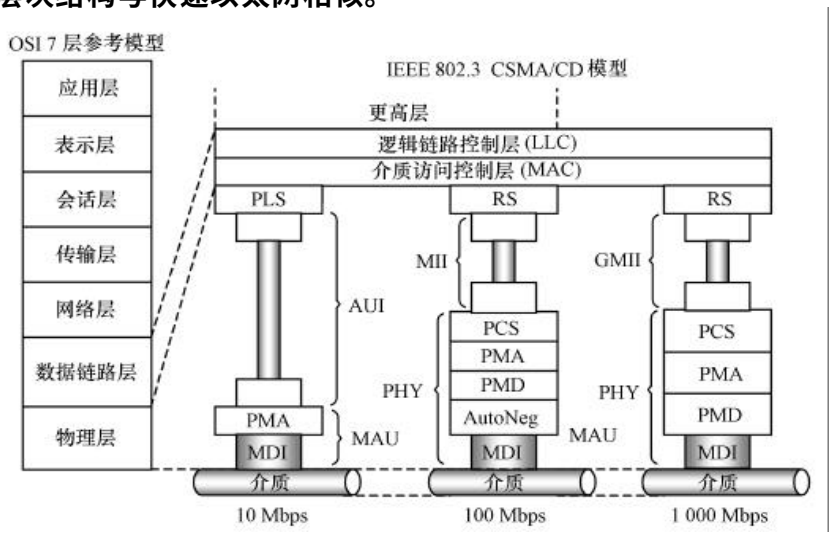


图 5.13 千兆位以太网的层次结构

IEEE 802.3z 标准任务组的首要目标之一，就是保持与现有十兆位以太网和百兆位以太网兼容。在结构上，千兆位以太网也与百兆位以太网相似。需要说明的是：

(a) 千兆介质无关接口 GMII 是千兆 MAC (GMAC) 与物理层间的逻辑接口。GMII 中的数据通路宽度为 8 位，发送和接收时钟工作在 125 MHz，因此数据率为 1 000 Mb/s。它也支持 25 MHz 和 2.5 MHz 的频率，用以支持 100 Mb/s 和 10 Mb/s。

(b) 物理编码子层 PCS 负责数据的发送、接收、同步、载波、侦听和自动协商，并使用 8B/10B 编码方法，将 8 位的数据转换成 10 位在物理介质上传输。

(c) 物理接入子层 PMA 负责将 PCS 传递给物理介质相关子层 PMD 的 10 位并行信号转换成串行位流，也可以把 PMD 提供的串行位流转换成并行的 10 位码交给 PCS。

(d) 物理介质相关子层 PMD 是物理层的最低子层，负责提供到物理介质的接口，在 PMD 子层和介质之间交换串行 8B/10B 编码位流。

2. 千兆位以太网的物理层标准

Gigabit Ethernet 与 Fast Ethernet 的区别仅在于把每个比特的发送时间进一步降低到 1ns。它可以支持多种介质，目前制定了 4 种有关通信介质的标准：

1000Base－SX，支持波长为 850 nm 的多模光纤，光纤长度可达 300～550 m。

1000Base－LX，支持波长为 1 300 nm 的单模光纤，光纤长度可达 3 000 m。

1000Base－CX，支持屏蔽双绞线，双绞线长度可达 25 m。

1000Base－T，支持 5 类非屏蔽双绞线，双绞线长度可达 100 m。

5.4　交换式局域网

5.4.1　局域网的分段与交换式局域网

1. 网段微化

在开始介绍共享介质网络时，读者大多都会担心，要是各个站点同时发送信息，岂不乱套了吗？学习了共享介质系统中多点发送的控制方法，这种担心似乎就小多了。不过，还是不可掉以轻心。实际上，由于多个节点共享同一介质，整个系统处于一个碰撞域范围内，对于有 *N 个节点的网络，每个站点发送的概率*只有 1/N，或者说每个站点平均只占有系统带宽的 1/N。随着站点的增加，碰撞的几率也会增加，监听/检测－退避－监听/检测－退避……又不断出现碰撞，网

络的效率大大降低,用户由于等待时间过长而难以忍耐。再加上退让时间,有可能使网络无法运行。解决这一问题的最初努力就是将网络分割成一些网段,使网络中的一部分站点能够并行地发送数据。

把一个网络分割成子网的设备称为网桥(Bridge),也称桥接器。从另一个角度来讲,使用网桥可以将多个局域网连接起来,实现距离的扩展。网桥的主要用途有:

用于同构型(第 3～7 层完全相同)LAN 间的连接;

扩展工作站的平均占有频带;

扩展 LAN 的地址范围;

进行网段微化,将局域网分段成几个子网,以提高信息流量和网络性能。

网桥可安装在文件服务器上,称为内桥;也可以安装在工作站上,称为外桥。图 5.14 为用网桥连接局域网和远程网的示意图,连接两个远程网时,每端各需要一个网桥。

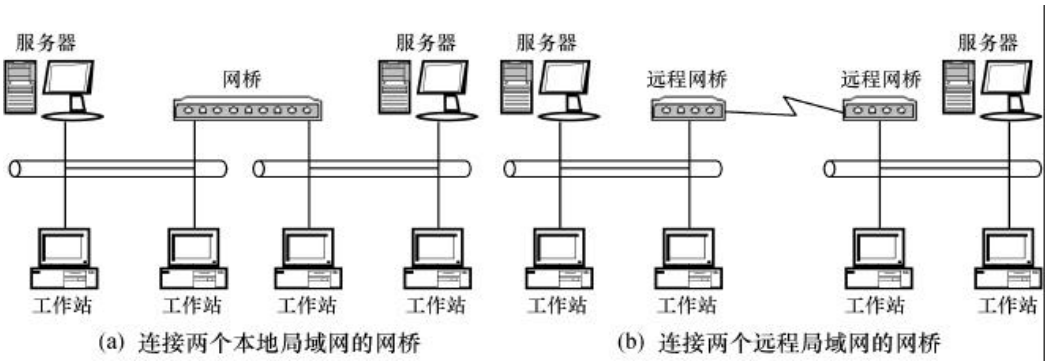


图 5.14　用网桥连接局域网和远程网

2. 网桥的工作原理

简单的网桥有两个端口,复杂的网桥可以有多个端口。下面以图 5.15 所示的两端口网桥为例来介绍网桥的基本原理。

两端口网桥连接两个网段。由于每段都分别是一种广播传送方式,系统工作时,网桥的每个端口都会接收到所连接的网段上传送的各种帧。每收到一个帧,将其先存放在网段的缓冲区中。如校验无差错,且该帧是要发送到别的网段的帧,则通过查找站表,将收到的帧发送到对应的端口,进入相应的网段中。若不是发往别的网段的帧,就将之丢弃,使之不发往别的网段,不增加别的网段的负担。网桥工作的特点是,收到 MAC 帧后,不剥离 MAC 域,只是完整地把帧中继到目的站点所在的网段中。

网桥是通过内部的端口管理软件和网桥协议实体来工作的。

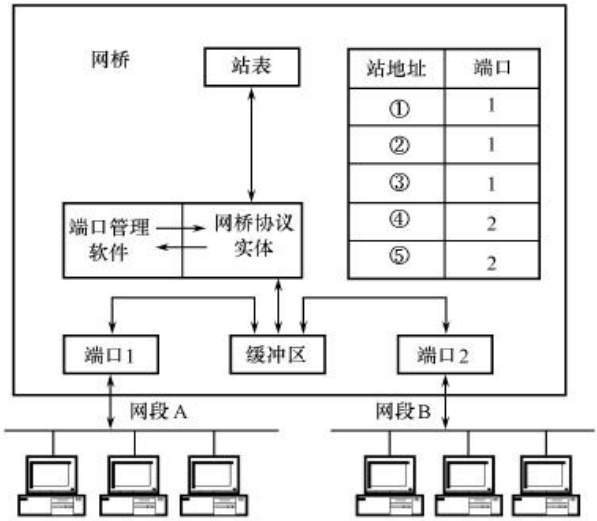


图 5.15　网桥的基本原理

3. 交换式以太网

网段微化减少了站点对总线的竞争，但在较大型的网络中，过多地分割会使得整个网络的结构和管理变得十分复杂，并使成本随之增加。交换式网络则从根本上改变了共享介质工作方式，它可以通过交换机在多端口之间实现多个并发连接，实现多个节点间的并发通信，以增加带宽，改善网络性能和服务质量。图 5.16 示出了一个交换式以太网结构。

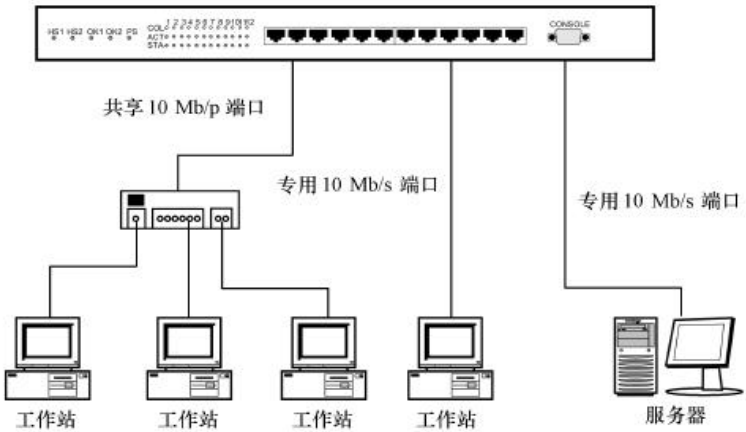


图 5.16　交换式以太网结构

5.4.2 交换式局域网工作原理

1. 交换设备的基本操作

交换技术是相对于共享技术而言的。共享技术是指多个计算机或设备共享一个传输介质。连接在 Hub 中的一个节点发送数据时,将以广播方式将数据传送到 Hub 的每一个端口。这样,当有多个设备同时发送数据时,就会产生碰撞。交换式局域网从根本上改变了“共享介质”的工作方式,它是“有目的地”转发数据,通过交换机端口之间的数据交换,形成多个并发连接,从一个端口进入的数据被送到相应的目的端口,不影响其他端口,因而不会发生碰撞。

交换式网络的核心部件是交换机。目前的局域网交换机主要是针对 Ethernet 设计的。一个交换机能将网络分成几个较小的碰撞域(如以太网)或较小的环(如环网),给每个站提供专用的信息通道,分配更多的带宽。大多数交换机具有将低速网络连接到高速网络的能力。

在第3章中已经介绍过,交换就是将数据从一条链路转发到另一条链路。局域网交换技术是 OSI 参考模型第2层——数据链路层上的交换技术,它所交换的对象是帧,也就是转发帧。

在图5.17所示的交换式局域网中,交换机共有6个端口,其中的4个端口1、4、5、6连接有节点A、B、C、D。交换机中的“端口号-MAC 地址映射表”可以建立端口号与节点的 MAC 地址间的对应关系。端口之间可以根据需要同时建立多条连接。当有两个节点要同时发送数据(如节点A要向节点C、节点D要向节点B发送数据)时,只要分别在帧中写上目标地址,交换机根据“端口号-MAC 地址映射表”就可以实现同时转发。

由上面的讨论可知,一个交换式局域网的交换设备——交换机必须保证两项基本的操作:

交换数据帧;

维护交换地址表。

2. 交换模式

交换机是构成整个交换网络的关键设备。交换机所使用的技术影响着交换网络的性能。根据具体采用的交换技术不同,交换机可分为不同的类型。目前常用3种交换模式:存储转发(Store and Forward)、直通(Cut Through)和无碎片直通(Fragment Free Cut Through)。

(1) 直通模式

直通方式也叫快速转发(Fast-Forward)模式,这种交换机接收到数据帧时,

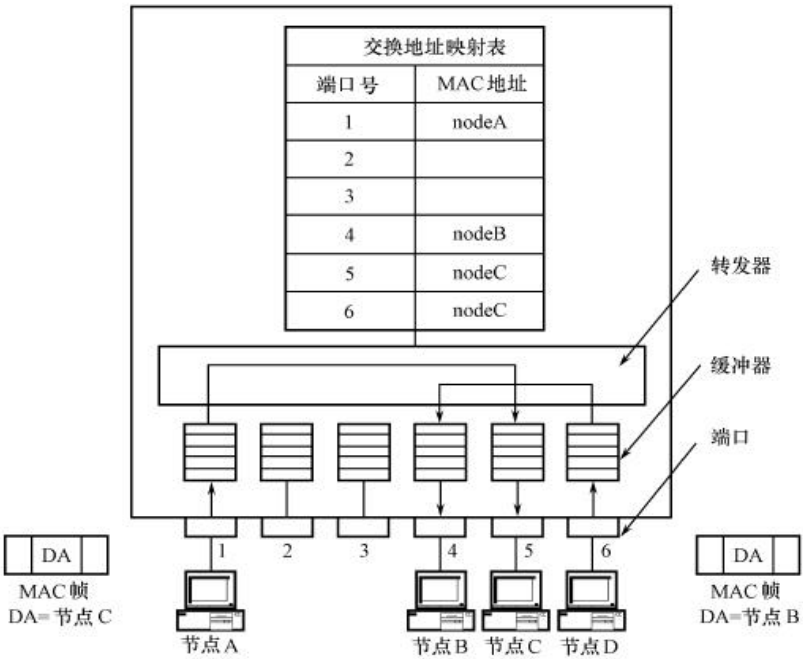


图 5.17　一个交换式局域网的工作过程

找到了的目的地址便立即将帧转发出去，不做差错和过滤处理。

⑵ 无碎片模式

“碎片”是指在信息发送过程中突然发生冲突时，由于双方立即停止发送数据帧在网上形成的残缺不全的帧。碎片是无用的信息垃圾，必须将它们清除。无碎片直通模式首先存储接收到的数据帧的部分字节（前 64 个字节），然后进行差错检验，发现有错，立即滤除，并要求发方重发该帧；如未发现错，则立即转发出去。

⑶ 存储转发模式

存储转发模式是指交换机接收到数据帧后，先存储在一个共享缓冲区中，然后进行过滤（滤掉不健全的帧和有冲突的帧）和 CRC 差错校验，之后才将数据按目的地址发送到指定的端口。如果 CRC 校验失败，即将该帧丢弃。

在 3 种交换模式中，存储转发模式具有较高的交换质量，但要检测的数据段多，处理速度慢，故适合于网络主干的连接。直通模式是其中速度最快的一种，但由于对任何帧都不做过滤处理，因而误码率较高。无碎片直通模式是前两种模式的折中。

3. 交换机与集线器间的区别

交换机的作用是对封装的数据包进行转发，并减少冲突域，隔离广播风暴。

从组网的形式看,交换机与集线器非常相似,但实际工作原理有很大不同。

从 OSI 体系结构看,集线器工作在 OSI/RM 的第 1 层,是一种物理层的连接设备,因而它只对数据的传输做同步、放大和整形处理,不能对数据传输的短帧、碎片等进行有效处理,不进行差错处理,不能保证数据的完整性和正确性。交换机工作在 OSI 的第 2 层,属于数据链路层的连接设备,不但可以对数据的传输进行同步、放大和整形,还提供完整性和正确性的保证。

从工作方式和带宽看,集线器是一种广播模式,一个端口发送信息,所有的端口都可以接收到,容易发生广播风暴;同时它共享带宽,当两个端口间通信时,其他端口只能等待。交换机是一种交换方式,一个端口发送信息,只有目的端口可以接收到,能够有效地隔离冲突域,抑制广播风暴;同时每个端口都有自己的独立带宽,两个端口间通信不影响其他端口间的通信。

4. 交换机与网桥的比较

局域网交换机(Ethernet Switch)在 OSI/RM 的第二层(或者说在 IEEE802.2 层)上运行,采用帧交换技术。它在功能上与多口网桥有些类似,但有显著不同。网桥一般用于连接各个网段,而局域网交换机既能连接网段又能连接单机,并且多是连接单机的。局域网交换机提供辨认 MAC 地址能力,将主机和它所连接的端口相联系,在收到数据帧时能根据目的地址将其转发到相应的端口。

5. 交换机和路由器、网桥相互之间的比较

(1) 要区别交换机和路由器,首先要区分广播域和碰撞域。交换机的主要功能是将 LAN 的碰撞域分段成一些较小的碰撞域,用于满足一个组织的各部分对带宽的要求。例如,可以将 100 Mb/s 的带宽分配给一些 100 Mb/s、10 Mb/s 的用户,但它们还属于同一广播域,一个广播域中产生的业务,仍然转发给它的各碰撞域。路由器将网络分解成多个广播域,它的数据包过滤作用,只允许特定的数据包通过,限制了广播风暴扩散的可能性,限制了不支持协议的数据包的发送,控制网络间的数据传输流量,并限制了以未知网络为目的地的数据包的发送,可以提供防火墙服务用的过滤方法。

(2) 路由器是位于第 3 层的网际的互联设备,可用于不同网络的互连。多协议路由器不仅可以实现异构网(不同类型局域网)间的互连,还可以实现局域网与广域网间的互连。总之,路由器所连接的物理网络的低 2 层协议可以各不相同,但第 3 层必须相同。多协议路由器可以实现网络层协议转换,它通过支持多种协议数据的转发,实现异种网互连。一般说来,异种网络互连与多个子网互连都应采用路由器来完成。因此路由器比交换机需要更多的信息。

而交换机的转发策略只按每一帧中的 MAC 地址相对简单地决定转发目的

地,不考虑数据帧中隐藏更深的其他信息。

(3) 网桥和交换机仅支持广播型拓扑结构,不支持环路型拓扑结构。路由器可以支持复杂网络拓扑结构。

(4) 路由器是一种具有一定智能的通用设备,它具有智能化的分组转发功能,可以选择最优路径转发分组,也可以限制路由选择信息的传播,藉此可以进行分组滤波,提供网络安全。交换机则是一种专用设备,它所连接的各碰撞域属于同一广播域,主要功能是提供附加带宽,能进行带宽分配。

(5) 从过滤网络流量的角度来看,路由器的作用与交换机和网桥非常相似。但是与工作在网络物理层、从物理上划分网段的交换机不同,路由器使用专门的软件协议从逻辑上对整个网络进行划分。例如,一台支持 IP 协议的路由器可以把网络划分成多个子网段,只有指向特殊 IP 地址的网络流量才可以通过该路由器。对于每一个接收到的数据包,路由器都会重新计算其校验值,并写入新的物理地址。因此,使用路由器转发和过滤数据的速度往往要比只查看数据包物理地址的交换机慢。但是,对于那些结构复杂的网络,使用路由器可以提高网络的整体效率。

应当说明的是,目前交换技术同路由技术正在相互融合,形成了交换式路由器和路由式交换机。

6. 局域网交换机的类型

目前,Ethernet 的物理层协议主要对应于 10 Mb/s、100 Mb/s、1 000 Mb/s 三种传输速率,所以 Ethernet 交换机有以下几种:

10M 交换机——只支持 10 Mb/s 端口。

100M 交换机——只支持 100 Mb/s 端口。

10M+100M 交换机——一部分端口支持 10 Mb/s,一部分端口支持 100 Mb/s。

100M+1 000M 交换机——一部分端口支持 100 Mb/s,一部分端口支持 1 000 Mb/s。

10M+100M+1000M 交换机——一部分端口支持 10 Mb/s,一部分端口支持 100 Mb/s,一部分端口支持 1 000 Mb/s。

有些交换机还带有 10 Mb/s/100 Mb/s 的自适应端口,有的厂家(如 Luncent Technologies)将交换机做成模块式的,插上不同的模块,就成为不同的端口。

5.4.3　虚拟局域网 VLAN

近年来,交换局域网由于其性能较高而交换成本低廉,已经基本上取代了共享介质局域网,得到了广泛的应用,同时也促进了交换技术的发展。在此基础上,

一种新的网络技术——虚拟局域网 VLAN (Virtual LAN) 又呈现在人们的面前。

VLAN 建立在交换技术的基础上,通过交换机“有目的地”发送数据,灵活地进行逻辑子网 (广播域) 的划分,而不像传统的局域网那样把站点束缚在所处的物理网络之中。

VLAN 的形成方法很多,区别主要在对其成员资格的定义上。下面介绍 4 种通用的方法。

1. 端口 VLAN

图 5.18 所示为两种形式的端口 VLAN:单交换机 VLAN 和多交换机 VLAN。它们都是 LAN 交换机。可以简单明了地看出,通过交换机的端口定义,可以将连接在一台交换机上的节点划分为不同的子网,也可以将连接在不同交换机上的节点划分在一个子网中。

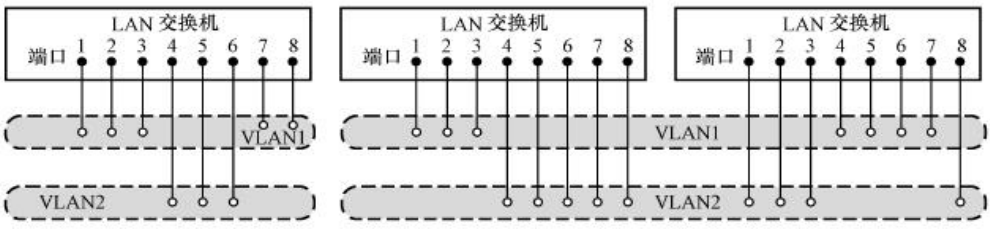


图 5.18　端口定义 VLAN

端口定义 VLAN 是早期的 VLAN 定义方法。它虽配置简单,但有一定的限制:不允许不同的 VLAN 包含相同的物理网段,并且要将用户从一个端口移动到另一个端口时,必须对 VLAN 的成员重新配置。

2. MAC VLAN

MAC VLAN 是一种基于用户的 VLAN,它用终端系统的 MAC 地址来定义 VLAN。由于 MAC 地址是与硬件相关、固定于工作站的网络接口卡 (NIC) 之内的,因此 MAC 定义的 VLAN 允许工作站移动到网络的其他物理网段中,又能保持原来的 VLAN 成员资格,因为它的 MAC 地址没有变。

MAC VLAN 的不足之处在于,它要求所有用户在初始阶段必须配置到至少一个 VLAN 中;初始配置必须由人工完成,然后才可以自动跟踪用户。这对用户较多的大型网络是非常烦琐的。

3. 第三层 VLAN

第三层 VLAN 是用物理层地址或协议类型 (若支持多协议) 来定义 VLAN 的成员资格。在第三层定义 VLAN 有以下优点:

减少交换机之间交换 VLAN 成员信息的工作量；

用户可以随意移动工作站而无须重新配置网络地址，特别适合 TCP/IP 用户（对其他协议，如 IPX、DECnet 要差一些）；

按协议类型定义 VLAN 成员，有利于组成面向业务或应用的 VLAN。

5.5　无线局域网 WLAN

5.5.1　WLAN 传输方式和频段范围

一般而言，凡是采用采用无线传输介质的计算机局域网都称为无线局域网 WLAN（Wireless Local Area Network）。表 5.3 列出了在 WLAN 中使用的无线频段范围与其他物理参数。

表 5.3　WLAN 介质物理参数

频　段	1 GHz～3 GHz		10 GHz～30 GHz	红　外		
传输技术	窄带调制	扩展频谱	窄带调制	定向波束红外线（DB/IR）方式		扩散红外线（DF/IR）方式
				点对点方式	反射方式	
传输速度	数百 kb/s～10 Mb/s		＞100 Mb/s	可达 50 Mb/s	可达 10 Mb/s	几十 kb/s～10 Mb/s
通信距离	100 m（无须视距）		几十米（无须视距）	＞50 m（视距）	—（无须视距）	数米～20 米（无须视距）
移动支持	一般	好	一般	不支持	不支持	差
成本	低	高	高	高	高	低
使用许可	需要	ISM 频段不需要	需要	不需要	不需要	不需要

由表中可以看出，目前 WLAN 的传输速率可以从几百千比特到十几兆比特，其传输方式基本上有三种。

1. 扩频方式

在扩频方式中，数据基带信号的频谱被扩展几倍到几十倍后被射频发射出去。这种方式以牺牲带宽为代价，换来通信系统高抗干扰能力和高安全性，并由于单位频带内的功率降低而减小了对其他电子设备的干扰。扩频方式的 WLAN 一般采用 ISM 频段（即工业、科学、医学频段——Industrial，Scientific，Medical 频

段)。欧美日等国家各设置有自己的 ISM 频段范围,如美国联邦通信委员会 FCC 规定的 ISM 频段为 902～928 MHz、3.4～3.484 MHz、4.725～4.850 MHz 三个频段。若发射功率以及带外辐射超过 FCC 的规定,应当向 FCC 提出专门申请。

2. 窄带调制方式

窄带调频方式的信号频谱不作任何扩展地直接发射出去。与扩频方式相比,它的频带利用率高,但要选用专用频段,并需要经 FCC 批准。

3. 红外线方式

用红外线作为 WLAN 的传输方式最大优点是不受其他无线电信号的干扰,也不对其他无线电设备造成干扰,因而不受国家无线电管理委员会的限制。但是,它几乎不能穿透非透明物体,使其应用受到较大限制。采用红外线作为通信介质,适合于低成本、跨平台、点到点的高速数据连接,尤其适合嵌入式的系统和设备,主要应用于设备互连(完成不同设备内文件与数据的交换)、信息开关(数据终端与 Internet 互连)。

5.5.2　无线局域网的结构

连接在无线局域网中的设备通常称为站。按照移动性可以把站分为三类。

固定站:如台式计算机和其他有线局域网中的设备。

半移动站:经常改变使用场所的站,一般在移动状态下不需要保持与网络通信。

移动站:在移动过程中也需要与网络通信的站,如车载计算机等。

无线局域网可以在普通局域网的基础上通过无线 Hub、无线接入站(AP)、无线网桥、无线 Modem 以及无线网卡等实现,形成不同的网络结构。下面介绍几种结构形式。

1. Hub 接入型

采用无线 Hub 可以组建星型 WLAN,并可以在此基础上组建类似于交换以太网工作方式的 WLAN。

2. 基站接入型

这是一种采用移动蜂窝通信网接入方式组建 WLAN 的方式。这时,各站点之间是通过基站接入－交换数据、互相连接。利用这种方式,可以实现各移动站通过交换中心的自组网,还可以通过广域网远地站点组建自己的工作网络。

3. 网桥接入型

网桥接入型可用于实现不同局域网之间的互连。它不仅提供两个局域网之

间的物理层与数据链路层的连接,还为用户提供较高层的路由和协议转换。

4. 无中心接入型

无中心结构允许网中任意两个站点间直接通信,是一种分布式对等结构方式。

5.5.3　IEEE 802.11 协议

IEEE 802 委员会的 802.11 工作委员会成立于 1990 年 7 月,1998 年完成 802.11 的制定工作。

如图 5.19 所示,IEEE 802.11 规定 WLAN 的最小构件是基本服务集 BSS (Basic Service Set)。在一个 BSS 内,所有的站均运行同样的 MAC 协议并且以争用方式共享同一介质。一个 BSS 可以是独立的系统;也可以通过接入点 (Access Pointer) 连接到主干网上,与其他 BSS 连接或与有线网络连接,组成扩展服务集 ESS (Extended Service Set)。

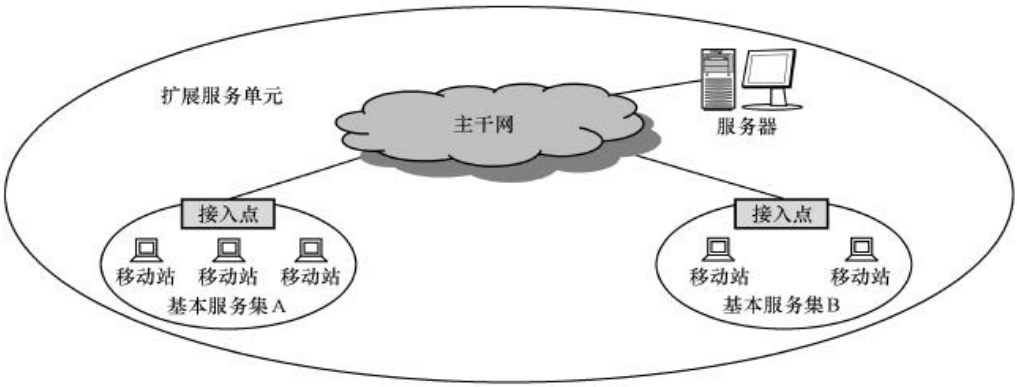


图 5.19　IEEE 802.11 结构

WLAN 协议栈是 IEEE 802 模型的一个子集。它的标准化也主要表现在 LLC 以下——MAC 层和物理层。这两层的标准由 IEEE 802.11 定义。

1. 物理层

IEEE 802.11 协议定义了 WLAN 所使用的无线频段以及调制方式,并进一步分为 802.11、802.11b 和 802.11a 三种类型:

IEEE802.11 使用 2.4 GHz 频带,传送速率为 1 Mb/s;

IEEE802.11b 使用 2.4 GHz 频带,标称传送速率为 11 Mb/s,实际为 7 Mb/s ~ 8 Mb/s;

IEEE802.11a 使用 5 GHz 频带,传送速率为 54 Mb/s。

2.4 GHz 频带是一个容易受微波炉、无绳电话和其他无线设备干扰的频带,

5 GHz 频带是一个干扰较小的频带。

下面说明 WLAN 的三种主要物理层实现方法。

(1) 跳频扩频 (FHSS)

跳频扩频使用 2.4 GHz 的 ISM 频带,共有 79 个信道频道可供跳频使用,第一个频道的中心频率为 2.402 GHz,以后每隔 1 MHz 为一个信道。视不同的频移键控技术,基本接入速率为 1 Mb/s 或 2 Mb/s。

(2) 直接序列扩频 (DSSS)

直接序列扩频使用 2.4 GHz 的 ISM 频带,接入速率为 1 Mb/s 或 2 Mb/s。

(3) 红外 (IR)

红外方式使用波长 850～950 nm 的红外线传送数据,速率为 1～2 Mb/s。

2. MAC 层

从原则上讲,IEEE 802.11 的 MAC 层协议与有线局域网的 MAC 协议并无本质上的区别。在图 5.20 中给出了 IEEE 802.11 的 MAC 层结构,称为 DFWMAC (分布式基础无线网 MAC)。它可以为本地链路控制层提供两种服务:竞争服务和无竞争服务。

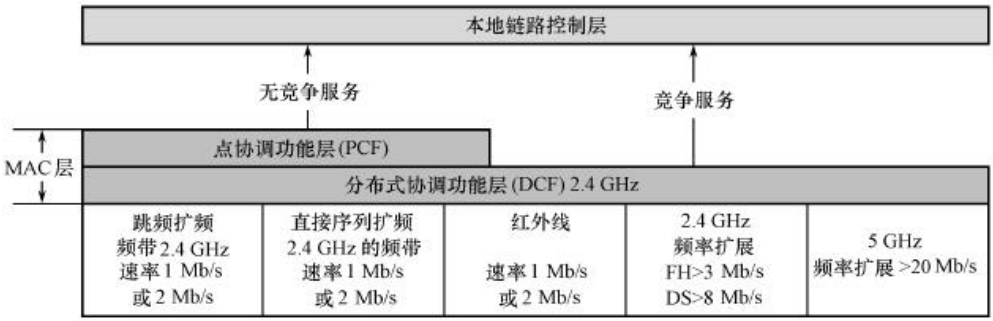


图 5.20 IEEE 802.11 协议结构

(1) 竞争服务

在有竞争的情况下,WLAN 像以太网一样,用载波侦听的方法将访问介质的决定分布到每个节点。但是由于在无线局域网上信号的动态范围很广,发送站难于有效地识别是噪音还是自己发送的信号,因而要检测冲突不现实,无法沿用原有的 CSMA/CD,而是采用了带有冲突避免的载波多路侦听协议 CSMA/CA (Collision Avoidance) 作为 MAC 层的协议。

CSMA/CA 并不能完全避免冲突,但可以减少碰撞几率。如图 5.21 所示,CSMA/CA 的访问规则如下:

任何一个站点在发送数据之前,还是要先监听载波,确认信道空闲时,发

送探询帧,仅当信道空闲一个 IFS (帧间隙) 的间隙时间后仍然空闲,才发送数据。

如果介质忙 (包括侦听中发现忙、在 IFS 时间内发现忙) ,站点要推迟一个随机时间后重新尝试。

一旦当前的数据传送完毕,站点要再延迟一个 IFS 时间 如果在这段时间内介质仍然忙,站点就使用二进制退避算法并继续监听介质,直到介质空闲。

接收端在收到数据后,等信道空闲一个 IFS 时间后才发出回答帧,否则推迟一个随机时间后重新尝试。

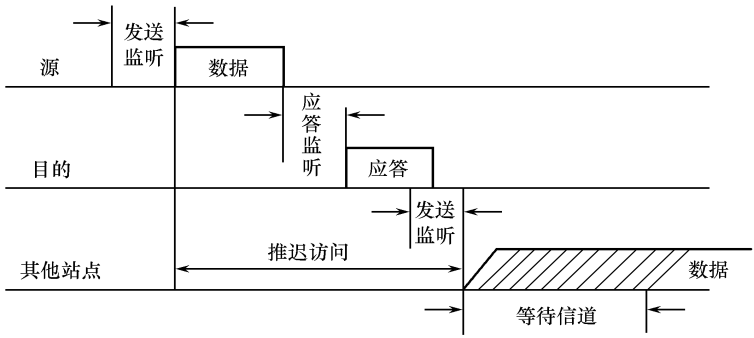


图 5.21　发送站点使用 IFS 的 CSMA 访问规则

分布式基础无线网 MAC 算法 DFWMAC 是一种分布式访问控制机制,由 MAC 的分布式协调功能子层 DCF 实现。为了实现优先级机制,DCF 使用了三种不同长度的 IFS。

SIFS (短 IFS) :最短 IFS,用于优先级最高、需要立即处理的情况,如发送确认 (ACK) 帧、发前清除 (CTS,先发一个小的发送要求——RTF,成功后接收方立即用 CTS 响应,迫使其他站点推迟对介质的占用) 以及轮询响应 (在无竞争服务中使用) 等。

PIFS (点协调 IFS) :中等优先级,在无竞争服务中使用。

DIFS (分布式协调 IFS) :最长 IFS,用于异步竞争访问的最小延时。

(2) 无竞争服务

无竞争服务采用集中访问控制,包括集中轮询主管的轮询,由一个中央的决策者协调访问请求,实现可以选择的访问——点协调功能 PCF。这种机制适合于下列情形:

几个互连的 WLAN;

一个与有线主干网相连的基站;

实时性强的点;

高优先级的站点。

PCF 在 DCF 的顶部实现，它在发出轮询时使用 PIFS，将所有异步帧都排除在外，并使优先级高的站点可以先发送。

习　　题

5.1　简述 CSMA/CD 工作原理以及其 MAC 帧结构中各字段的功能。

5.2　试述令牌总线网中逻辑环的形成。

5.3　试比较十兆、百兆、千兆位以太网层次结构和实现技术的异同。

5.4　为什么各种以太网都要定义多种物理层标准？

5.5　标准以太网的波特率是多少？

5.6　试比较 CSMA/CD 与 CSMA/CA。

5.7　比较网桥、路由器，指出它们的区别和联系。

5.8　查找资料，分析各高速以太网提高速率的技术手段。

5.9　局域网中都采用了哪些编码？这些编码各有什么特点？

第 6 章　Internet 与 TCP/IP 体系结构

6.1　Internet 概述

表面上看，Internet 与广域网差不多，但实际上相差很远。广域网是一个比较独立的网络，拥有一个连接非常紧密的结构，并且有一个专门的机构对整个网络进行维护。而 Internet 是由成千上万个网络松散地连接而成的，它不属于任何一个国家、部门、单位、团体，也没有一个专门的机构对它进行维护；任何一台计算机，任何一个网络，只要遵守共同的规则——TCP/IP，就可以与它连接在一起。所以，有人说它是一个虚拟网，也有人说它是一个网上之网。图 6.1 是 Internet 的结构示意图。

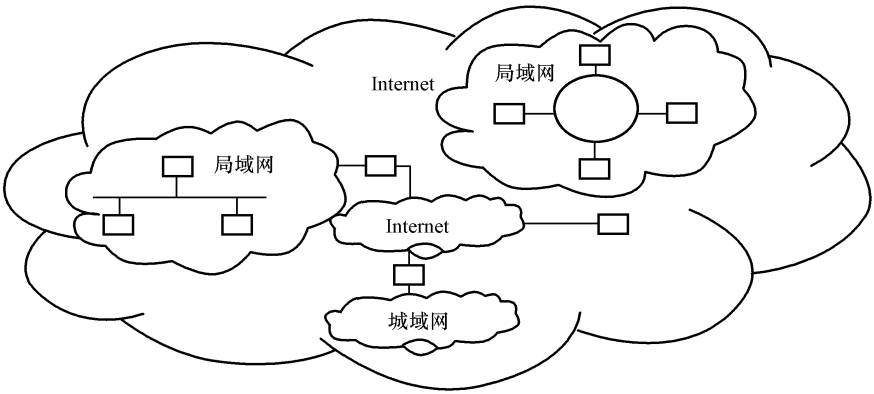


图 6.1　Internet 结构示意图

6.1.1　TCP/IP 概述

在 Internet 中包含的网络是形形色色的，它们的硬件组成不同，运行的协议也不同。要将它们连接起来协调工作，就需要一个大家都公认的协议。传输控制协议和网际协议 TCP/IP (Transmission Control Protocol/Internet Protocol) 就是这样的协议簇。它是美国国防部高级研究计划署为实现美国本土广域互联网

APARNet 而开发的通信传输协议。APARNet 于 1969 年 12 月投入运行，后来发展成为洲际 Internet。由于 Internet 的成功应用，TCP/IP 已成为世界公认的事实上的网络标准。它的特点是适应异种计算机和异种计算机网间的通信。随着 Internet 的广泛应用和被全世界承认，Internet 所使用的 TCP/IP 体系结构在计算机网络领域亦占有特殊重要的地位，成为一种实际的标准。TCP/IP 模型是在物理网基础上建立的，包括网络接口层、网际层（IP 层）、传输层（TCP 层）和应用层。

图 6.2 所示为 TCP/IP 的体系结构，同时给出了它与 ISO/OSI 之间的大致对应关系。其核心是 IP 层和 TCP 层。

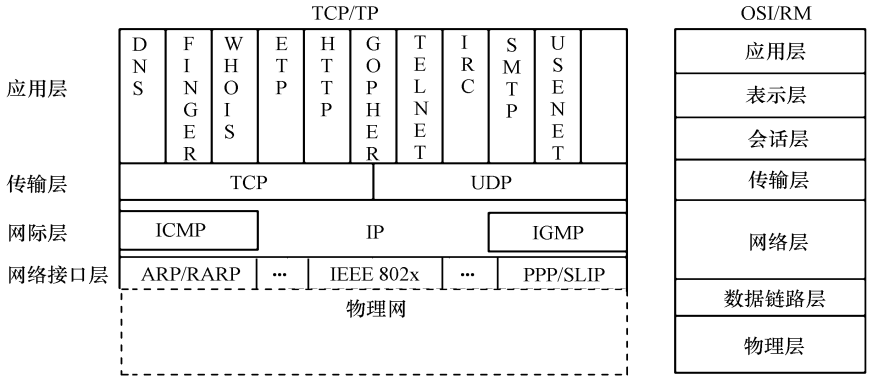


图 6.2　TCP/IP 四层参考模型

1. 网络接口层

TCP/IP 协议栈结构没有包含数据链路层和物理层，因此不能完成计算机网络的全部功能，必须与其他协议协同工作。网络接口层负责将 IP 分组封装成适合在具体的物理网络上传输的帧结构并交付传输。它包括：

用于协作 IP 分组在现有网络介质上传输的协议，如 IEEE 802.x；

IP 地址与实际物理网络地址间的转换协议——ARP（Address Resolution Protocol）和 RARP；

用于串行线路连接主机与网络或连接网络与网络的 SLIP 协议和 PPP 协议。

2. 网际层

网际层的主要作用是解决网络互连中的问题，即网际寻址（包括地址格式、地址转换等）。主要协议有：

IP 协议，规定一种统一的地址格式——协议地址；

网际控制信息协议（ICMP，Internet Control Message Protocol），提供差错报告机制；

路由信息协议（RIP，Routing Information Protocol），RIP 协议负责数据的包装、寻址和路由。

3. 传输层

传输层负责维护信息的完整性，它提供端到端的通信服务，即提供一个应用程序到另一个应用程序之间的通信服务。传输层有如下功能：

解决不同应用程序的识别问题，要附加从何处（信源）来到何处（信宿）去的应用程序信息；

提供可靠传输，为此传输层的每个分组均含有校验字段；

对信息流进行格式化。

传输层提供传输控制协议 TCP（Transmission Control Protocol）和用户数据报协议 UDP（User Data Protocol）。

4. 应用层

TCP/IP 的应用层是几个可以在各种机型上广泛实现的协议，如文件传输协议 FTP、远程终端访问协议 Telnet、域名服务程序 DNS、简单邮件传输协议 SMTP 等。

6.1.2　TCP/IP 与 OSI/RM 的比较

OSI/RM 与 TCP/IP 是两个独立协议体系结构模型，有许多相似之处，如主要层上有一定的对应关系。但是，OSI/RM 是作为计算机网络的标准制定出来的，TCP/IP 则是在 Internet 发展的实践中产生的。两者的背景不同，出发点不同，并且各具特色。除表现结构上的不同之外，还需要说明几点。

1. 层次性是否严格

OSI/RM 最大的贡献在于它作为一种理论模型，有清晰的层次结构，并且用服务、接口和协议三个基本概念作为每一层的核心。

协议：同层实体之间通信的规程。

服务：下一层实体对上一层实体的支持。

接口：上下层之间的通信界面。

OSI/RM 先于 TCP/IP 制定出来，并且它不基于某个具体的协议集。这使得 OSI/RM 非常一般化，并且制定者对将哪些功能放在哪一层还缺乏更多的实践经验。OSI/RM 这样强的层次结构缺少灵活性。例如它原先是基于点到点的结构考虑的，当出现广播式结构时，就需要插入一个新的子层，这在介绍局域网时已经看到。

TCP/IP 是实践中形成的，是经验的总结，虽然 TCP/IP 模型也分层次，但是层次间的依赖关系不像 OSI/RM 那样强。在 OSI/RM 中，上层实体之间的通信，

必然涉及下一层实体,并且要在下一层服务的支持下进行,不相邻层间的协议是隐藏的,随着技术的变化具有较好的可替换性。TCP/IP 模型则不同,它在传输层上面只设了一层——应用层,使结构比较实用,并且允许上一层越过紧靠的下一层去使用更低层提供的服务功能,这样有利于减少不必要的开销,提高协议的效率。

2. 可靠性第一还是效率第一

可靠性是指网络正确地传输数据的能力。OSI/RM 以可靠性第一作为其基本宗旨,每一层都要进行差错检测和处理,因而按 OSI/RM 构建的网络可以适应较恶劣的传输条件,但额外传输开销大,传输效率低。

TCP/IP 模型则以效率第一作为其基本宗旨,它不是在每一层中都进行差错检测和恢复,特别是在通信子网中不对传输数据进行差错检测和恢复;它把可靠性看做传输层的任务,丢失和损坏数据的恢复只由传输层完成,即由主机承担,而通信子网对可靠性不做检查。

3. 主机负担重还是通信子网负担重

OSI/RM 认为通信子网是提供传输信息的设置,它把与传输有关的问题都放在通信子网中处理,如监视数据流量、控制网络访问、记账收费、路径选择、流量控制等问题的处理,而给主机留下的事情并不太多。同时,它在运输层只提供面向连接的服务。因此,OSI/RM 系统中通信子网负担较重,主机负担较轻,即 OSI/RM 对主机的要求不高。

TCP/IP 主要的功能是实现异种网之间的互连。它没有定义自己的通信子网,而以其它通信子网作为考虑的基础并把它们都看成是不可靠的,不管通信子网是复杂还是简单,都要在传输层要求主机参与几乎所有复杂性问题的处理过程。因此,在 TCP/IP 模型中主机的负担较重。

4. 异种网互连的能力

TCP/IP 模型从一开始就考虑到异种网的互连问题,并用网际层来处理异种网的互连问题。而 OSI 最初并没有考虑网络互连问题,后来才不得不打一个补丁——在网络层中划分出一个子层来完成类似网际层的功能。

6.2 网络接口协议 SLIP 和 PPP

网络接口层的重要功能是将 IP 分组封装成适合在物理网络上传输的帧格式进行传输。SLIP 协议和 PPP 协议就是使用串行线路连接主机与网络,或连接

网络与网络的两个协议。

6.2.1　SLIP 协议

SLIP (Serial Line Internet Protocol) 提供在串行线路上封装 IP 分组的简单方法，使得远程用户通过电话线路和高速调制解调器方便地接入 TCP/IP 网络。图 6.3 表明了 SLIP 协议对 IP 分组的封装方法。

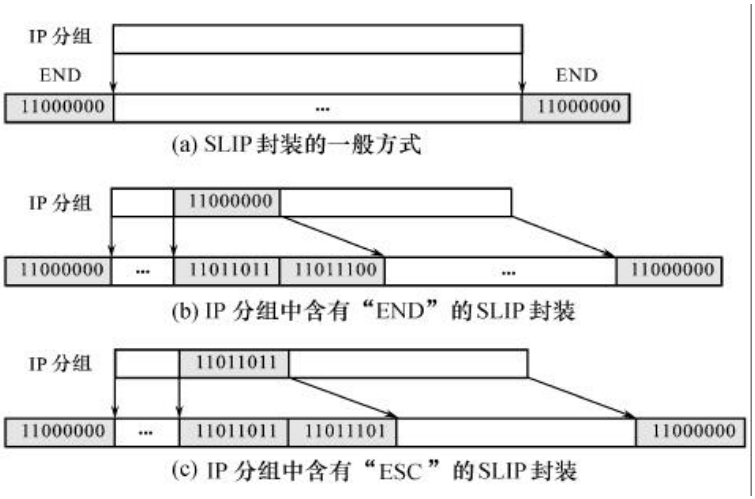


图 6.3　SLIP 封装方式

下面总结 SLIP 封装的基本方法：

(a) 在 IP 分组的前后各加一个称为 END 的特殊字符 (0xc0——11000000)，如图 6.3 (a) 所示；

(b) 当 IP 分组中含有“11000000” (END) 时，应在其前填充 0xdb (SLIP ESC)，并在传输时用 0xdb+0xdc——11011011 11011100 代替，如图 6.3 (b) 所示；

(c) 当 IP 分组中含有“11011011” (ESC) 时，应在其前填充 0xdb (SLIP ESC)，并在传输时用 0xdb+0xdd——11011011 11011101 代替，如图 6.3 (c) 所示。

SLIP 提供了一个简单的组帧方法，但是它存在如下一些问题：

SLIP 不支持在连接建立过程中动态分配 IP 地址，通信双方必须事先知道对方的 IP 地址，但又不可能为每一台连接的计算机分配一个 IP 地址；

SLIP 帧中无协议类型域，因此只能支持 IP 协议；

SLIP 帧中无校验字段，因此必须选用有校验功能的 MODEM，或由上层进行差错处理。

这些问题给实际应用带来不少限制。

6.2.2　PPP 协议

PPP (Point－to－Point Protocol) 是串行通信线路上一个有效的点对点通信协议，它的格式与 HDLC 相似，如图 6.4 所示。

字节数	1	1	1	1 或 2	≥1 600	2 或 4	1
	标志	地址	控制 00000011	协议	信息数据	校验 (CRC)	标志

图 6.4　PPP 帧格式

可以看出，PPP 帧格式在 IP 分组之上增加了起始和结束标志、地址、控制、协议和校验等字段，下面进一步对这些字段加以介绍。

(1) 起始和结束标志

PPP 协议帧的起始和结束标志都是“01111110” (0x7e) 。如果在信息字段中出现 0x7e，就要对该信息字段进行填充：在同步数据链路中，采用逢连续 5 个 1 就加 1 个 0 的比特填充法；在异步数据链路中，采用以 0x7d 为转义字符的字符填充法，同时要将第 6 个字符变反。例如，字符 0x7e 要转换成 0x7d＋0x5e (01111101 01011110) ，而字符 0x7d 要转换成 0x7d＋0x5d (01111101 01011101) 。

(2) 地址字段

地址字段的值总是 0xff，这是一个广播地址，表示将该帧发向所有的站点。

(3) 控制字段

控制字段的缺省值为 0x03，这时帧为无编号帧，即不提供使用帧编号和应答的可靠传输机制。

(4) 协议字段

协议字段的编码值指明信息字段中携带的数据的协议类型，使得 PPP 可以支持多种协议，其缺省长度为 2 个字节，可以用 LCP 协商成 1 个字节。

(5) 帧校验字段

设置帧校验字段，使 PPP 协议在链路上具有差错检测功能。

PPP 协议的上述优点，使其将 SLIP 协议取而代之。

6.3　网　际　层

网际层是 TCP/IP 协议簇中最重要的一层。它的主要功能是网络互联，或者

说是解决网络互联中的各种问题。

IP 协议等同地看待所有的物理网络，它定义了一个抽象的“网络”，屏蔽了物理网络连接的细节，为众多的不同类型的网络和计算机提供了一个单一的、无缝的通信系统。正因为如此，才把多个网络连成一个互联网。

这一节介绍这些功能。

6.3.1　IP 地址结构

Internet 是由许许多多的物理网络组成的网上之网，其中每一个小型网络都是由信道和节点组成。由于两个节点可能在同一个物理网络之中，也可能不在同一个物理网络之中，因此关键就是如何从源节点出发找到目标节点，这就是寻址问题。

寻址问题主要涉及 Internet 的地址结构、有关地址的协议以及对寻址过程进行控制的路由机制。

1. IP 编址方案

在 IP 层中传输的协议数据单元是 IP 分组，也称 IP 数据报。IP 分组从源主机向目的主机传送，依据就是 IP 地址。IP 地址是 Internet 协议地址的简称，用作 Internet 网络上独立的计算机惟一标识。通信时要利用 IP 地址来指定目的机地址，就像电话网中每台电话机必须有自己的电话号码一样。

IP 协议提供整个 Internet 通用的地址格式。为了确保一个 IP 地址对应一台主机，网络地址由 Internet 注册管理机构网络信息中心 NIC 分配，主机地址由网络管理机构负责分配。如图 6.5 所示，每个 IP 地址占用 32 位，并被分为 A、B、C、D 和 E 五类，分别用 0、10、110、1110 和 11110 标识。

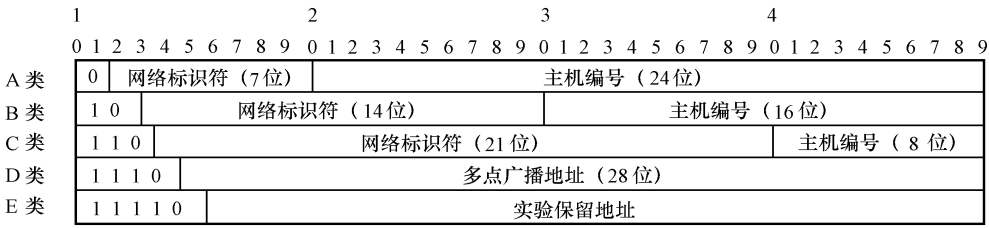


图 6.5　五类 IP 地址结构

IP 地址是 32 位的二进制地址，例如某地址为

10000000 00001010 00000010 00011110

由于它以“10”打头，所以是一个 B 类地址。

IP地址太长,而且不便于记忆,因而常用4个十进制数分别代表4个8位二进制数,在它们之间用圆点分隔,以X.X.X.X的格式表示,称为点分十进制计数法(Dotted Decimal Nottion)。

上述地址可以写为

128.10.2.30

其网络地址为128.10,网络内主机地址为2.30。

A、B、C是3类基本地址类型,都由3个部分IP数据组成:类型标志、网络标识符(Netid)和主机编号(Hostid)。这3类基本地址类型的区别仅在于网络大小不同。

A类地址是给巨型网络分配的IP地址,它用1位"0"作为类型标志,Hostid占24位,网内主机可达1 600万个;Netid占8位(实际是7位),因而在Internet中可以有126个(除去127)具有A类地址的网络,如ARPANet、NSFNet等。

B类地址是给大型网络分配的IP地址,它用"10"作为标志,Hostid占16位,网内主机最多65 534个;Netid占16位(实际是14位),取值范围为128.1~191.254,最多网络数为16 384个。

C类地址是给小型网络分配的IP地址,它用"110"作为标志,Hostid占8位,网内主机最多可达254个;Netid占24位(实际是21位),一般可以选用192.1.1~223.254.254之间的数,最多网络数达200万个。

D类地址是一种多址广播地址格式,用4位的"1110"作为标志。E类地址是为实验保留的地址。

2. 子网掩码

任何一个A、B、C类地址都对应着一定规模(主机数目)的网络。当某实际的网络规模接近IP地址的Hostid上限时,该IP地址就得到充分利用。如一个实际网络中的主机数为250台左右时,申请一个C类地址最为合理。若实际的网络规模较小,如只有30台左右的主机时,独自占用一个C类地址会造成地址资源的浪费,而改用D类地址又会给网关的路由选择表和寻址增加很大复杂性,较合理的做法是将一个C类地址分给若干个小的网络共同使用。具体办法是将这些较小的网络看作一个网络的子网,并从Hostid域中借用某几位高位作为子网的Subnetid域。于是,网关(连接物理网络的路由器)的路由表就被分成两级:先识别由Netid标识的路由,以确定一个逻辑的网络;再在该逻辑的网络内部用Subnetid来确定具体的子网。

网络管理员使用子网掩码来借用Hostid域中的某几位高位作为子网的Subnetid域,即用一个与IP地址格式相同的屏蔽码对网络IP地址进行"与"操作,用来限定一个网络的IP地址范围。利用子网掩码,可以把一个大的网络划

分为几个子网。

当网络中没有子网时，A、B、C 三类网络的缺省掩码分别为：

A 类网络，11111111.00000000.00000000.00000000，即 255.0.0.0；

B 类网络，11111111.11111111.00000000.00000000，即 255.255.0.0；

C 类网络，11111111.11111111.11111111.00000000，即 255.255.255.0。

这些数据与同类型的网络地址进行“与”运算时，所得出的值不变。而当网络中有子网时，就要在上述三类网络缺省掩码的“0”的部分从高到低占用几位。

例 6.1　将一个有 256 台主机、网络号为 200.15.192 的 C 类网络分为 2 个相同的各拥有 128 台主机的子网。

在网络号为 200.15.192 的 C 类网络中，主机的编号为 200.15.192.0～200.15.192.256。由于要将一个 C 类网络分为两个子网，因而要在其 Hostid 域中借用最高一位，子网掩码由 C 类的缺省掩码 255.255.255.0 变为 255.255.255.128，即

11111111.11111111.11111111.10000000

由此类推，要划分为 4 个子网，应借用 2 位；要划分为 8 个子网，应借用 3 位；……

例 6.2　对于一个 C 类网络 202.113.240，可以使用子网掩码 255.255.255.224 划分为 8 个子网，每个子网有 32 个 IP 地址：

202.113.240.0～202.113.240.31

202.113.240.32～202.113.240.63

……

202.113.240.224～202.113.240.255

应当注意，不管如何划分，一个网络中可容纳的主机总数不会增多。

6.3.2　IP 地址解析

1. 协议地址和物理地址

IP 地址称为高级协议地址，是站在 Internet 网络的角度按照 IP 协议为主机和路由器所做的编号，是抽象地址。

但是，每台主机都是属于某个特定的物理网络的。在每个具体的物理网络中，每台主机又各有一个物理地址。例如对以太网来说，每一个网卡的卡号就是使用该网卡的主机的物理地址，由于全世界的以太网卡是统一编号的，因此网卡号就可以惟一地指定一台机器。然而，在以太网中运行的协议是 IEEE 802.3，而 IEEE 802.3 所辨认的只是以太网地址，而非 IP 地址。也就是说，具体的物理网

(组成 Internet 的局域网或广域网) 并不知道如何通过协议地址来定位一台计算机。

2. 地址解析

IP 地址是由软件进行维护的软地址,而物理地址是由硬件管理的硬地址。物理网络无法直接根据软地址定位一台主机。为此,要想通过一个物理网进行帧的传送,必须含有目的地的硬件地址 (如以太网地址) 。将一台计算机的 IP 地址翻译成等价的硬件地址的过程称为地址解析或称地址映射,也称为地址绑定。

应当注意,地址解析是同一个网络内部的局部过程,即一台计算机只能够解析位于同一网络中的另一台计算机地址。如在图 6.6 中,主机 A 上的一个应用程序要传输一条报文到主机 F 上,由于 A 与 F 不在同一个网络上,因此 A 上的软件无法解析 F 的地址,于是这个报文的传输要经过如下过程:

① 主机 A 上的软件首先确定要将报文传输到 F 必须经过路由器 R1,于是解析位于同一网络中路由器 R1 的地址,将报文传送到 R1;

② 路由器 R1 上的软件确定要将报文传输到 F 必须经过路由器 R2,于是解析位于同一网络中路由器 R2 的地址,将报文传送到 R2;

③ 路由器 R2 上的软件确定 F 就在同一网络上,于是解析位于同一网络中主机 F 的地址,将报文传送到 F。

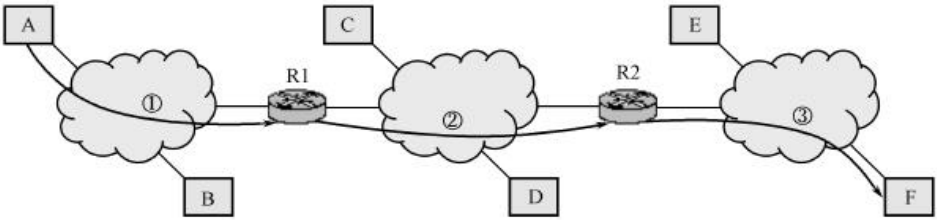


图 6.6　地址解析实例

3. 地址解析技术与地址解析协议

地址解析是通过软件实现的。地址解析软件要根据使用的协议和硬件编址方案来进行地址解析。对不同的物理网络,由于协议和编址方案不同,解析方法也不相同。例如,将 IP 地址解析为以太网地址与解析为 ATM 网地址的方法是不相同的。

一般说来,大体上有三种地址解析方法。

(1) 查表 (Table Lookup) 方法:将地址绑定信息存放在内存的一张表中,当要进行地址解析时,可以查表找到所需的结果。这种方法常用于 WAN。

(2) 相似形式计算 (Close－Form Computation):网络中主机的 IP 地址分配通

过对硬件地址的简单逻辑运算和算术运算得到,因而在 IP 地址与物理地址之间存在一种函数关系,可以直接运算求出。这种方法常用于可配置的网络。

(3) 报文交换 (Message Exchange) 法:前两种方法是集中式地址解析,而报文交换是分布式地址解析方法,即当一台机器要解析同一网络中另一台计算机的 IP 地址时,先通过网络发送一个请求报文——请求对指定 IP 地址的解析,之后收到一个应答——回答对应的硬件地址。这种方法常用于静态编址的 LAN。

那么,请求报文如何发送呢?通常有两种方法:一是在网络中设立一台或几台服务器,专门用来回答地址解析的请求;一是向全网广播,由各台计算机自己解析自己的 IP 地址。

为了使所有的计算机共同认可地址解析消息的精确格式和含义,TCP/IP 的地址解析协议 ARP 定义了两种基本的消息:ARP 请求消息 (一个请求包含一个 IP 地址和对相应硬件的请求,格式如图 6.7 所示) 和 ARP 应答消息 (一个应答消息既包含发来的 IP 地址,也包含相应的硬件地址)。ARP 规定:所有 ARP 请求消息都直接封装在 LAN 帧中,广播给网上的所有计算机,每台计算机收到这个请求消息后都要检测其中的 IP 地址;与 IP 地址匹配的计算机即发出一个应答消息,而其他计算机则丢弃收到的请求,不发出任何应答。

<table>
<tr><th colspan="3">0 … 7 8 … 15 16 … 31</th></tr>
<tr><td colspan="2">硬件类型</td><td>协议类型</td></tr>
<tr><td>硬件地址长度</td><td>协议地址长度</td><td>操作</td></tr>
<tr><td colspan="3">发送方硬件地址 (8 位组 0~3)</td></tr>
<tr><td colspan="2">发送方硬件地址
(8 位组 4~5)</td><td>发送方 IP 地址
(8 位组 0~1)</td></tr>
<tr><td colspan="2">发送方 IP 地址
(8 位组 2~3)</td><td>目的硬件地址
(8 位组 0~1)</td></tr>
<tr><td colspan="3">目的硬件地址 (8 位组 2~5)</td></tr>
<tr><td colspan="3">目的 IP 地址 (8 位组 0~3)</td></tr>
</table>

图 6.7　用于以太网的 ARP 分组格式

通过对一个帧中协议类型的判断,一台计算机就可以知道该帧中是否含有 ARP 报文。例如,当一个以太网帧携带有 ARP 报文时,类型字段中就包含十六进制数 0x806。

IP 层中还包含一个反向地址解析协议 RARP,用于规定硬件地址到等价的 IP 地址的翻译过程。

需要说明的是,ARP/RARP 可以看作 IP 层的协议,也可以看成网络接口层

的协议。有的资料也把网络接口层看成IP层的一部分。TCP/IP的分层是不太严格的。

6.3.3 IPv4分组格式

Internet是建立在多个物理网之上的。TCP/IP协议通过IP分组将各种物理网络统一起来,达到屏蔽细节、提供一致性的目的。IP协议的主要功能还包括无连接IP分组传送、IP分组寻址以及差错处理三部分。IP协议提供的是非可靠的无连接分组服务。非可靠是指无论传输正确与否,都不做验证、不发确认,也不保证IP分组的正确顺序。有关可靠性的工作都留给传输层处理。无连接是指每一个IP分组的处理都独立于其他IP分组,一串IP分组从一台计算机出发经过不同的路径到达另一计算机,中间可能部分丢失,而其他部分仍然被传送,IP协议不轻易丢弃IP分组。

IPv4分组由报头和数据区两部分组成,每部分的格式如图6.8所示。

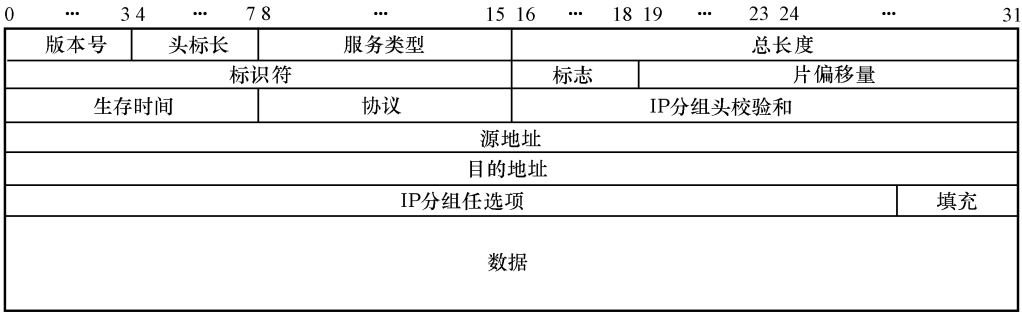


图6.8 IP分组格式

下面介绍各字段的含义。

(1) 版本号Ver(Version)

版本号占4位,用来指明所使用的IP协议的版本号,目前使用的是IPv4。所有的IP软件都要检查IP分组的版本号,若版本不同则需要转换,以保证IP分组的格式与软件期待的格式一致。

(2) 头标长IHL(Internet Header Length)

在IP分组的报头中,任选项和填充域是不固定的。当不含任选项和填充域时,头标长为5行(每行4个字节);任选项和填充域的最大长度是10行(占40字节)。报头的最大长度为15行(占60字节)。也就是说,头标的长度范围以行计算,在5~15之间,为此IHL占4位。

(3) 服务类型ToS(Type of Service)

描述 IP 分组所希望获得的服务质量，占 8 位，包括：

低延迟、高吞吐量、高可靠性，各占 1 位；

优先级，共 8 级，占 3 位；

未用 2 位。

(4) 总长度 TL (Total Length)

给出 IP 分组的总长度，占 16 位，即允许最长的 IP 分组为 65 535 字节。

(5) 标识符 ID (Identification)

ID 是源主机赋予 IP 分组的标识号。同一源主机所发出的各 IP 分组有惟一的标识。在 IP 分组传输的路径中，有些网络的处理能力有限，路由器必须将该 IP 分组分成若干片段。但无论分成多少片，每一片都有相同的 ID。目的主机根据 ID 和源主机号来判断所收到的片属于哪个 IP 分组。ID 占 16 位。

(6) 标志 F (Flag)

F 共占 3 位：

第 1 位恒为 0；

第 2 位为 0 时是可分片，为 1 时是不可分片；

第 3 位为 0 时是最后报片，为 1 时是非最后报片。

(7) 片偏移量 FO (Fragment Offset)

FO 占 13 位，用以标明当前段片在初始 IP 分组中的位置，目的主机可以根据 FO 来重新组合 IP 分组。

(8) 生存时间 TTL (Time To Live)

IP 分组在网络中传送的倒计时时钟，占 8 位，最大值为 255，缺省值为 64，单位为秒。当 TTL＝0 而尚未传送到时，该 IP 分组即被丢弃。

(9) 协议 Prot (Protocol)

协议段占 8 个字节，用以标识接收的 IP 分组中的数据的高层（传输层）协议。高层协议号由 TCP/IP 协议中央权威机构 NIC (Network Information Center) 分配，如：1——控制报文协议 ICMP，6——传输控制协议 TCP，8——外部网关协议 EGP，17——用户数据报协议 UDP，29——传输层协议第 4 类 ISO－TP4。

(10) IP 分组头校验和 (Header Cheksum)

IP 分组头校验和占 16 位，用以保证 IP 分组报头的完整性。校验方法是：先设初值为 0，然后对报头数据每 16 位求异或，结果取反。当目的地计算出的校验和与该域的值不符时，则丢弃该报文。

(11) 源地址 Source (Source Address)

源地址占 32 位。

(12) 目的地址 Dest (Destination Address)

目的地址占32位。

(13) 任选字段Opt(Options)

用于存放安全保密、报文经历、错误调试报告、时间戳等信息,其长度可变。

(14) 填充段(Padding)

IP分组头长必须是4字节的整数倍,填充字段就是为最后满足这一条件而设置的。

6.3.4 网际控制消息协议ICMP

IP协议的概念简明扼要,报文格式只有一种,它要求网络只需尽力传送报文包到目的地,而不必保证传递的正确性,所以是一种不可靠、无连接的包传送协议,也是一种尽最大努力服务的协议。这样,一个包在经过多个网际部件的传送途中,可能出现传送方向出错、目的主机不响应、包拥塞、超时等情况时,IP协议将无能为力。因为它没有差错报告和差错纠正机制。然而,这些问题又必须处理,为此在IP层引入了一个子协议:网际控制消息协议ICMP(Internet Control Message Protocol)。

1. ICMP提供的服务

ICMP是一种差错报告机制,它为路由器或目标主机提供了一种方法,使它们能把遇到的差错报告给源主机。具体地说,ICMP提供如下服务:

测试目的主机可到达性和状态,如接收设备接收IP分组时缓冲区是否够用;

将不可到达的目的主机报告给源主机;

进行IP分组流量控制;

向路由器发送路由改变请求;

检测循环(由此会引发"广播风暴")或超长路由;

报告错误IP分组头;

获取网络地址;

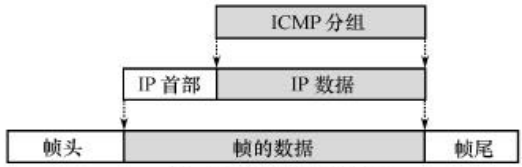
获取子网掩码。

2. ICMP分组

ICMP分组的格式如图6.9(a)所示。它由分组头和数据两部分组成,并且封装在IP分组中传输。

在ICMP的首部,首先标识的是ICMP分组的类型。按照协议的功能,ICMP分组可以分为三种类型:差错报告分组、控制分组和测试(请求/应答)分组。更详细的分类见表6.1。

(a) ICMP 分组的格式



(b) ICMP 分组的封装

图 6.9　ICMP 分组

表 6.1　主要 ICMP 分组

基本类	类型码	分组名称	代码	说　　明
差错报告	3	目的站不可达	0～15	代码段指明了 16 种不可达具体原因
	11	超时	0	TTL 超时 (只由路由器使用)
			1	重组超时 (只由目的主机使用)
	12	参数出错	0	在首部的一个字段中有差错或二义性
			1	缺少所需选项部分
控制	4	源站抑制	0	拥塞控制和源站抑制
	5	重定向	0	对特定网络路由的改变
			1	对特定主机路由的改变
			2	基于指明服务类型对特定网络路由的改变
			3	基于指明服务类型对特定主机路由的改变
测试	8	回应请求	0	测试可达性
	0	回应应答	0	
	13	时间戳请求	0	测试 IP 分组在两台机器之间往来所需时间
	14	时间戳应答	0	
	17	地址掩码请求	0	测试子网掩码
	18	地址掩码应答	0	
	10	路由器询问	0	测试路由器地址及其是否正常工作
	9	路由器通告	0	

应当注意,ICMP 虽然是第三层的一部分,但又好像第四层那样调用 IP 的服务,并且 ICMP 与 IP 一样,都是不可靠传输,ICMP 的信息也可能丢失。为了防止 ICMP 信息无限制地连续发送,对 ICMP 分组传输中的问题不能再使用 ICMP 解决。

3. ICMP 应用

ICMP 作为 IP 协议的补充,使一个路由器或一台目的主机可以通知源主机有关数据分组处理中的错误,并可以进行必要的处理。下面介绍它的两种简单而广泛的应用。

(1) ping

ping是TCP/IP网络中一个最简单而又非常有用的ICMP应用程序。它使用ICMP回应请求/应答,测试一台主机的可达性,验证一个IP安装是否正确,具体可以用于下列场合:

验证基础TCP/IP软件的操作;

验证DNS服务器的操作;

验证一个网络或网络中的设备是否可以被访问。

ping在不同的实现中语法格式有所不同。下面是在UNIX中的应用格式:

ping [-switches] host

host,目的主机的名字或其IP地址;

switches,参数选项,它是下列可选项的组合:

[-dfnqrvR] [-c *count*] [-i *wait*] [-l *preload*] [-p *pattern*] [-s *packtssize*]

表6.2给出了这些可选参数的意义。

表6.2 ping的可选参数

选项	意义
-d	设定下列测试参数
-f	洪泛ping——快速发出测试分组。只有超级用户可以选用此项
-n	只输出数字
-q	不显示任何传输分组的信息,只显示最后结果
-r	绕过正常的路由表,通过附加网络直接到达一台主机
-v	详细输出
-R	报告路由
-c	发送指定数目的分组后停止
-i	设定发送测试分组的时间间隔(秒数),预设值为1(每秒发送1个分组)
-l	预设在故障进入常规行为模式前ping能够尽快送出的分组数量。只有超级用户可以选用此项
-p	规定填充字符
-s	指定分组的数据部分大小(字节数)。预设值为56(加上8字节的ICMP头,共64字节)

(2) traceroute

traceroute程序用来确定通过网络的路由IP数据分组。它先把一个TTL=1的IP分组发送给目的主机,在经过第1个路由器时把TTL减到0,遂丢弃该分组并把ICMP超时消息返回给源主机,从而标识了第1个路由器。以后,不断增加TTL值重复上述过程,就可以依次标识出通向目的主机的路径上各路由器。

6.3.5　IPv6

1. IPv6 及其目标

迅速发展中的 Internet 将不再是仅仅连接计算机的网络，它将发展成能兼容电话网、有线电视网的通信基础设施。随着 Internet 的广泛应用和用户数量的急剧增加，只有 32 位（地址数量为 4.3×10^9）的 IPv4（IPv1～IPv3 从来没有被正式使用过，IPv5 仅用来命名 Internet 面向连接的协议 STP）地址危机已经展现在人们的眼前。面对这一危机，1990 年 Internet 工程任务组（IETF，Internet Engineer Task Force）开始着手一个新的 IP 版本——IPng（IP Next-Generation，下一代 IP 协议），其主要目标是：

具有非常充分的地址空间；

简化协议，允许路由器更好地处理 IP 分组；

减少路由表大小；

提供身份验证和保密等进一步的安全性能；

更多地关注服务类型，特别是实时性服务；

允许通过指定范围辅助多投点服务；

允许主机 IP 地址与地理位置无关；

可以承前启后，可以与 IPv4 共存，又允许进一步演变。

1992 年 6 月，IETF 公开征集对 IPng 的设计方案，收到了若干提案，到 1994 年形成最终方案，1995 年 1 月“下一代 IP 建议书”——RFC1752 发表，这就是现在的 IPv6。

IPv6 将 IP 地址扩充到 128 位，地址数增加到 4.3×10^{38} 个。同时，IPv6 简化了 IP 分组头，由 IPv4 的 12 个段减为 8 个段，从前的一些必要的段变为可选的段，使路由器能快速地处理 IP 分组，改善路由器的吞吐率。它还使用地址空间的扩充技术、使路由表减小地址构造和自动设定地址等技术。与 IPv4 相比路由数可以减少一个数量级，并能提高安全保密性。在主机数目大量增加，决定数据传输路由的路由表不断加大，路由器的处理性能跟不上这种迅速增长的形势下，这些技术的使用使 Internet 连接变得简单，而且使用容易。

IPv6 在路由技术上继承了 IPv4 的有利方面，代表未来路由技术的发展方向，许多路由器厂商目前已经投入很大力量来生产支持 IPv6 的路由器。IPv6 也有一些值得注意和效率不高的地方，因此 IPv4 和 IPv6 将会共存相当长的一段时间。

2. IPv6 分组结构

IPv6 协议数据单元（IP 分组）如图 6.10 所示，它的头由 IPv6 头和扩展头两

部分组成。

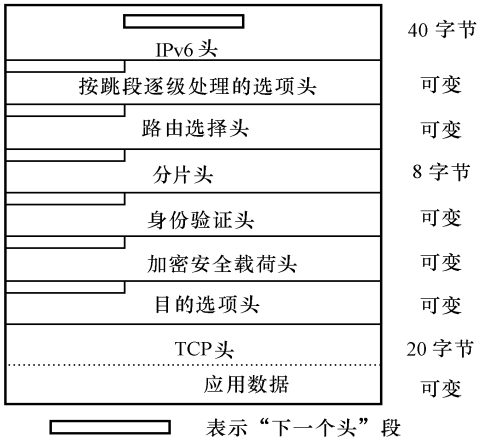


图 6.10　IPv6 的协议数据单元结构

带有可选扩展头是 IPv6 的重要特色，即 IPv6 头是必需的，而其他扩展头是可选的，这些扩展头有：

按跳段逐级处理的选项头，定义在每段都要予以处理的特别选项；

路由选择头，提供扩展的路由选择信息，相当于 IPv4 的源路由选择；

分片头，包含分割与重组信息；

身份验证头，提供 IP 分组完整性和身份验证；

加密安全载荷头，提供保密性；

目的地选项头，包含由目的节点查看的可选信息。

IPv6 分组固定头（即 IPv6 类）是 IPv6 分组所必需的，它具有固定的 40 字节长度，如图 6.11 所示。IPv6 分组固定头由版本号、优先级、流标记、载荷长度、下一个报头跳段限制、源和目的地址等 8 个段组成。

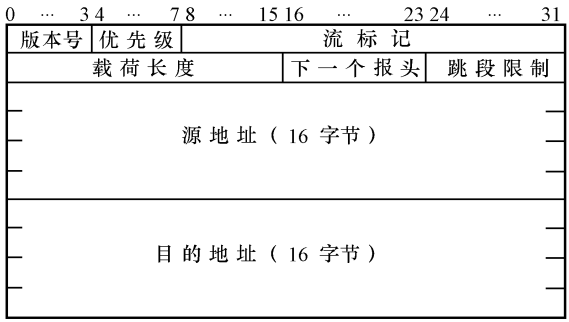


图 6.11　IPv6 固定头结构

下面分别介绍这些段。

(1) 版本号 (4 位)

版本号的值为 0110 (6) 。

(2) 优先级 (4 位)

当一个源发出多个 IP 分组时,该段可以为每个 IP 分组指定一个独立的传递和投递优先级。在指定时,首先区分 IP 分组是属于拥挤控制的交通,还是非拥挤控制的交通;然后将每一类分为 8 个优先级别。表 6.3 列出了优先级别类别和等级的含义。

表 6.3　优先级别类别和等级的含义

优先性	拥挤控制的交通		非拥挤控制的交通	
	级别	说　　明	级别	说　　明
低	0	非特征化交通	8	愿最先丢弃的交通 (如高保真视频)
	1	填充交通 (如网络新闻)	9	…
	2	无人值守交通 (如邮件)	10	…
	3	(保留)	11	…
	4	有人值守的大块数据传送 (如 FTP)	12	…
	5	(保留)	13	…
	6	交互交通 (如 Telnet)	14	…
高	7	Internet 控制交通 (如路由协议)	15	最不愿丢弃的交通 (如低保真视频)

非拥挤控制的交通是实时性交通,如实时视频和声频,它们需要恒定的数据速率和恒定的投递延迟,因此优先级别高于拥挤控制的交通。

(3) 流标记 (24 位)

流是主机为特别的源发往特别的目的地并希望中间路由器进行特别处理的 IP 分组序列所作的标记。

(4) 载荷长度 (16 位)

载荷长度指明除 IPv6 报头外其他部分 (扩展头和传输层 PDU) 的长度,单位为字节数。

(5) 下一个报头 (8 位)

下一个报头指明紧随该 IPv6 报头的扩展头的类型。

(6) 跳段限制 (8 位)

跳段限制相当于 IPv4 的生存时间段,由源设置最大值,然后被所转发的节点减 1。如果跳段限制被减至 0,该 IP 分组即被丢弃,这比 IPv4 的生存时间段所需的处理简单。

(7) 源地址 (128 位) 和目的地地址 (128 位)

这两个地址要比 IPv4 地址 (32 位) 长得多。

3. IPv6 地址

(1) 三种基本 IPv6 地址类型

在 IPv6 中,地址不是赋给某个节点,而是赋给节点上的具体接口。一般来讲,一个 IPv6 报文可以归纳为以下三种类型的地址:

单播 (Unicast) 地址,标识单个接口,IP 分组将选择一条最短路径到达目的接口。

多播 (Multicast) 地址,标识一组接口,IP 分组将发送给使用该多投点地址的所有接口。

任播 (Cluster) 地址,标识一组接口,IP 分组只发送给其中的一个接口。

图 6.12 所示为三种基本 IPv6 地址的格式。

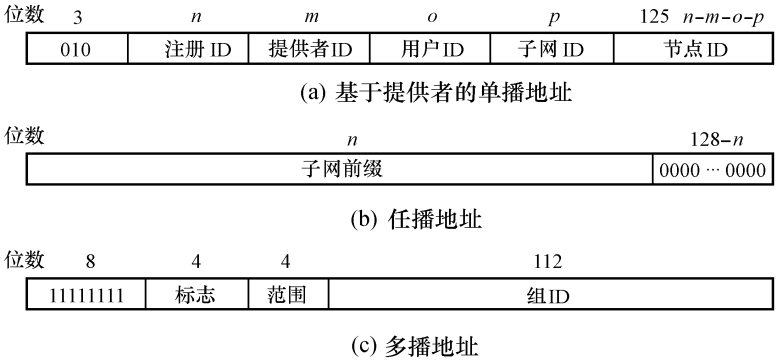


图 6.12　三种基本 IPv6 地址格式

下面介绍有关字段的含义。

(a) 单播地址中的有关字段

注册 ID:n=5,指出注册地址的机构。

提供者 ID:m= 变长,标识,用户的 ISP。

用户 ID:o= 变长,标识一个 ISP 的某个用户。

子网 ID:p= 变长,当用户可以有几个不同子网时,标识,其中某一子网。

节点 ID:定义连接到子网的单个节点接口。

(b) 多播地址中的有关字段

标志:由 3 个 0 加一个 T 位组成。T=0 表示一个永久分配的地址;T=1 表示一个非永久地址。

范围:限制多播组的范围,即 1——本地节点,2——本地链路,5——本地

场点。

组 ID 标识给定范围内的一个多播组，可以是永久的，也可以是暂时的。

(2) 本地地址

IPv6 还包括了本地范围的单播地址的前缀。它定义了两种类型：本地链路（Link－Local Address）和本地场点地址（Site－Local Address）。前者限制为单个网络，后者限制为单个站点。它们的格式如图 6.13 所示。

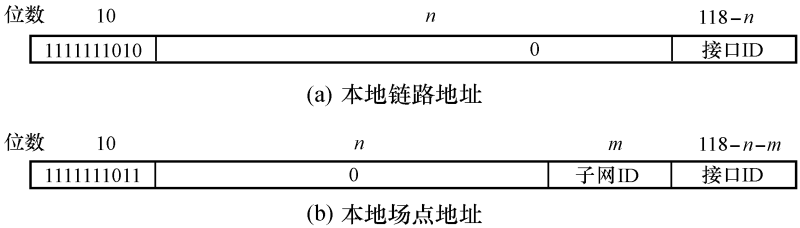


图 6.13　本地地址格式

(3) 地址的分级结构

IPv6 的地址格式与 IPv4 相比，一个重要的变化是单播地址结构层次的变化。IPv4 是两层次地址结构——网络地址和主机地址，而 IPv6 的单播地址为图 6.14 所示的三层次地址结构。

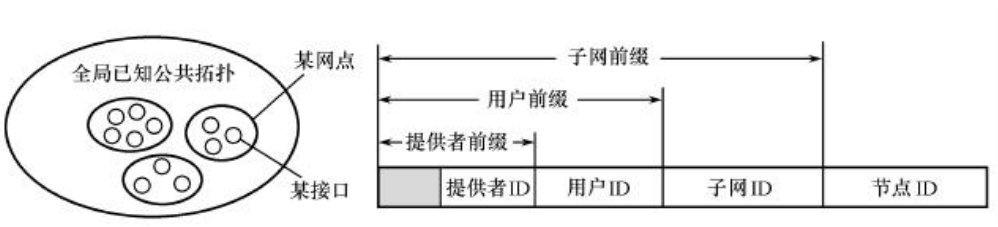


图 6.14　IPv6 单播地址的分级结构

网点和网络接口指定了可以确认的实体：网点对应了一组计算机和网络，隐含着邻近的物理连接以及拥有设备的单个组织；网络接口为最低层，对应附属于计算机和网络的单个附件。公共拓扑（Public Topology）是最高层，在 IPv6 中没有具体定义，现在仅预想了两种类型——ISP（提供远程服务）和交换（Exchange），与主 ISP 相连，并在它们之间传递通信量，也为单个订户（Subscriber）服务，为订户分配一个地址。

(4) IPv6 地址的冒分十六进制表示

一个 128 位的 IPv6 地址，即使用点分十进制写，也是相当长的。例如：

10.220.136.100.255.255.255.255.0.0.18.128.140.10.255.255

为了减少地址的书写长度,便于记忆,IPv6 的设计者们建议使用一种更紧凑的书写格式——冒分十六进制表示法(colon hexadecimal notation)。这样,上述地址就可以记为

69DC:8864:FFFF:FFFF:0:1280:8C0A:FFFF

在此基础上,人们又提出压缩零(zero compression)表示法。例如地址

69DC:0:0:0:0:0:0:B1

可以压缩地表示为

69DC::B1

4. 从 IPv4 向 IPv6 的过渡

随着 IPv4 地址即将枯竭(估计在 2005～2010 年间将分配完毕),如何从 IPv4 转向 IPv6,即从 IPv4 向 IPv6 过渡的问题越来越突出。但是由于 IPv6 与 IPv4 不兼容,这一转换过程有许多困难。目前,IETF 的研究从 IPv4 向 IPv6 过渡的专门工作组已经提出了许多方案,这些方案主要有以下几类。

(1) 双协议栈技术

如图 6.15 所示,IPv6 与 IPv4 虽然格式不兼容,但它们具有功能相近的网络层协议,都基于相同的物理平台,而且加载于其上的 TCP 和 UDP 完全相同。因此,如果一台主机能同时运行 IPv4 和 IPv6,就有可能逐渐实现从 IPv4 向 IPv6 过渡。

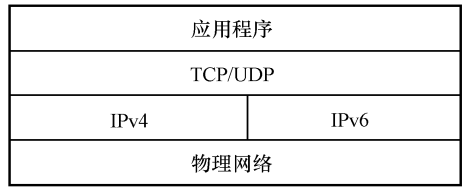


图 6.15　IPv4/v6 双协议栈的协议结构

(2) 网络地址转换—协议转换(NAT－PT)技术

NAT－PT(Network Address Translation－Protocol Translation)技术通过与 SIIT 协议转换和传统的 IPv4 下的动态地址翻译(NAT)以及适当的应用层网关(ALG)相结合,实现只安装了 IPv6 的主机与只安装了 IPv4 的主机间大部分应用的相互通信。

(3) 6 over 4 隧道技术

随着 IPv6 标准的推广,IPv6 的实验网络已经遍布全球。隧道技术就是设法在现有的 IPv4 网络上开辟一些“隧道”将这些局部的 IPv6 网络连接起来。具体方案是,将 IPv6 数据分组封装入 IPv4,送入隧道,IPv4 分组的源地址是隧道入口,IPv4 分组的目的地址是隧道出口。IPv4 分组穿过隧道后,在出口处再取出 IPv6 分组转发给目的站点。由于隧道技术只在隧道入口和出口处进行修改,因此实现起来比较容易。但无法实现 IPv6 主机与 IPv4 主机间的直接通信。

(4) 6 to 4 隧道技术

6 to 4 隧道技术简称 6 to 4 技术,它是一种自动构造隧道的技术。它在 IPv4 NAT 协议中加入对 IPv6 和 6 to 4 的支持,成为一个非常吸引人的方案。6 to 4 的

关键是它可以自动从 IPv6 地址的前缀中提取一个 IPv4 地址。这样,当用隧道将一个 IPv6 的出口路由器与其他 IPv6 域建立连接时,IPv4 隧道的末端就能从 IPv6 的地址中自动提取出来,从而在 IPv4 的海洋中将各个 IPv6 孤岛相互连接。

6.4　IP　路　由

简单地说,Internet 就是用路由器连接起来的网络。IP 层的一个重要功能是 IP 路由,它为路由器提供了互连不同物理网络的基本能力。

6.4.1　路由器工作概述

1. 路由器的基本工作流程

Internet 是由众多相对独立的子网连接起来的互联网络,各子网内部又由许多主机组成。子网内部主机间的通信按照链路层协议进行;子网之间的通信则要通过路由器实现。

路由器处在两个或多个网络的连接点上,作为不同网络之间互相连接的枢纽,路由器系统构成了 Internet 的主体脉络,也可以说,路由器系统构成了 Internet 的骨架。路由器的处理速度是网络通信的主要瓶颈之一,它的可靠性则直接影响着网络互连的质量。因此,在园区网、地区网乃至整个 Internet 研究领域中,路由器技术始终处于核心地位,其发展历程和方向成为整个 Internet 研究的一个缩影。

简单地说,路由器的主要工作是:

路径判断,使用一定的路由算法选择合适路径;

转发。

发送数据时,源主机知道目的主机地址和相邻路由器的地址,并在 IP 分组中用物理地址指明相邻路由器的地址,用协议地址指明目标主机的地址,接着按物理地址将 IP 分组发往第一个路由器。路由器收到 IP 分组后,按照路由表将物理地址修改为下一个节点的地址,直到将 IP 分组传送到目的主机。图 6.16 给出了路由器的工作流程。

下面分别介绍路由器工作流程中各步所进行的工作。

(1) 接收帧,分解出 IP 分组

当封装有 IP 分组的数据帧沿某个物理网传送到路由器的某个端口时,路由器的低层驱动程序按照相应的数据链路协议接收这个帧,并从中分解出 IP 分组交给 IP 层处理。

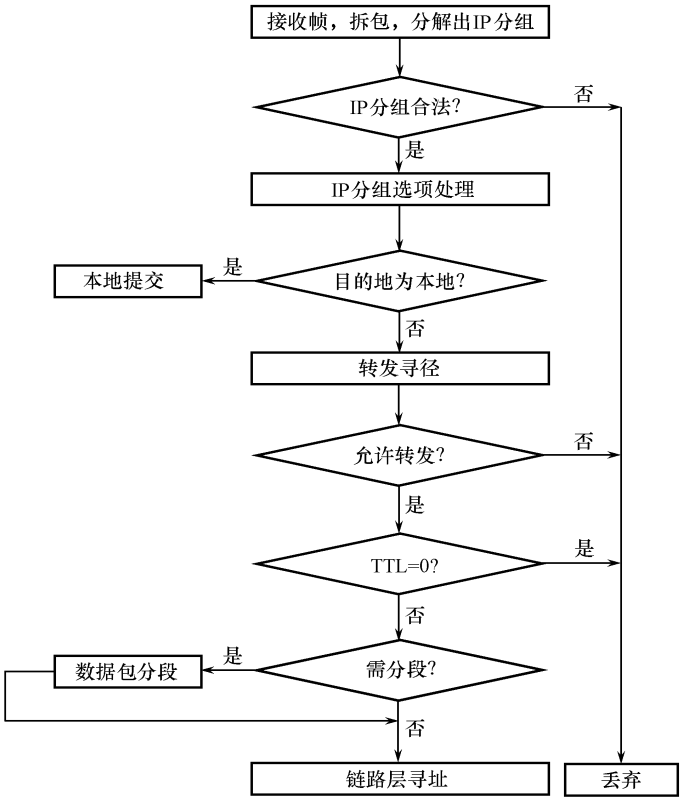


图 6.16　路由器工作流程

(2) 验证 IP 分组的合法性

对 IP 分组按下列项目进行合法性验证：

帧的长度必须能容纳下 IP 分组；

IP 校验和正确；

目前 IP 版本号为 4；

IP 报头长度必须足够大，可以容纳下最小的 IP 分组 (20 字节)；

IP 分组必须足够大，要能容纳下 IP 分组头。

(3) IP 分组选项处理

根据不同的路由选项类型，路由器在选项数据域中写入不同的内容：

记录路由选项，写入自己的 IP 地址；

时间戳选项，写入自己的 IP 地址以及当前世界标准时间计算值 (以毫秒为单位)；

源路由选项，先写入自己的 IP 地址，然后做进一步处理。

⑷ 确定 IP 分组是本地提交还是转发

路由器收到一个 IP 分组有三种处理方式：

转发，IP 分组中含有一个源路由选项时；

本地提交，IP 分组的目的地址或非转发组播地址中的一个与路由器一个端口地址相符；

既转发又提交，IP 目的地址是广播地址或是一个组播地址时。

⑸ 转发寻径

IP 选择路由是基于目的网络号，而不是基于目的主机。一个路由表包含许多 (N，G) 对，N 是目的 IP 网络号，*G* 是通过路由表可以将 IP 分组送到网络 *N* 的网关 (连接物理网络的路由器) 的 IP 地址。当收到一个 IP 分组时，首先通过网络掩码屏蔽操作从目的 IP 地址中找出目的网络号，接着按目的网络号在路由选择表中查找匹配的表项：

未找到相匹配的表项时，把该 IP 分组放入默认的下一路径的对应发送缓冲区中排队输出。

找到，把该 IP 分组放入路由选择表指定的发送缓冲区中排队输出。

⑹ 转发验证

在转发数据包之前，路由器要有选择地进行一些验证工作，提供一定的安全措施，防止外部主机伪装成内部主机的攻击，如：

将 IP 源地址和目的地址不合法的数据包丢弃；

将非法广播和组播数据包丢弃；

通过设置包过滤和访问列表功能，限制某些方向上的数据包转发。

⑺ TTL 处理

生存时间 (TTL，Time To Live) 用于限制数据包的生存时间，由一个倒计时时钟控制。通常数据包每经过一个路由器，该时钟减 1。当 TTL＝0 时，传送超时，该数据包即被丢弃。

⑻ 数据包分段

当要转发的 IP 分组长度大于输出到的物理网络的 MTU (Maximum Transfer Unit，最大传输单元) 时，路由器就要对该数据包分段。分段的原则是提高网络的传输效率，节省带宽，并有利于提高传输路径上路由器的处理效率。Internet 中常用的分段方法有：

第 1 段取当前路径上最有效的 MTU，其余平均分配在比 MTU 小一些的段中，这样可减少以后的分段操作。

按 MTU 大小分段，余下的为最后一段 (小于 MTU) 。

将 IP 分组都分作不大于 576 字节的段，这可以减少后面的分段，但会增

加网络的传输负荷,也会增加目的主机的计算工作量。

需要说明的是,在路由器初始化时,网络管理员为每个转发的网络都配置了一个相应的端口。由这个端口值,可以得到要输出的网络的物理网络类型和它的MTU以及相关驱动程序入口。

(9) 链路层寻址

路由器在完成了IP层的功能后,接下来是找一个相应的物理端口将数据包从链路层发送出去。具体实现时,IP层只要把IP分组的包长、目的物理网络地址、下一站的IP地址告诉驱动程序即可。驱动程序把IP分组封装在数据链路层的帧中,并利用ARP等地址解析协议把IP地址转换为物理地址。

2. 度量

路由器从一个网络接收分组,并将分组转发到另一个网络。但是,路由器通常连接到多个网络,并且都可能到达目的网络,这时它将把分组转发到哪个网络呢?

为了基于最优化原则决定一个可用的最佳路径,人们引入了度量(或称度,metric)。度量就是给通过的网络所指派的代价。一个特定路由的总度量等于组成该路由的所有网络的度量之和。按照最优化原则,路由器选择具有最小度量的路由。给每一个网络指派的度量取决于协议的类型。

3. 自治系统

在Internet中运行着大量路由协议,这些协议基本上属于动态自适应、分布式路由选择协议。一般说来,由于各ISP有自己的利益,不愿意提供自身网络详细的路由信息,因而整个Internet不适合跑单一的路由协议。为了保证各ISP利益,便于进行路由选择,Internet按运营被划分成许多较小的单位——自治系统(AS,Autonomous System)。每个AS通常由一个组织中的互联网络构成,由一个单独的管理机构管理(即由一个ISP运营),有权自主地决定本系统内部的路由协议。

划分了自治系统后,就可以把Internet中使用的协议分为两大类:

(1) 内部网关协议(IGP,Interior Gateway Protocol)——在一个自治系统内部使用的路由选择协议与其他自治系统中采用什么路由选择协议无关,主要有RIP、OSPF等。

(2) 外部网关协议(EGP,External Gateway Protocol)——当两个自治系统中使用不同的路由选择协议时,在两个自治系统之间进行数据报文转换的路由选择协议,主要有BGP。

6.4.2　路由信息协议 RIP

RIP (Routing Information Protocol) 是一种内部协议网关。它采用距离向量算法,用于小型自治系统中,并且将源代码嵌入了基于 BSD UNIX 的操作系统中,使人们容易接受。

1. RIP 的基本特点

(1) 动态更新路由

一个自治系统开始工作时,各路由器首先建立自己的初始路由表;然后周期性地(通常每隔 30 s)更新它的距离向量表,及时交换因拓扑结构变化引起的路由信息,维护相邻路由器的关系,同时根据收到的路由表计算自己的路由表。图 6.17 所示为使用 RIP 建立路由表的过程举例。

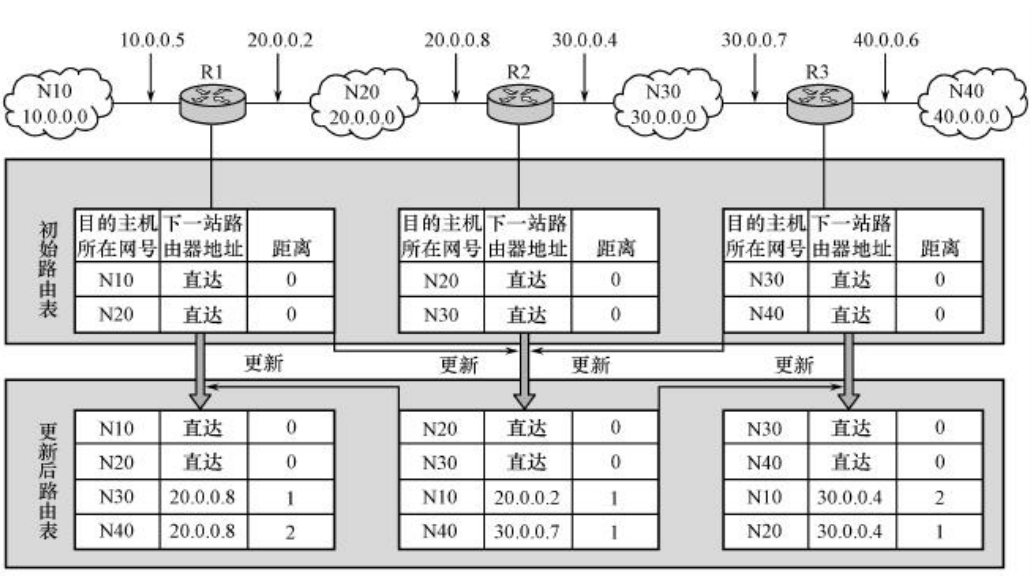


图 6.17　使用 RIP 建立路由表的过程举例

(2) 距离向量协议

RIP 按含最少路由器数选择路由,并且不允许同时使用多条路由,即使还存在另一条高速(低时延)但路由器较多的路由。

(3) 适用于小型自治网络

RIP 允许一条路由最多只能包含 15 个路由器。即最多 15 个跳数(Hop Count)。也就是说,当距离的最大值达到 16 时,就称为不可到达。可见 RIP 只适用于小型网络。

⑷ 好消息传得快,坏消息传得慢

如果路由器发现了一条更短的路由,更新消息就会传播得很快;而当网络出现故障时,要经过较长时间才能将此信息传送到所有路径。坏消息传播得慢,会使许多更新过程的收敛时间过长。

2. RIP 报文的类型

⑴ 请求报文

RIP 请求报文用于查询相邻 RIP 设备,以获得它们的距离向量表。

⑵ 响应报文

RIP 响应报文由一个 RIP 设备发出,用于公告它的本地距离向量表中维护的信息。这个表在如下几种情形下被发送:

如果网络中没有变化,每隔 30 s 自动发送一次更新信息;

如果网络有变化,路由器立即发送新的路由表——触发更新;

作为对另一个 RIP 节点的请求报文的响应。

当一个 RIP 设备接收到一个响应报文时,便将更新信息与本地距离向量表进行对比。如果更新信息中包含了一条到目的网络的代价更低的路由,则对本地表进行更新。

3. RIP 报文格式

RIP 报文格式如图 6.18 所示。其最大长度为 512bit,在一个单独的 RIP 通知中最多可以发送 25 个路由表项。大于这个值的更新必须用多个数据报进行公告。在局域网环境中,RIP 数据报使用 MAC 全站点广播地址和一个 IP 网络广播地址进行发送。在点到点或非广播环境中,数据报采用专门的编址方式发送到目的设备。

图 6.18　RIP 报文格式

6.4.3　开放式最短路由优先协议 OSPF

OSPF 是针对 RIP 的缺点于 1989 年开发出来的一种路由协议。“开放”的设

计思想,表明它不受任何一家厂商所左右,这正是它的强大生命力和广泛用途的源泉所在。在技术上,OSPF 是一种分布式链路状态协议 (Link State Protocol),即它是通过传递链路状态 (连接信息) 来得到网络信息,维护一张网络有向拓扑图,利用最小生成树算法得到路由表。

1. OSPF 区域

OSPF 是在一个 AS 内部运行的路由协议。由于 Internet 中许多 AS 很庞大,不便于管理,OSPF 允许将每一个 AS 分为一些编号区域 (Area),它们是包含在 AS 内部的一些网络、主机和路由器的集合,用 32bit 的区域标识符 RID (Area ID) 标识。区域通常对应地理边界或者行政边界。对于几个相邻网络,各区域之间不重叠,但也不彻底,有些路由器可能不属于任何区域。同一区域内的各路由器具有相同的链路状态数据库,并运行相同的最短路径算法。

所有的 OSPF 网络都至少要包含一个区域,并且要有一个主干区域。在具有多个区域的 OSPF 网络中,主干区域要与其他区域连接,以使其他区域直接把路由信息通知给主干区域,然后再由主干区域把此信息通知给其他区域。因此主干区域相当于一级区域,其他区域相当于二级区域。主干区域的编号为 0。

2. 路由器的分类

路由器按照其所处的位置可以分为 4 类。

(1) 内部路由器:这类路由器连接的网络都在一个区域内。

(2) 区域边界路由器 ABR (Area Border Router):把本区域的有关信息概括起来发给其他区域。

(3) 自治系统边界路由器 ASBR (AS Border Router):连接 AS 的路由器。一台路由器可以同时既是 ABR,又是 ASBR。

(4) 主干路由器:在主干中的路由器。一个主干路由器也可以同时是一个区域边界路由器。

3. 链路 (网络) 类型

OSPF 把物理网络称为链路。目前已经定义了 4 种类型的链路。

(1) 点到点链路 (或称点对点网络)

点到点链路直接连接两个路由器,中间没有其他主机或路由器。

(2) 过渡链路 (或称多路访问网络)

过渡链路是一种连接有若干个路由器的网络。数据可以从任何一个路由器进入网络,并从任何另一个路由器离开网络。这种链路又可以分为两种:

广播多路访问网络——支持与两个以上的路由器连接,并可以同时把一个报文发送到所有与之连接的路由器。

非广播多路访问网络——支持与两个以上的路由器连接，但无广播能力。其特例是点到多点网络。

(3) 残桩链路

残桩链路是只连接到一个路由器的网络。数据分组通过这个单一路由器进入网络，并通过这个路由器离开该网络。

(4) 虚拟链路

虚拟链路是当两个路由器之间的链路断开时管理员在它们之间使用一条更长的路径（可能经过几个路由器）创建的链路。

4. LSA（链路状态公告）及其类型

LSA 的内容描述了一个单独的网络组成成分（包括路由器、网段或者外部目的等）。LSA 在 OSPF 的邻接路由器间进行交换，以同步每个设备上的链路状态数据库。

LSA 包含 5 种信息类型：

路由器 LSA——由 OSPF 路由器产生，描述区域内路由器接口（链路）类型、地址、掩码等信息。

网络 LSA——在多路访问网络中列出与指定路由器（DR）连接的路由器 ID。

网络汇总 LSA——描述到其他区域中的网络的路由信息。

ASBR 汇总 LSA——描述到达 AS 边界路由器（ASBR）的路由信息。

AS 外部 LSA——由 ASBR 产生，描述到达 OSPF 网络外部目的地的路由信息。

每一个 LSA 报文都由头部和实例（Instance）组成。如图 6.19 所示，LSA 头部由 LSA 类型、广播路由器 ID、LS 序列号等信息组成。

LSA 年龄	选　项	LSA 类型
LSA 标志		
广播路由器 ID		
LS 序列号		
LSA 校验和	长　度	

图 6.19　LSA 头部格式

LSA 头部的 LS 类型、LSA 标志和广播路由器可以惟一地标识一个 LSA。

5. LSA 扩散

一旦某个路由器产生一个 LSA 或者修改一个 LSA，它就要把这一变化传送到网络上。这一传送过程称为扩散。扩散从一个路由器把 LSA 转发到了邻接

路由器开始。邻接路由器接收到 LSA 后,一方面要将它们保存到链路状态数据库中,一方面要继续将 LSA 传送到它们的邻接路由器。这种保存和转发一直要进行到所有的路由器都收到此更新。

6. Dijkstra 算法

运行 OSPF 的每个路由器都要将 Dijkstra 算法应用到其链路状态数据库中,以便计算出网络中两点之间的最短路径。Dijkstra 算法从源节点开始,将所连接的节点分为两个集合——试验的和确定的,即先将一些节点选入试验节点集合中,然后再检查它们,并把符合给定准则的节点移入确定节点集合中,从而找到最短路径中的一个节点。重复进行,直到到达目的节点。下面说明算法。

① 以本地节点(路由器)作为树根。

② 将代价 0 指派给该节点,并使之成为确定节点。

③ 重复进行下列步骤,对最后一个确定节点的每一个相邻节点进行检查,直到找到最短路径:

给每一个节点指派一个累计代价,并使之成为试验的;

在试验集合中寻找具有最小累计代价的节点,使其成为确定的。

7. 邻接路由器和 OSPF 报文类型

属于相同 OSPF 区域的两个相邻路由器(Neighbor Router),在同步了它们的拓扑结构数据库后,就可以成为邻接路由器。OSPF 通过邻接路由器交换链路状态信息。为了完成信息交换,OSPF 定义了如下一些活动:

发现邻居;

选择一个指定路由器;

建立邻接关系;

同步数据库。

这些活动通过表 6.4 所示的 5 类 OSPF 报文完成。

表 6.4　OSPF 的 5 类报文

报文类型	描　　述
Hello 报文	路由器启动时,用于发现邻接路由器
数据库描述报文	出现新线路时,给出报文发送者拥有的所有链路状态项序列号,接收者据此判定谁的数据最新
链路状态请求报文	路由器间相互发送该报文以从邻接路由器要求获取对应的链路状态信息,确定谁的数据最新
链路状态更新报文	用洪泛法向全网发送更新链路数据库状态
链路状态确认报文	对链路更新报文的确认

8. OSPF工作过程

(1) 建立邻接关系

一个路由器运行OSPF协议后,就试图与相邻路由器建立邻接关系。它定期地向各个网络接口(包括虚拟网络接口)发送Hello报文。在Hello报文中,含有自己的ID(即某一接口的IP地址)、优先权和相邻路由表。接收到Hello报文的路由器如果发现自己在对方的相邻路由表中,就表明双方都收到了对方的Hello报文。

(2) 同步LSA数据库

建立了邻接关系的路由器之间相互交换各自LSA数据库内信息的过程称为数据库的同步。每一个LSA数据库中包括许多LSA。每一个LSA代表某一个路由器的局部信息,由头部和实例组成。

每当路由器发送新的LSA时,其序列号将增1。路由器接收到一个新的LSA后,可以根据LSA头部的类型、标志和广播路由器判断其自身数据库中是否有相同的LSA,并根据LSA序号决定对哪一个LSA更新。

两台邻接路由器之间LSA数据库同步的过程如图6.20所示。

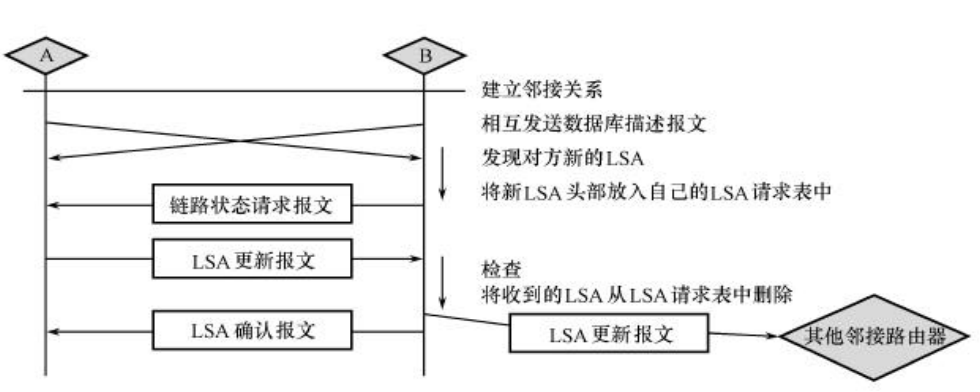


图6.20　两台邻接路由器间LSA数据库的同步过程

① 假设路由器A与路由器B刚建立起邻接关系,路由器A和B将相互发送数据库描述报文。在数据库描述报文中包括多个LSA头部。

② 一方(如B)发现对方(如A)的数据库描述报文中一些LSA头所代表的LSA自己的数据库中没有,或比自己的新,则将该LSA头部放入自己的LSA请求表中,然后向对方发送链路状态请求报文,要求得到具体的LSA信息(实例)。

③ 对方(如A)收到LSA请求报文后,将向请求方(如B)发送LSA更新报文。更新报文的数据部分是所请求LSA的完整信息。

④ 请求方(如B)收到每一个LSA更新报文后,进行检查,将收到的新的LSA

从自己的 LSA 请求表中删去,并向发送方 (如 A) 发出 LSA 确认报文。

⑤ 如果两个路由器间的 LSA 请求表为空,则表明两者的数据库达到一致,同步成功。

注意,同步是一个扩散过程。即一个路由器在其链路状态发生变化或收到其他路由器发送的 LSA 更新报文后,也要向其邻接路由器主动发送 LSA 更新报文,以便其他路由器尽快更新其拓扑数据库。

(3) 计算路由表

在接收到新的链路状态更新报文以及在实现了数据库同步后,路由器都要根据拓扑数据库重新计算它到各子网的最短路径,建立路径表。由于 OSPF 将 Internet 分成区域、AS 和 AS 外部三个层次,所以路由的计算也要分三个层次进行。

① 根据所在区域的数据库,利用本区域内的路由器 LSA 和网络 LSA,计算路由器到本区域内各网络的路由。

② 根据 ABR 向本区域散发的网络综合 LSA,计算路由器到本 AS 其他区域内各子网的可达信息。计算方法为:

本路由器到目的网络的距离＝边界网关到目的网络的距离＋本路由器到边界网关的距离

路由器选择到目的地的最优路径记录在路由表中。

③ 根据 ASBR 综合 LSA 和外部 LSA,计算路由器到 AS 外部的网络的可达性。计算方法为:

本路由器到目的网络的距离＝本路由器到 ASBR 的距离＋ASBR 到目的网络的距离

其中:

"本路由器到 ASBR 的距离"可以从前两步中得到;

"ASBR 到目的网络的距离"可以从 AS 外部 LSA 中得到。

6.4.4　边界网关协议 BGP

BGP (Border Gateway Protocol) 是一种外部网关协议。其开发初衷旨在提供一种自治系统之间交换路由信息的非循环方法,它基于路径向量路由选择,通过 ISP 边界的路由器加上一定的策略,选择过滤路由,把 RIP、OSPF、BGP 等的路由发送到对方,处理各 AS 之间的路由传递。Internet 是 BGP 处理多个 ISP 间路由的实例。BGP 的出现,引起了 Internet 的重大变革,它把多个 ISP 有机连接起来,真正成为全球范围内的网络。

1. BGP 网络结构

图 6.21 示出了一个 BGP 网络的组成。由于 BGP 是一种外部(边界)网关协议,因此它主要运行在每个 AS 的边界路由器间。在每个 AS 内部,可以运行 OSPF 或 RIP。运行 BGP 的路由器称为 BGP 说话者(BGP Speaker)。BGP 在一对邻接 BGP 说话者之间交换路由信息。这一对邻接的 BGP 称为 BGP 邻居(BGP Neighbor),BGP 邻居分为如下两种类型:

IBGP(内部 BGP 邻居)——位于同一 AS 中的一对 BGP 邻居;

EBGP(外部 BGP 邻居)——位于不同 AS 中的一对 BGP 邻居。

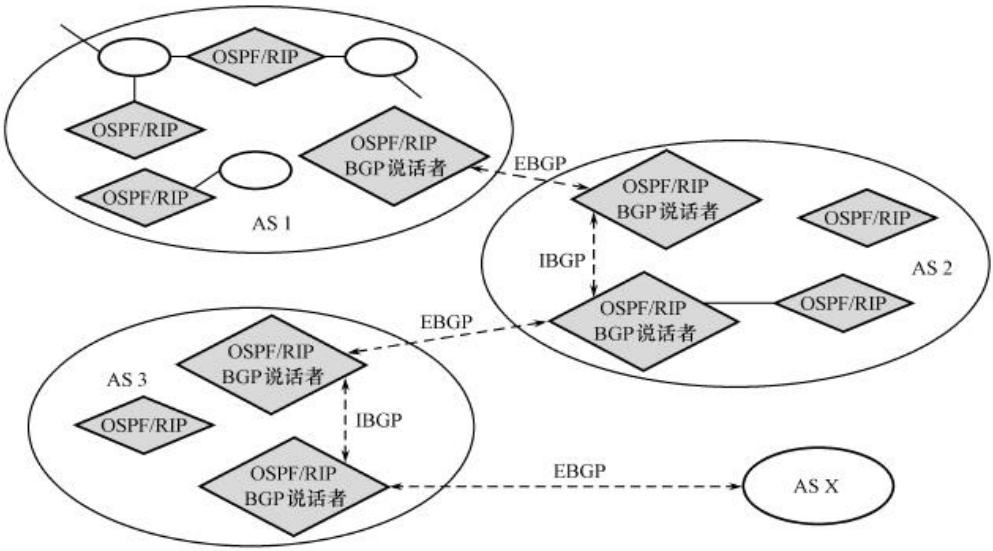


图 6.21　BGP 网络的组成

显然,一个 BGP 网络主要由一些 AS 和一些 BGP 说话者组成,并且通过 BGP 邻居交换路由信息。

2. 流量类型和 AS 类型

BGP 定义了两种类型的流量:本地流量和传输流量。本地流量是在 AS 中发出、在 AS 中终止的流量。除此之外的流量都是传输流量。BGP 的一个目标就是最小化传输流量的总量。

BGP 还定义了三种 AS:

Sub AS——只与另一个 AS 连接,并只承载本地流量;

多穴 AS——与一个或两个 AS 连接,但被配置成不转发传输流量;

传输 AS——与一个或两个 AS 连接,并被配置成承载本地流量和传输流量。

BGP 在处理路由时，使用 AS 号和 AS 路径：

AS 号——惟一标识一个 AS 的一个 16bit 串；

AS 路径——描述通过网络路由的 AS 号列表。

3. 路由策略

BGP 是外部网关协议，实现 AS 之间的路由。其路由策略的设置，一般要考虑政治、经济以及安全等多方面的因素。按照到达的目的地，可以将 BGP 的路由策略分为三类。

(1) 第一类策略：控制从本 AS 到其他 AS 的路径。例如，制定策略限制本 AS 发出的数据不能经过某些中间自治系统。

(2) 第二类策略：控制本 AS 是否为某相邻的 AS 传递过境数据。

(3) 第三类策略：实现 AS 内部的协调。

4. BGP 报文类型

所有 BGP 报文都包含了一个标准的报文头，报文头规定了 BGP 报文的类型。下面说明一些有效的 BGP 报文。

(1) OPEN (打开) 报文

一旦在两个对等的节点之间建立了连接，每个路由器就发送一个 OPEN 报文给邻居，请求建立邻接关系。一个 OPEN 报文由一个 KEEPALIVE 报文确认。

OPEN 报文中包含：

版本，1 bit；

本 AS 号，2 bit；

保持时间 (保持计数器的建议值)，2 bit，定义一方从另一方收到保活 (对 OPEN 的应答) 或更新报文之前所经过的最大秒数；

发端 BGP 路由器标识符，4 bit；

其他。

(2) KEEPALIVE (保活) 报文

运行 BGP 协议的各路由器 (BGP 中称对等路由器) 之间定期地相互交换保活信息，用来通报自己处于 BGP 工作状态，维护 BGP 连接。如果在保持计数器规定的时间内，发送路由器没有收到对等路由器的保活报文，则认为出现了一个错误，即向对等节点发送一个错误通知报文 NOTIFICATION 并关闭连接。

(3) UPDATE (更新) 报文

BGP 不定期地刷新整个 BGP 路由表。刷新使用 UPDATE (更新) 报文进行。一个 UPDATE 用来在对等 BGP 间传输路由信息：公告一个可行的路由，或者撤销那些不可行的路由。其报文中包含如下一些内容：

撤销的所有路由。

路径属性——用来评估一个路由,宣布本报文可达的网络路径。详见有关手册。

网络层可达信息(NLRI)——定义本报文真正通知的网络。

(4) NOTIFICATION(错误条件通知)报文

当一个 BGP 设备检测到影响与一个对等节点连接的错误条件时,就发送 NOTIFICATION(错误条件通知)报文,关闭与对等节点的连接,解除分配给该连接的所有资源,同时将与远程对等节点相关联的路由表项标记为无效,并通知给其他对等节点。

通知报文包括一个错误代码和一个错误子码。BGP 提供有如下一些错误代码:

消息报文头错误;

OPEN 消息错误;

UPDATE 消息错误;

保持计数器超时;

有限状态机错误;

停止。

6.4.5 基于路由器的防火墙技术

在建筑群中,防火墙(Fire Wall)用来防止火灾蔓延。在计算机网络中,防火墙是设置在可信任的内部网络和不可信任的外界之间的一道屏障,用来保护计算机网络的资源和用户的声誉,使一个网络不受来自另一个网络的攻击。从逻辑上讲,防火墙是一个分离器、一个限制器,也是一个分析器。具体地讲,防火墙有如下功能:它可以允许或禁止某一类具体 IP 地址的访问,例如某一台 IP 主机上有危险的用户或需要禁止的信息,则可以通过设置防火墙过滤掉从该 IP 地址发出的包;它也可以接收或拒绝某一类具体的应用,例如某一用户只使用电子邮件和 WWW 服务器,就可以通过设置,使得只有这两类应用的信息包可以通过。这种技术称为包过滤(Packet Filtering)防火墙技术。包过滤器逐一审查每份数据包并判断它是否与包过滤规则相匹配。过滤规则基于顺行处理数据包头信息,即通过对 IP 包头和 TCP 包头或 UDP 包头的检查实现,主要信息包括:

IP 源地址;

IP 目的地址;

封装协议(TCP、UDP、ICMP 或 IP Tunnel);

TCP/UDP 源端口;

TCP/UDP 目的端口;

ICMP 报文类型；

包输入/输出接口；

TCP 链路状态（TCP 包头的 ACK 位）。

通常，包过滤工作在网络层，故也称网络防火墙。

在 Internet 技术中还使用内容过滤技术。担任内容过滤的软件有："黑名单"软件、"白名单"软件和内容选择平台 PICS（Platform for Internet Content Selection）。

"黑名单"软件是第一代 Internet 内容过滤软件，其工作原理是封锁住不应检索的网址。其中最有名的是 Cyber NOT，它记录了大约 7 000 个网址，将有害和非法信息分为 13 类，如暴力/渎神、裸体、性行为、肉体描述、种族主义、对少数民族不恰当的评论、魔鬼崇拜、毒品、好战言论/极端主义、赌博、酒精、香烟。

"白名单"软件是第二代 Internet 内容过滤软件，其工作原理是先封锁全部网址，然后只开放应检索的网址。

数据包过滤防火墙网络逻辑简单、性能和透明性好，一般安装在路由器上。路由器是内部网络与 Internet 连接的必要设备，是一种"天然"的防火墙，它可以决定对到来的数据包是否要进行转发。这种防火墙实现方式相当简捷，效率较高，在应用环境比较简单的情况下，能够以较小的代价在一定程度上保证系统的安全。

包过滤技术的缺陷也是明显的。包过滤技术是一种完全基于网络层的安全技术，只能根据数据包的来源、目的和端口等网络信息进行判断，无法识别基于应用层的恶意侵入，如恶意的 Java 小程序以及电子邮件中附带的病毒。有经验的黑客很容易伪造 IP 地址，骗过包过滤型防火墙，一旦突破防火墙，即可对主机上的软件和配置漏洞进行攻击。进一步说，由于数据包的源地址、目的地址以及 IP 的端口号都在数据包的头部，很有可能被窃听或假冒；并且数据包缺乏用户日志（Log）和审计信息（Audit），不具备登录和报告性能，不能进行审核管理，因而过滤规则的完整性难以验证，安全性较差。

除了包过滤技术外，防火墙还采用代理服务（Proxy）技术和监测技术。

代理服务包括代理服务器和客户代理。代理服务器运行在防火墙上，客户代理运行在客户机上。外部客户访问内部服务器的请求要先发给防火墙的客户代理，通过防火墙的代理服务器将请求转发给内部服务器；服务器的应答则要先发给防火墙的代理服务器，再由防火墙的客户代理转发给外部客户。代理服务器的隔离作用强，具有对过往的数据包进行分析监控、注册登记、过滤、记录和报告等功能，可以针对应用层进行侦测和扫描，当发现被攻击迹象时会向网络管理员发出警报，并能保留攻击痕迹。代理服务器的缺点是必须针对客户机可能产生的所有应用类型逐一进行设置，大大增加了系统管理的复杂性。

监测型防火墙是新一代的产品,这一技术实际已经超越了最初的防火墙定义。监测型防火墙能够对各层的数据进行主动的、实时的监测,在对这些数据加以分析的基础上,监测型防火墙能够有效地判断出各层中的非法侵入。据权威机构统计,在针对网络系统的攻击中,有相当比例的攻击来自网络内部。而这种监测型防火墙产品一般还带有分布式探测器,这些探测器安置在各种应用服务器和其他网络的节点之中,不仅能够检测来自网络外部的攻击,同时对来自内部的恶意破坏也有极强的防范作用。因此,监测型防火墙不仅超越了传统防火墙的定义,而且在安全性上也超越了前两代产品。

6.5 TCP层

传输层位于通信子网与资源子网之间,对下面隔离通信子网的实现细节,对上面提供源主机与目的主机间的端对端通信连接服务。即从传输层开始,将进入主机级通信,而传输层的下面是网络级通信。

在TCP/IP的传输层中,有两个并行的独立协议:传输控制协议TCP和用户数据报协议UDP。

6.5.1 TCP协议

TCP是传输层的一个主要协议,图6.22说明了TCP与IP的关系。图中,两台主机通过两个网络和一个路由器进行通信。从TCP来看,只是两台主机间的一种连接,两台主机间的数据传输是通过调用IP来完成的。也就是说,TCP除了提供多个端口保证多进程通信外,主要提供端到端的面向连接的可靠的流服务。

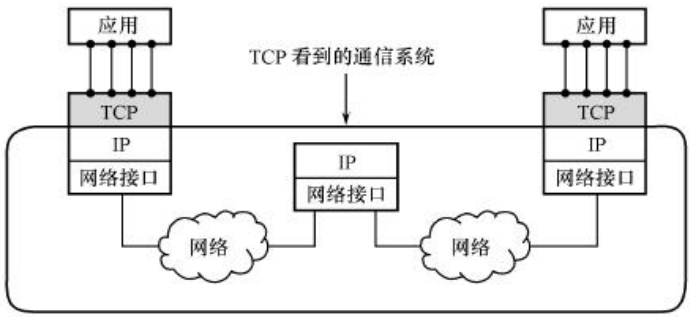


图6.22 TCP与IP的关系示例

1. TCP 服务特征

从应用程序的角度看,TCP 提供的服务有如下特征。

(1) 点对点的通信

TCP 是在网络层提供的服务基础上,提供一个直接从一台计算机上的应用到另一台远程计算机上的应用的连接。由于每一个 TCP 连接有两个端点,所以是一种端对端的协议。

(2) 虚电路连接

TCP 提供面向连接的服务,一个应用程序必须首先请求一个到目的地的连接,然后才能使用该连接传输数据。由于该连接是通过软件实现的,所以是虚连接(Virtual Connection)。

(3) 全双工通信

一个 TCP 连接允许任何一个应用程序在任何时刻发送数据,使数据在该 TCP 的任何一个方向上流动。因此,TCP 连接是由两个"半连接"组成的。

(4) 面向数据流的服务

当两个应用程序(用户进程)传输大量数据时,是以 8 位一组的数据流形式进行的。这种数据流是无结构的,既不提供记录式的表示法,也不确保数据传递到接收端应用进程时保持与发送端有同样的尺寸。因此,使用数据流的应用程序必须在开始连接之前就了解数据流的内容并对格式进行协商。

(5) 有缓冲的传输

当建立一个 TCP 连接时,连接的每一端分配一个缓冲区来保存输入的数据。通常把缓冲区中的空闲部分称为窗口。当交付的数据不够填满一个缓冲区时,流服务提供"推(Push)"机制,应用程序可以用其进行强迫传送。

(6) 完全可靠性

完全可靠性包括:

可靠连接的建立——当两个应用创建一个新连接时,两端必须遵从该新连接,旧的连接不影响新的连接;

可靠数据传输——TCP 确保通过一个连接发送的数据会同发送时一样正确地传送,而不发生数据丢失或乱序;

从容关闭——确保关闭连接之前传递所有的数据。

2. TCP 协议的主要功能

大致地讲,TCP 协议通过下面的功能来获得可靠性。

(1) TCP 连接的建立/释放与三次握手

所谓三次握手,就是通信双方在建立连接或释放连接时要经过三次交互。

TCP 采用三次握手来建立连接：

第一次握手——源主机发一个带本次连接序号的请求。

第二次握手——目的主机收到请求后，如同意连接，则发回一个带本次连接序号和源主机连接序号的确认。

第三次握手——源主机收到含两次初始序号的应答后，再向目的主机发一个带两次连接序号的确认。

目的主机收到确认，双方建立连接。由于连接是由软件完成的，所以称为虚连接。图 6.23 为 TCP 连接的三次握手示意图。其中，第一、第二次连接后，形成的是半连接。

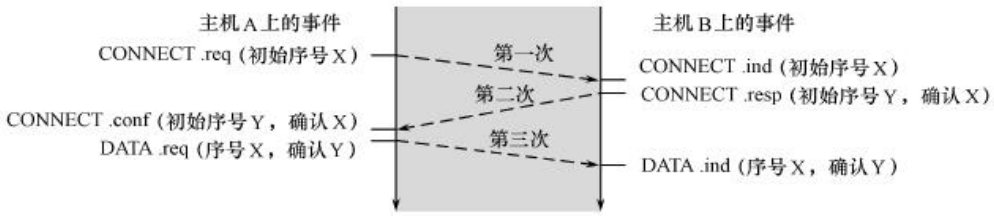


图 6.23　三次握手连接

同样，在释放连接时，TCP 也要采用三次握手方式。

下面通过释放时的情形来说明三次握手的必要性。

正常的释放连接是通过断连请求及断连确认实现的。但是，在某些情况下，没有经过断连确认，也可以释放连接。断连不当就有可能造成数据丢失。图 6.24 所示为一种断连不当引起数据丢失的情形：A 方连续发送两个数据后，发送了断连请求；B 方在收到第一个数据后，先发出了断连请求。结果第二个数据丢失。

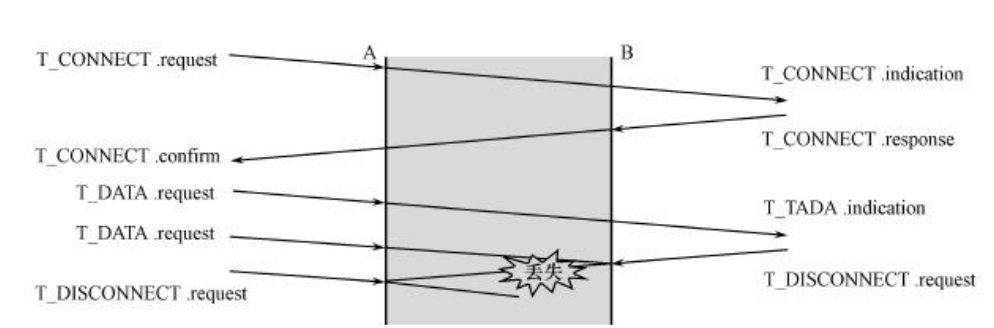


图 6.24　断连不当引起数据丢失

为了防止因断连不当引起的数据丢失，断连应选择在确信对方已经收到自己发送的数据并且自己和对方不再发送数据时。换句话说，就是要在征得对方

同意之后，才能执行断连操作。为了进行可靠的连接释放，比较有效的方法是三次握手（Three-Way Hand Shake）方法。

由于 TCP 连接由两个“半连接”组成，在撤销时，一方发起撤销连接但连接依然存在，所以 TCP 采用三次握手的方法来撤销这两个“半连接”：

① A 方发送断连请求 DR（DISCONNECT.request）TPDU（传输协议数据单元），并启动定时器；

②（DR 到达 B）B 送回断连证实 DC（DISCONNECT.confirm）TPDU，并启动定时器；

③（DC 到达 A）A 再送回一个 ACK TPDU，并删除连接；B 收到 ACK 后，删除连接。

图 6.25（a）是三次握手的示意图。其中，A 和 B 都进行了一个操作——启动定时器。那么，定时器有什么用途呢？它是在不正常情况下才起作用的，请看图 6.25（b）、（c）、（d）。

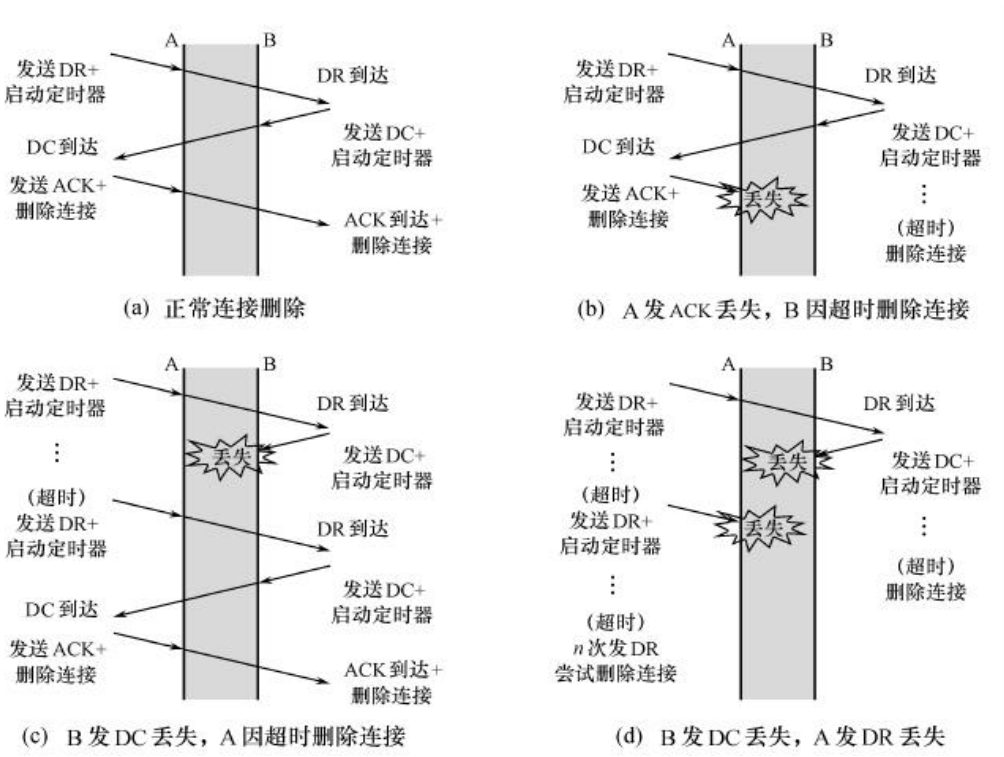


图 6.25　三次握手方法的连接释放

（2）确认和超时重传机制

确认就是目的主机每收到一个正确分组，即向源主机回送一个确认。

超时重传的基本思想是,设定一个时间片,如果一个 TCP 段在规定的时间片内收不到确认,就重传该分组。

(3) 流量与拥塞控制

TCP 协议采用滑动窗口进行流量和拥塞控制。TCP 的流量控制“窗口”是一种可变窗口。当接收方用户由于没有及时取走滞留在 TCP 缓冲区数据时,由于占用了系统资源,窗口将变小;当接收方用户取走在 TCP 缓冲区中的数据时,由于释放了系统资源,窗口将变大。也就是说,TCP 允许随时改变窗口大小。这样不仅可以提供可靠传输,还可以提供很好的流量控制。

与可变窗口相配套的是窗口通告(Window Advertisement)。即每个确认中,除了要指出已经收到的8位组序号外,还包括一个窗口通告,用于说明接收方窗口(接收缓冲区)还有多大——能接收多少个8位组数据。发送方要以当前记录的接收方最新窗口大小为依据决定发送多少8位组。

3. TCP 报文格式

TCP 的报文格式如图6.26所示。

图6.26 TCP 的报文格式

下面分别介绍各段的意义。

(1) 源端口(Source Port)

源端口即本地通信端口,支持 TCP 的多路复用机制。

(2) 目的端口(Destination Port)

目的端口即远地通信端口,支持 TCP 的多路复用机制。

(3) 序号(Sequence Number)

序号是数据段的第1个数据字节(除含有 SYN 的段外)。在 SYN 段中,该域是 SYN 的序号,即建立本次连接的初始序号,在该连接上发送的第1个数据字节的序号为初始序号+1。

(4) 数据偏移(Data Offset)

占 4 位。指出该段中数据的起始位置,以 4 字节为单位 (TCP 头总以 32 位边界对齐) 。

(5) 控制位 (Control Bits)

控制位共 6 位 :

URG——紧急指针域有效,即该段中携有紧急数据。

ACK——确认字段有效。

RST——连接复位,要求异常终止通信连接。

SYN——序号同步。

FIN——发送方字节流结束。

PSH——本报文段请求"推 (Push) "操作。即认定该段为"推进"段,段中数据是发送方当时发送缓冲区中的全部数据。对于收到这种数据的接收方来讲,应当把"推进"段中的数据尽快交给用户,并结束一次用户接收请求。

(6) 确认号 (Acknowledgment Number)

当 TCP 段头控制位中的 ACK 置位时,确认号域才有效。它表示本地希望接收的下一个数据字节的序号。对于收到有效确认号的发送者来说,其值表示接收者已经正确接收到了该序号以前的数据。

(7) 窗口 (Window)

表明该段的接收方当前能够接收的从确认号开始的最大数据长度,该值主要向对方通告本地接收缓冲区的使用情况。

(8) 校验和 (Checksum)

校验对象包括协议伪头、TCP 报头和数据。

(9) 选项 (Options)

选项位于 TCP 头的尾端,有单字节和多字节两种格式。单字节格式只有选项类型 ;多字节格式由一个字节的选项类型、多字节的实际选项数据和一个字节的选项长度 (三部分的长度) 组成。下面说明 TCP 协议必须实现的选项。

(a) 选项表尾选项 :KIND = 0。表示 TCP 头中由全部选项组成的选项表结束。其格式为 :

0	0	0	0	0	0	0	0

(b) 无操作选项 :KIND = 1。该选项可能出现在两个选项之间,作为一个选项分隔符,或提供一种选项字边界对齐的手段,本身无任何意义。其格式为 :

0	0	0	0	0	0	0	1

(c) 最大段长选项 :KIND = 2, LENGTH = 4。该选项主要用于通知通信连接的对方本地能够接收的最大段长。它只出现在 TCP 的初始建链请求中 (SYN

段)。如果在TCP的SYN段中没有给出该选项,就意味着有能力接收任何长度的段。其格式为:

0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	最大段长

(10) 填充(Padding)

当TCP头由于含有了选项而无法以32位边界对齐时,将会在TCP头的尾部出现若干字节的全0填充。

(11) URG位和紧急指针(Urgent Pointer)

传输层协议使用带外数据(Out-Of-Band,OOB)机制来传输一些重要数据,如通信的一方有重要的事情通知对方,需要加速传送这些通知数据。TCP支持一个字节的带外数据,并提供了一种紧急模式:在数据分组中设置URG=1,表示进入紧急模式,同时用紧急指针表明从该段序号开始的一个正向位移,指向紧急数据的最后一个字节。

6.5.2 UDP协议

UDP是一个无连接的协议,在发送时无须建立连接,仅仅向应用程序提供了一种发送封装的原始IP数据报的方法。图6.27所示为UDP数据报格式。其中,校验和是可选的,当不进行校验时,这个域为0。

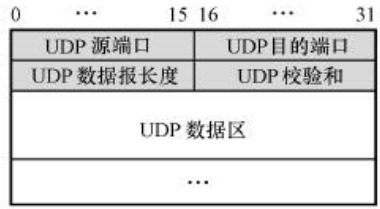


图6.27 UDP数据报格式

UDP是一个基于不可靠通信子网的不可靠传输层协议。因此,基于UDP的应用程序必须自己解决可靠性问题,如报文丢失、报文重复、报文失序、流量控制等。

应用程序可以使用UDP协议进行通信。不同的进程用不同的端口号进行标识。端口号分为公认(众所周知)端口号和自由端口号两种。

在UNIX系统中,一个UDP端口是一个可读和可写的软件结构,UDP协议为每个端口维护一个接收缓冲区。发送数据时,UDP协议将数据内容生成一个UDP数据报,然后交给网络层的IP协议发送。接收数据时,UDP协议从网络层IP协议接收到UDP,然后根据目的端口号将其放在相应的接收缓冲区中。如果没有匹配的端口号,UDP协议将丢弃该数据报,并向发送主机返回一个"不可到达"的ICMP消息。如果匹配端口号已满,UDP协议也丢弃该数据报,但不回送错误消息,将来靠超时重发。

UDP的优点在于高效率,通常用于交易型应用,一次交易只有一来一往两次报文交换。

6.5.3　TCP/UDP 端口号的分配方法

TCP 使用协议端口号（通常也简称端口号）来标识一台机器上的多个目的进程，每个协议端口号都是一个小的整数。协议端口号的分配有两种基本方式。

一种称为统一分配（Universal Assignment），是静态分配方式，由中央管理机构分配端口号。这些端口号要求所有的软件在设计时都要遵从。所以这些端口常被称为众所周知端口分配（Well-known Port Assignment）。TCP 与 UDP 的标准端口号是各自独立编号的。这两组标准端口号是所有采用 TCP/IP 协议的标准服务器必须遵守的，因而是固定的、全局性的公认端口，即“众所周知的端口”，也称“保留端口”。表 6.5 列出了一些当前分配的 TCP 和 UDP 端口号。

表 6.5　一些当前分配的 TCP 和 UDP 端口号

端口号	关键字	UNIX 关键字	说明	UDP	TCP
7	ECHO	echo	回显		Y
13	DAYTIME	daytime		Y	Y
19	CHARACTER GENERATOR	Character generator		Y	Y
20	FTP_DATA	ftp_data	文件传输协议（数据）		Y
21	FTP_CONTROL	ftp	文件传输协议（命令）		Y
22	SSH	ssh	安全命令解释程序		Y
23	TELNET	telnet	远程连接		Y
25	SMTP	smtp	简单邮件传输协议		Y
37	TIME	time	时间	Y	Y
42	NAMESERVER	name	主机名服务器	Y	Y
43	NICNAME	whois	找人	Y	Y
53	DOMAIN	nameserver	DNS（域名服务器）	Y	Y
69	TFTP	tftp	简单文件传输协议	Y	
70	GOPHER	gopher	Gopher		Y
79	FINGER	finger	Finger		Y
80	WWW	www	WWW 服务器		Y
101	HOSTNAME	hostname	NIC 主机名服务器		Y
103	X400	x400	X.400 邮件服务		Y
104	X400_SND	x400_snd	X.400 邮件发送		Y
110	POP3	pop3	邮局协议版本 3		Y
111	RPC	rpc	远程过程调用	Y	Y
119	NNTP	nntp	USENET 新闻传输协议	Y	Y
123	NTP	ntp	网络时间协议	Y	Y
161	SNMP	snmp	简单网络管理协议	Y	
179	BGP		边界网关协议		Y
520	RIP		路由信息协议	Y	

保留端口之外的其他端口称为自由端口。自由端口号是局部的,由本地主机随机地分配,用于区别一台主机中的多个进程。当一个进程发生时,本地主机的操作系统即给该进程分配一个自由端口号以与本主机上的其他进程相区别;该进程终止,此端口号即被释放,可以再分配给其他进程。而要知道另一台机器上的端口号,就必须送出一个请求报文询问,目的主机回答后,把正确的端口号送回来。这种分配方式称为动态绑定(Dynamic Binding)。

实际上,TCP/IP采用混合方式进行端口号的管理:分配某些端口号的同时,为本地网点和应用程序留下较大的端口号空间。

TCP是建立在所谓的"连接抽象"之上的,它所对应的对象是一个虚电路连接,而不是仅仅一个端口(UDP才将端口作为一个可供协议软件放置收到的数据报的队列)。TCP用一对整数(Host,Port)来标识一个虚电路连接。这一对整数称为端地址(Endpoint Address)。其中,Host为主机的IP地址,Port为该主机上的协议端口号。

习 题

6.1 查找资料,统计当前Internet上的主机数和用户数。

6.2 试将TCP/IP和OSI/RM结构进行比较,讨论其异同之处。

6.3 查找资料,分别说明在OSI、TCP/IP和IEEE 802模型中的每一层数据格式。

6.4 试比较SLIP与PPP。

6.5 某IP地址的十六进制表示为C22F1588,请将其转换成带点十进制表示。

6.6 写一个计算机程序,实现如下功能:

(1) 32位地址与点分十进制地址的相互转换;

(2) 判断该地址是A类地址、B类地址还是C类地址;

(3) 给出主机所在网络的网络号以及主机号。

6.7 举例说明子网屏蔽码的用途。

6.8 某B类网络,其子网掩码为255.255.240.0,每个子网的最大主机数是多少?

6.9 有一个网络ID为132.222.0.0,要将它分成多个子网,每个子网要容纳600台主机且要求有更多的子网,其子网掩码为( )。

A. 255.255.224.0　　B. 255.255.240.0

C. 255.255.248.0　　D. 255.255.252.0

6.10 地址解析协议的作用是什么?

6.11 当一台计算机广播了一个ARP请求之后,可能收到多少个应答?为什么?

6.12 ARP分组的长度是固定的吗?为什么?

6.13 当网际层使用IP协议而物理网为以太网时,一个ARP分组的长度是多少?携带ARP的以太网帧长度是多少?

6.14　试用 C 语言写出描述 ARP 分组的数据结构。

6.15　简述 IPv4 分组格式。

6.16　一个 IP 分组在到达时，其片偏移为 50。请问，在此分片的数据之前，源站已经发送了多少字节的数据？

6.17　试用 C 语言写出描述 IP 分组首部的数据结构。

6.18　试用 C 语言写出描述 IP 分组的数据结构。

6.19　试用 C 语言写出计算 IP 分组首部的校验和的函数。

6.20　试用 C 语言写出对 IP 分组进行分片的函数。

6.21　列表说明：

由路由器发出的 ICMP 分组；

由非目的主机发出的 ICMP 分组；

由目的主机发出的 ICMP 分组。

6.22　说明下列数值：

ICMP 分组的最小长度和最大长度；

携带 ICMP 的 IP 分组的最小长度和最大长度；

携带 ICMP 的以太网帧的最小长度和最大长度。

6.23　有这样一个 ICMP 分组：

类型，回送请求；标识符，123；序号，28；报文，Hello

请计算其校验和。

6.24　请根据下列到达 ICMP 报文首部（十六进制），说明其报文类型、代码、目的、最后 4 字节的值和最后 1 个字节的意义。

(1) 03 03 10 20 00 00 00 00

(2) 05 00 11 12 11 0B 03 02

6.25　试述 IPv6 IP 分组结构的特点。

6.26　试述 IPv6 地址分类特点。

6.27　请自己提出一种由 IPv4 向 IPv6 转换的好方法。

6.28　编写一个程序，将二进制的 IPv6 地址写成带有零压缩的冒分十六进制形式。

6.29　比较你所知道的几种路由协议的特点。

6.30　试用 C 语言写出路由表的数据结构。

6.31　试用 C 语言写出模拟路由选择的函数。

6.32　防火墙与路由器有何联系与区别？

6.33　你对防火墙的未来有何设想？

6.34　简要说明传输层协议的服务级别与网络层质量的关系。简述 TCP 协议的主要功能。

6.35　试写一个能可靠地进行通信的程序，并通过周期性地丢弃报文来模拟分组丢失。

6.36　写一个程序，用来提取并打印 TCP 报头内容。

第 7 章　ATM 网 络

7.1　ATM 技术特点

7.1.1　ATM 与 STM

传输模式是指在通信网中将信息从一点传到另一点或多点的方式。在同步传输模式 STM (Synchronous Transfer Mode) 中,为了在同一物理信道中传输多路信号,要给接入的每一路信号分配一个固定的时隙位置。当某一路无信息发送时,该时间片就空闲。异步传输模式 ATM (Asynchronous Transfer Mode) 采用标记多路复用技术,各路信息源没有固定的时隙位置,按需要分配时间片,这样就能充分利用带宽,提高传输效率。

7.1.2　信元传输

ATM 以信元 (Cell) 为基本单位进行信息的传递。信元是一种极短的、固定长度的数据分组。每个信元长 53 字节,包括 5 个字节的信头和 48 个字节的信息域。这种短小而固定的信元传输,灵活机动,不仅可以携带任何类型的信息 (数字、语音、图像、视频) ,支持多媒体通信,还能进一步降低传输时延,按业务的需要动态地分配网络带宽,既可以像线路交换那样传输话音业务,又可以像分组交换那样传输数字业务。同时,这种短小的固定长度的信元,使得交换可以由硬件进行处理,提高了处理速度,加大了传输容量。

7.1.3　虚路径和虚信道

ATM 交换机用于对虚信道 VC (Virtual Channel,也称虚通路) 中的数据进行交换,并且也是通过查阅表来确定交换机的出口以及所转接到的虚信道的标识符 VCI (Virtual Channel Identifier,占 16 位) 。在一个较大的 ATM 网络中通过 ATM 交换机的虚信道可能有上千条,当连接不断建立和释放时,交换机的存储和处理开销都是很大的。于是,ATM 中引入了虚通道 VP (Virtual Path,也称虚路径) ,并用

虚路径标识符 VPI (Virtual Path Identifier，占 8/12 位) 来标识。一条虚通道包括多个虚信道。于是，经过一个 ATM 交换机的交换就可以根据需要分为如图 7.1 所示的三种类型：

只交换 VPI，不交换 VCI。如图中端口 1 的 VCI/VPI=7/8 的信元，交换到了端口 2 的 VCI/VPI=7/16 连接上。

VPI 和 VCI 都交换。如图中端口 1 的 VCI/VPI=8/8 的信元，交换到了端口 3 的 VCI/VPI=75/17 连接上。

VPI 和 VCI 都不交换。如图中端口 1 的 VCI/VPI=7/9 和 VCI/VPI=8/9 的信元。

这样就可以大大减少交换机的开销，提高交换机的处理速度。这也是 ATM 技术的一大特点。

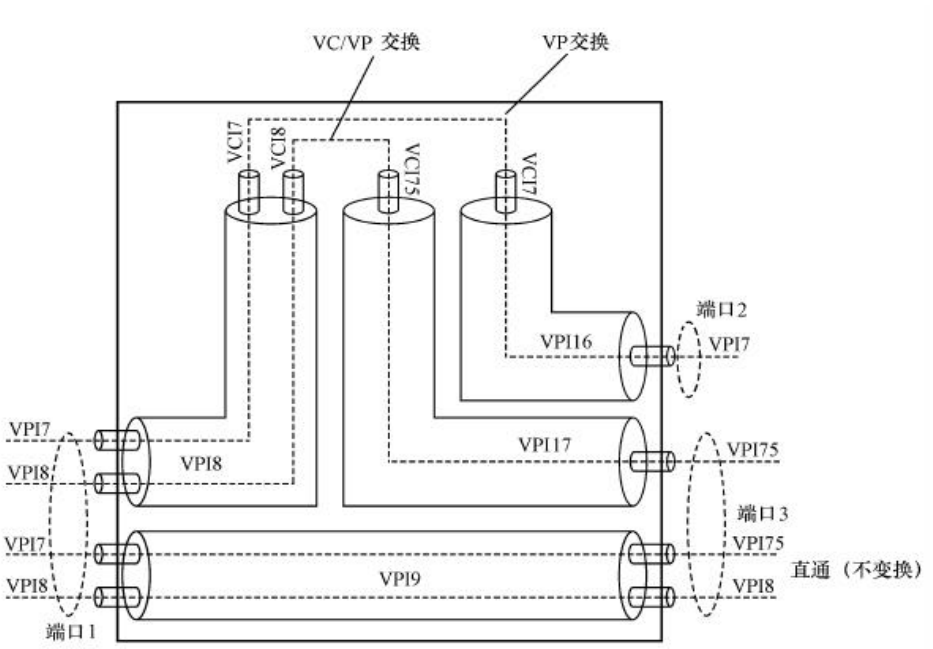


图 7.1　ATM 交换机对 VPI/VCI 的交换

ATM 是一种标记交换网络，靠标记进行路由选择，它的标记由虚路径标识符 VPI 和虚信道标识符 VCI 组成。由于 ATM 仅在数据链路层交换，处理节点 (ATM 交换机) 间的分组传输，因此 ATM 中虚路径标识符 VPI 的值和虚信道标识符 VCI 的值都只有本地意义。总的说来，VCI 仅在一个特定的 VPI 值中是惟一的，而 VPI 值仅在一条特定的物理链路上是惟一的。

ATM 是面向连接的网络，其虚电路的连接通过虚路径 VP 和虚信道 VC 两个

层次上的连接实现,即 ATM 中的连接分为两级:VPC (VP Connect,虚路径连接) 和 VCC (VC Connect,虚信道连接)。一个交换机外出链路上使用的 ATM 信元的 VPI/VCI 与该信元沿着它的端到端通路上访问的下一个相邻交换机上的入进 VPI/VCI 相同;依次类推,一段一段地链接,就能建立起两个端到端的链路。对于信元来说,每经过一个交换机,通过连接映射表查出对应的外出 VPI/VCI,将信元按此标记传送到下一交换机;依次类推,就能传送到目的端点。

图 7.2 所示为在一个 ATM 网络中建立 VPC 的过程。

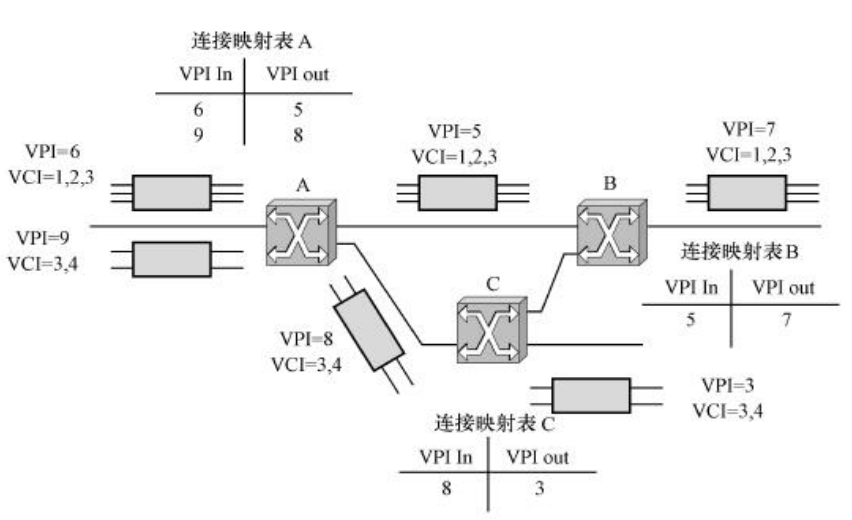


图 7.2 通过 VP 交换建立 VPC

ATM 交换技术是一种融合了线路交换方式和分组交换方式优点而形成的新型交换技术。ATM 交换有两个重要特征:

具有物理端口编号,为了提供交换功能,输入端口必须与输出端口相关联;

具有虚连接 (VP/VC) 标识符,输入 VPI/VCI 要与输出 VPI/VCI 相关。

也就是说,ATM 交换具有两方面的功能:一是空间交换,即将信元从一条传输线路转移到另一条传输线路上;另一方面是时间交换,即将信元从一个时隙改换到另一时隙。由于 ATM 的逻辑信道与时隙无固定关系,逻辑信道要靠信头来标识,时隙交换要靠信头翻译实现。

7.1.4 服务质量 QoS

服务质量 QoS (Quality of Service) 是网络管理的一个重要话题,部分原因是 ATM 主要用于实时服务,如音频和视频信号的传输。关于 QoS 的准确概念目前还难以给出。有人把 QoS 看作是用户对网络质量的评价,有人则认为 QoS 是网

络对业务做出的质量承诺。由于用户设备在请求建立连接时,传输层(准确地说是主机中的一个进程,泛称“客户”)中即提出了期望的 QoS 参数,为此就必须与网络进行协商。协商的内容包括:

需要的通信量;

同意的服务;

客户须遵从的要求。

最后的结果就是一种 QoS 契约:网络的承诺和对用户的要求。

为了使具体的通信量协定成为可能,ATM 标准定义了一系列 QoS 参数,如表 7.1 所示。

表 7.1　一些 QoS 参数

参数名	缩写名	含义
峰值信元速率	PCR	信元发送的最大速率
持续信元速率	SCR	长时间的平均信元传输速率
最小信元速率	MCR	最小可接收的信元传输速率
信元延迟变化极值	CDVT	最大可接收的信元抖动
信元丢失比率	CLR	信元丢失或提交太迟的比例
信元传输延迟	CTD	信元提交时拖延的时间(中间值和最大值)
信元延迟变化	CDV	信元提交时间的变化幅度
信元错误比率	CER	提交无错信元的比例
严重错误信元块比率	SECBR	出错信元的比例
信元错误目的地比率	CMR	信元提交至错误目的地的比例

在整个连接存在期间,网络应保证 QoS 执行以保障用户业务。当然,当用户的实际流量超过 QoS 的契约流量时,也要予以警告或采取措施,如设置信头中的 CLP 位,使之成为可优先丢弃的信元。

不同的业务类型具有不同的 QoS 类型。

7.1.5　ATM 网络接口

为了支持 ATM 网与 ATM 网和非 ATM 网的现行以及将来在网络应用方面的互连,ITU－T 和 ATM 论坛定义了 ATM 的网络接口,包括有:

用户－网络接口 UNI(User－Network Interface);

网络－节点接口 NNI(Network－Node Interface);

ATM 数据交换接口 DXI(Data Exchange Interface);

宽带互连接口 B－ICI(Broadband Inter－Carrier Interface)。

图 7.3 表明这些接口所处的位置。下面具体加以说明。

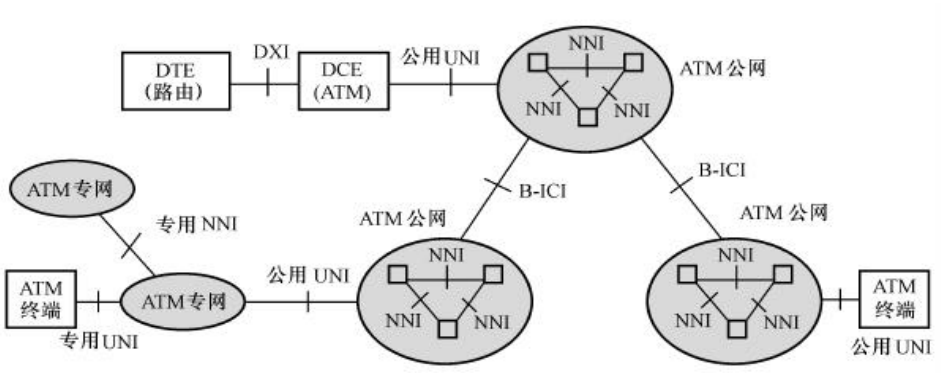


图 7.3　ATM 网络接口

1. 用户－网络接口 (UNI)

用户－网络接口 UNI 是 ATM 终端与其所连接的 ATM 网络之间的接口。终端设备是指将 ATM 信元传递到 ATM 网络的任何设备，可以是网间互通单元、ATM 交换机、ATM 工作站等。根据 ATM 网络性质的不同，UNI 可以分为：公共 UNI (连接 ATM 公网的接口) 和专用 UNI。

ITU－T 定义的 UNI 物理层接口速率主要有：1.5 Mb/s、2 Mb/s、25.6 Mb/s、51.8 Mb/s、155.520 Mb/s、622.080 Mb/s。

ATM 论坛定义的标准接口速率有：1.5 Mb/s、2 Mb/s、6.3 Mb/s、25 Mb/s、45 Mb/s、51.8 Mb/s、155.520 Mb/s、622.080 Mb/s。

2. 网络－节点接口 (NNI)

网络－节点接口 NNI 包括：

两个交换机间的接口；

两个 ATM 公网之间的接口；

两个 ATM 专网之间的接口。

3. ATM 数据交换接口 (DXI)

ATM 数据交换接口允许路由器等数据终端设备与 ATM 网络互连。

4. 宽带互连接口 (B－ICI)

宽带互连接口是 ATM 论坛定义的 ATM 公网的运营者 (Carrier) 到运营者的接口，不包括 ATM 专网之间的接口。

7.2　ATM 协议参考模型

ATM 协议参考模型与 OSI 参考模型有相似之处，又有本质差别。OSI 模型是一种平面结构，而 ATM 协议参考模型是一种立体结构。如图 7.4 所示，从顶上看，分为 3 个功能平面：用户平面、控制平面和管理平面；从侧面看，控制平面和用户平面都是分层结构；从正面看，分为 4 层：物理层、ATM 层、ATM 适配层和高层。

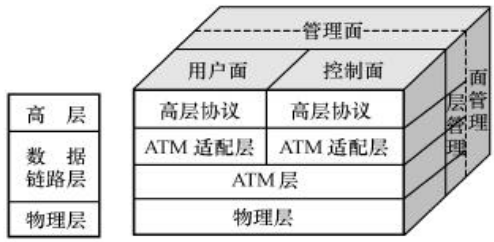


图 7.4　ATM 的协议参考模型

下面说明 ATM 的 3 个功能平面的作用。

(1) 用户面：采用分层结构，用于传送用户数据、流量控制、差错控制和其他用户功能信息。

(2) 控制面：采用分层结构，处理为建立和释放连接所需的信令（控制命令）。

(3) 管理面：提供层管理和面管理。管理面不分层次，执行与系统整体有关的管理功能，协调各平面之间的关系。层管理执行与各层实体中资源和参数有关的功能。这两种管理共同完成整个网络的维护和运行功能。

下面主要介绍 ATM 的各协议层次。

7.2.1　物理层

物理层的主要功能是控制数据的比特流在物理介质上传输。由于 ATM 信元可以通过多种不同的物理介质传输，物理层被分为两个子层：物理介质（PM）子层和传输收敛（TC）子层。

1. 物理介质（PM）子层

PM 子层主要负责在合适的介质上正确地发送和接收数字比特，并将比特流送到 TC 子层。具体地说，一是实现传送编码以及定时、同步等与物理介质相关的功能，如线路编码、解码、光电转换和比特定时等；二是定义物理传送接口。不同的传输系统要求不同的接口。目前 ATM 以 SDH 为基本传输系统，ITU－T 针对 UNI 规定了 155.520 Mb/s 和 622.080 Mb/s 的两种接口速率，以 SDH 帧结构或纯信元形式传送。

如图 7.5 所示，在 155.520 Mb/s 的接口上，ATM 信元装入 STM－1 帧的方法是：在 9 行×270 字节的 STM－1 帧中，除去每行 9 个字节用于传输开销 SOH 外，余下的 9 行×261 字节即为 VC－4。在 VC－4 中，再除去每行的第 1 个字节作为通道开销外，其余空间用于装载 ATM 信元。此时，ATM 信元的边界应当与 STM－1 中字节的边界对齐。在 STM－1 帧中，对信元定界的方法是：

① 首先利用 STM－1 的 SOH 的指针 AU－4 PTR 找到 VC－4 的起始位置；

② 利用信元信头差错控制（HEC，Header Error Control）机理，在 VC－4 中寻找信元的起始位置。

这样，即使信元跨越 STM－1 帧（由于 VC－4 的容量（2 340 字节）不是信元长度 53 字节的整数倍而造成）也不影响信元的读取。

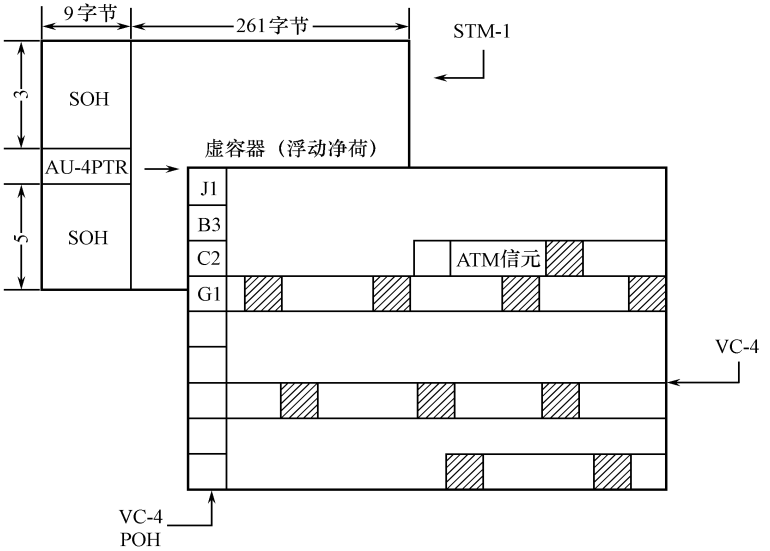


图 7.5　ATM 信元装入 STM－1 帧的方法

在 622.080 Mb/s 的接口上，ATM 信元装入 STM－4 帧中。其方法与 STM－1 相似。

2. 传输收敛（TC）子层

TC 子层主要完成以下 5 个方面的功能。

(1) 信元定界：按照一定的算法去搜索信元的边界，即信元的起始位置。在源端点定义信元边界，在接收端点恢复信元。

(2) 信头的差错控制（HEC）：信头差错控制技术采用 CRC 方式，生成多项式为 $g(x)=x^8+x^2+x+1$。发送端将信头前 4 个字节与对应的 x^8 多项式相乘

后,用 g(x) 去除,所得余数多项式就是所对应的 HEC 码,处于信头的第 5 个字节位置,在接收端进行校验。

(3) 传输帧的生成与恢复:将信元流封装成适合传输的帧,送到 PM 子层,以及将 PM 子层送来的传输帧恢复成信元流。

(4) 传输帧的自适应:实现信元流与传输帧转换时的格式自适应。在发送端把信元放到物理层的帧中,在接收端从帧中提取信元。

(5) 信元速率解耦:为了使 ATM 层信元流的速率适应传输介质的速率,在发送端插入一些空闲信元,在接收端识别并丢弃它们。

7.2.2　ATM 层

ATM 层是协议参考模型最重要的一层,其主要任务是给 ATM 信元流建立一个逻辑连接并通过 ATM 网络传送信元。

1. ATM 信元结构

ATM 信元由 5 字节的信头和 48 字节的负荷区组成。如图 7.6 所示,ATM 信头的结构因 UNI 和 NNI 而略有不同。

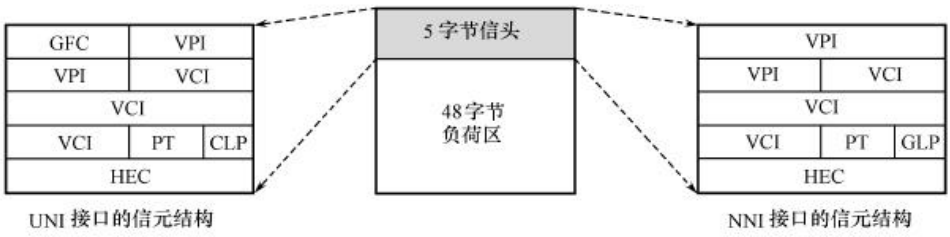


图 7.6　ATM 信元结构

下面对各字段加以说明。

① GFC (Generic Flow Control):4 位,只用于 UNI,进行一般流量控制,防止信道过载。

② VPI:虚路径——对于 UNI 为 8 位,每个主机可以定义 256 条虚路径。NNI 使用的 VPI 为 12 位,每个 ATM 交换机可以定义 4 096 条虚路径。

③ VCI:虚信道——16 位。

④ PT (Payload Type):3 位,净负荷类型。

⑤ CLP (Cell Loss Priority):1 位,发生信元冲突时,优先丢弃 CLP=1 的信元。

⑥ HEC (Header Error Control):8 位,信头校验码。

2. ATM 层的主要功能

(1) 信元的复用和分路：在发送端将不同连接（虚电路）的信元复用成单一信元流，在接收端分离恢复各连接的信元。

(2) 信元的添加和删除：在发送端将 AAL 层送来的信元净负荷添加信头，在接收端删除信头。

(3) 信元交换：一旦虚电路建立，VCI/VPI 接续路由表即建立。当信元进入交换机后，ATM 层读出信头中的 VCI/VPI，根据接续路由表找到它们的新值写入信头，信元被送往新值对应的 VC/VP 链路，交换完成。

(4) 一般流量控制：在 UNI 接口上，通过信头上的 GFC 码做一定的流量控制，以减轻可能发生的瞬间业务量超载。

ATM 层与上、下层间的关系如图 7.7 所示。

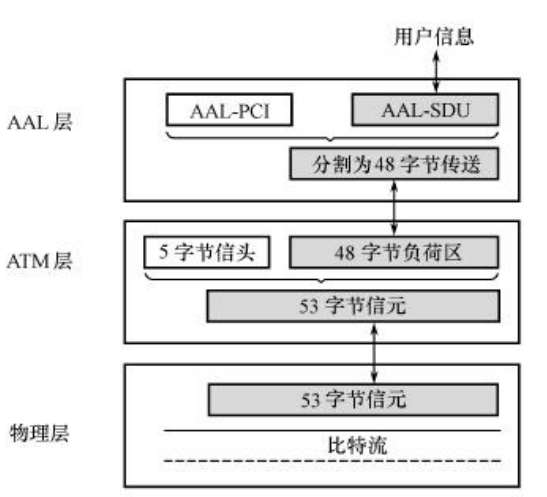


图 7.7　ATM 网络协议各层间的数据传输

7.2.3　ATM 适配层（AAL）

AAL 层介于 ATM 层和高层之间，它是为了使 ATM 层能适应不同业务类型的需要而设置的。ATM 按业务将 AAL 层分为表 7.2 所示的 4 类。

表 7.2　ATM 业务与 AAL 的对应关系

业务分类	A 类	B 类	C 类	D 类
AAL 类型	AAL1，AAL5	AAL2，AAL5	AAL3/4，AAL5	AAL3/4，AAL5
源和目的之间有无定时关系要求	要求		不要求	

续表

业务分类	A 类	B 类	C 类	D 类
AAL 类型	AAL1,AAL5	AAL2,AAL5	AAL3/4,AAL5	AAL3/4,AAL5
比特速率	固定	可变		
连接方式	面向连接			无连接
业务举例	固定比特率的话音、图像	可变比特率的话音、图像	X.25,帧中继	数据通信,LAN 间连接

最近,ATM 论坛又定义了一个新的服务类别——X 类。它是面向连接的,但通信量的类型和对定时的需求由用户定义。

7.3　ATM 局域网仿真 LANE

LANE 是 ATM 论坛提出的在 ATM 网上仿真局域网的规范,它把 ATM 仿真成数据链路层业务,不需要改变现有协议栈,ATM 仅作为第 2 层出现。

ATM 局域网是以 ATM 结构为基本框架,以 ATM 交换机作为局域网的交换中心,通过 ATM 设备将多台计算机互连到 ATM 交换机上的一类新型高速局域网。ATM 网络具有宽带、高速、提供 QoS 等多种优点,使 ATM 局域网在性能上大大优于传统的共享介质局域网。

组建一个 ATM 局域网需要两类硬件:ATM 主机接口卡和 ATM 交换机。ATM 主机接口卡用于将主机连接到 ATM 网络上,ATM 交换机用于作为 ATM 网络节点。

在将传统网络向 ATM 网络过渡的过程中,首要的问题是解决 ATM 与 LAN 接口、使两个 LAN 间经 ATM 进行交换和使 ATM 与各类广域网接口等。为此,ATM 论坛推出了 ATM 局域网仿真 LANE (LAN Emulation) 的标准和局域网仿真用户接口 LUNI (LAN Emulation User Network Interface)。通过这个接口,可以提供在 ATM 网上运行现有 LAN 协议 (IEEE802.3 和 IEEE802.5) 的手段;用 ATM 网络将传统的局域网与 ATM 主机互连起来,使局域网节点之间的通信过程与传统的局域网通信过程相同,不仅传统的应用程序及操作系统无须改动就可以在 ATM 网上运行,而且数据的传输速率也大大提高了。所以 LANE 又被称为 MAC over ATM。

7.3.1　LANE 网络的组成及体系结构

LANE 的核心是 LAN 仿真软件。该软件工作于 OSI 的数据链路层,独立于

上层协议。仿真软件由两部分组成：LAN 仿真客户机 LEC（LANE Client）和 LAN 仿真服务器 LES（LANE Server），如图 7.8 所示。

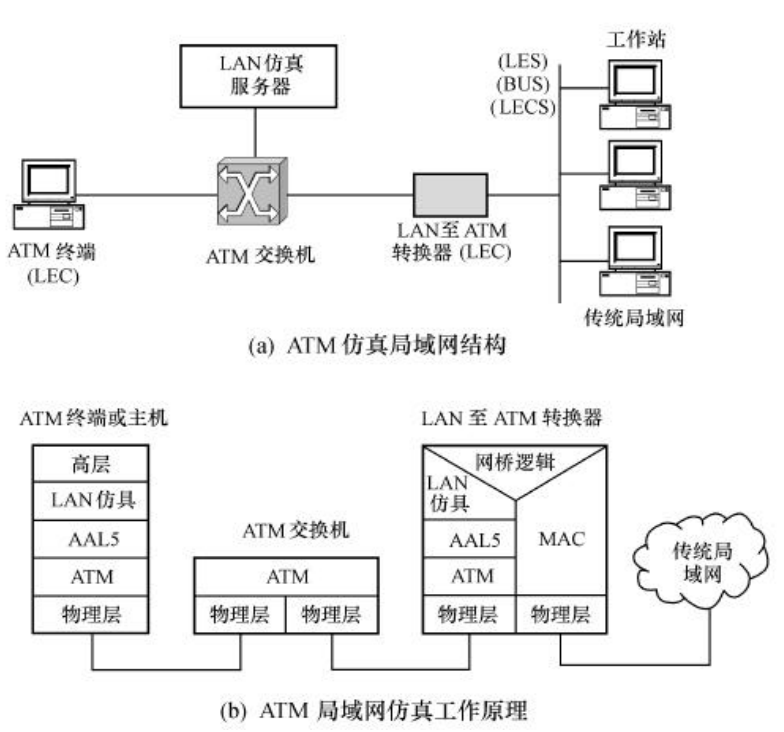


图 7.8　ATM 局域网仿真结构

1. LEC

LEC 安装在 ATM 网的终端系统（包括主机、转换器等）中，以代理身份加入到 LANE 中：

当它安装在有 ATM 网卡的 ATM 主机中时，代理主机接入 ATM 网，有一个 ATM 地址、一个 MAC 地址；

当它安装在 LAN/ATM 转换器中时，将代理整个传统 LAN 中的所有工作站接入 ATM 网，有一个 ATM 地址、多个 MAC 地址。

2. LES

LES 负责仿真网络的配置、地址解析和广播服务，为此可以分为三种逻辑服务器。

(1) 配置服务器 LECS（LANE Configure Server）：一个 ATM 网络上可以有多个 LANE。LANE 在加入仿真网络之前，要从 LECS 获得其 LES 的 ATM 地址，以便确定它要加入的仿真网络。LECS 提供 ATM 网的配置信息，并为 LEC 提供 LES 的

地址,负责将 LEC 分配到不同的仿真网络中去,并且它的功能对 LEC 是可选的,也允许 LEC 自行配置其 LES 地址。

(2) LES:用于登记或注册、解析 MAC 地址和 ATM 地址。

(3) 广播未知服务器 BUS (Broadcast and Unknown Server):用于 ATM 网卡上的广播和组播服务。

LES、LECS 和 BUS 可以安装在同一台设备(如 LAN 服务器)中,也可以安装在不同的设备中;在一个 LANE 中要有一个 LES 和一个 BUS,而一个 LECS 可以管理多个 LANE;LES 和 BUS 都至少要有一个 ATM 地址。

7.3.2　LANE 的帧格式

在 LANE 中,现有局域网的 MAC 帧在 ATM 桥上连接,利用 LANE 功能将 MAC 帧进行封装。如图 7.9 所示,去掉帧校验序列后,加上一个标志头,然后按 AAL5 CPCS－PDU (Common Part Convergence Sublayen－PDU) 的净荷字段进行承载,并在 ATM 上传送。

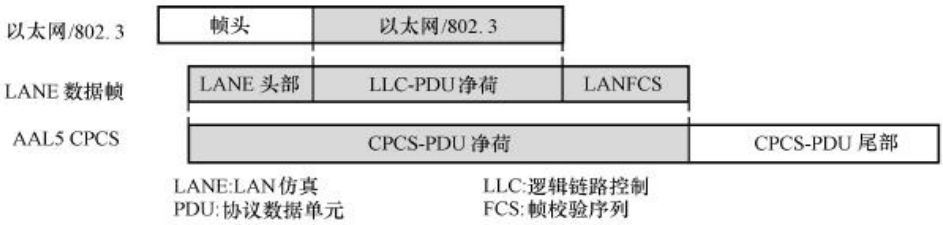


图 7.9　LANE 帧的封装

LANE 中的帧分为用户数据帧和控制信息帧两类。用户数据帧又因 MAC 层协议而不同。图 7.10 所示为它们的格式。

7.3.3　LANE 中的连接

在 LANE 中,用户数据和控制信息分别通过数据连接和控制连接进行传送。

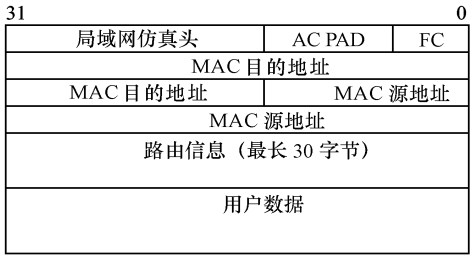
1. 控制连接

控制连接分为 3 种。

直接配置 VCC (Configuration Direct VCC):为使 LEC 从 LECS 获得配置信息而建立的双向点对点的连接。

直接控制 VCC (Control Direct VCC):为在 LEC 和 LES 之间传送控制信息而建立的双向点对点的连接。

(a) 以太网数据的 LANE 帧格式



(b) 令牌环网数据的 LANE 帧格式

(a) LANE 控制帧格式

图 7.10　LANE 的帧格式

控制分配 VCC (Control Distribute VCC)：由 LES 至各 LEC 的点对多点或单向点对点的连接。

2. 数据连接

数据连接也分为 3 种。

数据直接 VCC (Data Direct VCC)：为在有通信联系的 LEC 之间交换用户数据而建立的双向点对点的连接。

广播发送 VCC (Multicast Send VCC)：为将多发送类数据（包括广播类数据）由 LEC 递交给 BUS 而建立的双向点对点的连接。

广播转发 VCC (Multicast Forward VCC)：由 BUS 至各 LEC 的点对多点或单向点对点的连接，用来把多发送类数据由 BUS 转发给各个 LEC。

7.3.4　LANE 的初始化

LANE 是通过一套软件 LEC、LES、LECS 和 BUS 实现的。在正式通信之前先要初始化，以建立 LEC 与 LECS、LES 和 BUS 之间的控制虚通路连接 VCC，如图 7.11 所示。

下面介绍 LANE 的初始化过程。

① LEC 查询 LECS 的 ATM 地址（ATM 论坛分配的或从管理数据库中查询），建立 LECS 间的直接配置 VCC。

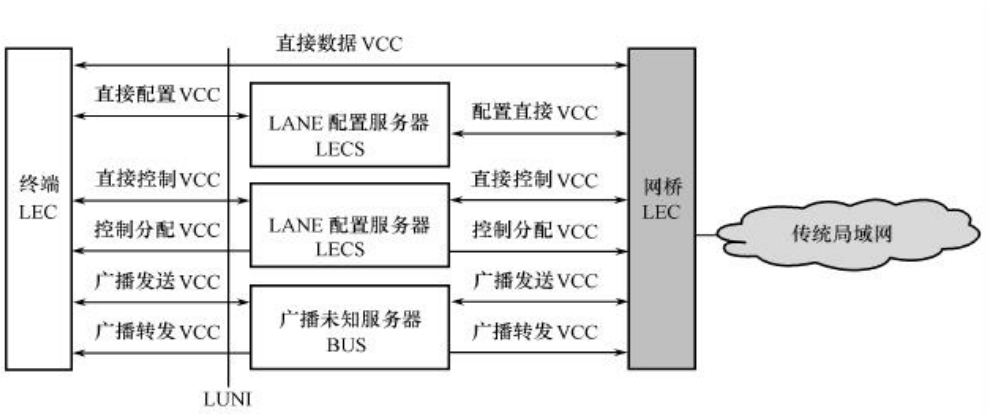


图 7.11　LANE 中的连接

② LEC 由已建立的 VCC 向 LECS 查询 LES 的地址及其他配置信息，以便在一个 ATM 网中配置多个仿真网络，实现 LANE。

③ LEC 获得 LES 的地址后，着手加入 LES 主管的 LANE。LEC 通过信令系统建立与 LES 之间的直接控制 VCC，并通过该 VCC 将自己的参数（LEC 的 ATM 地址、局域网类型、最大帧长、代理和 LANE 名）告诉 LES。LES 若接收 LEC 的这些信息，即建立与 LEC 间的控制分配 VCC，并通过该 VCC 或通过直接控制 VCC 将分配的 LEC 标识符通知 LEC。于是，LEC 加入了 LANE。

④ LEC 通过 LES 的直接控制 VCC 向 LES 询问 BUS 地址，待 LES 将 BUS 地址告诉 LEC 后，LEC 就通过信令系统建立与 BUS 的广播发送 VCC。同时，BUS 还建立一条到该 LEC 的单向广播转发 VCC。至此，LEC 已建立好与 LECS、LES 和 BUS 之间的控制虚通路连接。

⑤ 登记注册。LEC 通过直接控制 VCC 将自己的 MAC 地址及路由描述登记到 LES，LES 检查该 MAC 地址的惟一性后，在 LES 中建立 LEC 的 ATM 地址和 MAC 地址以及路由描述的对应关系。

到此为止，LEC 就可以加入网络正常运行了。

7.3.5　地址解析与数据传播

LEC 的功能是为上层发送数据和转发从上层收到的用户数据。当 LEC 收到上层的用户数据后，要根据数据帧中的地址判断通信的类型是广播通信，还是点对点通信。

1. 点对点通信

若属点对点通信，则需要进行地址解析。因为 LEC 从上层收到的数据帧中

只有目的站点的MAC地址,不是ATM地址。为了建立ATM连接,需要得到与MAC地址对应的ATM地址。这个对应关系存放在LEC内部的高速缓冲区中。为此LEC要先查询内部缓冲区。当查不到时,要通过直接控制虚通路连接VCC向LES查询。LES中的地址对应表是每个LEC注册时建立的,LES查到后即通知LEC。于是完成了地址解析。

LEC获得目的站点的ATM地址后,即通过信令系统与目的站点建立直接数据通信VCC,进行点对点的通信。

2. 广播通信

在广播通信的情况下,LEC可以将广播发送给初始化时建立的与BUS间的广播发送VCC,由BUS向LANE中的所有LEC广播。

由上述可知,LANE是利用LES和BUS来模拟以太网共享介质的结构和寻址方式。它向上层提供MAC层服务,上层用户看到的是传统的网络结构,真正的服务却是ATM的AAL5层和信令。

习 题

7.1 为什么ATM适合传输多媒体数据?

7.2 试述VC/VP连接过程。

7.3 试述ATM各层中数据的传输过程。

7.4 试述ATM网络中LES、LEC和BUS之间的关系。

7.5 AAL的服务类型有几种?请分别针对各类型的特性和典型服务应用说明之。

7.6 查阅资料,阐述当前IP技术发展趋势。

第 3 篇

计算机网络工作模式

计算机网络数据链路层的任务是解决网络中链路的通信问题。网络层是从一个网络的全局考虑通信问题;到了传输层,则主要考虑处于网络中两个主机之间的通信问题;由此向上,所考虑的就是进程间的通信问题了。这一篇从计算机网络工作过程中通信两端进程间关系的角度,讨论计算机网络的工作原理。内容主要包括:

客户机/服务器模式(第 8 章);

基于客户机/服务器的 Internet 应用(第 9 章)。

第 8 章　进程间通信的客户机/服务器模式

这一章介绍与计算机网络工作模式有关的几个基本概念：

进程；

客户机/服务器模型；

中间件。

8.1 进　　程

在客户机/服务器体系中，要使用进程。这一节对进程的一些概念做一点说明。

8.1.1 进程及其状态

《西游记》中描写的孙悟空要同众多的妖魔打斗，就拔下一根毫毛，变出一群自己的化身来。在多任务系统中，当只有一个任务（用户）时，计算机的资源（CPU、地址空间、全局变量、打开的文件、定时器、子进程、信号量等）都是分配给它的；但若有多个任务（用户）时，一个程序就有可能生成对应于不同任务的多个拷贝，使各个任务可以被并行地处理。于是引入了进程（Process）的概念。

进程是多任务系统中的一个重要概念。多任务系统是一种可以同时运行多道程序的系统。这多道程序共享着系统的资源——I/O 设备、存储器和 CPU，并且以一定的规则来占有 CPU。在这种情形下，用程序的概念已经难于准确地描述各个程序动态的运行过程，于是引入了进程的概念，并通常将其定义为程序关于某一任务的一次运行活动。一个程序为实现不同任务可以同时有多次运行活动，每个运行活动分别被作为不同的进程。在 UNIX 和 Linux 中，所有的程序运行都是以进程的形式得以执行的。一个程序为一个任务的启动执行，便是一个进程的建立。该程序的该次执行结束，便是该进程的撤销。

一个进程从建立到撤销，就是它的生存期，在其生存期内，视其对 CPU 的占有情形分为三种状态：

运行态——进程占有 CPU，正在运行；

就绪态——进程已经具备除 CPU 之外的其他资源和条件，等待 CPU；

等待态——进程等待CPU之外的其他资源或条件。

进程是一个随执行过程不断变化的实体,它们在各自的虚拟地址空间中运行,并通过一定的机制进行通信。

8.1.2 进程的特点

进程具有如下一些特点:

(1) 进程是程序在并发系统中的一次执行,它有一个从被创建到被撤销的生命周期。

(2) 进程是程序运行时系统进行资源分配的基本单位,每个进程具有自己的地址空间——自己的代码区、数据区、堆栈、进程控制块、程序计数器。

(3) 当一个进程要创建一个子进程时,内存映像要从父进程拷贝到子进程,所有描述字要在子进程中复制。

(4) 当一个程序按多任务组织时,在多任务操作系统内核的调度下,系统中就会生成程序的多个拷贝——多个进程,这些进程是同一程序的不同体现,并且并发地执行。所谓并发地执行,是指各进程间的指令空间是独立的,它们只会执行自己的数据,不能读写别的进程的数据,也不用跳转到另外的进程去执行,它们都可以按照自己的方向和速度独立地向前推进。

(5) 一个进程在处理一个操作时,如果还需要另外一个操作或者想执行另一个程序,这时就派生(Fork)一个子进程,让子进程去处理。

(6) 一个进程与其他进程或与外界的通信,通过系统调用进行。

8.2 客户机/服务器通信模型

8.2.1 客户机/服务器模型概述

现代应用程序一般具有两大功能:向用户提供界面和进行信息处理。随着计算机网络应用的不断深入和发展,网络程序的规模和数量越来越大。为了便于开发,提高程序的可靠性和运行效率,人们开始将程序按两大类功能分开进行设计。这两类程序分别称为客户机程序和服务器程序,从而形成一种新的软件体系。在这种软件体系中,一个计算任务被分布在两类协作单元上:一类称为"前台(Front-End)"或客户机(Client)程序,用于接待用户,负责提出计算请求;一类称为"后台(Back-End)"或服务器(Server)程序,处理客户机程序的请求,然

后将处理结果回送客户机。可见,C/S(客户与服务器)并非通常意义上的硬件或系统,而是程序,进一步说,应当是进程。它们可以运行在一台计算机中,也可以运行在网络环境中的两台或多台计算机中。

C/S结构好像酒店中的服务,分为前台接待顾客和后台制作两大部分一样,两部分之间分工、协同地工作,形成一般服务性系统内部实现对其用户提供应用服务的一种基本模式,称为客户/服务器计算模式或客户机/服务器计算模式,简称C/S模式。

与对等式工作模式相比,客户机/服务器计算模式最明显的特点是客户机的主动性和服务器的被动性。就是说,在C/S模式中,客户机和服务器不是平等工作的,而一定是先由客户机主动发出服务请求,服务器被动地响应。这也就是区分客户机与服务器的一条原则,看谁先发起通信,谁就是客户机。这种工作模式特别适合TCP/UDP工作方式。

如图8.1所示,在UDP协议的应用中,客户机首先向服务器的某个知名端口发送报文请求服务,服务器端的守候程序可以从收到的请求报文中的源地址识别出客户机的IP地址,然后根据决策的结果向客户机方返回服务应答。客户机与服务器之间可以直接联系,也可以通过若干中间件连接,中间连接件是一些标准接口,通常表现为一套API函数,并且对不同的应用有不同的接口。例如,对于数据库应用,标准接口可能是微软公司制定的ODBC等。

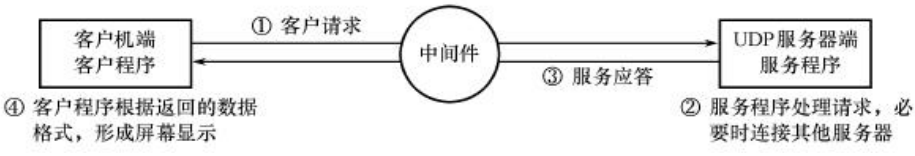


图8.1　使用UDP传递数据的客户机/服务器模型

再如图8.2所示,在TCP协议的应用中,连接的发动者是客户机端,服务器方则在知名端口上消极地等待连接,是连接的被动端。连接成功后,服务器方在连接上等待由客户机方发来的服务请求;服务器方收到请求后,返回应答报文;服务结束后,双方拆除连接。

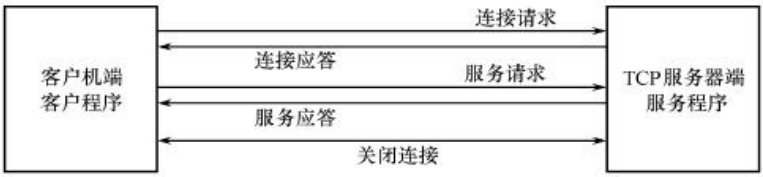


图8.2　使用TCP传递数据的客户机/服务器模型

8.2.2 客户机/服务器计算模式的优点

目前的计算机网络基本上都是采用客户机/服务器模式的,原因就是它能带来如下一些益处:

(1) 增强了系统的稳定性和灵活性

客户机/服务器模式将应用与服务相分离,使得系统具有即插即用的特点,减少了因系统变更带来的影响,模块易于替换、增减、移植,增强了系统的稳定性和灵活性。

(2) 能够为作业配备较佳资源

客户机/服务器模式可以针对应用和服务的不同要求,以及针对不同的处理要求来配置相应的资源,取得最佳的性能/价格比,提高了服务质量、集成水平和事务处理能力。

(3) 大大减低了系统的开发成本和风险

客户机/服务器模式便于类似系统的开发,它提供了一个开发框架,缩短了解决问题的时间,减少了风险,有利于快速解决问题,能将开发过程中的重复劳动减少到最少。同时,它可以在较低廉的工作站上开发完成,然后移植到较昂贵的产品系统中,大大减少了开发费用。

(4) 便于维护和应用

客户机/服务器模式为系统人员提供了一个共同的后台(服务器)环境,为用户提供了一个友好的操作环境,便于维护和使用。

8.2.3 客户机/服务器的应用方式

客户机/服务器计算模式可以有4种应用方式,下面分别介绍它们的特点。

1. 客户机类应用方式

在客户机类应用方式中,所有的处理均由客户机承担,服务器仅提供文件类服务,如磁盘服务、文件服务和打印服务等。这种应用方式也称为胖客户机方式。由于全部任务都放在客户机中运行,使得配置较高的服务器的CPU优势不能得以有效发挥,系统的效率不高(特别是在进行大量的数据检索时,要把每一条记录都读到工作站内存,通信的开销很大),数据的一致性、安全性难于保证(因为每一台计算机中都要存放数据库的一个副本)。

2. 服务器类应用方式

在服务器类应用方式中,所有的处理均由服务器承担,客户机仅作为仿真终端注册到服务器。这类应用方式是一种瘦客户机方式。所谓的Windows终端就

是一种最典型的瘦客户机,其100%的处理在服务器上进行。

3. 客户机/服务器类应用方式

这是最典型的客户机/服务器计算模式:处理由客户机和服务器共同承担,服务器运行后台进程、集中处理作业,客户机负责同用户打交道,从而真正发挥了网络优势。所谓的浏览器/服务器(Browser/Server)就是一种典型的客户机/服务器类应用。

4. 分布处理类应用方式

在分布处理类应用中,多台服务器协同承担处理工作。图8.3给出了集中式服务器结构与分布式服务器结构的区别。

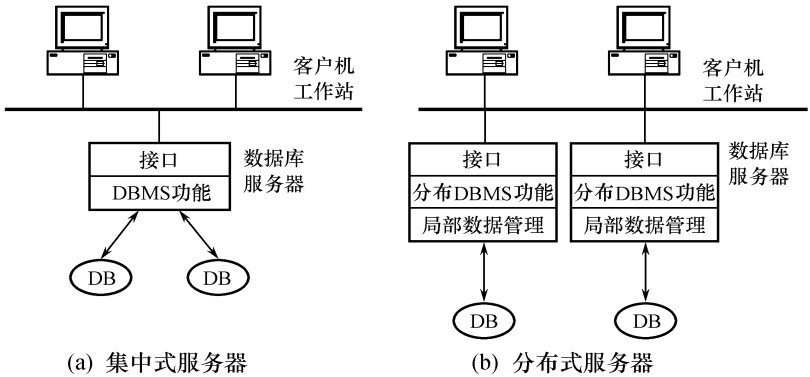


图8.3　集中式服务器结构与分布式服务器结构的区别

8.3　中　间　件

8.3.1　中间件的概念

客户机/服务器模式因其众多的优点,已经获得广泛的应用。但是,它给程序员带来不少困难。这是因为,在网络环境下要编写客户机端和服务器端的应用程序,将涉及多个平台、多个协议和多种程序设计语言。这对大多数程序员来说仍然是相当困难的:如果要针对网络的低层协议来编写应用程序,则将使程序过多地依赖低层网络技术,很难集成新的网络服务;如果程序在很多地方依赖于高层协议和网络软件,则程序将很难编写,也难于移植到其他操作系统环境中。

中间件(Middleware)就是解决这一难题的一条出路。如图8.4所示,它的基本思路是在客户机与服务器或应用程序与操作系统之间增加一层可复用的软

件,通过一个高层应用程序编程接口 API(Application Program Interface),将低层网络协议与实现技术(即网络与应用)隔离开来,使程序员能将精力集中在应用软件的设计上。

图8.4 中间件的作用

世界著名咨询机构IDC将中间件定义为:

中间件是一类软件,而非一种软件;

中间件是基于分布式处理的软件,它最突出的特点是其网络通信功能;

中间件不仅仅实现互连,还要实现应用间的互操作。

中间件可以为程序员提供高层、跨平台、多协议的接口,提供一些简单而功能强大的命令,使得在客户机/服务器模式下的编程变得简单而有效,同时还提供数据转换、差错检测、安全管理、队列和命令等多种功能。

世界著名咨询机构Standish Group在一份研究报告中归纳了中间件的十大优越性:

缩短应用的开发周期;

节约应用的开发成本;

减少系统初期的建设成本;

降低应用开发的失败率;

保护已有的投资;

简化应用集成;

减少维护费用;

提高应用的开发质量;

保证技术进步的连续性;

增强应用的生命力。

8.3.2 中间件的类型

随着计算机软件技术的发展,中间件技术已经日渐成熟,并开始商品化。按照IDC的分类方法,中间件可以分为6类。

(1) 消息中间件

消息中间件能够在客户和服务器之间提供同步或异步的连接,并且在任何

时刻都可以将消息进行传送或存储转发。消息中间件不会占用太多的网络带宽,可以跟踪事务,并通过将事务存储到磁盘上实现故障时的系统恢复。这类中间件的典型产品有 IBM CICS、BEA Message Q、东方通科技 TongLink/Q、清华紫光北美 TH-MT 等。

(2) 交易中间件

交易中间件适合于联机交易处理系统,主要功能是管理分布于不同计算机上数据的一致性、保障系统处理的效率和负载均衡。交易中间件遵循的主要标准是 X/OPEN DTP 模型,典型的产品有 IBM CICS、BEA Tuxedo、东方通科技 TongEasy。

(3) 对象中间件

就像用集成模块和扩展板装配计算机一样,对象中间件能为用户或开发者提供即插即用的互操作性。主要的对象中间件标准是 CORBA 的软构件框架。典型的对象中间件产品有 Borland VisiBroker、东方通科技 TongBroker。

(4) 应用服务器

应用服务器是一种基于组件的中间层集成框架,它为组件的运行提供运行时环境、基础服务和管理等功能。应用服务器和组件各司其职,它们之间由明确规定好的接口进行管理。这类中间件一般基于 J2EE 工业标准,主要典型产品有 IBM Websphere、BEA Webiogic、东方通科技 TongWeb、HP Total-e-Server。

(5) 安全中间件

安全中间件是以公钥基础设施(PKI)为核心,建立在一系列相关国际安全标准之上的一个开放式应用开发平台,它向上为应用系统提供开放的接口,向下提供统一的密码算法接口及各种 IC 卡、安全芯片等设备的驱动接口。典型产品有东方通科技 TongSec。

(6) 应用集成服务器

应用集成服务器能把工作流和应用开发技术(如消息及分布构件)结合在一起,能自动地将构件、Script 应用、工作流行为结合在一起处理,同时也集成文档和电子邮件。典型产品有 IBM Flowmark。

习　　题

8.1　说明进程与程序的关系。

8.2　说明进程之间有哪些通信方式。

8.3　试描述客户机/服务器环境的概念与体系结构。

8.4　使用客户机/服务器模式的主要优点有哪些?

8.5　试述中间件的用途。

第 9 章　客户机/服务器模式应用举例

在 Internet 的运输层(也称传输层)中,数据传输过程总是从一端首先发起请求开始的,因而基于传输服务的应用基本上都采用了客户机/服务器模式工作。下面介绍一些基本应用。

9.1　远程登录 TELNET

9.1.1　本地注册与远程登录

1. 注册

在一个计算机系统中,当用户从键盘上键入一个字符时,这个字符就会被发送到计算机主机中,同时回送到监视器。一个较大的计算机系统可以支持多个用户,每个用户通过一个终端(通常由键盘、监视器和鼠标组成)与计算机交互;计算机分时地处理每个用户的请求,并使每个用户都有独享计算机的感觉,可以使用系统资源,从一个程序切换到另一个程序,但是实际上,每个用户都只有系统分配的使用特定系统资源的权力。为了识别用户,在分时操作系统环境中,要给每一个授权用户一个标识,通常还有一个口令。用户标识定义用户是系统的一部分,用户口令用于安全性检查,防止非授权用户使用系统资源。因此,一个用户要接入到多用户系统中时,首先要使用用户标识或注册名字登录到系统,系统还要进行口令检查。

2. 本地注册

本地注册是指用户注册到本地的分时系统上。如图 9.1 所示,当用户在终端上键入字符时,键入的字符就被终端驱动程序接收;接着操作系统就解释字符组合,并调用所使用的应用程序。

3. 远程登录 TELNET

远程登录 TELNET 是终端网络(TErminaL NETwork)的缩写,它可以建立一个远程 TCP 连接,让用户(使用主机名和 IP 地址)注册到远地的一个主机上。这时,用户把击键信号传到远地主机,也把远地主机的输出通过 TCP 连接返回到

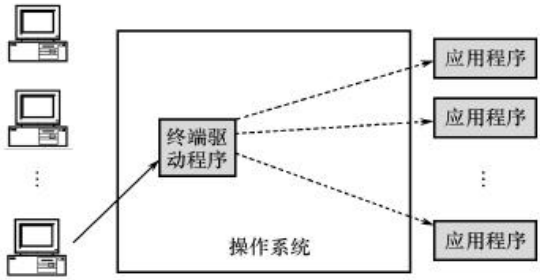


图9.1　本地注册

本地屏幕,好像使用的终端直接连接到远地主机上一样。

如图9.2所示,TELNET采用客户机/服务器模式。在本地系统上运行TELNET客户进程,在远地主机上运行TELNET服务器进程,并使用TCP/IP协议传输数据。连接由客户机发起,服务器在端口23上监听。用户在终端上键入字符发送给终端驱动程序,同时本地操作系统接收这些字符,但不解释它们。这些字符被送到TELNET客户程序,由客户程序将它们转换成网络虚拟终端(NVT,Network Virtual Terminal)字符的通用字符集,然后再送入本地TCP/IP堆栈。这些NVT形式的命令或正文通过Internet传送到远程的计算机中,由远程机器中的TCP/IP堆栈接收并交付操作系统,然后递交给TELNET服务器将这些字符转换为远程计算机可以解释的相应字符。这些字符还不能直接交给操作系统,因为远程操作系统不接收来自TELNET服务器的字符,只接收来自终端的字符。为此增加一个伪终端驱动程序模块,它将这些字符伪装成好像是从一个本地终端发来的,这样,操作系统就能将这些字符传递给适当的程序了。

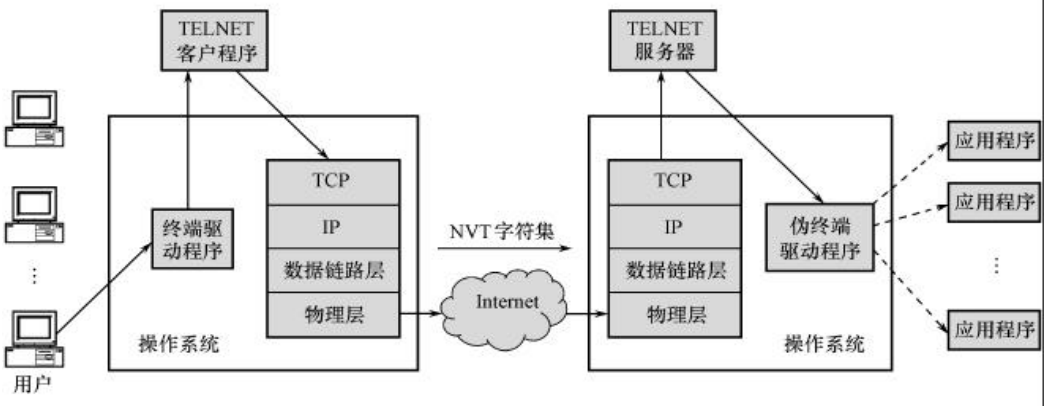


图9.2　远程登录

网络虚拟终端是一种双向字符设备。它包括一个字符输出(显示)设备和一个输入键盘。显示设备负责响应从 TELNET 连接上接收到的数据,键盘通过 TELNET 连接向对方发送数据。当要求回显时,显示设备也响应键盘产生的数据。

9.1.2　NVT 字符集

TELNET 之所以能够适应许多计算机和操作系统的差别,其核心在于网络虚拟终端 NVT。NVT 定义了一种服务器系统格式与客户机系统格式之间的中间格式——NVT 字符集,实现它们的相互转换,而不是主机或客户机间直接将本方的信息转换成对方设备能识别的信息。

NVT 字符集包括两个 8bit 的字符集,如图 9.3 所示,其中一个为数据字符集,供数据用;另一个为远程控制字符集,供远程控制用。

图 9.3　NVT 字符

1. 数据字符集

NVT 使用的数据字符集的低 7 位与 US ASCII 的低 7 位相同,但最高位为 0,称为 NVT ASCII,目的是在客户机与服务器之间使用选项,协商取得一致。

2. 远程控制字符集

控制字符集用于在客户机和服务器之间传送控制字符,为与数据字符相区别,最高位为 1。表 9.1 列出了一些 NVT 远程控制字符及其表示和意义。

表 9.1　一些 NVT 远程控制字符及其表示和意义

字符	十进制码	二进制码	意义
EOF	236	11101100	文件结束
EOR	239	11101111	记录结束
SE	240	11110000	子选项结束
NOP	241	11110001	无操作
DM	242	11110010	数据标记
BRK	243	11110011	断开
IP	244	11110100	中断进程
AO	245	11110101	异常中止输出

续表

字符	十进制码	二进制码	意义
AYT	246	11110110	对方是否还在运行
EC	247	11110111	擦除最后一个字符
EL	248	11111000	擦除行
GA	249	11111001	前进
SB	250	11111010	子选项开始
WILL	251	11111011	同意激活(enable)选项
WONT	252	11111100	拒绝激活选项
DO	253	11111101	认可选项请求
DONT	254	11111110	拒绝选项请求
IAC	255	11111111	作为控制解释(下一个字符)

说明：

(1) 在TELNET的默认实现中,用户不能在本地进行编辑,编辑工作必须在远程服务器处进行。因此,所有的编辑操作(如字符擦除)都要转换成控制字符(如擦除字符为EC)。

(2) 在客户机与服务器之间的数据流中,为了区分数据字符和远程控制字符,要在每个远程控制字符序列前面加一个特殊的控制字符——IAC(解释为控制)。例如,某用户想发送数据字符序列

cat file2

但在键盘上将f敲成了g,随后即改正时的敲键过程为

cat g<backspace>file2

而从客户机传向服务器端的字符流如图9.4所示。

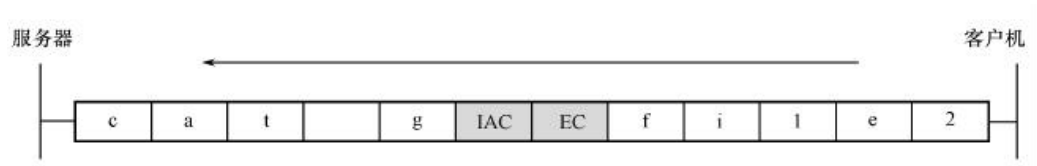


图9.4　将控制字符嵌入到数据流中的例子

3. 协商选项

(1) 选项

在远程登录过程中,使用选项可以使不同的终端获得不同的服务特性。表9.2为常用的一些协商选项。

表 9.2　一些常用协商选项

选项	代码	意义
二进制	0	使用 8 位二进制传输
回显	1	将收到的数据送回发送端显示
状态	5	请求 TELNET 的状态
定时标记	6	定义定时标记
终端类型	24	设置终端类型
终端速率	32	设置终端速率
行方式	34	改变到行方式

(2) 选项协商

客户与服务器在服务使用前或服务使用中首先要对选项进行协商。有的选项由服务器激活,有的仅能由客户激活,还有一些则可以由服务器或客户激活。一个选项需要通过提供(Offer)或请求激活。一方可以提供激活一个选项,另一方可以同意或不同意该提供。

选项协商要使用如下 4 种控制字符:

WILL　提供激活选项
　　接受请求激活选项
WONT　拒绝请求激活选项
　　提供禁止选项
　　接受禁止选项
DO　同意提供激活选项
　　请求激活选项
DONT　不同意提供激活选项
　　同意提供禁止选项
　　请求禁止选项

图 9.5 所示为一个选项协商的例子。在这个例子中,客户希望服务器将收到的每一个字符回显。

(3) 子选项协商

子选项是为某些选项附加的信息,如终端的具体类型或速率。在进行子选项协商时,要使用控制字符 SE(子选项结束)和 SB(子选项开始)。图 9.6 所示为一个子选项协商的例子。

4. 对服务器的控制

如同可以通过控制字符对本地程序进行中断、字符擦除等控制一样,在

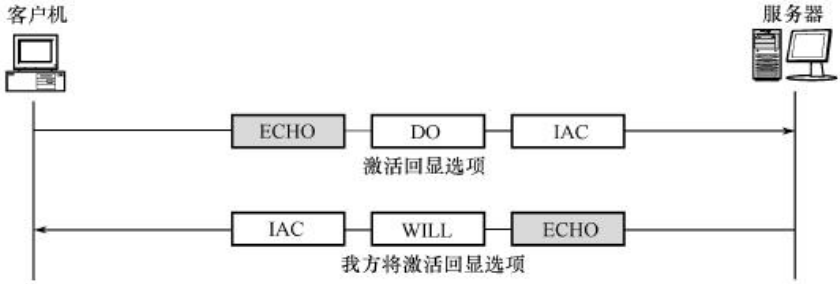


图9.5　一个回显选项协商的例子

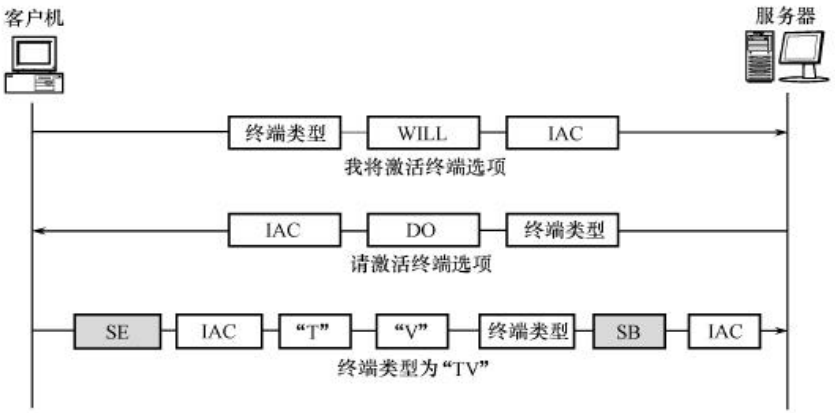


图9.6　一个子选项协商的例子

TELNET方式下,也可以运用控制字符对远程计算机——服务器上运行的程序进行控制。表9.1中的IP、AO、AYT、EC和EL即为可以发送到服务器上对所运行程序进行控制的字符。

5. 带外信令

在正常情况下,服务器端缓冲区满时,就不能再接收输入数据了。然而,当服务器应用程序进入无限循环后,即使服务器应用程序不再接收任何输入数据,用户也要设法使之中断。这时,客户机端就必须发送紧急TCP报文,迫使服务器接收。

TELNET用特殊字符DM(数据标记)表示紧急TCP报文。也就是说,当服务器应用程序看到嵌入有DM标记的报文时,就读取它,并丢弃DM前的其他数据,而把DM后面的数据按正常处理。

图9.7所示为一个带外指令传输的例子。服务器将接收DM后面的数据,而丢弃其前面的数据和字符。

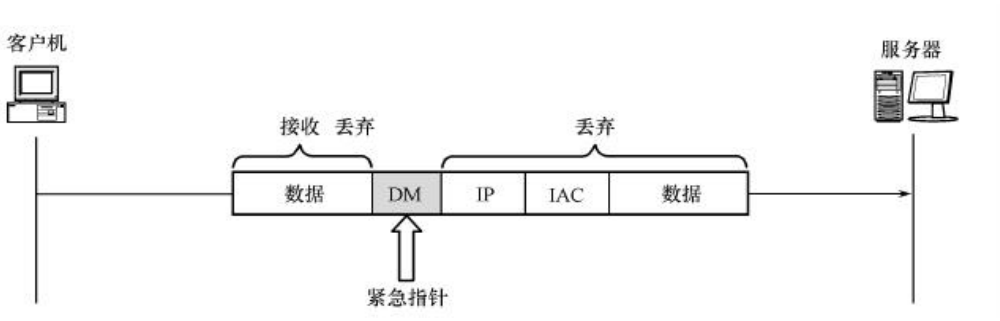


图 9.7　带外指令传输举例

6. 转义字符

在 TELNET 工作方式下,通常用户键入的字符是发给服务器进行解释的。有时,也希望由客户机解释。这时就要使用转义字符。转义字符通常以 Ctrl+] (^]) 打头,表示是给客户机解释的。

9.1.3　TELNET 工作方式和用户命令

1. 工作方式

TELNET 一般具有 3 种工作方式。

(1) 默认方式

在默认方式下,回显由客户机完成。用户键入一个字符,客户机将将其回显到显示器或打印机,并且要到一行完成才发送。发送后,在接受新行之前,要等待来自服务器的 GA (前进) 命令。它是一种半双工操作方式,当 TCP 连接是全双工时,效率较低,已经属于陈旧工作方式。

(2) 字符方式

在字符方式下,回显由服务器完成。用户键入一个字符,就从客户发给服务器,服务器再将该字符回显在客户的屏幕上。在这种方式下,对每一个字符,将在客户与服务器之间传送 3 个 TCP 报文段:

用户键入一个字符,发给服务器;

服务器确认收到的字符,将该字符回显回去;

客户确认收到回显的字符。

因此,这种方式的字符回显会被延迟,并将产生较大的网络开销。

(3) 行方式

在行方式下,行的编辑 (回显、字符擦除、行擦除等) 由客户完成,然后客户将整个行发给服务器。这种方式可以弥补默认方式和字符方式的不足。

2. 用户命令

前面的命令非常难记,因此用户实际使用的命令要由操作系统转换为用户友好的命令。表9.3列出了一组用户命令。

表9.3　一组用户命令

命令	意义	命令	意义
open	连接到远程计算机	set	设置操作参数
close	关闭当前连接	status	显示状态信息
display	显示操作参数	send	发送特殊字符
mode	改变到行方式或字符方式	quit	退出TELNET

9.2　文件传输协议FTP

FTP (File Transfer Protocol) 是TCP/IP提供的标准机制,用来从一个主机将文件复制到另一个主机。它提供交互式的访问,允许客户机指明文件的类型、格式(如是否用ASCII码等)、存取权限(授权,口令等)。

在网络环境中,由于众多计算机厂商研制的文件系统存在差异,给文件传输带来许多困难:

数据存储原型不一致;

文件命名规定不一致;

对于同一功能,操作命令因操作系统而异;

访问控制方法不一致。

FTP的功能就是减少或消除在不同操作系统下处理文件时的不兼容性。

9.2.1　FTP模型

1. FTP模型概述

FTP采用C/S模式。图9.8给出了FTP的基本模型。

可以看出,一个FTP基本模型由如下部分组成:

(1) 两种连接

控制连接;

数据连接。

(2) 3个客户组件

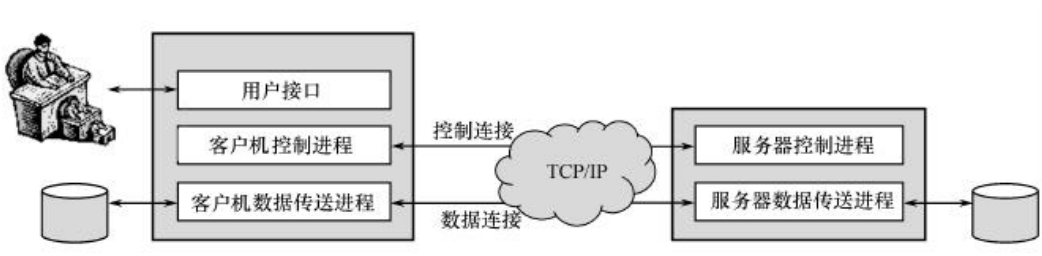


图 9.8　FTP 基本模型

用户接口；

客户机控制进程；

客户机数据传送进程。

(3) 2 个服务器组件

服务器控制进程；

服务器数据传送进程。

2. FTP 连接及其工作过程

FTP 的一个 FTP 服务器进程可以同时为多个客户机进程提供服务。FTP 的服务进程由两大部分组成：一个主进程——负责接受新的请求；多个从进程——处理单个请求。传送使用 TCP 可靠的传输服务。主进程工作时，要先打开众所周知端口 (21)，使客户机进程能够连接。然后，主进程与从进程并行地工作：主进程等待并接收客户机进程发出连接请求；从进程处理进程发来的请求。从进程对客户机进程的请求处理完后即终止，但从进程在运行期间可能会创建其他子进程。

FTP 的一个技术特点是具有两条连接：一条用于数据传送，使用众所周知端口 20；另一条用于控制信息 (命令和响应) 的传送，使用众所周知端口 21。将命令和数据传送分开，将使 FTP 的传送效率更高，并且控制连接与数据连接不会发生混乱。

9.2.2　FTP 文件传输过程

1. 数据连接过程

控制进程接收到客户机的文件传输请求后，即创建数据传送进程同时创建数据连接。数据连接创建过程如下：

① 客户机使用短暂端口发出被动打开命令 (客户机发出传送文件命令)；

② 客户机将该端口号发给服务器；

③ 服务器发出主动打开命令，建立众所周知端口 20 与客户使用的短暂端

口之间的连接。

此后便由数据传送进程完成文件的传送。传送结束后关闭数据传送连接并结束运行。

2. 文件属性

在数据传输之前,必须通过控制连接定义要传送文件的如下3个属性。

(1) 文件类型

ASCII文件:这是传送正文文件的默认格式,每一个字符使用NVT ASCII进行编码。

EBCDIC文件。

图像文件:作为连续的比特流传送,没有解释和编码,是二进制文件的默认格式。

若文件为ASCII或EBCDIC编码,还要定义文件的可打印性。

(2) 数据结构

文件结构(默认):无结构的字节流文件。

记录结构:记录型文件,用于正文文件。

页面结构:页面可以顺序地存取。

(3) 传输方式

流方式(默认):数据以字节流方式由FTP交给TCP,由TCP将数据划分为合适大小的报文块。对记录结构,每一记录要增加一个EOR(记录结束符),文件最后增加一个EOF(文件结束符);对文件结构,不需要文件结束符。

块方式:数据以块方式由FTP交给TCP。每块前增加3字节用做块描述和指示块大小。

压缩方式。

3. 文件传输方式

读取文件:从服务器将一个文件复制到客户机。

存储文件:从客户机将一个文件复制到服务器。

从服务器将目录列表或文件名以文件形式发送到客户机。

4. 控制连接上的通信

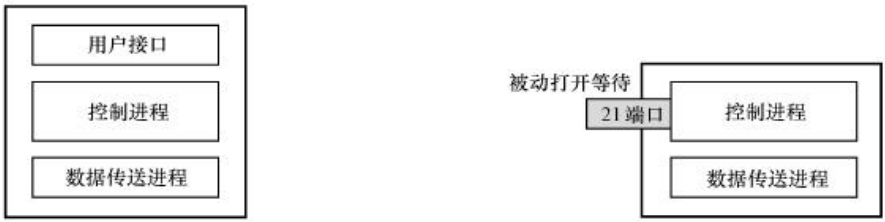
FTP控制连接上的通信方法与TELNET相同:使用NVT ASCII字符集,并通过命令和响应完成。

系统在创建从进程(控制进程)时随之创建控制连接。这个控制连接的创建过程如下:

① 服务器在众所周知端口21发出被动打开命令,等待客户机连接(如图

9.9(a)所示);

② 客户机使用短暂端口发出主动打开命令(如图9.9(b)所示)。



(a) 服务器在众所周知端口21发出被动打开命令

(b) 客户机使用短暂端口发出主动打开命令

图9.9　创建控制连接

控制连接在整个会话期间一直打开,随时准备接收客户机的文件传输请求。

在控制连接上传送的是命令与响应。命令由客户机发向服务器;响应由服务器发回客户机。每一个命令产生至少一个响应。

(1) 命令

FTP发送的命令用大写的ASCII码表示,后面可以带有参数。这些命令大致可以分为6组。

接入命令:使用户接入到远程系统。

文件管理命令:使用户接入到远程计算机的文件系统。

数据格式化命令:供用户定义数据结构、文件类型和传输方式。

端口定义命令:定义客户机端的端口号。

文件传输命令:供用户传输文件。

杂项命令:将信息交付给客户端的FTP用户。

表9.4列举了一些常用命令。

(2) 响应

响应是对命令的回应。每一个FTP命令至少产生一个响应。一个响应包含两个部分:正文和它后面的3个数字。正文部分定义所需的参数或额外的解释,数字部分定义代码。

表9.4　常用命令

类型	命令	参数	说明
接入	USER	用户标识符	用户信息
	PASS	用户口令	口令
	ACCT	应付费的账务	服务信息
	REIN		重新初始化
	QUIT		向系统注销
	ABOR		前面的命令异常中止
文件管理	CWD	目录名	改变到另一个目录
	CDUP		改变到父目录
	DELE	文件名	删除文件
	LIST	目录名	列出子目录或文件
	NLIST	目录名	列出子目录名或无其他属性的文件
	MKD	目录名	创建新目录
	PWD		显示当前目录名
	RMD	目录名	删除目录
	RNFR	文件名(旧文件名)	标识要重新命名的文件
	RNTO	文件名(新文件名)	重新命名文件
	SMNT	文件系统名	安装文件系统
数据格式化	TYPE	A(ASCII),E(EBCDIC),I(图像),N(非打印)或T(TELNET)	定义文件类型和当需要时定义打印格式
	STRU	F(文件),R(记录)或P(页面)	定义数据的组织
	MODE	S(流),B(块)或C(压缩)	定义传输方式
端口定义	PORT	6个数字的标识符	客户选择端口
	PASV		服务器选择端口
文件传输	RETR	文件名	读取文件,文件从服务器传送到客户
	STOR	文件名	存储文件,文件从客户传送到服务器
	APPE	文件名	与STOR相似,但文件存在就必须附加数据
	STOU	文件名	与STOR相似,但文件名在目录中须惟一,且已存在文件不能被重写
	ALLO	文件名	在服务器为文件分配存储空间
	REST	文件名	在指明的数据点给文件标记确定位置
	STAT	文件名	返回文件的状态
杂项	HELP		询问关于服务器的信息
	NOOP		检查服务器是否工作
	SITE	命令	指明特定网点
	SYST		询问服务器使用的操作系统

第 1 个数字 x 定义命令的状态：

x=1,正面初步回答——动作已经开始；

x=2,正面完成回答——动作已经完成；

x=3,正面中间回答——命令已经接受；

x=4,过渡负面完成回答——无动作发生；

x=5,永久负面完成回答——命令没有接受。

第 2 个数字 y 定义命令的下列状态：

y=0,语法；

y=1,信息；

y=2,连接；

y=3,鉴别和记费；

y=4,未指明；

y=5,文件系统。

第 3 个数字 z 提供附加信息（详细内容参阅有关资料）。

5. 文件传送举例

例 9.1 图 9.10 所示为一个使用 FTP 读取目录中项目清单的例子。

说明：

① 在端口 21 上创建了控制连接后，FTP 服务器在控制连接上发出 220（服务器就绪）响应。

② 客户机发送 USER，提供用户标识符。

③ 服务器发出 331（用户名 OK，口令？）响应。

④ 客户机用 PASS 命令回答口令。

⑤ 服务器发出 230（用户注册 OK）响应。

⑥ 客户机为在短暂端口上进行数据连接的被动打开，在控制连接上发送 PORT 命令，将该端口号送给服务器。

⑦ 服务器发出 150（数据连接即将打开）响应，即不立即打开客户机的被动数据连接，而是准备发出主动打开，实现在端口 20 与客户机收到的短暂端口之间的数据连接。

⑧ 客户机发送 LIST 报文。

⑨ 服务器发出 125（数据连接 OK）响应。

⑩ 服务器在数据连接上传送文件列表或目录。

⑪ 服务器在数据连接上传送文件列表或目录结束后，发出响应 226（关闭数据连接）。

⑫ 这时，客户机可以有两种选择——请求关闭控制连接或开始另一个活

动。这里选择用命令 QUIT 请求关闭控制连接。

⑬ 服务器发出响应 221（服务关闭）。

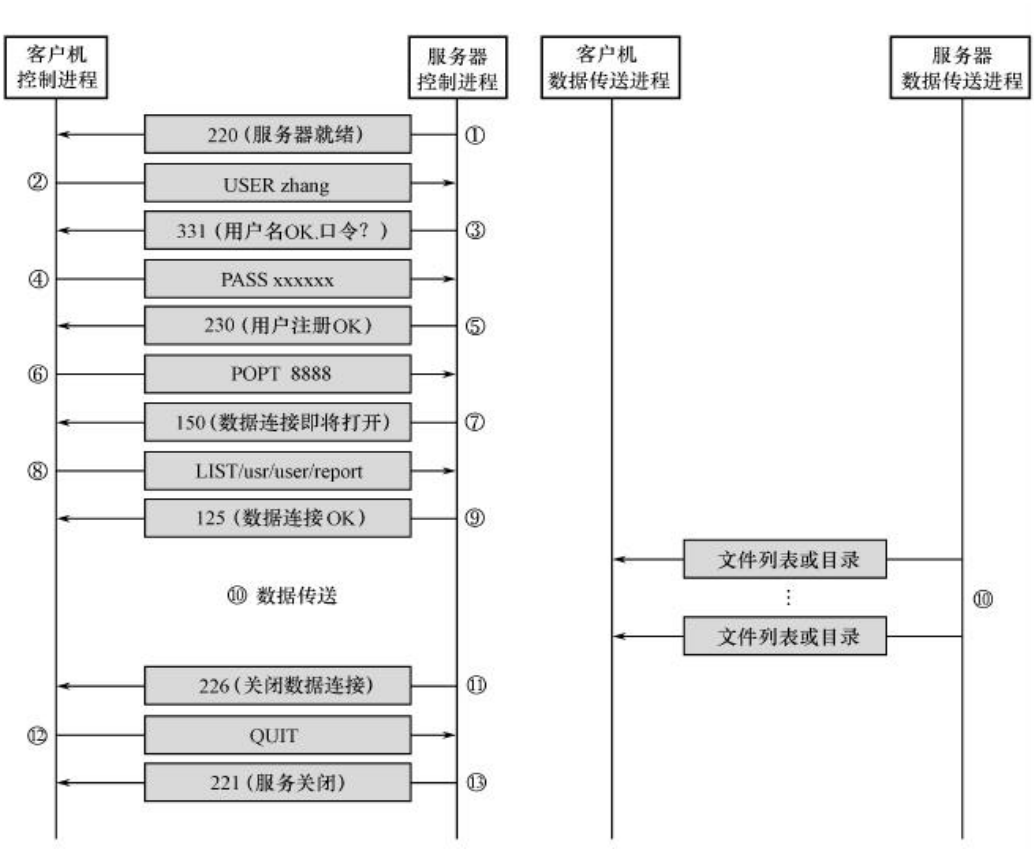


图 9.10　FTP 传送文件列表或目录

6. 用户接口

多数操作系统提供友好的用户接口来接入 FTP 服务。下面是 UNIX FTP 提供的用户接口命令清单：

!,$,account,append,ascii,bell,binary,bye,case,cd,cdup,close,cr,delete,debug,dir,disconnect,from,get,glob,hash,help,lcd,ls,macdef,mdelete,mdir,mget,mkdir,mls,mode,mput,nmap,ntrans,open,prompt,proxy,sendport,put,pwd,quit,quote,recv,remotehelp,rename,reset,rmdir,runique,send,status,struct,sunique,tenex,trace,type,user,verbose,?

例 9.2　例 9.1 的用户接口。

```
% ftp if.sytu.edu.cn
220 Server ready
```

```
Name :abcd
Password :xxxxxx
ftp> ls/usr/user/report
200 OK
150 Opening ASCII mode
……
……
226 transfer complete
ftp> close
221 Goodbye
ftp> quit
```

说明:加下划线的部分为用户键入的内容,其他为系统的提示或响应信息。

9.3 电子邮件传送协议

电子邮件(E-mail)是Internet上一种最广泛的应用。Internet的电子邮件系统采用了客户机/服务器方式,电子邮件的发送与接收由E-mail客户程序和服务程序共同完成。

9.3.1 电子邮件系统的基本原理

1. 电子邮件系统的一般构成

电子邮件系统的一般结构如图9.11所示。其最主要的部件是用户代理UA和邮件传送代理MTA。

(1) 用户代理UA(User Agent)

UA通常是一个用来发送和接收邮件的程序。它的主要功能是:

发送邮件——UA为用户准备报文、创建信封,并将报文装进信封,暂存起来。

接收邮件——UA定期检查邮箱,若有邮件到来,就向用户发出一个通知;若用户打开信箱准备读取邮件,则显示一个清单,给出邮件的有关信息,包括发信人地址、主题、发送时间等。

有些用户代理具有额外的用户接口,向用户提供可视化的交互服务。

(2) 邮件传送代理MTA(Mail Transfer Agent)

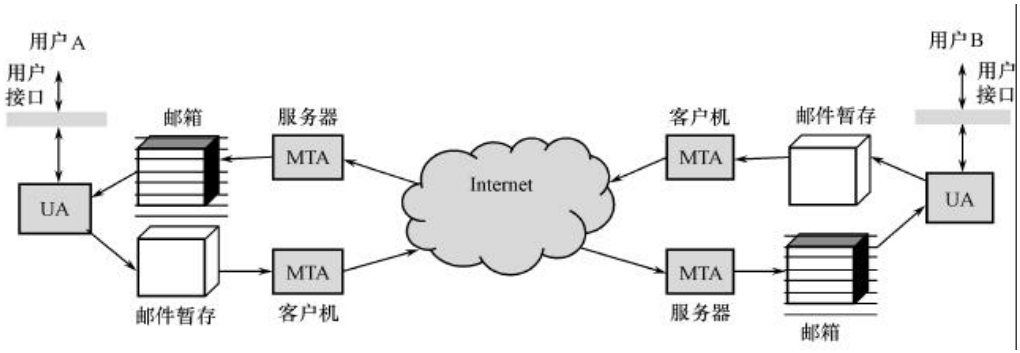


图9.11　E-mail的客户机/服务器工作方式

MTA相当于现实生活中的邮局,担任邮件传送工作。MTA分为客户机MTA和服务器MTA。客户机MTA的功能是将暂存的邮件传送到Internet上;服务器MTA的功能是将Internet上送来的邮件传送到客户端的邮箱中。

除此之外,为了让用户能与子网外的用户通信,还必须有一台中继主机(Relay Host)或邮件网关(Gateway)。Relay Host是一个运行一种或一种以上Mail相关协议的系统,这些协议称为Mailer。每个Mailer指定一种策略和机制,用于发送或接收邮件;并且接收方Relay Host的Mailer必须与发送方一致,两个系统才能互通邮件。Gateway负责在运行不同协议的网络间传递邮件,即允许发送方的Mailer与接收方不同。不过,网关的配置是非常复杂的。

2. 电子邮件访问模式

电子邮件一旦被传送到邮件存储服务器后,接收者就可以访问它。接收者对电子邮件服务器的访问方式有3种:离线(Offline)、在线(Online)和断线(Disconnected)。

(1) 离线模式

离线模式是最基本的模式。按照该模式工作时,客户与邮件存储服务器相连接后,下载所有的接收者邮件,同时从服务器中删除;然后在本地客户机上存储、处理。这种模式简单,仅需要最小的服务器连接时间。

(2) 在线模式

在线模式中所有的邮件处理和操作都在服务器上进行(尽管在某些情况下用户可以把邮件下载到本地客户机上,但服务器中仍然保留)。当用户需要访问并处理电子邮件时,必须与服务器保持连接。这样,客户机的配置可以很小,但服务器需要有足够的带宽和资源供多个用户分享。

(3) 断线模式

断线模式综合了离线模式和在线模式的特点。在这种模式中,用户能下载

邮件在本地客户机上离线处理,但服务器仍然是邮件的保存处。用户处理完邮件后,可再次与邮件服务器连接并上载所有的改变。采用这种模式,客户机与服务器的连接时间更短,但不论客户机还是服务器都需要足够的资源。

9.3.2　简单邮件传输协议 SMTP

SMTP (Simple Mail Transfer Protocol) 是 Internet 上各 MTA 站点之间的电子邮件传送协议。它规定了 14 条命令和 21 种响应,主要用于连接建立、邮件传送、连接释放。

1. 命令

SMTP 的 14 条命令分为 3 类:

(1) 强制性命令 (5 条) ——每一种实现都必须支持;

(2) 常用命令 (3 条) ;

(3) 极少使用的命令 (6 条)

表 9.5 为 SMTP 命令列表。

表 9.5　SMTP 命令列表

类　型	命令	变量	说明
强制命令	HELO	发送 (客户机) 端主机名	客户机用来标识自己
	MAIL FROM	发信人	客户机用来标识发信人
	RCPT TO	预期收信人	客户机用来标识预期收信人
	DATA	邮件主体	用来发送真正的报文
	QUIT		用来结束报文
常用命令	RSET		复位,用来使当前邮件事务异常中止
	VRFY	需要验证收信人的名字	用来验证收信人地址
	NOOP		客户机用来检查收信人状态
极少使用的命令	TURN		让收信人和发信人交换位置
	EXPN	需扩展的邮件发送清单	要求接收主机将作为变量的发送清单进行扩充,并返回清单收信人邮箱地址
	HELP	命令名	要求收信人发送关于作为变量的命令的信息
	SEND	预期的收信人	指明邮件要交付给收信人终端,而不是邮箱
	SMOL	预期的收信人	指明邮件要交付给收信人终端或邮箱
	SMAL	预期的收信人	指明邮件要交付给收信人终端和邮箱

2. 响应

响应由 3 位数字组成。第 1 位数字 x 的意义如下:

x=2,正面完成回答；

x=3,正面中间回答；

x=4,过渡负面完成回答；

x=5,永久负面完成回答。

后面两位提供更多的内容,这里不再介绍。

3. 邮件传送过程

一个邮件报文的传送过程需要经过3个阶段。

(1) 建立连接

在使用SMTP进行邮件传送过程中,客户机(发送)端使用短暂端口,服务器(接收)端使用众所周知端口25。当客户与服务器的众所周知端口25之间建立了一条TCP连接后,SMTP服务器就开始其连接阶段。下面说明其步骤。

① 服务器发送响应：

已经准备好,发送220(服务就绪)；

未准备好,发送421(服务不可用)。

② 接到响应220,客户机发送HELLO,并用域名标识自己。

③ 服务器发送响应250(请求命令完成)。

(2) 传送报文

SMTP在客户机与服务器之间建立连接后,发信人就可以向一个或多个收信人发送一个单独的报文了。步骤如下：

① 客户机发送MAIL报文,介绍发送者(包括邮件地址)；

② 服务器发送响应250(请求命令完成)或其他适当代码；

③ 客户机发送RCPT(收信人)报文,包括收信人的邮件地址；

④ 服务器发送响应250(请求命令完成)或其他适当代码；

⑤ 客户机发送DATA报文,对报文的传送初始化；

⑥ 服务器发送响应354(开始邮件输入)或其他适当代码；

⑦ 客户机用连续的行发送报文内容；

⑧ 服务器发送响应250(OK)或其他适当代码。

(3) 终止连接

报文传送成功后,客户就可以终止连接。步骤如下：

① 客户机发送QUIT命令；

② 服务器发送响应221(服务器关闭传输信道)或其他适当代码。

9.3.3　其他几个重要的电子邮件协议

1. 通用 Internet 邮件扩充标准 MIME (Multipurpose Internet Mail Extensions)

MIME 是一个辅助协议,它允许非 ASCII 数据通过 SMTP 传送。为适应任意信息数据类型的表示,每个 MIME 报文要包含告知收信人数据类型和所使用编码的信息。基本内容类型 (Content-Type) 用于说明邮件的性质。表 9.6 列出了 7 种基本内容类型及其含义。

表 9.6　7 种 MIME Content-Type 的基本内容类型及其含义

基本内容类型	子类型	说明
text	plain	无格式文本
	richtext	有少量格式命令的文本
image	gif	GIF 格式静止图像
	jpeg	JPEG 格式静止图像
audio	basic	可听见声音
video	mpeg	MPEG 格式影片
application	octet-stream	不间断字节序列
	postscript	PostScript 可打印文件
message	rfc822	MIME RFC 822 邮件
	partial	为传输而分割开的邮件
	external-body	从网上获取文件
multipart	mixed	按规定顺序的几个独立部分
	alternative	不同格式的同一文件
	parallel	必须同时读取的几部分
	digest	每一部分都是完整的 RFC 822 邮件

2. 邮局协议 POP (Post Office Protocol) 和 Internet 报文存取协议 IMAP (Internet Message Access Protocol)

以拨号连接方式进行电子邮件传送的用户一般都要使用 POP。这种情况下,用户要先拨号,与邮箱所在的计算机建立连接;连接成功就可以运行 POP 客户程序,与远地的 POP 服务器程序通信,发送或接收电子邮件。

POP 是一个脱机协议,它是一个具有存储转发功能的中间服务器,用户每次打开邮箱,即将邮箱中接收到的邮件一次性取回到自己的计算机,POP 服务器就不再保存这些邮件,用户与 POP 服务器的连接即中断。此后,用户便可以在自

己的计算机上自由地处理收到的邮件。

IMAP是美国斯坦福大学从1986年就开始开发的与POP3对应的一种多重邮箱电子邮件协议。它能从邮件服务器上获取有关E-mail信息或直接收取邮件,具有高性能和可扩展的优点。它提供在线、离线、分离三种操作模式,使用户可以在远地操纵服务器邮箱。所有收到的邮件要先送到IMAP服务器。用户需要打开某个邮件,才能将该邮件传到自己的计算机中,在用户未发出删除该邮件的操作之前,它一直保存在IMAP服务器上。显然,使用IMAP与使用POP相比,用户与服务器的连接频繁且时间长。

应当注意,IMAP和POP是用户与本地邮件服务器之间的邮件传送协议,而SMTP是Internet上各MTA站点之间的电子邮件传送协议。

3. 邮件转发(Mail Forwarding)

邮件转发软件可以将邮件中使用的邮件地址映射为一个或多个新的邮件地址。此外它还包含一个邮件别名扩展(Mail Alias Expansion)机制。使用别名系统允许单个用户拥有多个邮件标识符,于是就可以建立一些邮件发送清单(Mailing List),使一个标识符与一批收信人相关联,利用邮件分发器(Mail Exploder)将接收到的一个邮件发送给一大批人。在Internet上有许多开放的邮件发送清单。

4. 中继MTA与电子邮件网关(E-mail Gateway)

SMTP并非惟一的电子邮件协议。如图9.12所示,当一个专用网中不使用TCP/IP协议时,其所使用的电子邮件协议也会与SMTP不同。这时可以通过电子邮件网关实现不同协议之间的转换,将一种格式的邮件转换成另一种格式。

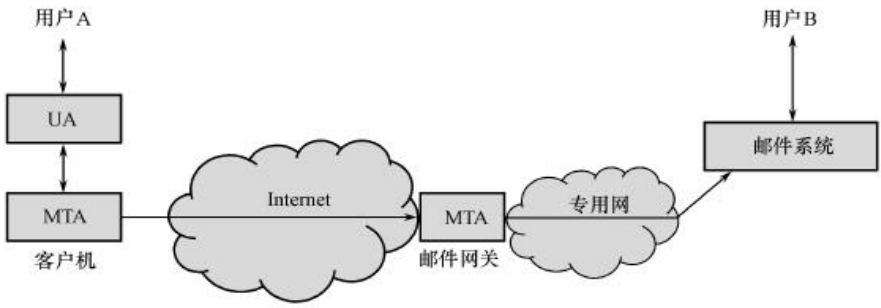


图9.12　电子邮件网关的工作过程

9.4 简单网络管理协议 SNMP

9.4.1 网络管理概述

1. 网络管理功能

网络管理是为保证网络在工作期内能正常地使用网络服务而进行的全部操作和维护性活动。在 OSI 网络管理框架模型中,基本的网络管理功能被划分为如下5个方面。

(1) 配置管理 (Configuration Management)

一个计算机网络是由多种多样的设备连接而成的,它们的参数、状态、名字以及物理结构和逻辑结构,就构成了计算机网络的配置。但是,由于网络运行环境的变化、系统本身用户的增减以及设备维修等原因,为了使网络有效地工作,网络的配置也要随之变化。为此而采取的各种手段就构成了网络管理的配置管理功能。

配置管理的主要功能有:

识别被管网络的拓扑结构;

识别网络中的各个对象;

激活跟踪程序;

保存网络配置文档、请求的服务与服务协议及软件分布情况;

动态维护网络的配置数据库;

自动修改制定设备的配置。

配置管理的主要目标是有效地维护网络历史的、当前的以及未来的应备配置的详细记录,随时掌握网络的变化或故障可能产生的影响,依据历史记录找出判断和排除网络故障的对策。所以,配置管理是网络管理的基本功能。

(2) 失效管理 (Fault Management)

失效管理也称故障管理或网络监控,其功能包括主动监控、被动监控 (故障检测和诊断) 、故障隔离与处理。失效管理的目的是保证网络能提供连续可靠的服务,能快速定位并隔离网络中的故障源或发现潜在的故障,并尽快将其排除。

(3) 性能管理 (Performance Management)

故障管理侧重于故障的诊断与排除,而性能管理侧重于故障的预防。为了防患于未然,故障管理要收集流量、使用者、访问的资源等信息,并解释周期性性

能指标度量和验证网络瓶颈,为未来网络的性能提供预测。其目的是在使用最少的网络资源和具有最小延迟的前提下,保证网络提供可靠和连续的通信能力,并使网络资源达到最优化的程度。

(4) 计费管理(Accounting Management)

从本质上讲,计费管理是以网络用户使用网络资源的情况为依据的管理。对于商业化的计算机网络,计费管理要求记录每个用户每次通信的开始时间、结束时间、通信中使用的服务等级、访问的信息资源类型和流量,以便摊派网络运行费用,收取回报。从非商业化的目的出发,计费管理的职能是审计,用来统计不同线路的使用情况、不同资源的利用情况等,以便改善网络的运行质量。

(5) 安全管理(Security Management)

安全管理具有三层含义:

保证用户的权益不受损害,如账号被盗用等;

保证网络上的信息资源不受侵害,如被窃用、篡改、破坏等;

保证网络上的设备不被滥用和破坏。

为了保证网络的安全,除了要采取技术措施、法律手段、道德约束外,还应实施有效的管理措施,管理的内容包括:

主动监控措施,利用网络的监测和审计设施实时记录网络资源的使用情况,报告越轨行为或发出危险行为警报;

被动限制措施,通过对用户注册及其时间、地址位置的限制,口令及其加密控制,对不良行为加以限制;

防御式补救措施,主要包括设置资源访问权限、目录与文件属性控制、备份机制与数据加密。

一般说来,一个具体的网络管理系统不一定都要包含以上的5项功能,不同的系统可能会侧重其中的某几项,但几乎所有的系统都会包括失效管理功能。

2. 网络管理机构

(1) IANA

IANA(Internet Assigned Numbers Authority)是管理IP地址、分配Internet RFC(Request For Comments)序号以及决定其他与Internet运行和服务有关的序号与定义的机构。

(2) 网络信息中心NIC

InterNIC(Internet Network Information Center)是IANA的运行机构,具体负责IP地址的分配、域名注册、协调、目录服务、信息统计和发布服务。

但是,由于Internet是一个世界范围的计算机互联网络,它按照自治域方式管理和运行不同地域和组织的网络。因此,网络信息中心也是按照自治域和ISP

分布构建的。

(3) 网络运行中心 NOC

NOC (Network Operation Center) 是各自治域和 ISP 用于完成网络之间的路由、报文转发、计费、安全、与用户相连接以及网络实际运行和维护等有关功能的机构。NOC 对网络和设备的管理方式有 3 种。

① 本地终端方式：通过被管理设备的 RS－232 接口，对被管理计算机系统进行监控、配置、计费以及性能和安全管理。

② 远程 telnet 方式：通过远程登录方式，用命令操作对已知地址和管理口令的设备进行管理。

③ 基于 SNMP 的代理/管理器方式：详细内容将在后面几小节介绍。

9.4.2 SNMP 管理模型

SNMP 是使用 TCP/IP 协议族对 Internet 上的设备进行管理的一个框架，它提供一组基本的操作对 Internet 进行监视和维护，并使用两个基本概念——管理器 (Manager) 和代理 (Agent)。如图 9.13 所示，一个网络管理系统由管理站、代理和被管理设备组成。被管设备可以是网络中的任何类型的节点，包括计算机、通信服务器、打印机、路由器、网桥和集线器等。代理是运行 SNMP 服务器程序的主机或路由器，它随时记录网络上的各种事件，并随时向管理器报告网络的使用情况以及各项参数。管理器是运行 SNMP 客户程序的主机，获得代理提供的有关信息后，向代理程序下达有关命令，对网络进行适当调整，使网络达到最佳运行状态。由于有些被管设备运行网管软件的能力有限，因此必须设定一个最低基准，定义一个最基本的性能指标。

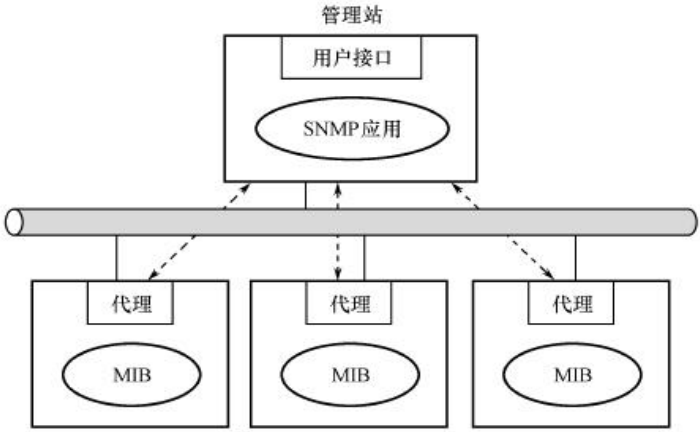


图 9.13　SNMP 管理模型

需要特别强调的是,管理器是通过发出请求得到能够反映代理行为的信息进行检查的,所以它是一个客户机端,而代理是服务器端。

此外,还须注意的是,Internet中的管理不仅是通过SNMP完成的,还要使用另外一些协议。其中主要是管理信息标准SMI(Standard of Management Information)和管理信息库MIB(Management Information Base)。

9.4.3　SMI

SMI是SNMP的一个指南,它给出描述管理对象3个属性的方法:

给对象命名的方法;

定义可以在对象中存储的数据类型的方法;

对在网络上传输的数据进行编码的方法。

SMI在RFC1155中详细定义。下面仅做简要介绍。

1. 名字

SMI要求每一个被管对象(如一个路由器、一个路由器中的变量、一个值等)具有一个惟一的名字。为了使名字具有全局性,如图9.14所示,SMI采用了基

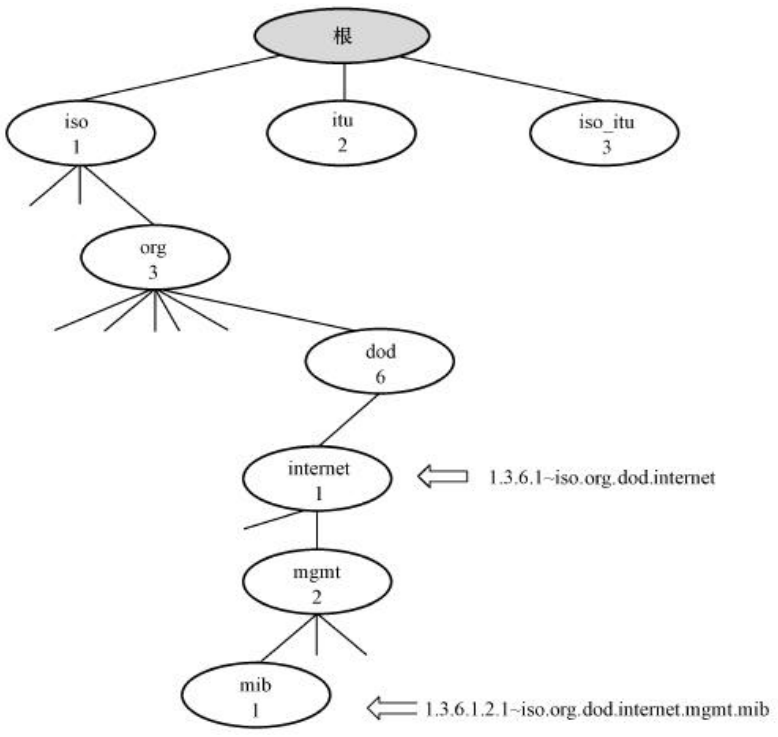


图9.14　SMI基于树结构的分级命名体制

于树结构的分级命名体制,并且可以采用点分十进制或点分名字的记法。

2. 类型

SMI 定义对象存储的数据的数据类型的方法是基于 ASN.1 (Abstract Syntan Notation,抽象语法标记法) 的,并且分为两大数据类型:简单类型和构造类型。

(1) 简单类型

简单类型是原子类型,有表 9.7 列出的 7 种重要简单数据类型。

表 9.7　SMI 定义的重要简单数据类型

类型	大小	说明
Interger	4 字节	在 0 到 $2^{32}-1$ 之间的一个整数
String	可变	零个或多个 ASCII 字符
ObjectIdentifier	可变	用 ASCII 数字表示的对象标识符
IPAddress	4 字节	由 4 个整数组成 IP 地址
Counter	4 字节	一个整数,其值可从 0 增加到 4 294 967 295;当它到达最大值时就返回到 0
Gauge	4 字节	与 Counter 相同,但当它到达最大值时不返回;它保持在这个数值直到复位
TimeTicks	4 字节	记录时间的计数值,以 1/100 s 为单位

(2) 构造类型

SMI 定义了两种构造数据类型:

Sequence:一些简单数据类型 (不要求相同) 的组合。

Sequence of:一些相同数据类型的组合。

3. 编码

SMI 使用基本编码规则 BER,将数据编码后在网络上传输。BER 要求每一块数据都要编码成如图 9.15 所示的三元组格式:标记、长度和值。

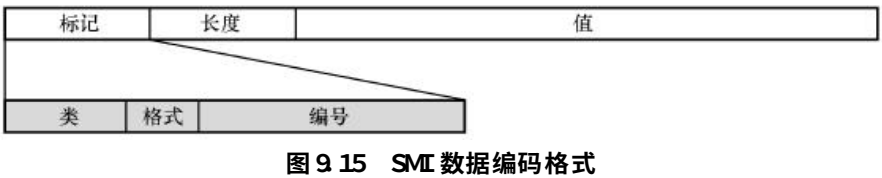


图 9.15　SMI 数据编码格式

(1) 标记

标记是一个描述数据类型的 1 字节字段。它由 3 个子字段组成:

类 (2bit)。定义数据的作用域——通用类 (00)、应用类 (01)、特定上下文

类(10)和专用类(11)；

格式(1bit)。指出数据是简单数据类型还是构造数据类型；

编号(5bit)。将数据类型进一步编号。

(2) 长度

长度为一个字节或多个字节。若为一个字节,其最高位必为0;若为多个字节,则前面几个字节的最高位必为1,以说明后面的字节还是长度子字段,最后一个字节的最高位为0。

(3) 值

值字段按照BER中定义的规则编码。图9.16所示为几个SMI数据编码的例子。

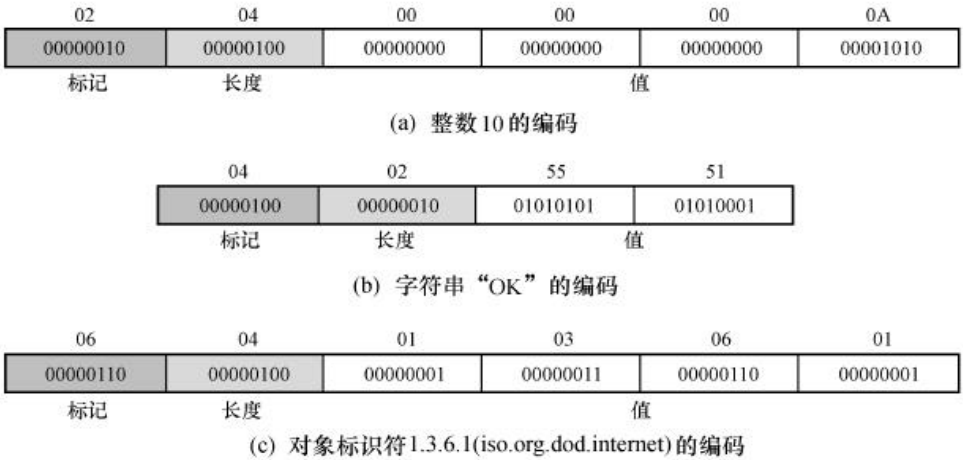


图9.16　SMI数据编码示例

9.4.4　MIB

1. MIB概述

MIB是管理器能够管理的对象的集合。每一个代理都有自己的MIB。MIB中所有对象的定义规则和语法,都在SMI中有详细的描述,并且应当具备如下几个项目：

对象名称；

对象类型；

访问状态——对象是只读、只写或可擦写等；

对象描述——描述对象的各种信息,如设备规格等可提供管理器解读的

数据；

对象识别码——即OID对象识别码，用于保证对象在MIB中的惟一性。

MIB有许多版本。其中，ISO制定的MIB－Ⅰ和MIB－Ⅱ是两个重要的标准。

2. MIB－Ⅱ

MIB－Ⅱ是SNMP协议进行管理时所运用的变量集合的定义标准，也是使用性最高的网络数据结构标准。图9.17所示为MIB－Ⅱ对象标识码树结构。例如，Internet对象的OID定义便为{ISO(1) ORG(3) DOD(6) Internet(1)}，即1.3.6.1。

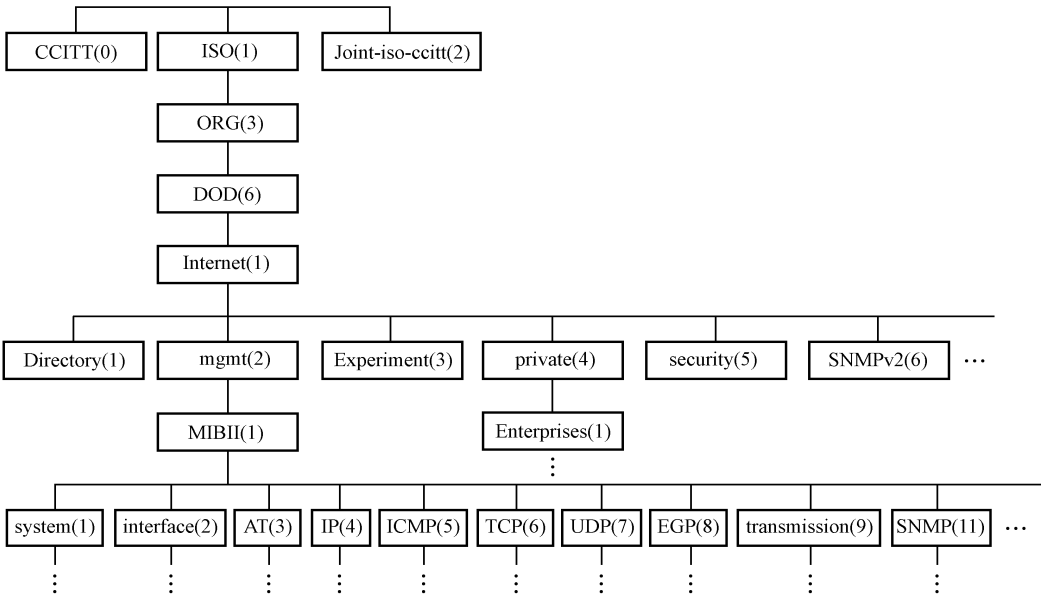


图9.17　MIB－Ⅱ对象识别码树结构

MIB将所有管理的对象都按一定的标准分为不同群组。图9.17中最下面一行便是MIB－Ⅱ定义的各功能群组。表9.8对一些重要的MIB－Ⅱ功能群组进行了说明。

表9.8　较重要的MIB－Ⅱ功能群组

群组名称	对象数量	说明
system	7	描述设备名称(sysName)、识别码(sysObjectID)、激活所需时间(sysUpTime)、管理者名称(sysContact)、位置(sysLocaton)、定义(sysDescr)、网络层级(sysServices)
internet	23	描述网络装置接口的各项数据及网络接口个数，前缀为if，如ifNumber(网络接口个数)

续表

群组名称	对象数量	说明
AT	3	地址转译,如地址解析的相关信息,前缀为 at
IP	42	描述与 IP 相关的流量、寻址、重组、舍弃等信息,前缀为 ip
ICMP	26	包含 ICMP 的各项信息,如网络状态、错误、计数等,前缀为 icmp
TCP	19	描述 TCP 协议连接、送收、错误等信息,前缀为 tcp
UDP	7	描述 UDP 协议送收包数量、故障原因等信息,前缀为 udp
EGP	20	描述外部网关和路由器的各项信息,前缀为 egp
transmission	0	保留给特定介质
SNMP	30	描述 SNMP 协议运作、信息数量等各项信息统计,前缀为 snmp

9.4.5　SNMP 的工作机制

1. UDP 端口

SNMP 在两个众所周知端口 161 和 162 上使用 UDP 服务。众所周知端口 161 由服务器(代理)使用,162 由客户(管理器)使用。

2. 管理器轮询,代理报告异常

SNMP 协议是一个异步的请求/响应协议,它的管理功能是通过轮询操作实现的。管理器周期地向被管设备的代理发送轮询信息,并根据各代理回复的响应进行处理。这时,代理(服务器)在端口 161 上发出主动打开,然后等待;而管理器(客户机)使用短暂端口发出主动打开。客户机向服务器发送请求报文,以短暂端口为源端口,以众所周知端口为目的端口;服务器向客户机发送响应报文,以众所周知端口为源端口,以短暂端口为目的端口。

除了管理器发送轮询信息外,被管设备也通过代理发送异常信息来中断管理器的工作处理流程。

这种管理器周期地发送轮询信息以监视和维持网络资源,被管设备的代理进行异常报告的机制,使 SNMP 成为一种实现简单、维护容易和非常有效的管理协议。

3. SNMP 的通信原语

SNMP 使用 5 种原语实现网络管理功能:

(1) get_request——访问代理,并从 MIB 树上得到指定的对象实例值;

(2) get_reponse——响应管理器来的请求,包含响应标识和响应状态信息;

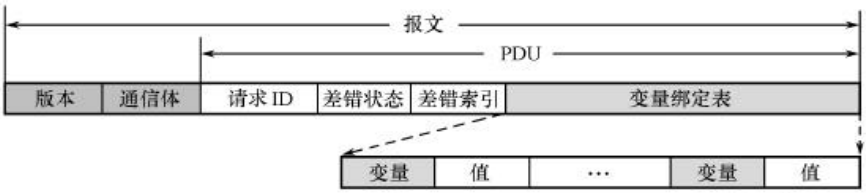
(3) get_next_request——访问代理,并从 MIB 树上检索指定对象的下一个对象实例;

(4) set_request——描述在一个对象实例上执行的行动；

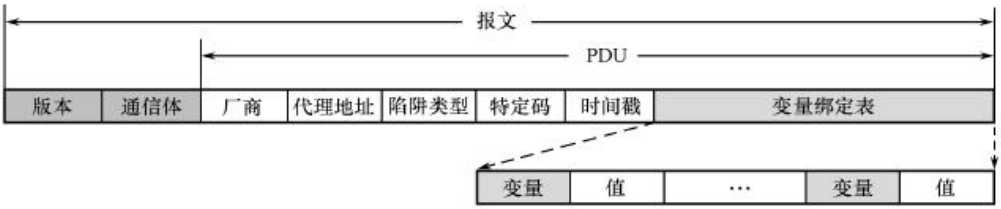
(5) trap——代理向管理器发送异常事件。

4. 格式

SNMP报文的前4种格式相似，如图9.18(a)所示；惟trap报文格式有所不同，如图9.18(b)所示。



(a) get_request\get_response\set_request和get_next_request报文格式



(b) trap报文格式

图9.18 SNMP报文格式

下面对SNMP格式中的有关字段作扼要介绍。

(1) 版本(Version)：实际值为版本号减1。

(2) 通信体(Community)：定义口令。缺省时为“public”。

(3) 请求ID(Request ID)：管理器在请求报文中使用的一个序号，代理将在响应中重复。

(4) 差错状态(Error status)：只用于响应报文，给出代理报告的差错的类型。在请求报文中为0。可能出现的差错状态有表9.9中所示的下几种：

表9.9 可能出现的差错状态类型

状态	名称	意义
0	noError	无差错
1	tooBig	响应太大，无法放进一个报文内
2	noSuchName	变量不存在
3	badName	要存储的值无效
4	readOnly	企图修改只读对象
5	genErr	其他

(5) 差错索引(Error index):是一个偏移,告诉管理器引起差错的是哪个变量。

(6) 变量绑定表(VarBindList):管理器希望读取或设置的一组具有对应值的变量。

(7) 厂商(Enterprise):定义产生陷阱的软件包标识(ObjectID)。

(8) 代理地址(Agent address):定义陷阱的代理IP地址。

(9) 陷阱类型(Trap type):这些类型如表9.10所示。

表9.10　陷阱类型

类型	名称	意义
0	coldStart	代理已被引导
1	warmStart	代理已被重新引导
2	lirkDown	接口出故障
3	lirkUp	接口已正常工作
4	authenticationFailure	检测出无效通信体
5	egpNeighborLose	EGP路由器变为故障状态
6	enterPriseSpecific	其他报文

(10) 特定码(Specific code):陷阱类型为6时,定义厂商使用的特定码。

(11) 时间戳(Time stamp):给出引起陷阱的事件所经历的时间。

5. 编码

(1) 报文结构

如图9.18所示,一个报文是一个三元组:版本、通信体和PDU。它们又各是一个三元组:

版本——标记、长度和值;

通信体——标记、长度和值;

PDU——代码、长度代码和PDU数据代码。

而PDU数据又包括请求ID(标记、长度和值)、差错状态、差错索引和变量绑定表。

(2) 编码规则

SNMP采用BER标准对报文进行编码,并且采用如下规则:

报文用标记定义;

类是上下文敏感的(context-sensitive,值为10);

格式是结构化的(取1);

编号视报文类型分别为0(00000),1(00001),2(00010),3(00011),4(00100)。

下面给出不同类型报文的整个标记。

get_request:A0 (10100000)

get_next_request:A1 (10100001)

get_response:A2 (10100010)

set_request:A3 (10100011)

trap:A4 (10100100)

6. 举例

例 9.3　管理器(SNMP 客户)发送 get_request 报文,希望知道一个特定的路由器已经收到了多少 UDP 数据报;代理用 get_response 报文响应。

(1) get_request 报文

要发送报文获知路由器已经收到了多少 UDP 数据报,与此信息相关的响应是 udpInDatagram,其标识符为 1.3.6.1.2.1.7.1。由于管理器打算读取一个值,因此最后一部分定义一个值为 0 的空实体。报文编码如下:

30 2A	Sequence,长度为 $2A_{16}$
02 01 00	Integer,长度为 01_{16},版本=0
04 06 70 75 62 6C 69 63	String,长度为 06_{16},通信体默认为:public
A0 1D	get_request,长度为 $1D_{16}$
02 04 00 01 06 11	Integer,长度为 04_{16},请求 ID=00010611_{16}
02 01 00	Integer,长度为 01_{16},差错状态=00_{16}
02 01 00	Integer,长度为 01_{16},差错索引=00_{16}
30 0F	Sequence,长度为 $0F_{16}$
30 0D	Sequence,长度为 $0D_{16}$
06 09 01 03 06 01 02 01 07 01 00	ObjectID,长度为 09_{16},响应为 udpInDatagram
05 00	空实体,长度为 00_{16}

(2) get_response 报文

用以送出所收到的 UDP 数据报数量。与之相关联的 MIB 变量也是 udpInDatagram,而 VarBindList 是对象标识符,其后跟对象的值。报文编码如下:

30 2E	Sequence,长度为 $2E_{16}$
02 01 00	Integer,长度为 01_{16},版本=0
04 06 70 75 62 6C 69 63	String,长度为 06_{16},通信体默认为:public
A0 1D	get_request,长度为 21_{16}
02 04 00 01 06 11	Integer,长度为 04_{16},请求 ID=00010611_{16}
02 01 00	Integer,长度为 01_{16},差错状态=00_{16}
02 01 00	Integer,长度为 01_{16},差错索引=00_{16}

30 0F　　Sequence,长度为 13_{16}

30 0D　　Sequence,长度为 11_{16}

06 09 01 03 06 01 02 07 01 00　ObjectID,长度为 09_{16},MIB 变量为 udpInDatagram

05 00　　Counter,长度为 04_{16},值 12 11

9.5　超文本传输协议 HTTP

9.5.1　超文本与 WWW

1. 概述

毫无疑问,阅读本书的读者都已经使用过 Internet 上的浏览器了,也体味过浏览时从一个网页跳跃到另一个网页,从某处跳跃到他处的奇妙功能了。这就是超文本技术。它是指一个含有多个指针(Link)的文本文件,每一指针可以指向任何形式的、计算机可以处理的其他信息源,即它可以通过热链路(Hotlink)或热字(词)链接到其他文本、图像、声音、动画等任何形式的文件中,形成一种"联想"关系。一个超文本由多个信息源链接(Hyper Link)而成,并且这些信息源的数目可以不受限制,从一个文档链接到另一个文档,形成遍布世界的 WWW(World Wide Web)。

WWW 简称 Web,这个名字本身就非常形象地定义了用链接技术组织的全球性信息资源。它所使用的服务器称为 WWW 服务器或 Web 服务器,每个 Web 服务器都是一个信息源。遍布全球的 Web 服务器,通过链接把各种形式的信息,如文本、图像、声音、视频等无缝隙地集成在一起,构筑成密布全球的信息资源。Web 浏览器则提供以页面为单位的信息显示。用户在自己的计算机上安装一个 WWW 浏览程序和相应的通信软件后,只需要提出自己的查询要求,就可以轻松地从一幅页面跳到另一幅页面,从一台 Web 服务器跳到另一台 Web 服务器上,自由自在地漫游 Internet 世界了。用户无需关心这些文件存放在 Internet 上的哪台计算机中,具体到什么地方、如何取回信息都由 WWW 自动完成。

2. URL(Uniform Resource Locator)

任何一个信息文档、图像、视频或音频图片都可以被看成是一种资源。为了引用资源,应当使用惟一的标识来描述它放在何处以及软件如何存取它,当前使用的机制称为统一资源定位器 URL。URL 地址既可以是本地硬盘上的某个文件,也可以是 Internet 上的一个网点。

简单地说,URL由两部分组成：

sckema:path

这里,sckema表示连接模式。连接模式是资源或协议的类型。WWW浏览器将多种信息服务集成在同一软件中,用户无需在各个应用程序之间转换,界面统一,使用方便。目前支持的连接模式有：

http(超文本传输协议)；

ftp(远程文件传输协议)；

gopher(信息鼠)；

WAIS(广域信息查询系统)；

news(用户新闻讨论组)；

mailto(电子邮件)；

……

path部分一般包含有主机全名、端口号、类型和文件名、目录号等。其中,主机全名以双斜杠"//"打头,一般为资源所在的服务器名,也可以直接使用该Web服务器的IP地址,但一般采用域名体系。

path部分的具体结构形式随连接模式而异,下面介绍两种URL格式。

(1) HTTP URL格式

http://主机全名[:端口号]/文件路径和文件名

由于HTTP的端口号默认为80,因而可以不指明。HTTP协议缺省的文件名为index.html,这部分也可以不写。

(2) FTP URL格式

ftp://[用户名[:口令]@]主机全名/路径/文件名

其中,缺省的用户名为"anonymous",用它可以做匿名文件传输。如果账户要求口令,口令应在URL中编写。

9.5.2 B/S计算模式与浏览器结构

1. B/S模式及其特点

Web以浏览器/服务器模式(简称B/S模式)工作。B/S模式是在C/S模式基础上发展起来的一种适合WWW工作的模式。它一方面继承和共融了C/S模式中的网络软硬件平台和应用,又有了一些新的发展和C/S模式所不及的特点。下面介绍这些特点。

(1) 用户访问方式

在C/S计算模式中,一般采用具有图形用户接口(GUI)的PC机作为客户机

端设备;用户在客户机上以事件驱动方式1对M地访问应用服务器上的资源。

在B/S模式中,用户在基于浏览器的客户机上以网络用户界面(NUI)方式N对M地访问服务器上的资源。

(2) 体系结构

C/S模式一般采用图9.19所示的两层结构,其应用程序分为两大部分:一部分是由多个用户共享的信息与功能,这部分称为服务器部分;另一部分是为每个用户所专有,称为客户机部分。客户机部分负责执行前台功能,如管理用户接口、数据处理和报告请求等。而服务器部分执行后台服务,如管理共享外设、控制对共享数据库的操纵、接受并应答客户机的请求等。虽然C/S体系结构采用的是开放模式,但这只是系统开发一级的开放性,在特定的应用中无论是Client端还是Server端都还需要特定的软件,没能提供用户真正期望的开放环境;应用软件系统一般要在客户端和服务器端分别安装。

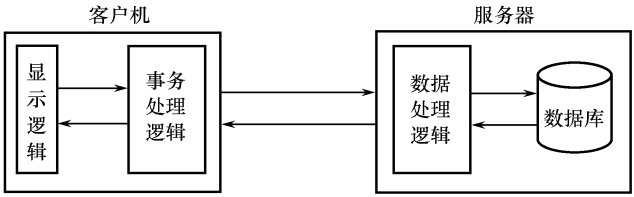


图9.19　C/S系统的结构

B/C模式则形成图9.20所示的浏览器—Web服务器—应用数据库服务器的三层结构。这时,在用户端只需要安装一个通用的浏览器软件,不需要安装应用软件,做到了与软硬件平台无关,为用户提供了方便。并且,它的前端是以TCP/IP协议为基础的,企业内的WWW服务器可以接受安装有Web浏览程序的Internet终端的访问。作为最终用户,只要通过Web浏览器,各种处理任务都可以调用系统资源来完成,这样大大简化了客户端,减轻了系统维护与升级的成本和工作量,降低了用户的总体拥有成本(TCO)。

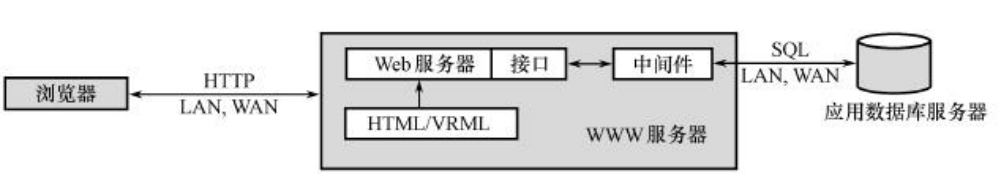


图9.20　WWW信息服务框架

(3) “瘦”的客户机

C/S模式的1对M特点,要求把对数据库的访问形成一个统一的连接平台,

客户必须到这个平台上才能访问到服务器资源。因此,客户机端除了要负责图形显示和事件输入外,还要配备应用逻辑和业务处理软件以及开发工具。这样,软件的变动与版本的升级以及硬件平台的适应能力就必然会牵动系统中所有的客户机,并使服务器越来越“胖”。而在 B/S 模式下,由于平台的无关性,可以把应用逻辑和业务处理放置在服务器侧,于是客户机就可以做得尽可能的“瘦”。

⑷ 开发、维护和升级

在 C/S 模式中,客户端承担着显示逻辑和事务处理逻辑双重功能。系统设计中软件开发的工作量主要在客户端。相对而言,服务器端只承担数据处理逻辑,是一种通用功能,开发相对简单。因此,系统维护的工作量也主要在客户端。

在 B/S 模式中,客户端是一种通用的浏览器,没有多少开发量,主要工作在服务器端。随着软件系统的改进和升级越来越频繁,B/S 架构的产品明显体现更方便的特性。无论用户的规模有多大,有多少分支机构,都不会增加任何维护升级的工作量,所有的操作只需要针对服务器进行。如果是异地,只需要把服务器连接上网即可立即进行维护和升级,这对人力、时间、费用的节省是相当惊人的。

表 9.11 列出了 B/S 模式与 C/S 模式特点的对比。

表 9.11 B/S 模式与 C/S 模式特点的对比

	结 构	应用软件分布	客户机	客户访问形式	数据流	平台无关性	开发点
C/S	两层结构:客户机—服务器	客户端和服务器端	胖客户机	1:M/GUI	突发性	否	客户机
B/S	三层结构:浏览器—Web 服务器—应用数据库服务器	服务器端	瘦客户机	N:M	不可预测	是	服务器

2. 浏览器结构

在 B/S 模式中,用户的本地计算机或经远程登录的主机中运行有 WWW 的客户程序,即 Web 浏览器。Web 浏览器主要提供两种功能:一方面向用户提供风格统一的、使用方便的信息查询界面;另一方面将用户的信息查询请求转换成 Internet 的查询命令,传送到网上相应的 Web 站点进行处理。Web 服务器完成规定的工作就将所查询结果返回客户机,通过客户机程序把返回数据格式化为屏幕显示的格式,显示给用户。客户机与 Web 服务器间的交互是通过 HTTP 实现的。

图 9.21 所示为浏览器的组成。可以看出,一个浏览器有一组客户机、一组解释程序、一个管理客户机和解释程序的控制程序。

控制程序是浏览器的核心部件,它解释鼠标命令和键盘输入,并调用有关组

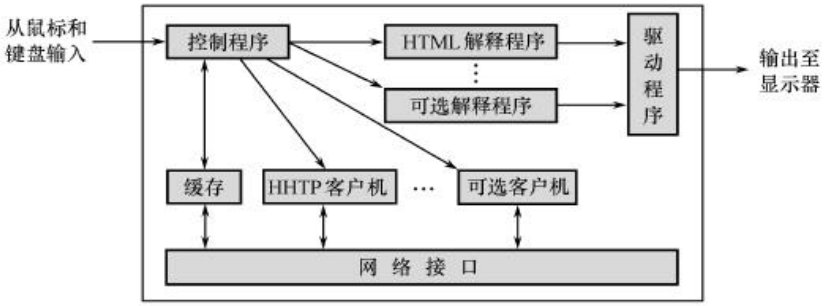


图9.21　浏览器的主要组成

件执行用户指定的操作。例如,当用户点击一个链接的起点时,控制程序就调用一个客户机程序从远程服务器上将所需文档取回,并调用解释程序向用户显示该文档。

缓存在浏览器中用于存放页面的副本。每当用户点击某一选项时,浏览器首先检查磁盘中的缓存内是否有该项,避免网络过多的传输,以改善浏览器的运行特性。

9.5.3　HTTP的工作机制

1. HTTP的特点

在WWW系统中,浏览器是一个网络客户机,用户可以用浏览器通过HTTP协议并根据某资源的URL向Web服务器提出请求;Web服务器一方面通过接口和中间件与网上数据库资源连接,另一方面按用户请求的URL将用HTML/VRML书写的HTML/VRML页面返回给客户机端,由浏览器负责解释执行,将最终结果显示在用户面前。因此,浏览器实际上就是一个HTML/VRML解释器。

实现WWW的通信协议HTTP是为分布式超媒体信息系统设计的一个协议,它定义了HTTP的通信交换机制、请求及响应消息的格式等。下面说明其特点。

(1) 以B/S模式为基础

HTTP支持浏览器与服务器之间通信及相互传送数据。一个服务器可以为分布在世界各地的许多浏览器服务,因而它是一种分布式信息系统,并能对多重协议提供一个统一的通用接口。

(2) 简易性

HTTP被设计成一个非常简单的协议,使得Web服务器能高效地处理大量请求,客户机要连接到服务器,只需发送请求方式和URL路径等少量信息。由于HTTP协议简单,HTTP的通信与FTP、TELNET等协议的通信相比,速度快而且

开销小。

(3) 可扩充性与内容－类型(content－type)标识

HTTP具有较好的可扩充性,能够支持所有的数据格式,使用该协议传输的信息不仅仅是超文本,也可以是简单文本、声音信号、图像以及可以在Internet上访问的任何其他信息,并让客户机程序能够恰当地处理它们。

(4) 无连接性

无连接性的含义是限制每次连接只处理一个请求,服务器处理完客户机的请求并收到客户机的应答后,即断开连接,不会继续为这个请求负责,从而不用为保留历史请求而耗费宝贵的资源。这样,实现起来效率十分高,可以节省传输时间。

(5) 可靠性

HTTP是一种建立在TCP协议上的协议。它使用TCP协议来确保自身的可靠性。

(6) 无状态性

HTTP是一种无状态的协议。无状态是指协议处理事务没有记忆能力。这意味着:由于缺少状态使得HTTP累赘少,系统运行效率高,服务器应答快;另一方面,由于没有状态,协议对事务处理没有记忆能力,若后续事务处理需要有关前面处理的信息,那么这些信息必须在协议外面保存;另外,缺少状态意味着所需的前面信息必须重现,导致每次连接需要传送较多的信息。

2. HTTP的通信端口

HTTP通信建立在TCP/IP连接上,缺省的TCP端口号是80,但也可以使用其他端口号。Web服务器运行着一个守护进程(HTTP Daemon),它始终在端口80监听来自远程客户机的请求。当一个请求发来时,它就会产生一个子进程来处理当前请求,守护进程继续以后台方式运行,在端口80监听来自远程的连接请求。

HTTP通信中客户机提出请求应该带上全部必要的信息,客户机和服务器之间不能对不明确的问题进行磋商。一旦客户机提出请求,服务器感到信息不够时没有办法要求客户机给出进一步的信息。

HTTP具有两种报文:请求和响应。

3. 请求报文

图9.22给出了请求报文的结构。一个请求报文包含一个请求行、一个首部,还可能有一个主体。

请求行由URL字段、HTTP版本号字段和请求类型字段三部分组成。请求类型字段是其核心,它定义了几种不同方法的报文。请求方法是客户机向服务器发出的真正命令或请求。HTTP 1.1中定义的几种方法如下:

图9.22　请求报文结构

GET——客户机要从服务器中读取文档；

HEAD——客户机只想得到文档的某些信息，而非文档本身；

POST——客户机要给服务器提供某些信息；

PUT——客户机要将一个新的或替换的文档存储在服务器上；

PATCH——与PUT相似，但请求只包含在现有文件中出现的差异清单；

COPY——要将文件复制到另一个位置；

MOVE——要将文件移动到另一个位置；

DELETE——要将服务器上的一个文件删除；

LINK——要创建从一个文档到另一个位置的链接；

UNLINK——删除由LINK建立的链接；

OPTION——客户机向服务器询问可用的选项。

4. 响应报文

响应报文的组成与请求报文相似：一个状态行、一个首部，有时也包含一个主体。

状态行用于定义响应报文的状态，它包括HTTP版本、一个空格、一个状态码、一个空格和一个状态短语。状态短语是以文本形式解释状态码。表9.12列出了一些最常用的HTTP状态码以及状态短语。

表9.12　一些最常用的HTTP状态码以及状态短语

系列	状态码	状态短语	说明
非正式	100	Continue	已经开始接收请求，用户可以继续发送请求
	101	Switching	同意客户请求，切换到更新首部中定义的协议
成功请求	200	OK	请求成功
	201	Created	创建了新的URL
	202	Accepted	请求刚被接收，还没有起作用
	204	No content	主体空

续表

系列	状态码	状态短语	说明
重定向	301	Multiple choices	请求的 URL 多于一个资源
	302	Moved permanently	服务器不再使用所请求的 URL
	304	Moved temporarily	所请求的 URL 已经暂时被移动
客户差错	400	Bad request	请求中有语法错误
	401	Unauthorized	请求缺少合适的授权
	403	Forbidden	拒绝服务
	404	Not found	文档未找到
	405	Method not allowed	URL 不支持该方法
	406	Not acceptable	请求的格式不能被接受
服务器差错	500	Internal server error	服务器端出错,如崩溃
	501	Not implemented	请求的动作无法完成
	503	Service unavailable	服务暂不可用,但以后可能被请求

5. 首部

无论是请求报文中的首部,还是响应报文中的首部,都是用来在客户机和服务器之间传送附加信息。首部可以由一个首部行组成,也可以由多个首部行组成。每个首部行都有如图 9.22 所示的结构:由一个首部名、一个冒号、一个空格和一个首部值组成。

首部行可以分为如下 4 类。

(1) 通用首部,给出关于报文的通用信息,可以出现在请求报文和响应报文中,如:

Date——给出当前日期;

MIME-version——给出所使用的 MIME 版本。

(2) 请求首部,指明客户机的配置和客户机优先使用的文档格式,仅出现在请求报文中,如:

Accept——给出客户机能够接受的媒体格式。

(3) 响应首部,指明服务器的配置和关于请求的特殊信息,如:

Server——给出服务器名和版本号。

(4) 实体首部,给出关于文档主体的信息,如:

Connect-length——指出文档长度;

Connect-type——指明媒体类型。

6. 举例

例 9.4　用 HEAD 方法读取一个 HTML 文档信息。具体过程见图 9.23。

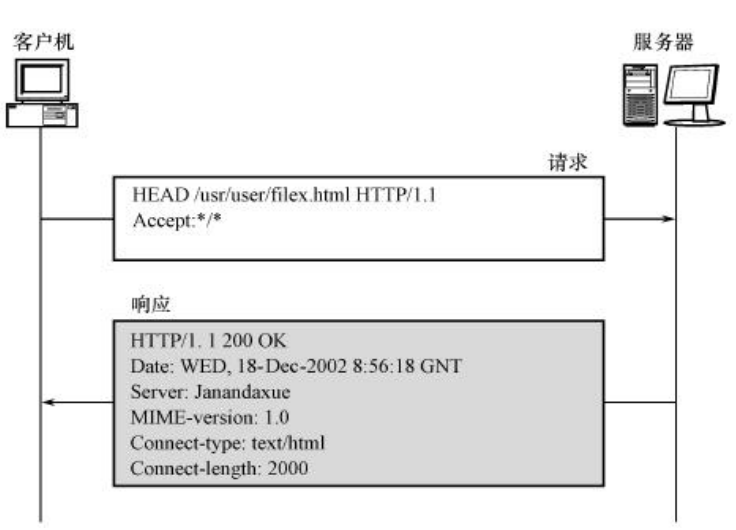


图 9.23　用 HEAD 方法读取一个 HTML 文档信息

说明：

(1) 请求报文

请求行给出方法 (HEAD)、URL (/usr/user/filex.html) 和 HTTP 版本 (1.1)。

首部为一行请求首部 (Accept)，用通配符给出客户机可以接受任何格式的文档。

不包含主体。

(2) 响应报文

状态行给出 HTTP 版本 (1.1)、状态码 (200) 和状态短语 (OK)。

首部共 5 行，定义日期、服务器、MIME 版本、文档类型和文档长度。

不包含主体。

习　　题

9.1　试给出 TELNET 对二进制数据 00111100 11110011 11111111 进行传输时，从客户机发出的比特序列。

9.2　试给出在下列方式下，TELNET 客户机和服务器之间交换的字符序列：

(1) 从默认方式切换到字符方式；

(2) 从默认方式切换到行方式；

(3) 从字符方式切换到行方式；

(4) 从行方式切换到字符方式；

(5) 从字符方式切换到默认方式；

(6) 从行方式切换到默认方式。

9.3　为什么在 FTP 中,客户机对控制连接发出主动打开,而对数据连接要发出被动打开?

9.4　试给出携带一种 FTP 数据的 TCP 报文段。

9.5　一个用户想把主机"aaaHost.sytu.edu.cn"中目录/usr/usrs/myRepot1 下的文件 file1 移动到目录/usr/usrs/myRepot2,请给出所有 FTP 命令和响应。

9.6　要将一个报文从地址 aaa@xxx.com 发送到地址 bbb@yyy.com,试给出下列阶段中的 SMTP 命令和响应:

(1) 从 aaa@xxx.com 到 bbb@yyy.com 的连接阶段;

(2) 从 aaa@xxx.com 到 bbb@yyy.com 的报文发送阶段;

(3) 从 aaa@xxx.com 到 bbb@yyy.com 的连接终止阶段;

(4) 用户 bbb@yyy.com 收到 aaa@xxx.com 的报文后,再转发给 ccc@zzz.com 的整个过程。

9.7　请写出下列情况下的 MIME 首部:

(1) 发送无格式的正文;

(2) 发送一个 JPEG 报文。

9.8　一个管理器希望知道一个路由器转发的 IP 分组数量。若此值为 1 200,试给出报文 get_request 和 get_response 的编码。

9.9　简述 B/S 工作模式与 C/S 工作模式有何异同。

9.10　试实现下面的 HTTP 交互过程。

(1) 读取文档/usr/user/sytu/doc1;

(2) 分别给出下列情况下的响应:

成功的请求;

请求中有语法错误;

客户机无权访问该文档;

文档永久地移到了/usr/jndx/doc1。

第 4 篇

计算机网络应用程序设计

这一篇从计算机网络协议实现的角度来介绍计算机网络的工作和应用开发原理。

建造计算机网络的目的是应用。随着计算机网络的飞速发展和日益普及,越来越多的应用转移到计算机网络平台上来,如丰富多彩的 Web 主页、经济快捷的电子邮件、轻松愉快的网上聊天、自由自在的 BBS 等。这些应用都是工作在客户机/服务器模式下,其开发的关键是针对应用的计算机网络进行通信程序设计。

计算机网络程序设计就在于实现网络应用进程间的信息交互。一般说来,应用进程间的通信可以分为两种:同一系统中的不同应用进程间的通信和不同系统中的应用进程间的通信。

为了支持网络应用程序的开发,UNIX 不仅支持多种网络协议,还提供了功能强大的网络程序设计接口。在这些网络

应用程序设计接口中,最常用的有：

插口(Socket)；

传输层界面(TLI, Transport Layer Interface)；

远程过程调用(RPC, Remote Procedure Call)。

本篇主要讨论不同系统中进程间通信的 Socket 技术。

第 10 章　插口 API 基础

这一章介绍与计算机网络程序设计有关的相关知识：

插口；

基本插口函数。

10.1　基本概念

10.1.1　插口 API

1. 端口与插口

在计算机网络体系结构中，传输层是非常重要的一层：在其上，是主机之间（严格地说是进程之间）的通信；在其下，是网络中的数据传输。在 TCP/IP 中，TCP 和 UDP 都使用了端口来为上层应用进程提供通信服务，即所有的应用进程必须通过相应的端口才能与运输层实体进行交互。

端口用 16 bit 的地址码——端口号进行标识。

但是，一个进程为了进行连接，不仅需要指定对方的端口号，还需要指定对方主机的 IP 地址。于是，就把一个二元组（IP 地址，端口号）称为一个插口（socket），也称一个套接口。

一条连接需要用一对插口标识。这对插口组成一个四元组：

（本地 IP 地址，本地端口号，远程 IP 地址，远程端口号）

对于整个 Internet 来说，在传输层通信的一对插口必须是惟一的，这样才能区分开同时通信的多个主机中的多个进程。

2. 插口 API

20 世纪 80 年代初期，加利福尼亚州伯克利（Berkeley）分校的一个研究组在远景研究规划署（Advanced Research Projects Agency，ARPA）的资助下，将 TCP/IP 移植到了 UNIX（称为 Berkeley UNIX 或 BSD UNIX）在网络中。在计算机通信中，用户直接打交道的用户界面是应用层进程。这些应用进程要通过一个通信信道与另一方的应用进程进行通信。这两个进程在通信时，要涉及复杂的各层协议

的实现细节。为了简化网络应用程序的设计,使进程对通信信道的操作能够像对文件一样简单,伯克利的开发者们开发了一种称为插口(socket)的逻辑文件。当系统把一个 socket 分配给一个进程时,就是将一个进程连接到了一个信道的端点上,好像一个用户把一台电话机接到了电话线上,只要对方也把电话机接到电话线上,双方就可以进行连接和通话了。

插口 API 由一些插口函数实现。如图 10.1 所示,这些函数屏蔽了协议的实现细节,使应用程序变得简单。

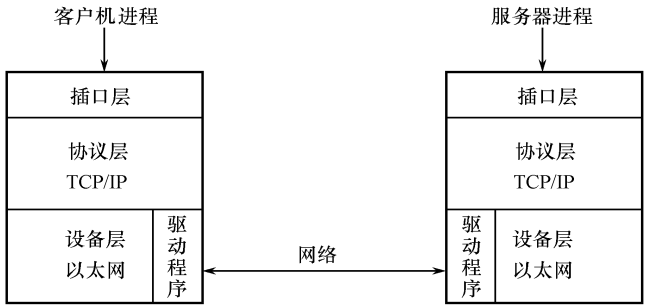


图 10.1 插口通信模型

socket 为整个网络通信提供协议基础。进程间的通信就通过连接两个进程的通路进行,软件设计人员不必考虑这个通路是什么,只要知道如何把一个进程连接到通路的端点即可。通路端点的分配由逻辑文件 socket 进行,使得进程用文件描述符访问通路就像访问普通文件一样,大大简化了通信程序的设计。因此,尽管插口最早是作为 BSD 规范提出来的,但它已经成为 UNIX 操作系统环境下 TCP/IP 网络编程标准,并且应用范围扩大到 Windows,Windows NT,OS/2,Sun OS,NetWare 等。

10.1.2 UNIX/Linux 进程

1. UNIX 的进程层次和初启过程

图 10.2 所示为 UNIX 的进程层次结构。对于 UNIX 系统中的每个终端端口,都有一个 getty 程序在运行。只要系统允许注册,UNIX 的 init 进程便自动为每个端口启动 getty 程序运行。

getty 程序的作用是:

决定传输速率;

显示"login:"消息;

等待用户的输入。

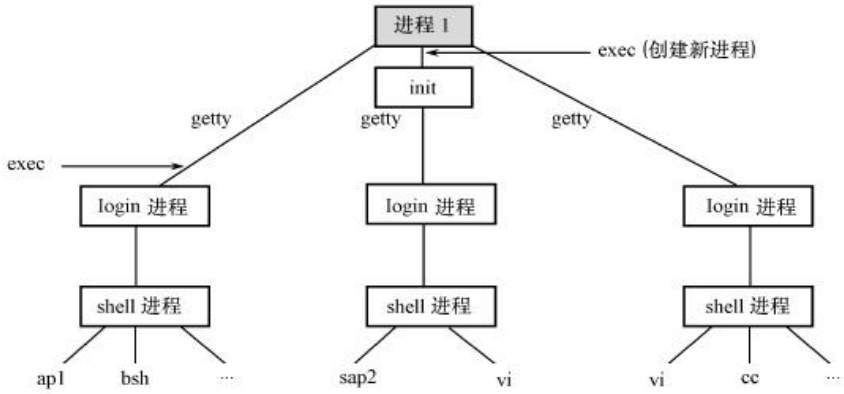


图 10.2　UNIX 的进程层次

只要用户输入一串以回车键结束的字符，getty 立即用其读入的字符串作参数，调用 login 程序，并用 login 覆盖自己——getty 进程变为 login 进程。

当 login 开始运行时，首先在终端上显示"Password："，并等待用户输入口令。用户输入口令后，login 根据/etc/password 文件中的响应项验证用户输入的注册名和口令。验证通过，即启动 shell，并在用户终端上显示命令提示符"$"（超级用户提示符为"#"），然后等待用户输入命令。

2. 任务结构体和进程表

任务（task）与进程在多任务系统中是同一事物的外部观点和内部观点，在 UNIX 中一般不作区分，并且为了管理上的方便将每一个进程用一个 task_struct 结构体表示。

进程表是由 NR_TASKS 个 task_struct 组成的静态数组：

```
struct task_struct *task[NR_TASKS];
```

每个进程在进程表中占有一项（一个 task_struct 结构体）。在进程表中，所有进程被组织成一个双向环形链，链头由外部变量 init_task 指出。系统初启时，init_task 指向 0 号进程（即系统的第一个任务）。下面是 init_task 的定义：

```
struct task_struct init_task;
```

当前进程由变量 current 指出：

```
struct task_struct current;
```

3. 进程的识别

在 Linux 系统中，为了管理进程使用了进程识别号（process ID）。与进程识别号相联系的还有下列识别号：

实际用户识别号（real user ID），标识真正运行进程的用户；

实际组识别号（real group ID）；

有效用户识别号（effective user ID），用于给新创建的文件赋所有权，检查文件的存取权限，常用来判断读写文件的优先权限；

有效组识别号（effective group ID）。

此外还设置了两个标记位：

set－user－ID——real user ID 与 effective user ID 是否相同。0，相同；1，不同。

set－group－ID——real group ID 与 effective group ID 是否相同。0，相同；1，不同。

例 10.1 用户 A 运行程序 X，产生一进程 P。用户 A 的用户识别号为 10，组识别号为 1；程序 X 的所有者识别号为 100，组识别号为 2；当前 set－user－ID 位为 1；set－group－ID 为 0。那么，进程 P 的实际用户识别号为 10，有效用户识别号为 100，实际组识别号和有效组识别号都为 1。

进程的有关识别号可以用下列函数取得：

uid_t getuid(void)；	返回实际用户识别号
uid_t getpid(void)；	返回实际组识别号
uid_t geteuid(void)；	返回有效用户识别号
uid_t getepid(void)；	返回有效组识别号
pid_t getppid(void)；	返回父进程识别号
pid_t getpgrp(void)；	返回进程组识别号

4. 进程的阻塞与非阻塞

对于运行状态中的进程来说，当某些条件不具备，如输入／输出条件还没有准备好时，该进程可以有两种处理方式：阻塞与非阻塞。

(1) 阻塞方式

在阻塞方式下，当输入／输出条件还没有准备好时，就将进程转入阻塞状态，标记阻塞原因，并保留当前进程现场信息，然后将控制转入进程调度程序，等阻塞原因解除后，将进程状态由阻塞变成就绪等待处理机，或直接获得由调度程序再次分配的处理机，恢复阻塞的现场继续执行。

(2) 非阻塞方式

在非阻塞方式下，当输入／输出条件还没有准备好时，进程仍处于运行状态，通过循环，反复进行某条件的查询。一般说来，阻塞机制可以防止进程在循环反复的查询等待中造成资源浪费。但是有时在应用中需要进程一直处于运行状态，不希望进程在对 socket 的操作中阻塞，这时就要使用非阻塞机制。

10.1.3　信号及其处理

UNIX 系统用信号来通知一个或多个进程异步事件的发生,例如键盘上某个键被按下,某种严重错误发生等。

1. 信号的概念

Linux 系统为进程在正常执行过程中可能发生的各种软件和硬件状态定义了一组信号,它们与硬件中断源相似,故也称为“软中断”。信号是异步发生的,进程无法预先确定何时接受某个信号。例如,SIGINT 是用户在终端输入中断键时产生的终止进程运行的信号;SIGABRT 是进程在调用了 abort 函数时产生的退出信号等。因而信号机制提供了一种处理异步事件的方法。

异步事件和同步事件的区别在于,前者是不可预见的,而后者是可以预见的。

每个信号都有一个名字,这些名字都是以 SIG 开头,例如上面的 SIGINT 和 SIGABRT。信号的名字在头文件中分别被定义为一些正整数——信号编号(Signal Number)。但是,没有编号为 0 的信号。因为 0 在 kill 函数中有特殊的用途。

2. 信号的产生

信号可以由内核发往一个进程,也可以从一个进程发往另一个进程。具体地说,信号发生在下列情况下:

在终端输入某些终端字符。如键入 DELETE 键,产生中断信号 SIGINT。

硬件出现意外(如除零、非法内存地址等),被识别后通知内核;内核会给情况发生时正在运行的进程生成相应的信号。例如进程在执行过程中使用了非法地址时,就会产生 SIGSEGV 信号。

软件产生的信号。例如,进程设置的时间片终止时会产生 SIGALRM 信号。

用 kill 函数向其他进程或进程组发送信号。

用 raise 函数给进程自己发送信号。

3. 信号的属性

对进程来说,信号是随机产生的。不可能仅通过对某个变量的检测来看信号是否发生;进程只能告诉内核信号出现时就进行指定的处理。这些处理有 3 种情形,也称信号的 3 种属性。

(1) 忽略信号

大多数信号是可以忽略的。只有 SIGKILL 和 SIGSTOP 两个信号不可以忽略。这两个信号能够给超级用户一个杀死或中断任何进程的简洁方法。

(2) 接受(Catch)信号

如果进程要接受某个信号,它就必须告诉内核当信号出现时要调用的函数。

这些函数用于对所发生的信号进行相应的处理。例如,进程接受到信号SIGCHLD,就意味着子进程运行结束,这时处理函数就可以调用waitpid来获得子进程的进程号以及结束状态。再如,子进程创建了临时文件,就有可能要为信号SIGTERM写一个处理函数来清除临时文件。

(3) 使用缺省处理函数

每个信号都有一个缺省处理函数,并且大多数信号的缺省处理函数都是结束进程运行。

4. 信号描述

下面给出部分信号的描述。

SIGABRT:调用abort时产生,进程将非正常结束。

SIGALRM:用alarm设置的时钟的时间段终止时产生。

SIGCHLD:当一个进程被中断或运行结束时,会给父进程发送该信号。一般该信号将被忽略。

SIGCONT:一个作业控制信号。当被中断的进程再执行时该信号被发送给进程。

SIGFPE:数学计算出现意外(如除零、浮点溢出)时,产生该信号。

SIGHUP:终端有断线等,就会给与控制终端相连的控制进程(session leader)发该信号。

SIGKILL:进程执行了非法硬件指令时产生该信号。

SIGINT:在终端输入中断键(Control-C或Del)时产生该信号,并被送往前台进程组的每一个进程。

SIGKILL和SIGINT是两个不可忽略的信号。系统管理者可以利用它们杀死任何进程。

SIGPIPE:向读进程已经运行结束的管道里写数据,就会产生该信号。

SIGQUIT:在终端输入退出键(通常为Control-\)时,产生该信号。

SIGSEGV:进程使用了非法内存地址时产生该信号。

SIGSTOP:一个中断进程运行的作业控制信号。

SIGTSTP:一个交互中断信号,在终端输入中断键时产生。它被送往前台进程组的每个进程。

SIGTERM:调用kill(1)命令时产生。

SIGTRAP:跟踪中断时产生。

SIGTTIN:一个后台进程组的进程想要从终端读取数据时产生。

SIGTTOU:一个后台进程组的进程要向控制终端写数据时,由终端驱动器产生。

SIGUSR1:用于应用程序的用户自定义信号。

SIGUSR2:用于应用程序的用户自定义信号。

5. 信号处理函数

当一个导致信号产生的事件发生时,信号就会发给进程。而当进程开始对信号进行处理时,就称信号被发送(deliverred)给了进程。在信号产生到信号被发送的时间段内,称信号被挂起(pending)。

下面给出与信号处理有关的系统调用。

signal:建立信号和信号处理函数的连接,返回上一次信号的属性(dispostion)。

kill:给指定进程号的进程或指定进程组号的进程组发送信号。

raise:给调用进程发送信号。

alarm:设定时间片,在一定时间间隔内产生时钟信号。

pause:使调用进程进入睡眠状态。

sigprocmask:设置调用进程的信号掩码。

sigpending:获得进程当前挂起的信号的集合。

sigaction:建立信号和信号处理函数的另一种系统调用。

sigsetjmp:用于非局部跳转的系统调用。

sigsuspend:自动保存信号掩码使调用进程进入睡眠状态的系统调用。

下面对信号处理函数作几点说明。

(1) 信号与信号处理函数的连接

只有建立了信号与信号处理函数之间的连接,用户自定义的信号处理函数才有意义。信号与信号处理函数间连接可以通过 signal 函数或 sigaction 函数来实现。

(2) 信号的阻塞

进程有一个选项来决定是否阻塞(blocking)信号的发送(delivered)。如果为进程产生的信号被阻塞,则无论信号的处理是缺省的还是自定义的,信号将一直处于挂起(pending)状态,直至对信号的阻塞被解除或将信号处理改为忽略信号。

对信号的阻塞是在信号被发送(delivered)时决定的,而不是在信号产生时决定的。这样可以在信号发送前改变对信号的处理。

(3) 进程的信号掩码

每一个进程都有一个信号掩码,它是当前进程阻塞信号的集合。

10.2　基本插口函数

插口 API 的实现,就是调用 socket 函数,实现一些功能。下面介绍几个基本

的 socket 调用。

10.2.1 创建 socket——服务绑定 :socket()

1. 功能

在应用程序中使用插口 API 时,首先要创建一个插口——socket。为了说明什么是 socket,可以打个比方 :一个电话插座,既可以接电话机打电话,也可以接传真机发传真,还可以接计算机上网。接上什么设备,就是绑定了什么服务。创建一个插口,就是创建一个通信,需要指定提供什么服务,即为一个标识符绑定某个服务,并为之分配资源。这个标识就是该插口标识。

应用程序调用 socket() 函数将创建一个插口。

2. 原型

```
#include<sys/socket.h>
int socket (int family,int type,int protocol) ;
```

返加 :非负描述字——成功,-1——出错

3. 参数说明

(1) family (在 Linux 中为 domain)

family 用来指定插口使用的协议族。它可以是下列值 :

AF_INET	IPv4 版本的 TCP/IP 协议族。
AF_INET6	IPv6 版本的 TCP/IP 协议族。
AF_OSI (AF_IPX)	ISO 协议族。
AF_CCITT	ITU-T 协议,Xerox 网络系统协议。
AF_LOCAL (AF_UNIX)	UNIX 域协议族,本机的进程间通信时使用。
AF_ROUTE	路由插口。
AF_KEY	密钥插口。

这些常量的名字在不同的实现中可能有些差别 (如有可能是括号中的名字) 。

(2) type

type 表示插口要使用的通信类型。根据传输数据的类型,插口可分如下 5 种类型。

SOCK_STREAM :面向连接的数据字节流插口 (Stream Socket) 。字节流插口是最常用的插口类型,是由 TCP 协议使用的接口。它提供面向连接的 (建立虚电路) 、可靠、有序且不重复的无记录边界的数据流。

SOCK_DGRAM:无连接的数据报插口(Datagram Socket)。数据报插口是UDP协议使用的接口。它提供无连接的服务,以独立的信包进行网络传输,信包最大长度32KB,传输不保证顺序性、可靠性和无重复性,通常用于单个报文传输或不太重要的通信中。

SOCK_RAW:原始数据报插口(Raw Socket)。原始数据报插口只对Internet有效,可以用来直接对数据链路层编程,一般不提供给普通用户,主要用于开发新的网络层协议或用于提取协议较隐蔽的功能。

SOCK_SEQPACKET:提供顺序的、可靠的、基于固定最大长度数据报的、有传输路径的双向连接。大致与SOCK_STREAM相同。

SOCK_RDM:提供一个可靠的数据报层,但不保证报文到达的顺序。

应当注意,在socket()函数中,并非所有family和type的取值都是可以组合的。表10.1列出了一些常见的组合及其对应的协议。

表10.1　socket函数中的family和type一些常见组合

类型(type)	AF_INET	AF_INET6	AF_LOCAL	AF_ROUTE	AF_KEY
SOCK_STREAM	TCP	TCP	支持	不支持	不支持
SOCK_DGRAM	UDP	UDP	支持	不支持	不支持
SOCK_RAW	IPv4	IPv6	不支持	支持	支持

(3) protocol

protocol表示插口所使用的特定传输协议。通常设置为0,表示使用默认协议。例如对Internet:

当参数type为流插口时,默认为TCP协议;

当参数type为数据报插口时,默认为UDP协议。

当参数type为原始插口类型时,由于对多种协议有效(如ICMP、IGMP等),需要指定protocol参数。

Protocol的常量定义在头文件<netinet.h>中,并以IPPROTO_开头。如IPPROTO_IGMP等。

4. 应用举例

例10.2　创建一个TCP插口的操作。

```
sockfd=socket(AF_INET,SOCK_STREAM,0);
if(sockfd<0){/* 创建socket不成功时的错误处理 */
    fprintf(stderr, socket error:%s\n ,strerror(errno));
```

```
        exit (1);
    }
```

由 socket( )使用的参数可以看出,它在创建时只与协议族、通信类型和特定传输协议相关。这些参数就是特定服务的提供者。

通常,跨机应用进程要在网络环境下通信,因此必须在网络的每一端都建立一个插口,通过两个插口间的读、写操作实现网络的通信功能。总之,插口是网络通信的基本单元,它提供了不同主机间进程双向通信的端点。这些进程在通信前各自建立一个 socket,并通过对 socket 的读/写操作实现网络的通信功能。

10.2.2 本地地址绑定(bind( ))

插口被创建后,仅仅是定义了一个特定服务的描述符,既没有本地地址,也没有远程地址。为了进行通信,要将 socket 绑定在某一地址上,就好像将电话插座与某个设备连接后,还要给出用户账号和主机号一样。在介绍绑定方法之前,首先介绍一下插口地址的数据结构。

1. 插口地址

插口的地址绑定就是给一个插口赋予特定插口二元组的值——插口地址。由于插口可以被任意协议使用,因此插口地址因协议族而不同,也随版本而异。目前插口的地址格式已经有了许多版本。最近 Berkeley 代码定义的插口地址的数据结构如下:

```
struct sockaddr {
  u_char        sa_len;          /* 地址总长 */
  u_char        sa_family;       /* 地址族 */
  char          sa_data[14];     /* 地址本身 */
};
```

(1) sa_family 是地址族,指出通信域的地址格式,常用的变量有:

AF_ISO(或 AF_OSI) ——ISO 协议地址格式;

AF_UNIX——UNIX 协议地址格式;

AF_INET——TCP/IP 协议地址格式;

AF_NS——XEROX 协议地址格式;

AF_INET6——IPv6 协议地址格式;

AF_IPX——Novell 网络地址格式;

AF_ROUTE——内部路由协议地址格式;

AF_APPLETALK——Apple Talk 网络地址格式;

AF_DECnet——DEC 网络地址格式。

(2) sa_data 字段为实际地址。插口 API 还要求每个协议族为 sockaddr 结构体中的 sa_data 字段定义自己的确切格式。IPv4 协议族插口地址的结构体名为 sockaddr_in，定义在头文件 netinet/in.h 中，其定义如下：

```
struct in_addr {
 in_addr_t        s_addr;              /* 32 位 IPv4 地址 */
}
struct sockaddr_in {
 u_char           sin_len;             /* 地址总长 */
 u_char           sin_family;          /* 地址族 */
 u_short          sin_port;            /* 协议端口号 */
 struct in_addr   sin_addr;            /* 计算机 IP 地址 */
 char             sin_zero[8];         /* 未用 (全置为 0) */
};
```

我们注意到，sockaddr_in 的前两个字段对应一般 sockaddr 的前两个字段，后三个字段定义了 TCP/IP 协议所希望的确切地址格式。

2. 函数 bind()的功能和原型

(1) 功能

调用函数 bind()，可以将本地地址与插口绑定在一起。调用成功时，返回 0；否则，返回－1，并设置全局变量 errno 为错误类型 EADDRINUSER。

(2) 原型

```
#include <sys/socket.h>
int bind (int sockfd, const costruct sockadd * myaddr, socklen_t addrlen);
                                    返回 0——成功，－1——出错
```

(3) 参数说明

sockfd 是函数 socket 返回的插口描述字；

myaddr 是特定于协议的地址结构体的指针，指向本地插口地址；

addrlen 是插口地址结构体的长度。

3. 使用说明

服务器程序和客户机程序都可以调用 bind()来绑定插口地址。表 10.2 列出了地址绑定时 IP 地址和协议端口号的可行组合方式，其他 IP 地址和端口号的组合没有实际意义。

表 10.2　地址绑定时 IP 地址和协议端口号的可行组合方式

程序类型	IP 地址	端口号	说明
服务器	INADDR_ANY (通配地址)	非 0	服务器指定公认端口号,不指定 IP 地址
服务器	本地 IP 地址	非 0	服务器指定 IP 地址和公认端口号
客户机	INADDR_ANY (通配地址)	非 0	指定客户机连接端口号,不指定 IP 地址
客户机	本地 IP 地址	非 0	指定客户机 IP 地址和连接端口号
客户机	本地 IP 地址	0	指定客户机 IP 地址

下面对上述 5 种情形进行说明。

(1) 服务器指定公认端口号,不指定 IP 地址

服务器调用 bind() 函数时,如果设置的 IP 地址为 INADDR_ANY (通配地址),则表示它可以接收来自任何 IP 地址的客户机的连接。这是服务器最经常使用的绑定方式。

(2) 服务器指定 IP 地址和公认端口号

服务器调用 bind() 函数时,如果设置了某个本地 IP 地址,就表示只接收来自该 IP 地址指定的客户机的连接。

(3) 指定客户机连接端口号

在一般情形下,客户机一般不把 IP 地址捆绑到它的插口上,当客户机调用 connect() 进行 TCP 连接时,系统内核会自动为它选择一个未用的端口号,并用本地 IP 地址来填充插口地址中的相应项。在特殊情形下,才需要指定一个未用的端口号。

(4) 指定客户机的 IP 地址和连接端口号

表示客户机使用指定的 IP 地址和端口号进行通信。

(5) 指定客户机的 IP 地址

表示客户机使用指定的 IP 地址进行通信,而由系统内核自动为客户机选择一个未用的端口号。这种方式用于主机有多个 IP 地址的情形。

4. 应用举例

例 10.3

```
struct sockaddr_in serv_addr;
int sockfd;                          /* 插口描述符 */
short port=XXXX;                     /* 端口号 */

bzero (&myaddr, sizeof(myaddr));     /* 将地址数据结构缓存清零 */
myaddr.sin_family=AF_IENT;
```

```
myaddr.sin_port=htons (PORT) ; /* 将端口号转换成网络字节顺序 */
myaddr.sin_addr.s_addr=htonl(INADDR_ANY) ;  /* 设置IP 地址 */

if (bind (sockfd, (struct sockaddr * ) & myaddr, sizeof (myaddr) ) < 0) {
  fprintf (stderr, Bind to port %d error \ n , PORT) ;
  exit (1) ;
}
```

说明：

(1) 函数 bzero() 的原型为

```
#include <string.h>
void bzero (void *dest,size_t nbytes) ;
```

该函数的功能是将 dest 指定的对象中 nbyte 指定数目的字节初始化为 0。在本例中为将地址结构体中所有的字节都初始化为 0。

(2) 函数 htons() 和 htonl() 的原型为

```
#include <netinet/in.h>
unit16_t htons (unit16_t host16bitvalue) ;
/* 将 16 位主机字节序转换为网络字节序 */
unit32_t htonl (unit32_t host32bitvalue) ;
/* 将 32 位主机字节序转换为网络字节序 */
unit16_t ntohs (unit16_t net16bitvalue) ;
/* 将 16 位网络字节序转换为主机字节序 */
unit32_t ntohl (unit32_t net16bitvalue) ;
/* 将 32 位网络字节序转换为主机字节序 */
```

这里所谓字节序值是指对于多字节的数据而言，在内存中可以有两种存储方式：

小端 (little-endian) 字节序——主机字节序，以低端字节作为起始地址 (小地址)；

大端 (big-endian) 字节序——网络字节序，以高端字节作为起始地址 (小地址)。

具体采用哪种方式，依具体的系统而异。

然而在一个 IPv4 插口地址中，包含一个 16 位的端口号和一个 32 位的 IP 地址。发送协议栈和接收协议栈必须对这些多字节的传送字节序协调一致。为此需要进行转换。

网际协议在处理这些多字节整数时，使用大端字节序——网络字节序。在

那些与网际协议字节序相同的系统中,这四个函数通常被定义成空宏。

10.2.3 建立插口连接——绑定远端服务器地址:connect()

1. 功能

函数 connect()可以让客户机程序建立一个与 TCP 服务器的连接。

2. 原型

```
#include <sys/socket.h>
int connect (int sockfd, const struct sockaddr * servaddr, socklen_t addrlen);
```

参数:

sockfd 是函数 socket 返回的插口描述符;

servaddr 指定远程服务器的插口地址;

addrlen 是插口地址结构体的长度。

3. 错误代码

函数 connect()激发 TCP 的三次握手过程,且在连接建立成功或出错时才返回。返回错误的错误代码保存在全局变量 errno 中。错误代码为下面的任何一种。

(1) ETIMEOUT:TCP 客户机没有收到 SYN 响应时返回这类错误。例如在 4.4BSD 中,当调用 connect()函数时,发出一个 SYN;若无响应,等待 6 s 后再一个;若无响应,等待 24 s 后再一个。总共等了 75 s 后仍未收到响应,则返回错误 ETIMEOUT。

(2) ECONNREFUSED:硬错(hard error),即服务器对客户机 SYN 的响应是 RST(复位),表明该服务器在客户机端指定的端口上没有进程在等待连接请求。这时,客户机端一收到 RST,就立刻返回错误 ECONNREFUSED。

(3) EHOSTUNREACH 或 ENETUNREACH:硬错(soft error),即如果客户机发出的 SYN 在中间路由器上引发了一个目的地不可达 ICMP 错误,内核将保存此消息,并按一定间隔连续发出 SYN(如 4.4BSD 按上述规律发 SYN)。若在规定的时间内仍未收到响应,则返回错误 EHOSTUNREACH 或 ENETUNREACH。

4. connect()工作过程

connect()的工作过程如图 10.3 所示。

TCP 协议为每个被动(倾听)插口维护两个队列:

(1) 未完成队列——暂存尚未完成三次握手操作的 TCP 连接的队列。TCP 服务器每接到一个客户机 SYN 数据段,就会在未完成队列中创建一个新条目,

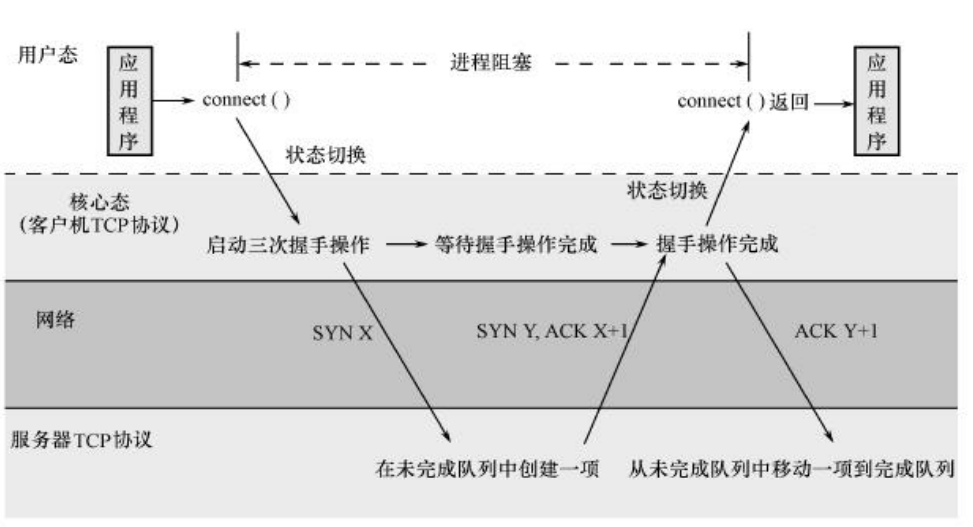


图 10.3　connect( ) 的工作过程

然后发送对客户机 SYN 的确认 (ACK) 和自己的 SYN 数据段 (ACK + SYN) ,等待客户机对自己的 SYN 数据段的确认。此时,插口处于 SYN _ RCVD 状态。该队列中的每个条目一直保存到客户机返回对服务器 SYN 的确认或连接超时。

(2) 完成队列——暂存已经完成三次握手操作但尚未被应用程序处理的 TCP 连接的队列。此时,插口处于 ESTABLISHED 状态。该队列中的每个条目一直保存到应用程序调用函数 accept 来接收它。

5. 应用举例

例 10.4

```
struct sockaddr _ in serv _ addr ;
char srvname [] =  XX.XX.XX.XX  ;/* 服务器地址 */
int sockfd ;                    /* 插口描述符 */
                                /* 已调用 socket 产生 sockfd */
bzero ((char*) &serv _ addr, sizeof (serv _ addr)) ;
                                /* 初始化服务器地址结构 */
serv _ addr.sin _ family= AF _INET ;      /* 协议地址类型 */
serv _ addr.sin _ addr.s _ addr= inet _ addr (srvname) ;
                                /* 将点分 IP 地址转换为整型地址 */
serv _ addr.sin _ port= htons (port) ;
                                /* 将端口号转换为网络字节顺序 */

if (connect (sockif, (struct sockaddr*) &serv _ addr,sizeof (serv _ addr)) <0)
```

```
    {/* 错误处理 */}
    else {/* 建立连接成功 */}
```

6. 注意事项

(1) 客户机在调用 connect() 前不一定非要先调用 bind 函数,因为必要时系统内核会自动给出源 IP 地址并选择一个临时端口。

(2) 如果函数 connect() 调用失败,则插口不可使用,必须关闭,不能对此插口再调用 connect() 函数。如果要重新建立连接,就要重新调用 socket 函数。

(3) 虽然 connect() 的主要作用是建立连接,但是它也可以由无连接的客户机程序调用。在面向连接的传输中使用时,connect() 与指定的服务器建立一个传输连接;在无连接的传输中使用时,connect() 在插口中记录服务器地址,使客户机可以向同一服务器传输多条信息而不必在每条信息中都指明目的地。

10.2.4 插口被动转换:listen()

1. 功能

函数 listen() 有两个功能。

(1) 主动插口的被动转换

socket() 创建的插口都是主动插口,只可以用来(调用 connect())进行主动连接(发出请求),不能接收连接请求。listen() 函数将未连接的主动插口转换为被动插口,即倾听插口(listening socket),告诉操作系统该插口可以接收连接请求。所以 listen() 也称为创建倾听插口函数。

(2) 设置最大请求连接数

listen() 用一个参数(backlog)指定完成队列的最大长度。如果一个客户机的 SYN 数据段到达时,倾听插口的完成队列已经满了,TCP 则忽略这个 SYN 数据段。

listen() 函数执行成功,返回 0,服务器的 TCP 状态由 CLOSED 转变成 LISTEN,成为被动模式;执行失败,返回 -1。

2. 原型

```
#include <sys/socket.h>
int listen(int sockfd, int backlog);
                                    返回 0——成功,-1——出错
```

参数:

sockfd 为要转换的已绑定本地地址的插口描述符;

参数 backlog 设置请求队列的最大长度,表示被动(倾听)插口能够接收

的最大数目的未接收连接(相当于窗口通告)。

3. 创建 TCP 服务器倾听插口的过程

(1) 首先调用 socket()函数创建一个主动插口;

(2) 调用 bind()函数进行插口地址绑定;

(3) 调用 listen()函数进行转换。

4. 应用举例

例 10.5　函数 listen()的包裹函数——允许用环境变量 LISTENQ 指定 backlog 值。

```
Void
Listen(int fd, int backlog)
 {
    char *ptr;
    /* 下面用环境变量包裹第 2 个参数 */
    if ((ptr=getenv( LISTENQ )) != NULL)
                                        /* 判定环境变量是否为空 */
      backlog=atoi (ptr) ;              /* 转换 ASCII 数字串为整数 */
    if (listen (fd,backlog) <0)         /* 如果转换成负数,出错 */
      err_sys( listen error ) ;
}
```

例 10.6　如何利用 socket、bind 和 listen 函数建立连接并接受数据。

```
int establish (unsigned short portnum) /* 建立 socket 的代码 */
 {
    char myname [MAXHOSTNAME+1] ;
    int s;
    struct sockaddr_in sa;
    struct hostent *hp;
    memset(&sa,0,sizeof (struct sockaddr_in)) ;
                                  /* 清除地址结构内容 */
    gethostname (myname,MAXHOSTNAME)得到主机名 */
    hp=gethostname (myname) ;   /* 得到地址信息 */
    if (hp==NULL) return (-1) ;/* 如果地址不存在,返回-1 */
    sa.sin_family=hp->h_addrtype*; 主机地址 */
    sa.sin_port=htons (portnum) ; /* 端口号 */
    if ((s=socket (AF_INET,SOCK_STREAM,0)) < 0) return (-1) ;
```

```
                                /* 如果创建 socket 不成功,返回-1 */
    if (bind (s, &sa, sizeof(struct sockaddr_in)) <0)
                                /* 把地址捆绑到 socket 对象上 */
    {
        close (s) ;
        return (-1)
    }
    listen(s,3) ;               /* 设置请求队列中最大请求数为 3 */
    return (s) ;
}
```

说明:

(1) 函数 memset( ) 的原型为

```
#include <string.h>
void *memset (void * dest, int c, size_t len) ;
```

它的功能是将 dest 指定的对象中数目为 len 个字节的部分置为 c。在本例中,是将插口地址中的所有字节都置为 0。

(2) 函数 gethostname( ) 的原型为

```
#include <unistd.h>
int gethostname(char * name, size_t namelen) ;
                        返回 0——成功,-1——出错
```

它的功能是返回当前主机的名字。其中,namelen 通常为定义在头文件 sys/param.h 中的常值 MAXHOSTNAMELEN。

10.2.5 从被动插口的完成队列中接收一个连接请求 accept( )

1. 功能

一个插口被创建、用 bind 绑定本地地址并转换成被动模式后,就可以接收一个连接请求。一旦接收了连接,服务器就能使用该连接与客户机进行通信。面向连接传输协议的服务器调用 accept 从被动(倾听)插口的完成连接队列中接收下一个连接请求。如果该完成连接队列空,则使这个进程睡眠。

2. 原型

函数 accept 的原型为

```
#include <sys/socket.h>
```

```
int accept (int sockfd, struct sockaddr * addr, int * addrlen) ;
                    返回 :非负描述符——成功,-1——出错
```

参数：

sockfd 为服务器已经创建并绑定到指定插口地址的插口描述符；

addr 是指向一个 Internet 插口地址结构体的指针；

addrlen 是指向一个整型数的指针。

当函数执行成功时,将返回 3 个结果：

函数返回值为一个标识所接收连接的新的插口描述符；

addr 指向的变量中存储客户机地址；

addrlen 指向的整数变量中存储客户机地址的长度。

在对客户机地址不感兴趣时,可以将参数 addr 和 addrlen 设置为 NULL。

3. accept() 工作过程

TCP 使用倾听插口接收客户机的连接请求。如果调用 accept() 时,倾听插口的完成队列中有已经完成 3 次握手的 TCP 连接,accept() 就从队列的首部读取第一个连接,并返回该连接的描述符。如果调用 accept() 时,倾听插口的完成队列为空,进程将阻塞,等待出现新的完成 3 次握手的 TCP 连接送到完成队列 ;TCP 协议为服务器创建一个新的连接插口来标识这条新的连接,accept() 返回该新连接的描述符。图 10.4 描述了阻塞时 accept() 的工作过程。

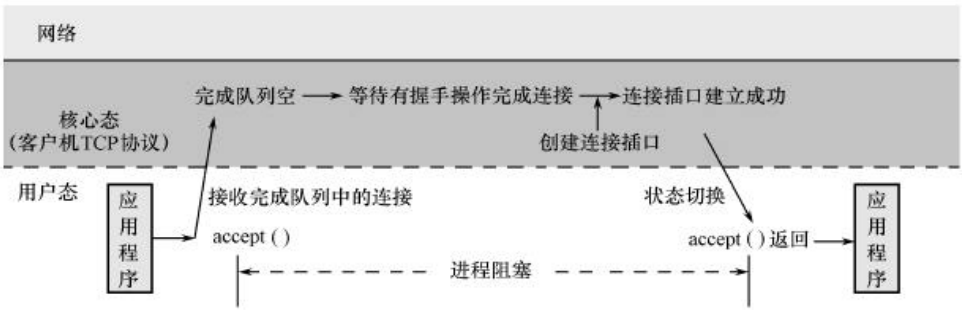


图 10.4　阻塞时 accept() 的工作过程

4. 应用举例

例 10.7　返回一个不存在的插口描述符,即出错。

```
int get_connection (int s)   /* s为完成连接队列中的一个连接请求 */
 {
    int t;    /* t为所连接的插口描述符 */
    /* 如果连接存在,就接受连接,并返回所连接的插口描述符 ;否则返回-1
```

```
    */
    if (t=accept(s,NULL,NULL))<0) return (-1);
    return t;                    /* 返回所连接的插口描述符 */
}
```

10.2.6 基本插口 I/O 函数

1. 基本插口 I/O 函数列表

UNIX 系统中将插口作为文件句柄(文件描述符)进行处理,因此用于普通文件读写的函数也可以用来进行插口的 I/O 操作。表 10.3 为用于 I/O 操作的系统调用函数。

表 10.3 插口 I/O 函数列表

操作方式	函数名	插口类型	需要指定目的地址	标志位	控制信息	要求头文件	功 能
写	write	TCP	否	无	无	unistd.h	发送一个缓冲区中的数据
	writev	TCP	否	无	无	sys/uio.h	发送多个缓冲区中的数据
	send	TCP	否	无	无	sys/socket.h	按指定选项发送数据
	sendto	UDP(TCP)	是	有	无	sys/socket.h	发送数据到指定地址
	sendmsg	UDP(TCP)	是	有	有	sys/socket.h	从多个缓冲区发送数据、控制信息和地址
读	read	TCP	否	无	无	unistd.h	接收数据到一个缓冲区
	readv	TCP	否	无	无	sys/uio.h	接收数据到多个缓冲区
	recv	TCP	否	无	无	sys/socket.h	按指定选项接收数据
	recvfrom	UDP(TCP)	是(返回报文源地址)	有	无	sys/socket.h	接收数据和发送者地址
	recvmsg	UDP(TCP)	是(返回报文源地址)	有	有	sys/socket.h	接收数据、控制信息和地址到多个缓冲区

下面对表中的标志位和控制信息做一点说明。

(1) 标志位

标志位在 I/O 函数中用参数 flags 描述,用于控制数据的接收/发送方式,具体见表 10.4 中的说明。

表 10.4　插口 I/O 函数中的标志位

flags	说　明	使用函数
MSG_DONTROUTE	不查路由表：告诉内核，目标主机在本地网上，不需查路由表	send，sendto，sendmsg
MSG_DONTWAIT	本操作不阻塞	recv，recvfrom，recvmsg，send，sendto，sendmsg
MSG_OOB	发送或接收带外数据 OOB（也称加速数据）——处理紧急事件	
MSG_PEEK	查看但不读取到达数据：系统不会将接收的数据丢弃	recv，recvfrom，recvmsg，
MSG_WAITALL	等待要读的所有数据：告诉内核，没有读够请求的字节数前不使操作返回	

(2) 控制信息

msghdr 把插口 I/O 时用到的有关报文的大部分信息都包装到了一个构造体中。这些信息组成了 I/O 操作中要使用的控制信息。不同的 UNIX 版本中定义的构造体 msghdr 有一定差别。下面是 Posix.1g 定义的 msghdr。

```
struct msghdr {
  void          msg_name;          /* 协议地址 */
  socklen_t     msg_namelen;       /* 协议地址长度 */
  struct iovec  msg_iov;           /* I/O 缓冲区构造体数组 */
  size_t        msg_iovlen;        /* I/O 缓冲区构造体数组长度 */
  void          msg_control;       /* 可选辅助数据的位置 */
  socklen_t     msg_controllen;    /* 可选辅助数据的长度 */
  int           msg_flags;         /* 报文标志位 */
};
```

2. I/O 函数说明

(1) 标准 I/O 函数（read() 和 write()）

原型：

```
#include <unistd.h>
ssize_t read(int filedes, void * buf, size_t nbytes);
ssize_t write(int filedes, void * buf, size_t nbytes);
```

返回：读/写字节数——成功，-1——失败

其中，buf 分别为接收/发送数据缓冲区，nbytes 为接收/发送的数据量大小。

(2) 从 socket 发送消息到另一个 socket

函数 send 和 sendto 用来从 socket 发送消息,它们的原型如下:

```
#include <sys/socket.h>
ssize_t send(int sockfd, const void * buff, size_t nbytes,
             int flags);
ssize_t sendto(int sockfd, const void * msg, size_t nbytes, int flags, const struct
               sockaddr * to, size_t tolen);
```

返回:发送字节数——成功,-1——失败

(3) 从 socket 接收消息

函数 recv 和 recvfrom 用来从 socket 接收消息,它们的原型如下:

```
#include <sys/socket.h>
ssize_t recv(int sockfd, void * buf, size_t len, int flags);
ssize_t recvfrom(int sockfd, void * buf, size_t len, int flags, struct sockaddr * from,
                 size_t fromlen);
```

返回:接收字节数——成功,-1——失败

下面给出发生错误时可能返回的一些错误代码。

EBADF:无效的描述符。

ENOTCONN:socket 指定的是一个面向连接的协议,但没有连接。

ENOTSTOCK:参数 s 指向一个非 socket 描述符。

EAGAIN:socket 标记为非阻塞,而该接收操作应该被阻塞(没有数据可接收的情况)或者接收操作设置了超时(timeout)时间,而在这段时间内没有数据可接收。

EINTR:在接收到数据之前,接收操作被信号中断。

EFAULT:指向接收缓冲正的指针指向进程空间之外。

EINVAL:参数无效。

(4) 用报文构造体收发数据

函数 recvmsg()和 sendmsg()是使用 msghdr 构造体从/往指定的地址收/发数据,它们的原型如下:

```
#include <sys/socket.h>
ssize_t recvmsg(int sockfd, struct msghdr * msg, int flags);
ssize_t sendmsg(int sockfd, const struct msghdr * msg, int flags);
```

返回:接收字节数——成功,-1——失败

10.2.7 关闭插口通道与撤销插口

1. 概述

建立一对插口的连接之后,就可以进行通信了。由于 TCP 连接是全双工

的,对每一个插口来说,都可以看作在使用读、写两个通道进行通信。于是,关闭一个插口通信可以有两种不同级别的方法。

(1) 关闭插口通道,具体有三种方式:

SHUT_RD——关闭连接的读通道;

SHUT_WR——关闭连接的写通道;

SHUT_RDWR——关闭连接的读/写通道。

(2) 撤销插口——当然,同时关闭了连接的读/写通道。

这两种方式分别用两个不同的函数 shutdown() 和 close() 实现。

2. close() 原型

函数 close() 用于撤销一个插口,终止其 TCP 连接,其原型为:

```
#include <unistd.h>
int close (int sockfd) ;
```

返回 0——成功,-1——出错

3. shutdown() 原型

```
#include <sys/socket.h>
int shutdown (int sockfd, int how to) ;
```

返回 0——成功,-1——出错

参数 how to 用于指定关闭方式。

4. 两个函数的区别

(1) 一个文件或插口可能同时有多个进程访问。为此,系统为每个文件或插口都设置有一个访问计数器,表示当前指向该文件或插口的打开的描述符个数。close() 函数每被调用一次,就将访问计数器减 1;当计数器为 0 时,才真正将插口关闭。于是就会出现调用了 close(),但没有真正将插口关闭的情形。shutdown() 函数则不管访问计数器,而是立即将指定通道关闭。

(2) 一旦使用 close() 撤销了插口,即在读/写两个方向终止了数据传送,这会造成一方撤销了插口,另一方想发送也不可能的情形。使用 shutdown() 函数可以使 TCP 处于半开连接状态,如图 10.5 所示。

5. close() 关闭过程

假设某插口描述符只有一个进程使用,客户机应用程序调用 close() 函数,主动关闭其连接的过程如图 10.6 所示。

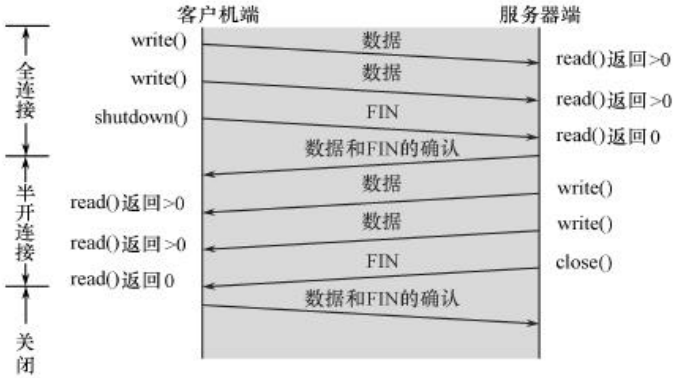


图 10.5　调用 shutdown()函数使 TCP 处于半开连接

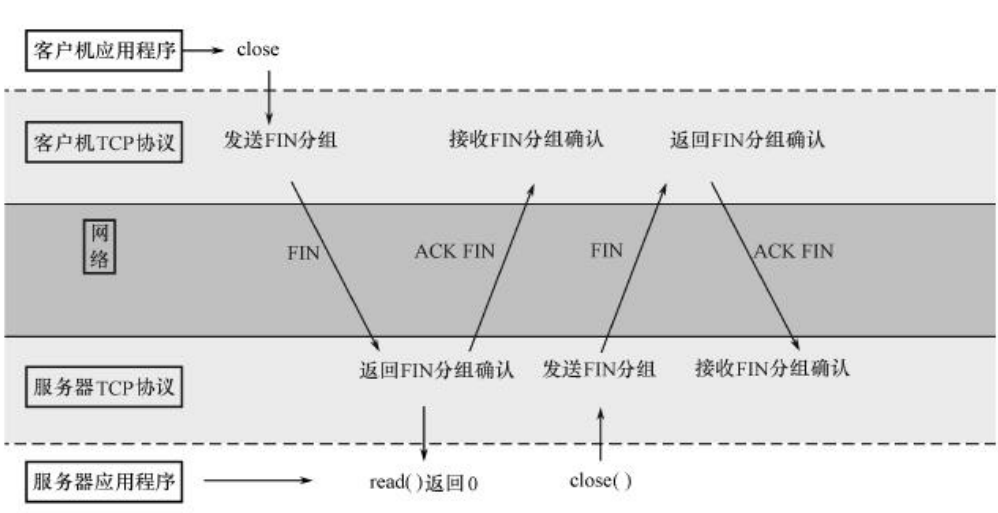


图 10.6　客户机执行主动关闭过程

习　　题

10.1　试说明 socket API、socket 地址的基本概念。

10.2　说明进程与信号之间的关系。

10.3　试画出服务器执行主动关闭过程的时序图。

10.4　试说明函数 bind、connect 和 listen 之间的区别。

10.5　试说明函数 sendto 和 write 之间的区别。

10.6　试说明函数 recvfrom 和 read 之间的区别。

第 11 章　TCP/UDP 的插口程序设计

11.1　TCP 有限状态机

11.1.1　TCP 有限状态机的概念

在网络通信中，一个健壮的插口应用程序必须能够处理通信过程中可能出

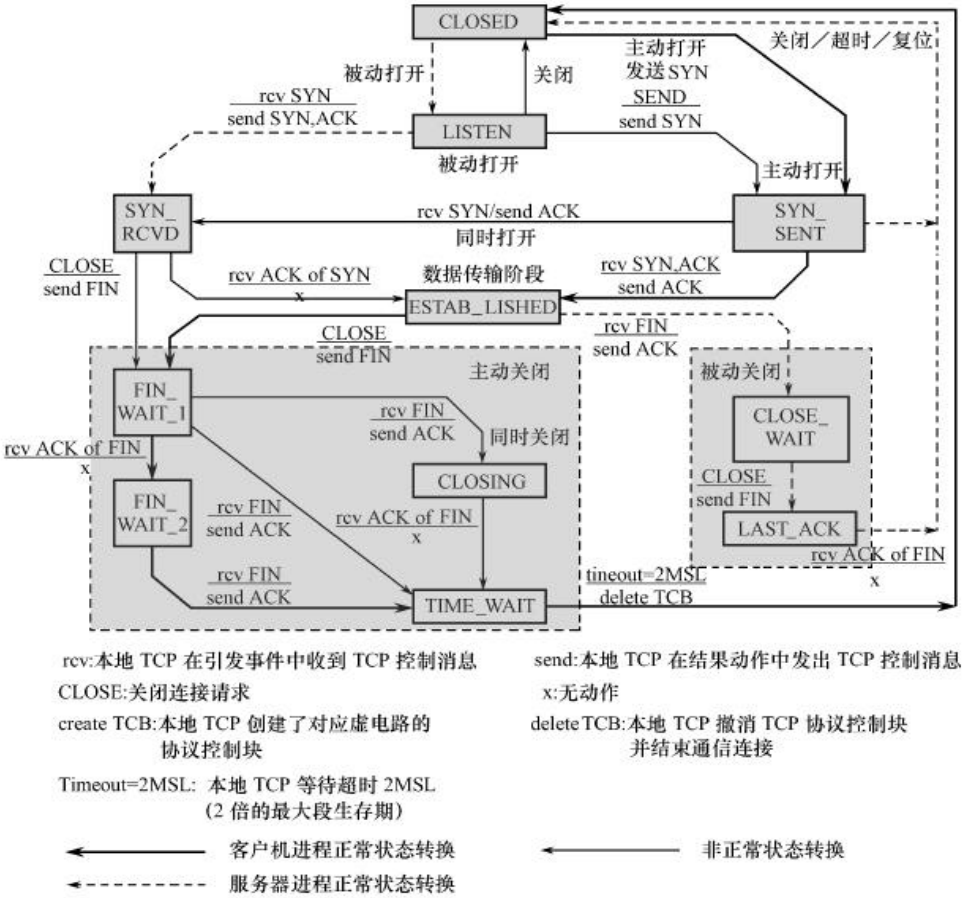


图 11.1　TCP 的有限状态机

现的各种状态。对于 TCP 来说,不同的状态将决定传输实体对一些事件的响应。描述 TCP 在响应各种事件时,各个宏观状态之间相互作用的规则称为 TCP 有限状态机(或称 TCP 状态转换图),它由图 11.1 所示的 11 种状态组成。

在 TCP 有限状态机中,每个连接可以处于如下一种状态。表 11.1 中介绍了这些状态的意义。

表 11.1 TCP 状态及其含义

状态名	说明
CLOSED	连接处于关闭状态——用户在对应通信插口上无任何动作
LISTEN	本地 TCP 正在等待远方的连接请求
SYN_SENT	本地 TCP 已发连接请求,正在等待远方 TCP 对本地请求的确认,或正在等待远方 TCP 发来的相匹配的连接请求
SYN_RCVD	本地 TCP 已经收到并发送了一个连接请求,确认了对方的请求,正在等待对方 TCP 对本地请求的确认
ESTAB-LISHED	连接已经建立,处于数据传输阶段
FIN_WAIT_1	本地 TCP 已经向对方发出了终止连接请求,正在等待远方 TCP 对本地终止连接请求的确认,或正在等待远方 TCP 发来的终止连接请求
FIN_WAIT_2	本地单向终止连接请求已经完成,正在等待远方 TCP 发来的终止连接请求
CLOSE_WAIT	本地 TCP 已经收到并确认了对方的终止连接请求,正在等待本地用户的关闭连接请求
CLOSING	通信双方同时请求关闭连接,正在等待远方 TCP 对本地终止连接请求的确认
LAST_ACK	本地 TCP 收到并确认了对方的终止连接请求后,又向远地发送了本地终止连接请求,正在等待远地 TCP 对本地终止连接请求的确认
TIME_WAIT	连接已经完全关闭,等待所有数据段从网络中消失

11.1.2 状态转换分析

1. 客户机进程发起连接的状态转换过程

① 一个主机的客户机进程要发起连接,其 TCP 实体就发一个 SYN 置 1 的分组 → 由 CLOSED 状态进入 SYN_SENT 状态。

② 收到来自进程的 SYN 和 ACK,TCP 就发出三次握手中的最后一个 ACK → 进入 ESTABLISHED 状态。这时就可以收发数据了。

2. 客户机进程发起断连请求(主动关闭)的状态转换过程

① 客户机进程要发起断连,其 TCP 实体就发一个 SYN 置 1 的分组,等待确

认 ACK 的到达 → 由 ESTABLISHED 状态变为 FIN_WAIT_1 状态。

② 运行客户机进程的主机收到确认 ACK,一个方向的连接关闭 → 进入 FIN_WAIT_2 状态。

③ 运行客户机进程的主机收到运行服务器的主机发送的 FIN 置 1 的分组后,应响应确认 ACK。

④ 收到确认 ACK,连接不是立即进入到关闭 CLOSED,而是先进入 TIME_WAIT 状态。

⑤ TCP 在 TIME_WAIT 状态等待 2MSL(MSL 表示分组在网络中的寿命),才删除原来建立的连接记录 → 返回 CLOSED 状态。

11.1.3　TCP 连接的建立和释放

TCP 是面向连接的协议,连接的建立和释放是每一次面向连接的通信中都不可少的。现在假定主机 A 中运行一个客户机程序,主机 B 中运行一个服务器程序。

1. TCP 连接的建立过程

TCP 采用"三次握手"协议建立连接,下面介绍其工作过程。

① 做准备工作

服务器端 B:

服务器进程首先调用 listen(),向其 TCP 层发出一个被动打开命令,要 TCP 做好接收客户机进程连接请求的准备;

进入"监听"状态,不断检测有无客户机进程发起的连接请求,有就作出响应。

客户机端 A:

客户机进程调用 connect(),向其 TCP 层发出一个主动打开命令,表明向某个 IP 地址的某个端口发起传输连接;

A 端的 TCP 向 B 端的 TCP 发出连接请求分组,其首部同步比特 SYN 置为 1,给出后面传输的第一个数据字节的序号(设为 x)。

② 主机 B 的 TCP 收到连接请求分组后,如同意,则发回确认分组:将 SYN 置 1,确认序号为 x+1,自己的序号为 y。

③ 主机 A 的 TCP 收到来自 B 的连接请求确认分组后:

向 B 发回确认,确认序号为 y+1;

同时通知上层应用进程连接已经建立。

④ 主机 B 的 TCP 收到主机 A 的确认,也通知其上层的应用进程连接已经建立。

2. TCP 连接的释放过程

当数据传输结束后,需要释放连接。通信的双方都可以发出释放连接的请

求。现在假设：

最后发送的数据序号为 m；

主机 A 不再向 TCP 层传送数据；

首先由运行在主机 A 上的进程向其 TCP 层发出连接释放请求。

下面介绍此后的过程。

① 主机 A 上的 TCP 通知 B 方的 TCP 要求释放从 A 到 B 的连接：

将发往 B 上 TCP 的分组首部的终止比特 FIN 置 1；

该终止连接报文序号为 u=m+1。

② 主机 B 的 TCP 协议层收到释放连接通知后，即发出确认报文：

确认报文序号为 u+1；

同时通知上层的应用进程。

这时，连接处于半关闭状态——B 不能再接收数据，但若还有一些数据要发往 A，则仍可以向 A 发送数据，A 接收到数据仍应向 B 发送确认。

③ 当主机 B 没有数据要发往 A 时：

B 的应用进程就通知 TCP 层释放连接；

主机 B 的 TCP 层发出连接释放分组（终止比特 FIN=1，序号 v 为已传送数据最后字节序号加 1；

重复上次已发过的 ACK=u+1。

④ 主机 A 收到 B 发来的连接释放分组后：

对由 B 发来的连接释放分组进行确认（ACK=v+1），这时 B 到 A 的反方向连接被释放；

A 的 TCP 向应用层报告，整个连接全部释放。

图 11.2 表明了上述 TCP 连接的建立和释放过程。

3. TIME_WAIT 状态的必要性

在上述过程中，TIME_WAIT 是一个很特殊的状态：假设执行主动关闭连接操作的一方在接收到对方的关闭请求（FIN 数据分组）后发送确认分组并进入这个状态。它的必要性表现在下述两个方面。

(1) 保证 TCP 协议的全双工连接能够可靠地关闭

为了说明这一必要性，假设：

① 客户机进程收到服务器进程发送的数据分组（FIN，SEQ=v，ACK=u），并发送了确认分组（ACK=v+1）后立即释放连接，并直接进入了 CLOSED 状态。

② 由于 IP 的不可靠性使这个 ACK 在传输中丢失，而服务器进程还在等待这个 ACK。超时后，服务器进程将重发其未被应答的分组（FIN，SEQ=v，ACK=u）。

③ 由于客户机进程已经释放了连接，服务器进程得到的将是 RST 响应。这

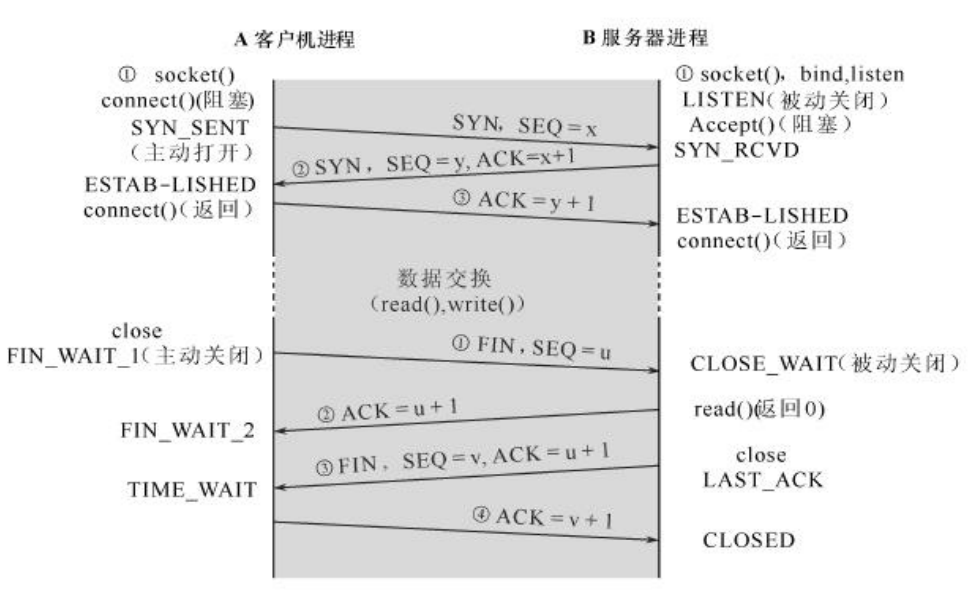


图 11.2　TCP 连接的建立和释放过程

个响应被服务器解释为错误。

结论：导致 TCP 不能保证彻底终止两个方向上的数据流，即不能保证全双工释放。

对策：设置 TIME_WAIT 状态，并在这一状态等待最大分组生存时间的两倍，就可以确保全双工释放。

(2) 保证本次连接的重复分组从网络中完全消失

客户机系统选择端口号的原则是保证所有活动的连接都具有惟一的端口号，但不保证前后两次与同一服务器的连接具有不同的端口号。这样就有可能出现前后两次与同一服务器的连接使用了相同的端口号，产生相同的连接插口对。在一般情形下，这没有多大问题。但是若由于 IP 传输的不可靠性使得前一次连接中的最后数据分组还在网络中滞留（如在某个路由器中被缓存）时，即被关闭而后一个连接插口对已经建立，残留的分组就会把后一个连接当作原先连接的化身（incarnation）。

设置 TIME_WAIT 状态，并在这一状态等待最大分组生存时间的两倍，就可以确保本次连接的分组全部从网络中消失。

4. 三次握手建立方式的分析

在每创建一个连接的三次握手中，要求每一端都产生一个随机的 32 位序列号，以便应用程序能通过 TCP 进行通信。

TCP 连接的三次握手机制可以通过如下一种方式建立：

由一方 TCP 发起握手过程，另一方 TCP 响应该同步过程；

由通信双方同时发起连接的同步握手。

(1) 由一方 TCP 发起握手过程，另一方 TCP 响应该同步过程

图 11.3 描述了一个最常见的、最简单的三次握手过程。图中，所有的 TCP 段都用序号（SEQ）、控制标志（CTL）和确认号（ACK）三个域简要地给出。

```
TCP A                                                    TCP B
1. CLOSED                                                LISTEN
2. SYN-SENT      →<SEQ=100><CTL=SYN>→                    SYN-RECEIVED
3. ESTABLISHED   ←<SEQ= 00><ACK=101><CTL=SYN,ACK>←       SYN-RECEIVED
4. ESTABLISHED   →<SEQ=101><ACK=301><CTL=ACK>→           ESTABLISHED
5. ESTABLISHED   →<SEQ=101><ACK=301><CTL=ACK><DATA>→     ESTABLISHED
```

图 11.3 基本的连接同步三次握手

说明：

第 2 行，TCP A 向 TCP B 发送 SYN 段请求建立连接；

第 3 行，TCP B 确认了 TCP A 的请求，并向 TCP A 发出了同步请求；

第 4 行，TCP A 确认了 TCP B 的 SYN，至此，TCP A 与 TCP B 间的通信连接正常建立；

第 5 行，通信连接上的数据传送开始。

(2) 通信双方同时发起连接的同步握手

图 11.4 描述了通信双方同时发起建立连接的过程。它比前面的连接过程稍复杂一些。图中的“…”表示某一数据正在网络传输途中。

```
TCP A                                                    TCP B
1. CLOSED                                                LISTEN
2. SYN-SENT      →<SEQ=100><CTL=SYN>                     …
3. SYN-RECEIVED  ←<SEQ=300><CTL=SYN> ←                   SYN-SENT
4.                    …<SEQ=100><CTL=SYN>      ←         SYN-RECEIVED
5. SYN-RECEIVED  →<SEQ=100><ACK=301><CTL=SYN,ACK>…
6. ESTABLISHED   ←<SEQ=300><ACK=101><CTL=SYN,ACK>←       SYN-RECEIVED
7.               …<SEQ=101><ACK=301><CTL=ACK>      →     ESTABLISHED
```

图 11.4 同时的连接请求

在这种形式的连接建立过程中，会出现本次同步请求与遗留在网络中的同步连接请求序号重复的现象。针对这一问题，TCP 协议使用了 RST 段（重复段）。这样，对于收到重复段的 TCP 接收方来说，可以考虑如下两种情况：如果接收方

TCP 是在某非同步状态 SYN _ RECEIVED 下收到重复段的，并且其前阶段状态为 LISTEN，那么接收方 TCP 将回到 LISTEN 状态重新开始侦听网络连接请求；如果接收方 TCP 是在其他同步状态下收到重复段的，接收方 TCP 将异常终止连接并通知本地用户。在图 11.5 中，如果第 6 行的 SYN 比第 5 行的 RST 到达 TCP B 早，连接的双方都将收到 RST 段；接着，TCP A 由于连接异常终止而回到 CLOSED 状态，TCP B 则重新回到 LISTEN 状态。

TCP A		TCP B
1. CLOSED		LISTEN
2. SYN-SENT	→<SEQ=100><CTL=SYN>···	
3. （duplicate）	···<SEQ=90><CTL=SYN> →	SYN-RECEIVED
4. SYN-SENT	←<SEQ=300><ACK=91><CTL=SYN,ACK> ←	SYN-RECEIVED
5. SYN-SENT	→<SEQ=91><CTL=RST> →	LISTEN
6.	···<SEQ=100><CTL=SYN> →	SYN-RECEIVED
7. SYN-SENT	←<SEQ=400><ACK=101><CTL=SYN,ACK>←	SYN-RECEIVED
8. ESTABLISHED	→<SEQ=101><ACK=401><CTL=ACK>→	ESTABLISHED

图 11.5　旧的重复 SYN 的恢复

(3) “半开”连接

下面的连接状态称为“半开”连接，是特殊连接状态：

TCP 连接的一方在没有对方消息的情况下已经关闭或异常终止了连接；由于系统崩溃，连接的双方处于一种非同步状态。

在“半开”状态下，只要具有通信能力的任一方向对方发送了数据，都将使连接自动重置或复位。

图 11.6 表明 TCP A 崩溃以后使 TCP A 与 TCP B 之间处于“半开”连接状态。这时，假定 TCP A 一端的用户企图重新打开通信连接，就会使 TCP B 感到它与 TCP A 之间仍然处于同步状态。图中的第 3 行表明，TCP A 方的请求到达 TCP B 时，由 TCP B 再一次地收到对方的同步请求，可以判断出连接已经处于非同步状态中，于是 TCP B 将异常终止连接并向 TCPA 发送 RST 段使连接复位；而 TCP A 在收到 RST 段时马上终止本地的连接企图。

TCP A		TCP B
1. (CRASH)		(SEND 300，RECEIVE 100)
2. CLOSED		ESTABLISHED
3. SYN-SENT	→<SEQ=400><CTL=SYN> → (??)	
4. (ABORT!!)	←<SEQ=0><ACK=301><CTL=SYN,ACK> ←	(ABORT!!)
5. CLOSED		CLOSED

图 11.6　“半开”连接状态的出现

图 11.7 给出了 TCP B 首先在“半开”连接上发起数据传输的情形。第 2 行表明,当 TCP A 收到从 TCP B 发来的数据段后,该数据段的目的方已经不存在。这时,TCP A 将在第 3 行向 TCP B 发送 RST 段,复位“半开”连接。

```
      TCP A                                                    TCP B
1. (CRASH)                                                     (SEND 300, RECEIVE 100)
2. (??)      ←<SEQ=300><ACK=100><DATA=10><CTL=ACK>  ←          ESTABLISHED
3.           →<SEQ=100><CTL=RST>   →                           (ABORT!!)
```

图 11.7 活动方使“半开”连接状态被发现

图 11.8 给出了在“半开”连接状态下一种有趣的现象:当 TCP A 和 TCP B 都处于 LISTEN 状态下侦听从其他目的地来的 SYN 同步连接请求时,一个在以前通信信道上迟滞的 SYN 段到达 TCP B 并激发 TCP B 开始响应三次握手过程(第 2 行)。接着 TCP A (第 3 行) 收到了其在 LISTEN 状态下不可接收的 ACK 标志,对于 TCP A 的动作将是复位双方无法同步的连接。其结果是使 TCP A 和 TCP B 都再一次回到 LISTEN 状态。

```
      TCP A                                                    TCP B
1. LISTEN                                                      LISTEN
2.                  …<SEQ=Z><CTL=SYN>    →                         SYN-RECEIVED
3.   (??)     ←<SEQ=X><ACK=Z+1><CTL=SYN,ACK>  ←                 SYN-RECEIVED
4.            →<SEQ=Z+1><CTL=RST>                  →           (return to LISTEN)
5. LISTEN                                                      LISTEN
```

图 11.8 旧的重复 SYN 的复位的“半开”连接

(4) 连接关闭

在一个已经建立的 TCP 连接中,一旦一端 TCP 用户向 TCP 协议模块递交了一条关闭连接请求——单向关闭连接的请求,就用 FIN (No more data from sender, 发送者无数据发送) 段通知通信的对等实体方它没有数据要发送。然而,一个用户端递交了关闭连接请求仅仅表明不再向对方发送数据,并不意味着它不能在连接的本地端继续接收数据。只有通信的另一端也递交了关闭连接的请求时,一条 TCP 会话连接才完全关闭。

TCP 连接的关闭过程在本质上也是一次“三次握手”过程。它有三种不同的情形。

(a) 本地用户首先关闭

本地 TCP 在收到用户的关闭请求后,将构造一个 FIN 段,然后把含有 FIN 标

志的数据段挂在 TCP 发送队列的尾部，并在第 1 次发出 FIN 段时使连接转入 FIN_WAIT_1 状态。如果这时用户不但确认了本地的关闭请求，而且也发起了一次关闭过程，连接才算正常关闭。

(b) 本地 TCP 收到远地传来的 FIN 段

当本地 TCP 收到从远地 TCP 传来的 FIN 段时，本地 TCP 将首先确认远地的关闭请求，然后通知本地用户（通信对方开始关闭连接）。此后，如果本地用户也向 TCP 实体递交了关闭连接请求，则本地 TCP 在发送完其内部缓冲的用户数据后才向对方 TCP 实体发出一 FIN 段，并且要等到该 FIN 段被确认后才删去原来的连接。如果出现的本地 FIN 得不到对方的确认，则本地 TCP 在用户超时后会异常结束连接，然后再通知本地用户。

(c) 通信双方同时关闭连接

连接两端的用户同时递交关闭连接的请求时，双方的 TCP 实体在发完各自的用户数据后，将分别向对方发出启动关闭过程的 FIN 段，并在得到各自的确认后删去原来的通信连接。

图 11.9 和图 11.10 给出了两种关闭 TCP 连接的过程。

```
      TCP A                                                    TCP B
1. ESTABLISHED                                              ESTABLISHED
2.  (Close)
     FIN-WAIT-1  →  <SEQ=100><ACK=300><CTL=FIN,ACK>   →  CLOSE-WAIT
3. FIN-WAIT-2  ←  <SEQ=300><ACK=101><CTL=ACK>          ←  CLOSE-WAI
4.                                                          (Close)
     TIME-WAIT   ←  <SEQ=300><ACK=101><CTL=FIN,ACK>   ←  LAST_ACK
5.   TIME-WAIT   →  <SEQ=101><ACK=301><CTL=ACK>        →  CLOSED
6. ( 2 MSL)
CLOSED
```

图 11.9　正常关闭连接的顺序

(5) 会话连接的重置

重置位 RST 通常用在接收端接收到一个与当前连接不相关的数据包时发送。例如，应用程序在使用完一条连接时就会使用关断操作将连接关闭，也就是说关断连接应当是正常使用中的一个操作，犹如使用完文件后的关闭文件操作一样。然而，并非所有的使用都能在正常条件下进行。有时候，非正常条件的出现会迫使应用程序或网络软件断开一条连接。重置就是 TCP 为非正常断连提供的一个重要设施，是一种立即的非正常中止，即要求两个方向上的传送都立即停止，并释放所占用的缓冲区等资源。

为重置一条连接，连接的一侧要发送一个报文段，将其 CODE 段中的 RST 位

```
      TCP A                                                    TCP B
1. ESTABLISHED                                                   ESTABLISHED
2. (ULP A issues CLOSE)                                          (ULP A issues CLOSE)
   FIN-WAIT-1 → <SEQ=100><ACK=300><CTL=FIN,ACK>  …  FIN-WAIT-1
              ← <SEQ=300><ACK=100><CTL=FIN,ACK>  ←
              … <SEQ=100><ACK=300><CTL=FIN,ACK>  →
3. CLOSING    → <SEQ=101><ACK=301><CTL=ACK>      …      CLOSING
              ← <SEQ=301><ACK=101><CTL=ACK>      ←
              … <SEQ=101><ACK=301><CTL=ACK>      →

4. TIME-WAIT                                                     TIME-WAIT
   (2 MSL)                                                       (2 MSL)
   (CLOSE)                                                       CLOSE
```

图 11.10 两端同时关闭连接的顺序

置 1,以此来启动一次终止过程。这种带 RST 标志的报文段称为重置段。连接的对方收到重置段后,立即使连接非正常中止,以此作为对重置段的响应。

下面先讨论一个 TCP 端发出重置段的几种情形。

(a) 关闭连接时的重置

若出现下面的一种情形:

上层协议用户通过 ABORT 服务请求希望终止已经建立的通信连接

TCP 实体的某一会话连接由于异常超时而无法继续正常通信

这时 TCP 将主动关闭本地连接,并发出一个 RES (重置) 段通知连接的对方 (重置段的序号等于对应会话连接控制块中的 snd_nxt,重置段的确认号应等于对应协议控制块中的 rcv_nxt) 。

(b) 连接不存在时的重置

当本地 TCP 在一条不存在的会话连接上收到某一输入段 (不包括输入的重置段) 时,将用 RES 向对方发出连接不存在的通知,而本地 TCP 仍保持原来状态。这时,如果输入段中含有 ACK 标志,则重置段的序号应等于输入段的确认号,而重置段的确认号应等于 0。否则,重置段的序号应等于 0 而重置段的确认号等于输入段的序号与数据长度之和。

(c) 本地连接处于非同步状态时的重置

当本地 TCP 的某一连接处于非同步状态 (LISTEN,SYN_SENT,SYN_RECEIVED) 时,若出现了情况:

输入段没有确认本地目前在该连接上发送的同步请求 (SYN)

输入段载带的安全性信息与在该连接上指定的安全性信息不匹配

那么本地 TCP 将用重置段响应对方,而本地 TCP 仍保持原来状态——非同步状

态。这时,如果输入段中含有 ACK 标志,则重置段的序号应等于输入段的确认号,而重置段的确认号应等于 0。否则,重置段的序号应等于 0 而重置段的确认号等于输入段的序号与数据长度之和。

(d) 本地连接失去同步状态时的重置

当本地 TCP 的某一连接处于同步状态 (ESTABLISHED,FIN_WAIT_1,FIN_WAIT_2,CLOSE_WAIT,CLOSEING,LAST_ACK 和 TIME_WAIT) 时,如果出现输入段载带的安全性信息与在该连接上指定的安全性信息不匹配,以及再一次收到了落在本地接收窗口内的 SYN 同步请求等现象,本地 TCP 将会用重置段通知对方,异常终止已经失去同步的连接,本地 TCP 将转入 CLOSED 状态。这时,如果输入段中含有 ACK 标志,则重置段的序号应等于输入段的确认号,而重置段的确认号应等于 0。否则,重置段的序号应等于 0 而重置段的确认号等于输入段的序号与数据长度之和。

下面再讨论一个本地 TCP 在某一通信连接上收到从远地传来的重置段时的处理过程。

① 首先对重置段的序号进行合法性检查,看该序号是否落在接收窗口之中。在 SYN_SENT 状态下,本地 TCP 将检查重置段的确认号,看其是否确认了本地发送的同步请求 SYN。

② 重置段的合法性得到保证后,本地 TCP 将按通信连接的当前状态进行处理:

LISTEN——不做任何处理,继续保持 LISTEN 状态;

SYN_RECEIVED 且其先前状态为 LISTEN——使当前状态返回 LISTEN;

其他状态下——将异常终止此会话连接,继而通知本地的上层协议用户。

11.2　TCP 插口编程初步

11.2.1　TCP 的 C/S 模型时序图

根据图 11.1 可以进一步画出图 11.11 所示的面向连接的 C/S 模式的典型时序图。该图说明,服务器必须首先启动,直到它执行 accept() 调用,进入等待状态,方可接收客户机请求。否则,客户机的 connect() 调用将返回出错代码,连接宣告失败。

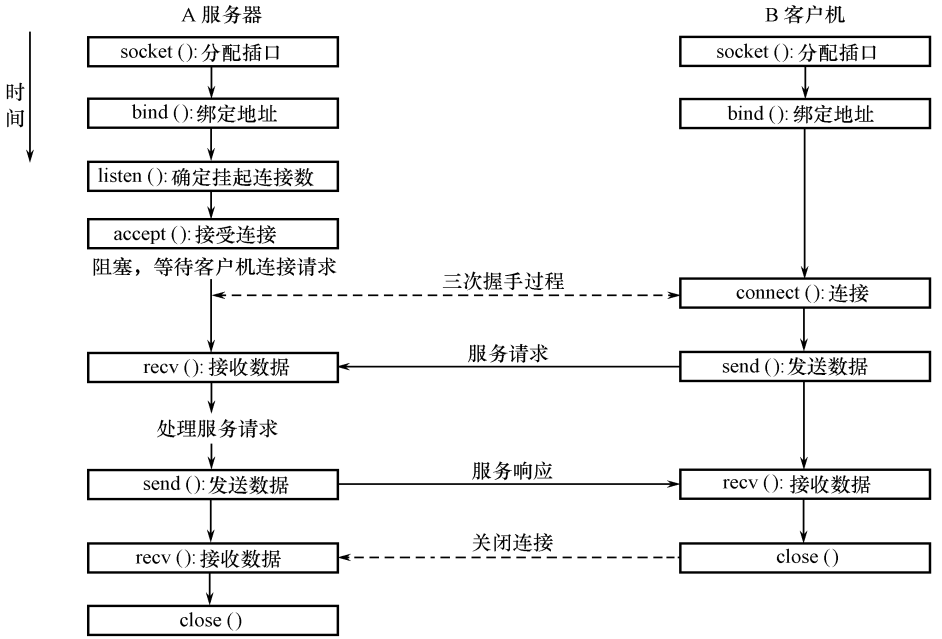


图 11.11 面向连接的 C/S 模式的典型时序图

11.2.2 一个简单的 TCP 网络通信程序

网络通信程序的最大特点就是必须遵循特定的通信协议，或者说，网络通信程序只不过是特定的通信协议的实现。

1. 程序功能

服务器将客户机发送来的数据返回给客户机。

2. 服务器端程序

(1) 服务器主程序

```
/******* 函数名 echo_serv *******/
#include  <string.h>
#include  <sys/types.h>
#include  <sys/socket.h>
#include  <netinet/in.h>
#include  <stdio.h>
#include  <signal.h>
```

```
#define SERVER_PORT 8080
#define BACKLOG 5

int
main()
{
    int       listenfd,connfd;
    struct    sockaddr_in servaddr;

    /*****创建插口*****/
    listenfd=socket(AF_INET,SOCK_STREAM,0);/*创建插口*/
    if(listenfd<0)
     {
        fprintf(stderr, "Socket error\n");
        exit(1);
    }

    bzero(&servaddr,sizeof(servaddr));
    server.sin_family=AF_INET;
    server.sin_addr.s_addr=htonl(INADDR_ANY);
    server.sin_port=htons(SERVER_PORT);

    /*****绑定服务器众所周知插口*****/
    if(bind(listenfd,(struct sockaddr *)&servaddr,
          sizeof(servaddr))<0)
     {
        fprintf(stderr, "Bind error\n");
        exit(1);
    }

    /*****建立长度为5的监听队列,从插口上倾听连接请求*****/
    if(listen(listenfd,BACKLOG) <0)
     {
        fprintf(stderr, "Listen error\n");
        exit(1);
    }
```

```
    for(;;)
     {
        /*****客户机连接建立*****/
        connfd=accept(listenfd,NULL,NULL);
        if(connfd<0)
         {
            fprintf(stderr, Accept error\n);
            exit(1);
        }

        /*****服务器通信*****/
        serv_respon(connfd);/* 通信函数 */
        close(connfd);
    }

    /*****服务器终止*****/
    close(listenfd);
}
```

选择端口号为 8080 的原因：

8080 > 1024,即不属于系统保留端口号,否则要使用超级用户权限；

8080 > 5000,即不属于系统自由端口号；

8080 不属于已经注册的端口号,否则 bind()调用会失败。

(2) 通信函数 serv_respon()

```
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>
#include <signal.h>

#define SERVER_PORT 8080
#define BACKLOG 5

void
serv_respon(int sockfd)
 {
```

```
    int     nbytes;
    char    buf[1024];

    for(;;)
    {
        /* 读取客户机请求,保存在 buf 中,返回数据内容大小 */
        nbytes=read_requ(sockfd,buf,1024);
        if(nbytes==0)     /* 无内容,表示客户机已经关闭了连接 */
            return;
        else if(nbytes<0)  /* 读错误 */
        {
            fprintf(stderr, "read error:%s\n",strerror(errno));
            return;
        }

        /* 将数据返回给客户机 */
        if(write_all(sockfd,buf,nbytes)<0)
            fprintf(stderr, "write error;%s\n",strerror(errno));
    }
}
```

(3) 函数 read_requ 的定义

```
int
read_requ(int sockfd,char * buf,int size)
{
    char        inbuf[256];
    int         n;
    int         i;

    /* 处理客户机请求首部(占 1 行)并检查 */
    i=read_line(sockfd,inbuf,256);
    /* 从插口上读取客户机请求首部 */
    if(i<0)
        return(1);
    else if(i==0)
        return(0);
    if(strncmp(inbuf, "Length",6)==0)
```

```
                                    /* 域内容为 Length ,则读取数据内容长度 */
            sscanf(buf+6, %d ,&n) ;
        else
         {
            sprintf(buf, Invalid option ,14) ;
                                    /* 域内容不为 Length */
            return(14) ;
        }
        return(read_all(sockfd,buf,n)) ;
                                    /* 按首部指定数据内容长度读取客户机数据
        */
    }
```

(4) 函数 read_line() 的定义

每次读 1 个字符(用函数 get_char() 从内核读取数据),读到回车符(\n)为止,就是读了 1 行。

```
# include <errno.h>
# define  MAX_BUFF_SIZE     1024
int
get_char(int fd,char *ch)
 {
    static int          offset=0;
    static int          zero=0;
    static char         buf[MAX_BUF_SIZE] ;

    for(;size <=0 || offset==size;)
     {
        size=read(fd,buf,MAX_BUF_SIZE) ;
                    /* 从内核读取 1 块数据 */
        if(size<0)
         {
            if(errno== EINTR)
             {
                size=0;
                continue;
            } else
                return(-1) ;
```

```
        }
        offset=0;
    }
    * ch=buf[offset++];              /* 从缓冲区返回一个字符*/
    return(1);
}

int
read_line(int fd,char *buf,int maxlen)
{
    int             i,n;
    char            ch;

    for(i=0;i<maxlen)
    {
        n=get_char(fd,&ch)
        if(n==1)
        {
            buf[i++]=ch;
            if(ch==  \n )
                break;
        }
        else if(n<0)
            return(-1);
        else
            break;
    }
    buf[i]=  \0  ;
    return(i);
}
```

3. 客户机端程序

(1) 客户机端主程序

```
/******函数名:echo_cli******/
#include  <sys/socket.h>
#include  <netinet/in.h>
#include  <stdio.h>
```

```
#define SERVER_PORT 8080
void
main(int argc,char * *argv)
{
    int        sockfd;
    struct     sockaddr_in servaddr;        /* 存服务器 IP 地址 */

    if(argc !=2)
    {
        fprintf(stderr, usage:client <IPaddress>\n);
        exit(1);
    }

    /***** 创建插口,并测试 *****/
    sockfd=socket(AF_INET,SOCK_STREAM,0);
    if(sockfd<0)
    {
        fprintf(stderr, Socket error\n);
        exit(1;
    }

    /***** 连接服务器 *****/
    bzero(&servaddr,sizeof(servaddr));    /* 初始化服务器地址结构 */
    servaddr.sin_family=AF_INET;      /* 协议地址类型 */
    servaddr.sin_port=htons(server_port);
                                          /* 将端口号转换为网络字节顺序 */
    if(inet_aton(argv[1],&servaddr.sin_addr)==0)
    {
        fprintf(stderr, Inet_aton error\n);
        exit(1);
    }

    if(connect(sockfd,(struct sockaddr *) &servaddr,
            sizeof(servaddr))<0)
    {
        fprintf(stderr, connect error %s\n ,strerror(errno));
```

```
        exit(1);
    }

    /*****与服务器通信*****/
    cli_requ(sockfd);

    /*****客户机终止*****/
    close(sockfd);
}
```

(2) 客户机通信函数

构造一个无限循环：

先从标准输入读取 1 行数据；

根据该行数据生成请求首部；

将这个首部和读取的数据发送到服务器；

等待服务器的返回数据；

读取到返回数据后，将之显示在标准输出上；

当服务器关闭连接（对插口的读操作不返回数据——返回 0 字节数据）时，cli_requ 返回。

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>

#define SERVER_PORT 8080

void
cli_requ(int sockfd)
{
    char    intbuf[256],outbuf[256];
    int     n;

    for(;;)
    {
        if(gets(intbuf) == NULL) return;
                                        /* 读取 1 行字符串到 intbuf 中 */
        n=strlen(intbuf);
        sprintf(outbuf, "Length %d\n",n);
```

```
            write_all(sockfd,outbuf,strlen(outbuf));
            write_all(sockfd,intbuf,n);
            n=read_all(sockfd,intbuf,n);
            if(n==0) return;
            else if(n<0)
             {
                sprintf(strerr, "read_line error",n);
                return;
            }
            write(1,buf,n);
        }
    }
```

4. 程序的执行

(1) 启动服务器(后台方式)

```
bash & echo-serv&
[1] 283
```

由于客户机程序还没有启动,服务器程序启动后,调用 socket()、bind()、listen()到 accept()处阻塞,等待接收连接。

(2) 查看服务器倾听插口状态(使用命令 netstat)

```
bash $ netstat -a |grep 8080
tcp        0        0        * 8080        * :*        LISTEN
```

用 netstat 查看插口状态必须指定"-a"选项。

本地插口地址为* 8080,远程插口地址为* :*(任何网络设备接口)。

(3) 启动客户机程序

```
bash $ echo-cli 127.0.0.1
```

127.*.*.*代表本机器。

客户机启动后,调用 socket()和 connect()。connect()启动 TCP 的 3 次握手协议,与服务器建立连接。

现在,有了 3 个插口:服务器的倾听插口和连接插口,客户机的连接插口。用 netstat 命令可以查看它们。

注意:不同的系统中,netstat 命令的格式不完全相同。

(4) 向客户机输入数据

```
bash $ echo-cli 127.0.0.1
Hello China! (ctrl+D)
Hello China! (服务器回显)
```

“ctrl+D”是文件结束符。

11.3　TCP 输入/输出模式

11.3.1　阻塞模式下的输入/输出与超时控制

1. 阻塞式输入/输出模式概述

在默认情形下,所有的插口都是阻塞式的。下面介绍在 TCP 插口上产生阻塞的 4 种情形。

(1) 阻塞式读操作

如果插口的接收缓冲区中已经没有数据可读,读函数(read()、readv()、recv()、recvfrom()和 recvmsg())将阻塞,进程进入休眠状态。

TCP 协议以字节为单位进行数据的发送和接收,只要接收缓冲区中有数据出现,进程将被唤醒,读函数的工作是将接收缓冲区中的数据拷贝到进程的缓冲区中。在数据拷贝结束之后,读函数成功返回,进程继续执行。

(2) 阻塞式写操作

如果插口的发送缓冲区空间小于要写的数据量时,写函数(write()、writev()、send()、sendto()和 sendmsg())将不进行任何拷贝操作,进程将阻塞,进入休眠状态。一旦发送缓冲区有足够的空间,内核将唤醒进程,将数据从用户缓冲区中拷贝到插口的发送缓冲区空闲部分,然后立即成功地返回,并不等待 TCP 协议将这些数据成功发送。当对方 TCP 协议返回数据确认时,便会释放一部分发送缓冲区。

(3) 阻塞式接收连接

TCP 协议使用倾听插口接收客户机的连接请求 SYN,完成 3 次握手操作的 TCP 连接被保存在倾听插口的完成连接队列中。如果调用 accept()时完成队列为空,进程即被阻塞,直到完成队列中出现新的 TCP 连接。接收成功,accept()将返回该新连接插口的描述符。

(4) 阻塞式连接

当 TCP 的客户机插口调用 connect()与服务器建立连接时,进程至少需要阻

塞一个往返时间 (RTT) 。

2. 阻塞模式的优点

阻塞式输入/输出有如下一些优点：

结构简单；

通信双方比较容易保持同步；

编程逻辑清晰,程序员不用考虑太多的情况。函数成功返回,则进程继续;函数错误返回,则检查错误类型,进行错误处理。

3. 阻塞模式的缺点

(1) 在读操作时进程可能永远阻塞

在读操作时,由于没有数据可读而产生阻塞后,假如对方主机崩溃,进程将无法接收到任何数据,将会产生永远阻塞。其他操作一般不会永远阻塞,但可能阻塞较长的时间。

(2) 进程效率比较低

当进程阻塞在一个读进程时,必须等待读操作的返回,等待过程中不能进行其他操作。如果一个进程同时从多个插口读数据,只能串行地进行:首先读第一个插口,进程阻塞,等待该插口的数据到达。在此过程中,即使其他插口有数据到达,进程也不能被唤醒,只能等到第一个插口数据到达后被唤醒。

防止进程阻塞时间过长的一种措施是引入超时控制。下面介绍两种超时控制方法。

4. 用函数 alarm() 实现超时控制

(1) 系统调用函数 alarm()

系统调用函数 alarm() 的原型：

```
#include <unistd.h>
unsigned int alarm(unsigned int seconds);
```

返回:非 0——上次定时调用到发送之间剩余的时间 0——上次没有定时

功能：

为调用进程在指定的 seconds 秒后发送一个 SIGALRM 信号；

如果指定 seconds 为 0,则不再发送 SIGALRM 信号；

后一次设定取消前一次设定；

只设定发送一次信号,要多次发送,就要对 alarm() 调用多次。

(2) 信号 SIGALARM 及其处理函数

UNIX 为进程在正常执行中可能发生的各种软件和硬件状态定义了一组信

号。这些信号好像是中断源,每一个信号都有一个相对应的动作。当进程接收到一个信号之后,将调用相应的动作。其中的信号 SIGALARM 是在定时器时间到时发生,默认的调用动作是终止进程。

如果一个进程在睡眠中可以中断优先,进程捕获的信号将中断进程的睡眠。信号捕获通过设置信号的处理函数 sigaction() 进行。这个处理函数的原型如下:

```
#include <signal.h>
int sigaction(int signum, const struct sigaction * act,
struct sigaction * oldact) ;
返回 0——成功, -1——失败
```

参数:

signum——指定信号;

act——指定新动作;

oldact——存储旧动作,以便以后恢复。

结构 sigaction 的定义:

```
sturct sigaction
 {
    void (* sa_handler)(int) ;      /* 指定信号的动作 */
    sigset_t  sa_mask;              /* 指定被屏蔽的信号 */
    int             sa_flags;       /* 设置一些标志 */
    void (* sa_restorer)(void) ;    /* 已经过时 */
}
```

(3) 源代码片段

```
#include <signal.h>
…
int      to_flag;                   /* 全局变量:标志 */

int
main()
 {
     int              sockfd;
     struct sigaction act;
     int              to=30;
     …
     act.sa_handler=sigalrm_handler; /* 指定信号的动作 */
```

```
        act.sa_mask=0;                  /* 不屏蔽信号 */
        act.sa_flags=0;
        sigaction(SIGALRM,&act,NULL);   /* 设置信号处理函数 */
                                        /* 捕获:SIGALRM */
        ...
        for(;)
         {
            to_flag=0;
            alarm(to);                  /* 30 秒时发送信号 SIGALRM */

            n=read(sockfd,buf,sizeof(buf));
            /* to 时间内读到数据:read()返回 0;
             * 阻塞:read 返回-1,错误代码为 EINTR;
             * 超时:SIGALRM 中断 read,调用 sigalrm_handler 置 to_flag 为 1 */
            alarm();                    /* 不再发送 SIGALRM 信号 */

            if(n<0 && errno==EINTR)     /* 读失败 */
             {
                if(to_flag==1)          /* 超时 */
                   proc.timeout();      /* 超时处理 */
                else  continue;         /* 非超时读失败,继续循环 */
            }
            else
             {
                ...
            }
        }
    }

    void
    sigalrm_handler(int sign)
     {
        to_flag=1;
    }
```

5. 在插口上设置选项 SO_RCVTIMEO 和 SO_SNDTIMEO

(1) 函数 getsockopt()和 setsockopt()

函数 getsockopt() 和 setsockopt() 只作用于插口。getsockopt() 用于获得当前插口选项，setsockopt() 用于设置当前插口选项。其原型为：

```
#include <sys/socket.h>
int getsockopt(int sockfd, int level, int optname,
void * optval, sochken_t * optlen);
int setsockopt(int sockfd, int level, int optname,
const void * optval, sochken_t * optlen);
```

参数：

(a) sockfd——一个打开的插口描述符。

(b) level——指定选项层次：

SOL_SOCKET 为通用插口；

IPPROTO_IP 为 IP 选项；

IPPROTO_TCP 为 TCP 选项。

(c) optname——选项的名称。

(d) optval——存储选项的值。

(e) optlen——存放参数 optval 指向的选项的长度：

在 getsockopt() 中，返回参数 optval 指向的选项的长度；

在 setsockopt() 中，指定参数 optval 指向的选项的长度。

当 level 选定 SOL_SOCKET（通用插口）时，optname（选项的名称）可以有 16 种，其中：

SO_RCVTIMEO——接收超时选项；

SO_SNDTIMEO——发送超时选项。

两个选项的值是指向 timeval 结构体的指针。这个结构体定义在 <sys/time.h> 中。如果超时值设置为 0 s 或 0 ms，超时被禁止。设置了这两个选项，在当前插口上的所有读、写操作都可以保证超时返回。当读或写超时后，函数 getsockopt() 或 setsockopt() 将返回错误，错误类型为 ETIMEOUT。通过检查这个错误类型，可以进行超时处理。

(2) 例程

```
/* 以下代码监听到来的 TCP 连接，使用 setsocket 在读操作上设置了一个超时选项
*/
#include <sys/param.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/time.h>
```

```
#include <netinet/in.h>
#include <arpa/inet.h>

#include <err.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main (int argc,char *argv[]) {
  char buffer[1024] ;
  int listening_fd,
      connection_fd,
      port,
      size ;
  struct sockaddr_in local_address,
                     remote_address ;
  struct timeval tv ;
  if (argc != 2) {
      fprintf (stderr, usage : %s port\n ,getprogname()) ;
      exit (1) ;
  } else {
      port= atoi (argv[1]) ;
  }

  /* 创建插口 */
  if ((listening_fd=socket (PF_INET,SOCK_STREAM,0)) == -1)
   {
      err (1, socket ) ;
  }

  /* 即使在老的客户连接仍处于挂起状态的情况下,也允许我们再次绑定 */
  size=1 ;/* 1 秒 */
  if (setsockopt (listening_fd,SOL_SOCKET,SO_REUSEADDR,&size,sizeof (size)) !=
    0)
   {
```

```
    err(1, setsockopt);
}

bzero(&local_address,sizeof(local_address));
local_address.sin_family=AF_INET;
local_address.sin_addr.s_addr=htonl(INADDR_ANY);
local_address.sin_port=htons(port);

/* 将插口绑定到地址 */
if(bind(listening_fd,(struct sockaddr *) &local_address, sizeof(local_address)) ==
  -1)
 {
    err(1, bind);
}

/* 使其成为一个监听端口 */
if(listen(listening_fd,-1) !=0)
 {
    err(1, listen);
}

/* 接受连接 */
for(;)
 {
    size=sizeof(remote_address);
    if((connection_fd=accept(listening_fd,
        (struct sockaddr *) &remote_address,&size)) == -1)
     {
        err(1, accept);
    }

    printf("connected...\n");

    /* 建立超时 */
    tv.tv_sec=15L;
    tv.tv_usec=0L;
    if(setsockopt(connection_fd,SOL_SOCKET,SO_RCVTIMEO,&tv,sizeof(tv)) !=
```

```
    0)
  {
      err(1, setsockopt);
}
if(setsockopt(connection_fd,SOL_SOCKET,SO_SNDTIMEO,&tv,sizeof(tv)) !=
    0)
  {
      err(1, setsockopt);
}

/* 做一些读和写的工作 */
#define STRING hello\n
if(write(connection_fd,STRING,sizeof(STRING)) == -1)
  {
      err(1, write);
}

#undef STRING

if(read(connection_fd,buffer,sizeof(buffer)) == -1)
  {
      warn(read);
      goto failure;
} else {
    printf("received %s",buffer);
}

#define STRING goodbye\n
if(write(connection_fd,STRING,sizeof(STRING)) == -1)
  {
      err(1, write);
}
#undef STRING

failure:
if(close(connection_fd) == -1)
  {
```

```
            err(1, close);
        }

        printf("disconnected...\n");
    }

    return 0;
}
```

采用这种方法,服务器可以处理长时间没有响应的客户机,为了安全,服务器(如 TELNET 服务器)应当将与它们的连接断开。

但是,这种方法只适合读、写函数,对接收函数 accept()和 connect()无效。同时,需要事先估计可能的超时时间,如果超时时间设置得过小,则可能提前中断。

11.3.2　非阻塞模式下的输入/输出模式

1. 非阻塞输入/输出模式的概念

当一个插口被设置为非阻塞插口后,在该插口上进行输入/输出操作失败时,并不进入阻塞状态,而是让输入/输出函数返回错误,错误的类型为 EWOULDBLOCK。通过轮询式(polling)检查,当发现 EWOULDBLOCK 错误时,即忽略之,直到输入/输出操作成功为止。

下面是一段轮询程序:

```
for(;;)
 {
  if(read(sockfd,buf,nbytes) <0)
   {
       if(errno==EWOULDBLOCK)
           continue;
       else
        {
           fprintf(stderr, "read error:%s\n",strerror(errno));
       }
  } else
       break;
}
```

2. 非阻塞输入/输出模式的设置方法

新创建的插口在默认情形下是阻塞插口。为了实现非阻塞输入/输出,应当先把插口设置成为非阻塞式插口。方法有下面两种。

(1) 函数 fcntl()

函数 fcntl() 是 Posix 标准中使用的方法,其操作为:

```
int flag;
flag=fcntl(sockfd,F_GETFL,0);
fcntl(sockfd,F_SETFL,flag|O_NONBLOCK);
```

控制 flag,就可以控制将插口设置为 O_NONBLOCK。

(2) 函数 ioctl()

函数 ioctl() 的 FIONBIO 操作,可以设置一个插口为非阻塞插口,其操作为:

```
int on=1;
ioctl(sockfd,FIONBIO,&ON);
```

非阻塞输入/输出模式可以保证进程永远不会阻塞,并且进程可以同时处理多个描述符的输入/输出操作。它的缺点是:

编程方法比较复杂,不符合普通的编程习惯;

占用 CPU 时间较长,即使在插口上没有数据可读,仍然要不停地执行,而阻塞方式在插口上没有数据可读时就会放弃 CPU。

3. 例程

```
#include common.h

char buff[MAXBUFSIZE];                    /* 发送和接收缓存 */
int send_size[5] = {10,100,1000,10000,100000};/* 发送数据大小 */

void
main(int argc,char ** argv)
{
  int                 sockfd=0;       /* 插口描述符 */
  struct sockaddr_in  serveraddr;     /* 服务器地址 */
  int                 mode=0;         /* 阻塞模式 */
  int                 rcount=0,scount=0;/* 接收次数,发送次数 */
  int                 slen,rlen,len;  /* 发送和接收长度变量 */
  int                 rdt;            /* 记录 RoundTrip 时间 */
```

```
int                     i,j;            /* 循环变量 */
struct timeval          tv1,tv2;        /* 时间结构变量 */

if(argc<2)
 {
    printf("使用方法:tcp_client<server_IP_addlerss\n");
    exit(0);
}

for(;;)  /* 循环等待连接成功 */
 {
    /* 产生 TCP 插口 */
    if((sockfd=socket(AF_INET,SOK_STREAM,0))<0)
     {
        printf("产生插口失败,退出\n");
        exit(1);
    }

    /* 填写服务器地址结构 */
    bzero((char*)&serveraddr,sizeof(struct sockaddr_in));
    serveraddr.sin_family=Af_INET;
    serveraddr.sin_port=htons(SERVER_PORT);
    inet_aton(argv[1],&serveraddr.sin_addr);

    /* 发起连接请求 */
    if(connect(sockfd,&serveraddr,sizeof(serveraddr))<0)
     {
        printf("连接请求失败:errno=%d\n",errno);
        close(sockfd);/* 重建连接,必须重新产生新插口,关闭原插口 */
    }
    else break;
}

ioctl(sockfd,FIONBIO,&MODE);        /* 将插口置成非阻塞模式 */

for(i=0;i<5;i++)  /* 测试不同大小的报文 RoundTrip 时间 */
 {
```

```
slen=send_size[i];                    /* 本次测试数据大小 */
scount=rcount=0;

gettimeofday(&tv1,(struct timezone *)0);
for(j=0;j<50;j++)   /* 每一种大小测试 50 次 */
{
    len=slen;
    for(;)
    {
        if((rlen=send(sockfd,buff,len,0))<=0)
        {
            if((rlen<0) && (rlen==EWOULDBLOCK)) continue;
            printf("接收失败,程序退出,errno=%d\n",errno);
            close(sockfd);
            exit(1);
        }/* if */
        else
        {
            len=len-rlen;
            scount++;
            if (len==0) break;  /* 发送完成 */
        }/* else */
    }/* for(;) */
    len=slen;
    for(;)                            /* 接收数据 */
    {
        rlen=recv(sockfd,buff,len,0);
        if(rlen<=0)
        {
            if ((rlen<0) && (errno==EWOULDBLOCK)) continue;
            printf("接收失败,程序退出,errno=%d\n",errno);
            close(sockfd);
            exit(2);
        }
        len=len-rlen;
        rcount++;                     /* 接收次数加 1 */
        if (len==0) break;/* 本次接收完毕,进行下一次发送 */
```

```
            }
        }
        gettimeofday(&tv2, (struct timezone *)0);
        rdt=((tv2.tv_sec-tv1.tv_sec) * 1000000+
        tv2.tv_usec-tv1.tv_usec)/50;
        printf("报文大小=%d,RoundTripTime=%d 微秒,发送次数=%d,
                接收次数=%d\n",slen,rdt,scount,rcount);
        sleep(3)
    }
    close(sockfd);
}
```

11.3.3　TCP 输入/输出多路复用

输入/输出多路复用是一种特殊的阻塞式输入/输出模式,其特点是:用 select()指示内核监视多个事件(插口描述符有无就绪),当一个或多个事件发生或经过某个指定时间后,就唤醒进程,就绪的插口描述符可以进行相应的输入/输出操作。这种方法综合了阻塞式输入/输出和非阻塞式输入/输出的优点,比阻塞方式效率高,又能在多个插口描述符上进行输入/输出操作,提高了 CPU 的利用率。

1. select()函数原型

select()的功能是监视一组插口的状态——可读、可写或出现带外数据,直到指定插口状态改变或阻塞时间到。函数的原型如下:

```
#include <sys/time.h>
#include <sys/select.h>
int select(int nfds,
fd_set * readfds, fd_set * writefds, fd_set * exceptfds,
struct timeval * timeout);
```

2. select()函数参数

(1) 内核监视的描述符个数的参数:nfds,其值为最大描述符的值加 1(描述符最小值为 0)。

(2) 三个指向被监视描述符集(fd_set * 类型)的指针参数:

指向一组读条件描述符的指针——readfds;

指向一组写条件描述符的指针——writefds;

指向一组异常条件(带外数据)描述符的指针——exceptfds。

这三个参数中的每一个都可以指定一个或多个描述符值。由于这三个参数是指针,所以它们既可以向内核指示要监视的描述符,又可以返回状态变化的描述符。

为了能简单地指定一个或多个描述符,可以用整型数的每一位对应一个描述符;对某一位设置 1 或 0,就可以将该位对应的描述符设置为被监视或不被监视。设置采用如下宏定义进行:

```
void FD_ZERO(fd_set * fdset);  /* 将描述符集 fdset 全关闭 */
void FD_SET(int fd,fd_set * fdset);
                                /* 将指定描述符 fd 对应比特置 1 */
void FD_CLR(int fd,fd_set * fdset);
                                /* 将指定描述符 fd 对应比特置 0 */
```

例如语句:

```
fd_set rset;
FD_ZERO(&rset);          /* 将 rset 初始化——全部关闭 */
FD_SET(2,&rset);         /* 将 rset 中的 1 号描述符打开 */
FD_SET(5,&rset);         /* 将 rset 中的 5 号描述符打开 */
FD_SET(8,&rset);         /* 将 rset 中的 8 号描述符打开 */
```

这些指定工作应当在调用 select()前进行。

但是,整数的位长是有限的。在 32 位的计算机内,int 型整数占用 32 位。为了定义更多的描述符,可以使用整型数组:用其第 1 个元素指定描述符 0～31,用第 2 个元素指定描述符 32～63,……。

select()返回后,在指定的描述符集中检查状态发生变化的描述符,使用的宏定义为:

```
int FD_ISSET(int fd,fd_set * fdset);
                                /* 查看描述符 fd 是否在描述符集中 */
```

readfds、writefds 和 exceptfds 不仅可以监视读、写和带外数据的条件,还可以监视更多的状态变化条件。表 11.2 列出了用 select()可以检查的插口状态变化条件。

表 11.2 用 select()可以检查的插口状态变化条件

条件	可读	可写	异常
有数据可读	是	否	否
关闭连接的读通道	是	否	否

续表

条件	可读	可写	异常
给监听插口准备好新连接	是	否	否
有可用于写的缓存空间	否	是	否
关闭连接的写通道	否	是	否
待处理错误	是	是	否
TCP 带外数据	否	否	是

(3) 参数 timeout 是一个指向结构体类型 timeval 的指针，用于给定一组阻塞等待的指定描述符中任何一个状态发生变化的时间间隔。结构体 timeval 的定义如下：

```
struct timeval {
  long tv_sec;        /* 秒 */
  long tv_usec;       /* 微秒 */
};
```

在调用函数 select() 时，参数 timeout 的设置可分为如下 3 种情形：

将 timeout 置为空指针——等待无时间限制；

置 timeout.tv_sec=0，timeout.tv_usec=0——论询方式，select() 检查完描述符后立即返回，不等待；

timeout.tv_sec 和 timeout.tv_usec 不全为 0——select() 检查完描述符后，在规定的秒数和微秒数内返回。这时，将参数 readfds、writefds 和 exceptfds 置空，select() 也可以作为定时器使用。

3. select() 函数的返回

函数的返回值是描述符集内的描述符数；

当因超时而不是状态改变导致返回时，返回 0；

当发生错误时，返回 -1，错误代码存在 errno 中；

正常返回时，参数 readfds、writefds 和 exceptfds 返回状态变化的描述符；出错时，后 4 个参数均是未定义的。

4. select() 函数使用的基本方法

(1) select() 调用前的操作：

用 FD_ZERO() 初始化描述符集变量；

用 FD_SET() 和 FD_CLR() 设置要监视的描述符。

(2) select() 调用后的操作：

用 FD_ISSET()在指定的描述符集中检查状态发生变化的描述符。

5. 例程片段

程序功能:用 select()测试两个插口是否可写。

```
struct fd_ufds;                /* 声明插口描述符集 */
struct timeval tv;             /* 超时间隔 */
int fd1,fd2,maxfd,status;

…/* 其他代码 */
for(;)
 {
    FD_ZERO(&ufds);            /* 将 ufds 全部关闭 */
    FD_SET(fd1,&ufds);         /* 将 rset 中的 1 号描述符打开 */
    FD_SET(fd2,&ufds);         /* 将 rset 中的 2 号描述符打开 */
    maxfd=((fd1 > fd2) ? (fd1) : (fd2))+1;

    /* 每次调用前均要重新设置超时间隔 */
    tv.tv_sec=0;
    tv.tv_usec=2000;           /* 超时间隔为 2000 微秒 */

    status=select(maxfd,NULL,&ufds,NULL,&tv);

    if(status==0)
     {
        /* 超时处理 */
    }
    else if(status==-1)
     {
        /* 出错处理 */
    }
    else
     {
        if(FD_ISSET(fd1,&ufds))
         {
            /* 可以向 fd1 上发送数据 */
        }
        if(FD_ISSET(fd2,&ufds))
```

```
            {
                /* 可以向 fd2 上发送数据 */
            }
        }
    }
```

11.4　基于 UDP 的插口程序设计

11.4.1　UDP 编程模式

UDP 提供不保证顺序的用户数据报传输服务。在比较简单的应用中，客户机常常只用单个 UDP 报文来发送请求，服务器也用单个报文回送应答。这种情况下，UDP 服务器和客户机间的交互程序采用循环结构是非常有利的。图 11.12 表明了这种处理结构。

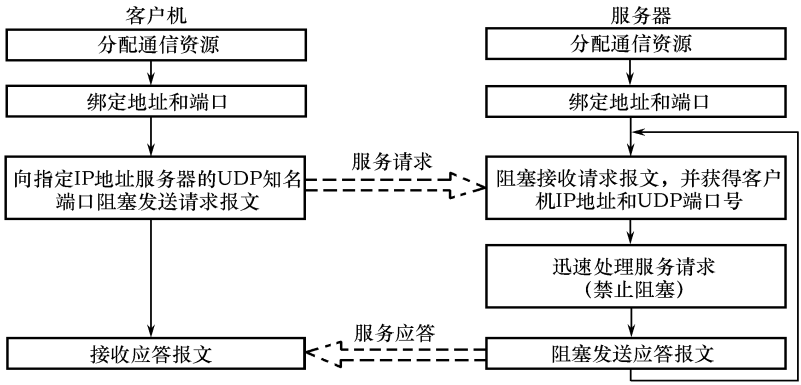


图 11.12　采用循环结构的 UDP 服务器与客户机间的交互

显然，UDP 插口实现循环服务器的方法非常简单：UDP 每次从插口上读取一个客户数据报，然后对它进行处理，接着进入下一循环。

计算机网络中两个节点间要进行 UDP 通信，发送数据的源节点必须知道接收数据的目的节点的 IP 地址和 UDP 端口号。一般说来，服务器具有相对固定的 IP 地址（或域名），并且一直在知名端口守候，因而客户机可以首先向服务器发送请求。对于服务器端来说，在接收 UDP 数据报时功能调用会同时给出发送方的 IP 地址。从通信的角度看，UDP 应用中客户机与服务器的概念不像 TCP 那样清晰：在任何时候，发送数据的一方可以认为是客户机端，接收数据的一方可以

认为是服务器端,而不像 TCP 应用那样,要把发起连接的一方作为客户机端,把接受连接的一方作为服务器端,并且始终保持这种关系。

下面介绍 UDP 客户机/服务器通信过程的基本特征。

(1) UDP 服务器是无连接的,它可以直接使用 UDP 插口接收客户机发送的数据报,而无须在处理客户机请求之前用倾听插口来建立连接,UDP 服务器也不用将 UDP 插口转换为倾听插口。但是在数据传输完成之后需要关闭插口,释放网络资源。

由于 UDP 服务器与任何客户机不需建立连接,所以系统不在插口数据结构中记录接收者的地址信息,服务器只能从每个到达的数据报中获得客户机的地址,并可以用函数 recvfrom() 的参数 from 返回该地址,然后使用这个地址并用 sendto() 向客户机返回数据。

(2) 无连接服务器通过 socket() 和 bind() 建立一个本地三元组。客户机调用 bind() 也建立一个三元组。无连接的两个端点一建立起来,分别用 socket 标识。然后,在发送数据阶段,发送方指定本地和远方的 socket 号,于是一个完整的五元组在数据收发过程中动态建立起来,实现了无连接的客户机和服务器的相互识别。

UDP 要求在通信的两方中,只有一方需要显式地绑定地址和端口。也就是说,在某些情形下,有一方可以不显式地进行指定地址和端口的绑定,而是由内核自动分配。但是,这时要求不绑定地址的一方(客户机端)必须首先向绑定地址的一方(服务器端)发送数据。

(3) UDP 没有为 UDP 插口设置发送缓冲区。当应用程序调用 sendto() 发送数据时,sendto() 便将要发送的数据从用户缓冲区拷贝到系统内核的缓冲区中,然后返回。UDP 协议将这些数据生成一个 UDP 数据报,传送给低层的 IP 协议,就完成了 UDP 数据报的发送。由于 UDP 是不可靠的协议,所以它不保留已经发送的 UDP 数据报的内容。

但是,UDP 插口数据结构中包含有发送缓冲区的大小一项。这项数据只用来限制可以发送的最大 UDP 数据报的大小。

(4) UDP 为每个 UDP 插口设置一个接收缓冲区队列。当一个数据报到达时,UDP 协议根据该数据报的目的端口号将它存放在对应的插口接收缓冲区队列中。

UDP 插口的接收缓冲区有最大限制。由于 UDP 不保证可靠传输,所以当插口的接收缓冲区已满时,就将再接收到的数据报丢弃。

(5) UDP 应用也可以调用 connect() 函数。但是,这时 connect() 函数的功能并不是向对方发送连接请求(启动 3 次连接协议),而只是让内核记录 connect()

调用中使用的 IP 地址和端口号，并立即返回给调用进程。这种情况下，UDP 使用的数据交换函数就不再是 recvfrom（）和 sendto（），而要使用 TCP 应用中常用的 write（）或 send（）、read（）或 recv（）。

(6) UDP 协议是面向记录的协议，没有文件结束符的概念，如果读操作返回 0，仅表示这个记录的长度为 0，并不表示对方已经结束了数据的发送。

11.4.2　一个简单的 UDP 客户机/服务器程序

1. UDP 服务器程序

(1) UDP 服务器主程序

```
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<stdio.h>

#define SERVER_PORT 8080

int
main()
 {
    int                    sockfd;
    struct  sockaddr_in addr;
    sockfd=socket(AF_INET,SOCK_DGRAM,0);
    if(sockfd<0)
     {
        fprintf(stderr, socket error.\n);
        exit(1);
    }
    bzero(&addr,sizeof(addr));
    addr.sin_family=AF_INET;
    addr.sin_addr.s_addr=htonl(INADDR_ANY);
    addr.sin_port=htons(SERV_PORT);
    if(bind(sockfd, (struct sockaddr *) &addr,sizeof(addr))<0)
     {
        fprintf(stderr, bind error.\n);
        exit(1);
```

```
    }
    udps_respon(sockfd);
    close(sockfd);
}
```

(2) UDP 服务器的通信程序

```
#define MAX_MSG_SIZE        1024

void
udps_respon(int sockfd)
{
  struct sockaddr_in        addr;
  int                       addrlen;
  char                      msg[MAX_MSG_SIZE];
  for(;;)
  {
      n=recvfrom(sockfd,msg,MAX_MSG_SIZE,0,
                 (struct sockaddr *)&addr,&addrlen);
      sendto(sockfd,msg,n,0,addr,addrlen);
  }
```

2. UDP 客户机程序

(1) UDP 客户机主程序

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>

int
main(int argc,char **argv)
{
  int                       sockfd;
  struct sockadd_in         addr;

  if(argc != 3)
  {
      fprintf(stderr, usage:client ipaddr port\n);
```

```
        exit(1);
    }

    sockfd=socket(AF_INET,SOCK_DGRAM,0);
    if(sockfd<0)
     {
        fprintf(stderr, socket error.\n);
        exit(1);
    }

    bzero(&addr,sizeof(addr));
    addr.sin_family=AF_INET;
    addr.sin_port=htons(atoi(argv[2]));
    if(inet_aton(argv[1],&addr.sin_addr)<0)
     {
        fprintf(stderr, inet_aton error.\n);
        exit(1);
    }

    udpc_requ(sockfd,&addr,sizeof(addr));
    close(sockfd);
}
```

(2) UDP 客户机通信函数

```
#define MAX_BUF_SIZE 1024

void
udpc_requ(int sockfd,const struct sockaddr_in *addr,int len)
 {
    char buf[MAX_BUF_SIZE];
    int n;
    for(;fgets(buf,MAX_BUF_SIZE,stdin)!=NULL;)
     {
        sendto(sockfd,buf,strlen(buf),0,addr,len);
        n=recvfrom(sockfd,buf,MAX_BUF_SIZE,0,NULL,NULL);
        buf[n]=0;
        fputs(buf,stdout);
```

```
    }
}
```

11.4.3 非阻塞模式下的 UDP 客户机/服务器程序

与 TCP 相比,UDP 的输入/输出操作有如下特点:

(1) UDP 插口没有实际的发送缓冲区,进行写操作时,内核将数据从用户缓冲区拷贝到系统缓冲区,然后将这些数据生成 UDP 数据报传递给 IP 协议发送。所以,UDP 协议不会出现发送缓冲区满的情况,在 UDP 插口上执行写操作时不会出现阻塞。

(2) UDP 是非连接的协议,不需要调用 accept(),虽然可以在 UDP 插口上调用 connect(),但仅仅是用 connect()记录对方地址,因此在 UDP 插口上不会发生调用 accept()和调用 connect()造成的阻塞。

(3) 在 UDP 插口上仅可能出现读操作阻塞。插口的接收缓冲区中没有数据可读时,读函数将阻塞;当一个完整的 UDP 数据报到达时,进程将被唤醒。

非阻塞模式 UDP 服务器例程:

```
/* UDP echo server 非阻塞模式*/
#include <stdio.h>        /* for printf() and fprintf() */
#include <sys/socket.h>   /* for socket(),bind,and connect() */
#include <arpa/inet.h>    /* for sockaddr_in and inet_ntoa() */
#include <stdlib.h>       /* for atoi() and exit() */
#include <string.h>       /* for memset() */
#include <unistd.h>       /* for close() and getpid() */
#include <fcntl.h>        /* for fcntl() */
#include <sys/file.h>     /* for O_NONBLOCK and FASYNC */
#include <signal.h>       /* for signal() and SIGALRM */
#include <errno.h>        /* for errno */

#define ECHOMAX 255                          /* 回显的最长字符串 */

void DieWithError(char *errorMessage);       /* 错误处理函数*/
void UseIdleTime();                          /* 使用空闲时间的函数 */
void SIGIOHandler(int signalType);           /* 处理 SIGIO 的函数 */

int sock;                    /* Socket—信号处理器的全局变量 */
```

```
int main (int argc,char * argv[])
 {
    struct sockaddr_in echoServAddr;            /* 服务器地址 */
    unsigned short echoServPort;                /* 服务器端口 */
    struct sigaction handler;                   /* 信号处理动作定义 */

    /* 测试参数数目的正确性 */
    if (argc != 2)
     {
        fprintf (stderr, Usage:%s <SERVER PORT>\n ,argv[0]);
        exit(1) ;
    }

    echoServPort=atoi (argv[1]);                /* 第一个参数:本地端口 */

    /* 创建用于发送和接收数据报的插口 */
    if ((sock=socket(PF_INET,SOCK_DGRAM,IPPROTO_UDP)) <0)
      DieWithError( socket() failed );

    /* 设置服务器地址结构 */
    memset (&echoServAddr,0,sizeof (echoServAddr)) /* 将结构清零 */
    echoServAddr.sin_family= AF_INET; /* Internet family */
    echoServAddr.sin_addr.s_addr=htonl(INADDR_ANY) ;
    /* 任何到来的连接 */
    echoServAddr.sin_port=htons (echoServPort); /* 端口 */
    /* 绑定本地地址 */
    if (bind (sock, (struct sockaddr *) &echoServAddr,
                sizeof (echoServAddr)) <0)
      DieWithError( bind() failed );

    /* 为 SIGIO 设置信号处理器 */
    handler.sa_handler=SIGIOHandler;
    /* 创建屏蔽所有信号的掩码 */
    if (sigfillset (&handler.sa_mask) <0)
      DieWithError( sigfillset() failed );
    /* 没有标记 */
    handler.sa_flags=0;
```

```
    if (sigaction (SIGIO,&handler,0) <0)
      DieWithError ( sigaction() failed for SIGIO ) ;

    /* 必须让 socket 接收 SIGIO 消息 */
    if (fcntl (sock,F_SETOWN,getpid()) <0)
      DieWithError ( Unable to set process owner to us ) ;

    /* 安排非阻塞的 I/O 和 SIGIO 递交 */
    if (fcntl (sock,F_SETFL,O_NONBLOCK | FASYNC) <0)
      DieWithError ( Unable to put client sock into non-blocking/async mode ) ;

    /* 做实际的工作;在后台回显 */

    for (;)
      UseIdleTime() ;
    /* 程序流不到这里 */
}

void UseIdleTime()
 {
    printf ( .\n ) ;
    sleep (3) ;        /* 睡眠 3 秒 */
}

void SIGIOHandler (int signalType)
 {
    struct sockaddr_in echoClntAddr ;            /* 数据报源地址 */
    unsigned int clntLen ;                       /* 地址长度 */
    int recvMsgSize ;                            /* 数据报长度 */
    char echoBuffer [ECHOMAX] ;                  /* 数据报缓冲区 */

    do /* 只要有输入... */
     {
        /* 设置输入/输出参数的大小 */
        clntLen = sizeof (echoClntAddr) ;

        if ((recvMsgSize = recvfrom (sock ,echoBuffer, ECHOMAX,0,
```

```
                (struct sockaddr *) &echoClntAddr,&clntLen)) < 0)
            {
                /* 惟一可接收的错误：recvfrom() 将会阻塞 */
                if (errno != EWOULDBLOCK)
                    DieWithError( recvfrom() failed );
            }
            else
            {
                printf( Handling client %s\n ,
                        inet_ntoa(echoClntAddr.sin_addr));

                if (sendto(sock,echoBuffer,recvMsgSize,0,
                    (struct sockaddr *) &echoClntAddr,
                    sizeof(echoClntAddr)) != recvMsgSize)
                    DieWithError( sendto() failed );
            }
        } while (recvMsgSize >= 0) ;
        /* 接收完毕 */
    }
```

习　　题

11.1　试写出将一个插口连接到网址为 xxx.yyy.edu 的 TELNET 服务器所需的代码。

11.2　试写出将一个插口连接到网址为 xxx.yyy.edu 的 FTP 服务器所需的代码。

11.3　试写出将一个插口绑定到本地服务器所需的代码。

11.4　试写出创建一个具有队列大小为 150 的倾听插口所需的代码。

11.5　要将一个字符串从客户机发送到服务器，请写出相应的 TCP 客户机和服务器程序。

11.6　要将一个字符串从客户机发送到服务器，请写出相应的 UDP 客户机和服务器程序。

11.7　从客户机输入“Hello,Server”传送到服务器端，变成“Hello,Client”送回客户机端。

附录　名词术语英汉对照表

AAL (ATM Adaptation Layer)	ATM 适配层
ABM (Asynchronous Balance Mode)	异步平衡方式
ABR (Area Border Router)	区域边界路由器
ACK (Acknowledgment)	确认
ACKN (Acknowledgment Number)	确认号
ADSL (Asymmetric Digital Subscriber Line)	非对称数字用户线
AG (Application Gate)	应用层网关
API (Application Program Interface)	应用程序接口
ARM (Asynchronous Response Mode)	异步响应模式
ARP (Address Resolution Protocol)	地址解析协议
ARPA (Advanced Research Projects Agency)	美国国防部高级研究计划署
ARPANet (Advanced Research Projects Agency Network)	ARPA 网
ARQ (Automatic Request for Repeat)	自动重复请求
AS (Autonomous System)	自治系统
ASIC (Application－Specific Intergrated Circuit)	专用集成电路
ASBR (AS Border Router)	自治系统边界路由器
ASCII (American Standard Code for Information Interchange)	美国信息交换标准代码
ASK (Amplitude－Shift Keying)	幅移键控
ASN (Abstract Syntax Notation)	抽象语法
ATM (Asynchronous Transfer Mode)	异步传输模式
AU (Arrangement Unit)	管理单元
AUI (Attachment Unit Interface)	连接单元接口
BaseBand	基带
BGP (Border Gateway Protocol)	边界网关协议
BroadBand	宽带
BS (Basis Station)	无线基地站
BSC (Binary Synchronous Communication)	二进制同步通信（协议）
BSC 或 Bisync	一种代表性的面向字符的同步传输规程
BSS (Basic Service Set)	基本服务集
BUS (Broadcast and Unknown Server)	广播未知服务器

CCITT (Consultative Committee for International Telegraph and Telephone)	国际电报电话咨询委员会
CD (Carrier Detect)	载波检测
CD (Collision Detection)	冲突检测
CDMA (Code Division Multiple Access)	码分多址
CPU (Center Process Unit)	中央处理器
CRC (Cyclic Redundancy Check)	循环冗余校验
C/S (Customer/Server)	客户机/服务器
CSMA (Carrier Sense Multiple Access)	载波侦听多路访问
CSMA/CD (Carrier Sense Multiple Access Collision Detection)	载波侦听多路访问/冲突检测
Datagram	数据报
DCC (Data Cross Connect)	数字交叉连接 (设备)
DCE (Data Terminal Equipment)	数据线路终接设备
DD (Delay Distortion)	延迟失真
DDN (Dotted Decimal Notation)	点分十进制计数法
DEST (Destination Address)	目的地址
DFWMAC (Distrubuted Foundation Wireless Medium Access Control)	分布式基础无线网 MAC
DIX (Digital Intel Xerox)	一种以三个公司名命名的以太网标准
DL (Data Link)	数据链路
DLL (Data Link Layer)	数据链路层
DM (Different Manchester)	差分曼彻斯特码
DNS (Domain Name System)	域名服务系统
DRR (Data Redundancy Reduction)	冗余数据简化
DS (Datagram Socket)	数据报插口
DT (Display Terminal)	显示终端
DTE (Data Terminal Equipment)	数据终端设备
EBCDIC (Extended Binary Coded Decimal Interchange Code)	扩充二－十进制交换码
EGP (Exterior Gateway Protocol)	外部网关协议
E－Mail	电子邮件
Ethernet	以太网
Fast－Forward	快速转发
FC	美国信息技术安全评价联邦准则
FCS (Frame Check Sequence)	帧校验序列
FDM (Frequency Division Multiplexing)	频分多路复用

FEP (Front-End Processor)	前端机
Fire Wall	防火墙
FR (Frame Relay)	帧中继
Frame	帧
FSK (Frequency-Shift Keying)	频移键控
FTP (File Translation Protocol)	文件传输协议
HDLC (High-Level Data Link Control)	高级数据链路控制 (协议)
Hot Potato	热土豆 (固定路由) 算法
HTML (Hypertext Markup Language)	超文本链接标示语言
HTTP (Hyper Text Transfer Protocol)	超文本传输协议
Hub	集线器
IANA (Internet Assigned Numbers Authority)	是管理IP地址、分配Internet RFC (Request For Comments) 序号以及决定其他与Internet运行和服务有关的序号与定义的机构
ICMP (Internet Control Message Protocol)	网际控制报文协议
ID (Identification)	标识符
IEEE (Institute of Electrical and Electronic Engineers)	美国电子与电气工程师协会
IETF (Internet Engineer Task Force)	Internet工程任务组
IGP (Interior Gateway Protocol)	内部网关协议
IGRP (Interior Gateway Routing Protocol)	内部网关路由协议
IMAP (Internet Message Access Protocol)	因特网报文访问协议
IP (Internet Protocol)	互联网络协议
ISA (Industry Standard Architecture)	工业标准体系结构
ISO (International Standardization Organization)	国际标准化组织
ISP (Internet Service Provider)	Internet服务供货商
ITU-T (International Telecommunication Union Telecommunication)	国际电信联盟电信委员会的标准化部门
LAN (Local Area Network)	局域网
LANE (LAN Emulation)	局域网仿真
LAP (Link Access Protocol)	链路访问协议
LECS (LANE Configure Server)	局域网仿真配置服务器
LES (LANE Server)	LAN仿真服务器
Linux (Linus Torvald's UNIX)	Linus Torvald研制的UNIX
LLC (Logical Link Control)	逻辑链路控制
LSA (Link State Advertisement)	链路状态公告
MAC (Medium Access Control)	媒体访问控制

MAN (Metropolitan Area Network)	城域网
MAU (Medium Attachment Unit)	介质附件单元
MDI (Medium Dependent Interface)	介质相关接口
MF (Mail Forwarding)	邮件转发
MIB (Management Information Base)	管理信息库
MIME (Multipurpose Internet Mail Extensions)	多用途 Internet 邮件扩充协议
MIN (Multi－stage Interconnect Network)	多级互联网络
MII (Medium Independent Interface)	介质无关接口
Modem (Modulator－Demodulator)	调制解调器
MTA (Mail Transfer Agent)	邮件传送代理
MUX (Multiplexing device)	多路复用器
NIC (Network Interface Card)	网络接口卡/网卡/网络适配器
NMP (Network Management Protocol)	网络管理协议
NOC (Network Operation Center)	网络运行中心
Nonpersisten CSMA	非坚持 CSMA
NRM (Normal Response Mode)	正常响应方式
NRZ (Non－Return to Zero)	不归零
NSAP (Network Service Access Point)	网络服务访问点
NVT (Network Virtual Terminal)	网络虚拟终端
OEC (Odd Even Check)	奇偶校验
ODBC (Open Database Connectivity)	开放式数据库互接
OSI/RM (Open System Interconnection/Reference Model)	开放系统互连参考模型
PCM (Pulse Code Modulation)	脉冲编码调制
PCS (Personal Communication System)	个人通信系统——个人通信网
PDH (Plesiochronous Digital Hierarchy)	准同步数字系列
PDU (Protocol Data Unit)	协议数据单元
PICS (Platform for Internet Content Selection)	内容选择平台
PL (Physical Layer)	物理层
PLS (Physical Signaling)	物理信令
PMA (Physical Medium Attachment)	物理媒体
PMD (Physical Medium－Dependent Sublayer)	物理介质相关子层
POH (Path Overhead)	通道开销
POP (Post Office Protocol)	邮局协议
POTS (Plain Old Telephone Service)	简单老式电话服务
PPP (Point－to－Point Protocol)	点对点协议
PSK (Phase－Shift Keying)	相移键控

PT (Payload Type)	净负荷类型
PVC (Permanent Virtual Circuit)	永久虚电路
QoS (Quality of Service)	服务质量
RARP (Reverse Address Resolution Protocol)	反向地址解析协议
RIP (Routing Information Protocol)	路由信息协议
RZ (Return to Zero)	归零
SDH (Synchronous Digital Hierarchy)	同步数字系列
SDLC (Synchronous Data Link Control)	(IBM 研制的) 同步数据链路控制 (协议)
SF (Store and Forward)	存储转发
SLIP (Serial Line Internet Protocol)	串行线网际协定
SMI (Standard of Management Information)	管理信息标准
SMTP (Simple Mail Transfer Protocol)	简单邮件传输协议
SPF (Shortest Path First)	最短路径优先
STM (Synchronous Transfer Mode)	同步传输模式
SVC (Switched Virtual Circuit)	交换虚电路
SYN (Synchronous)	同步
TB (Token Bus)	令牌总线
TC (Transmission Convergence)	传输收敛 (子层)
TCP (Transmission Control Protocol)	传输控制协议
TDD (Time Division Duplex)	时分双工
TELNET (TerminaL NETwork)	远程登录
TL (Total Length)	总长度
TLI (Transport Layer Interface)	传输层界面
TOS (Type Of Service)	服务类型
TPDU (Transport Protocol Data Unit)	传输协议数据单元
TPR (Token Passing Ring)	令牌通行环
TR (Token Ring)	令牌环
TS (Time Stamp)	时间戳
TSAP (Transport Service Access Pointer)	运输服务访问点
TTL (Time To Live)	生存时间
TUG (Tributary Unit Group)	支路单元群
UDP (User Data Protocol)	用户数据报协议
UNIX	贝尔实验室开发的操作系统
URL (Universal Resource Locator)	统一资源定位器
VC (Virtual Calls)	虚呼叫
VC (Virtual Channel)	虚通道
VC (Virtual Circuit)	虚电路

VCC (Virtual Channel Connect)	虚通道连接
WAN (Wide Area Network)	广域网
WDM (Wavelength Division Multiplexing)	光波分多路复用
WWW (World Wide Web)	万维网

参 考 文 献

1 张基温. 信息网络技术原理. 北京:电子工业出版社,2002

2 谢希仁. 计算机网络. 北京:电子工业出版社(第2版),1999

3 张基温. 现代计算机网络教程. 北京:人民邮电出版社,2001

4 Andrew S Tanenbaum. 计算机网络. 第3版. 熊桂喜等译. 北京:清华大学出版社,1998

5 朱稼兴. 计算机网络概念、原理、技术及应用. 北京:北京航空航天大学出版社,1999

6 王钦笙. 数字通信(修订本). 北京:人民邮电出版社,1999

7 陈云志等. SDH & WDM 设备与系统. 北京:人民邮电出版社,1999

8 陈锡生. ATM 交换技术. 北京:人民邮电出版社,2000

9 Adolfo Rodriguez,Tone Gatrell. TCP/IP 权威教程. 第7版. 杨铁勇,李增民译. 北京:清华大学出版社,2002

10 Behrouz A Forouzan,Sophia Chung Fegan. TCP/IP 协议族. 谢希仁译. 北京:清华大学出版社,2001

11 张尧学,王晓春,赵艳标. 计算机网络与 Internet 教程. 北京:清华大学出版社,1999

12 鲁士文. 计算机网络协议和实现技术. 北京:清华大学出版社,2000

13 赵慧玲等. 宽带 Internet 网络技术. 北京:电子工业出版社,1999

14 张公忠,陈锦章. 当代组网技术. 北京:清华大学出版社,2000

15 吴礼发. 网络程序设计教程. 北京:北京希望电子出版社,2002

16 陈远森,邓可,杜威等. UNIX 网络实用编程技术. 北京:中国水利水电出版社,2000

17 W Richard Stevens. UNIX 网络编程. 第1卷. 施振川等译. 北京:清华大学出版社,1999

18 汪蒲阳. 因特网应用编程. 北京:清华大学出版社,2000

19 张斌,高波等. Linux 网络编程. 北京:清华大学出版社,2000